

QUANTIFYING UNCERTAINTY. BAYESIAN NETWORKS

CHAPTER 13.1–5 14.1–2
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Administrativa
- ◇ Uncertainty
- ◇ Probability
- ◇ Syntax and Semantics
- ◇ Inference by enumeration
- ◇ Independence and Bayes' Rule
- ◇ Bayesian networks structure and semantics

Admin

Moodle: moodle.cs.utcluj.ro, Sisteme Inteligente, Seria B, Sem. 2, 2020/2021
(main way of communication)

References:

- 1) R. R. Slavescu
Sisteme Inteligente 2020/2021 - Lecture Notes
- 2) S. Russell, P. Norvig.
Artificial Intelligence: A Modern Approach (3rd ed.). Prentice Hall, 2009.
- 2') S. Russell, P. Norvig.
Artificial Intelligence: A Modern Approach (4rd ed.). Prentice Hall, 2020.
- 3) A. Géron.
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow.
O'Reilly Media, 2019

More references might be added in the lectures to come

Admin

Grades:

Lab L: 40% (2 assignments, A1 and A2, deadlines: TBA)

Final exam E: 60% (2 marks, E1 and E2)

To pass:

$$(0.25*A1 + 0.75*A2 \geq 5.00) \text{ AND} \\ (0.1*A1 + 0.3*A2 + 0.3*E1 + 0.3*E2) \geq 5.00$$

Bonuses for Kahoot competition (round winners 1p / Yellow Jersey top 10: 1.5, 1.4, ... , 0.6p), lecture contributions or original solutions to challenges.

Content

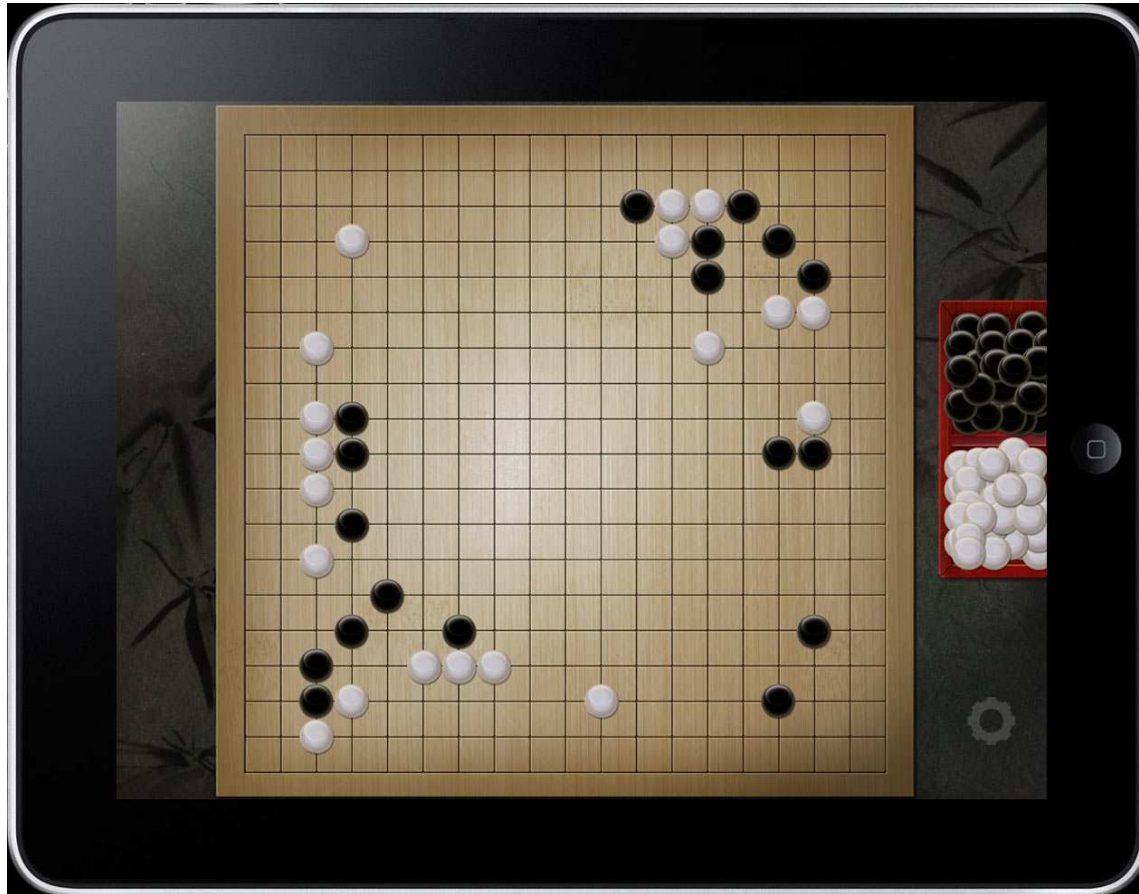
- ◇ Uncertainty: exact reasoning on inexact data
- ◇ Machine Learning ML: learn from and make predictions on data with no instructions flow
- ◇ Natural Language Processing NLP: interaction computers – NL texts:
 - part-of-speech tagging
 - parsing
 - named entity recognition
 - understanding
 - sentiment analysis
 - co-reference resolution etc.

Some state-of-the-art applications

- ◇ Watson: question answering system which (who?) can play Jeopardy!
- ◇ Netflix prize: predict movies' user ratings, based on prior user rating
- ◇ Libratus: plays Poker (Texas hold 'em) against humans
- ◇ AlphaGo (by Alphabet's DeepMind): plays Go

Some state-of-the-art applications

AlphaGo (by Alphabet's DeepMind): plays Go



Go game example (image source: <https://www.gametableapp.com>)

Some state-of-the-art applications

AlphaGo (by Alphabet's DeepMind): plays Go

1) "AlphaGo Lee"

March 2016: defeats 4-1 Lee Sedol (9 dan, 18 international titles)

2) AlphaGo Master

Dec 2016: 60 victories in a row against human professionals

May 2017: defeats World Champion Ke Jie in a 3-game match

3) AlphaGo Zero: no data from human games, just self-playing
surpasses: AlphaGo "Lee" in 3 days, AlphaGo Master in 21 days,
to become arguably the best ever player after 40 days

4) AlphaZero: Chess, Go, Shogi

AlphaGo = reinforcement learning + search + neural networks

5) MuZero: Chess, Go, Shogi without knowing the rules

Uncertainty

Let action A_t = leave for airport t minutes before flight departure.
Will A_t get me there on time?

Problems:

- 1) impossibility to specify all preconditions of an action (if no engine trouble and no traffic jam etc.)
- 2) partial observability (road state, other drivers' plans, etc.)
- 3) uncertainty in action outcomes (flat tire, etc.)

Hence a purely logical approach either

- 1) risks falsehood: " A_{25} will get me there on time"
- or 2) leads to conclusions that are too weak for decision making:
- " A_{25} will get me there on time if no accident and no rain etc etc."

(A_{1440} might reasonably be said to get me there on time but I'd have to stay overnight in the airport ...)

Rational decision: takes into account both goal importance and its likelihood

Probability

A sentence can have a **degree of belief** (not just true/false):

"Given the available evidence, A_{25} will get me to the airport on time with probability 0.04."

Probabilistic assertions **summarize** effects of

laziness: failure to enumerate exceptions, qualifications, etc.

practical ignorance: lack of relevant facts, initial conditions, etc.

theoretical ignorance: lack of a comprehensive domain theory

Subjective or **Bayesian** probabilities relate propositions to one's state of knowledge, e.g., $P(A_{25}|\text{no reported accidents}) = 0.06$

These are **not** claims of a "probabilistic tendency" in the current situation (but might be learned from past experience of similar situations)

Probabilities of propositions change with new evidence:

e.g., $P(A_{25}|\text{no reported accidents, 5 a.m.}) = 0.15$

Making decisions under uncertainty

Suppose I believe the following:

$$P(A_{25} \text{ gets me there on time} | \dots) = 0.04$$

$$P(A_{90} \text{ gets me there on time} | \dots) = 0.70$$

$$P(A_{120} \text{ gets me there on time} | \dots) = 0.95$$

$$P(A_{1440} \text{ gets me there on time} | \dots) = 0.9999$$

Which action to choose?

Depends on my **preferences** for missing flight vs. airport cuisine, etc.

Utility theory is used to represent and infer preferences

Decision theory = utility theory + probability theory

Probability basics

Sample space Ω : set of all possible worlds, e.g., 36 possible rolls of 2 dice.

Sample point/possible world/atomic event is any $\omega \in \Omega$, e.g., 1:6.

A probability space or probability model is a sample space with an assignment $P(\omega)$ for every $\omega \in \Omega$ s.t. the following axioms of probability hold:

(1) $0 \leq P(\omega) \leq 1$; (2) $\sum_{\omega} P(\omega) = 1$; (3) $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$
e.g., $P(1 : 6) = P(2 : 6) = \dots = 1/36$.

An event A is any subset of Ω . We have $P(A) = \sum_{\{\omega \in A\}} P(\omega)$

e.g., $P(\text{sum}11) = P(5 : 6) + P(6 : 5) = 1/36 + 1/36 = 1/18$

A random variable has a name (e.g. Sum7or11) and a domain (e.g., {True,False})

A probability model P gives a probability distribution for any $X = x_i$.

$P(\text{Sum7or11}=\text{True}) = ?$

Propositions

Proposition = the **set of sample points** where the proposition is true.

Given Boolean random variables A and B :

event a = set of sample points where $A(\omega) = \text{true}$

event $\neg a$ = set of sample points where $A(\omega) = \text{false}$

event $a \wedge b$ = points where $A(\omega) = \text{true}$ and $B(\omega) = \text{true}$

Often in AI applications, the sample points are **defined** by the values of a set of random variables, i.e., the sample space is the Cartesian product of the ranges of the variables

With Boolean variables, sample point = propositional logic model

e.g., $A = \text{true}$, $B = \text{false}$, or $a \wedge \neg b$.

Proposition = disjunction of atomic events in which it is true

e.g., $(a \vee b) \equiv (\neg a \wedge b) \vee (a \wedge \neg b) \vee (a \wedge b)$

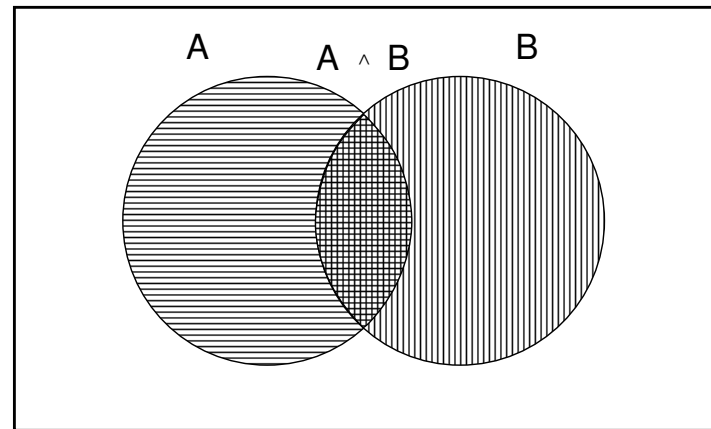
$\Rightarrow P(a \vee b) = P(\neg a \wedge b) + P(a \wedge \neg b) + P(a \wedge b)$

Why use probability?

The definitions imply that certain logically related events must have related probabilities

E.g., for the last **axiom of probability** : $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$

True



de Finetti (1931): an agent who bets according to probabilities that violate these axioms can be forced to bet so as to lose money regardless of outcome.

Syntax for propositions

Propositional or Boolean random variables

e.g., *Cavity* (do I have a cavity?)

Cavity = true is a proposition, also written *cavity*

Discrete random variables (finite or infinite)

e.g., *Weather* is one of $\langle \textit{sunny}, \textit{rain}, \textit{cloudy}, \textit{snow} \rangle$

Weather = rain is a proposition

Values must be exhaustive and mutually exclusive

Continuous random variables (bounded or unbounded)

e.g., *Temp = 21.6*; also allow, e.g., *Temp < 22.0*.

Arbitrary Boolean combinations of basic propositions

Prior probability

Prior or unconditional probabilities of propositions

e.g., $P(\text{Cavity} = \text{true}) = 0.1$ and $P(\text{Weather} = \text{sunny}) = 0.72$
correspond to belief prior to arrival of any (new) evidence

Probability distribution gives values for all possible assignments:

$\mathbf{P}(\text{Weather}) = \langle 0.72, 0.1, 0.08, 0.1 \rangle$ (normalized, i.e., sums to 1)

Joint probability distribution for a set of r.v.s gives the probability of every atomic event on those r.v.s (i.e., every sample point)

$\mathbf{P}(\text{Weather}, \text{Cavity}) =$ a 4×2 matrix of values:

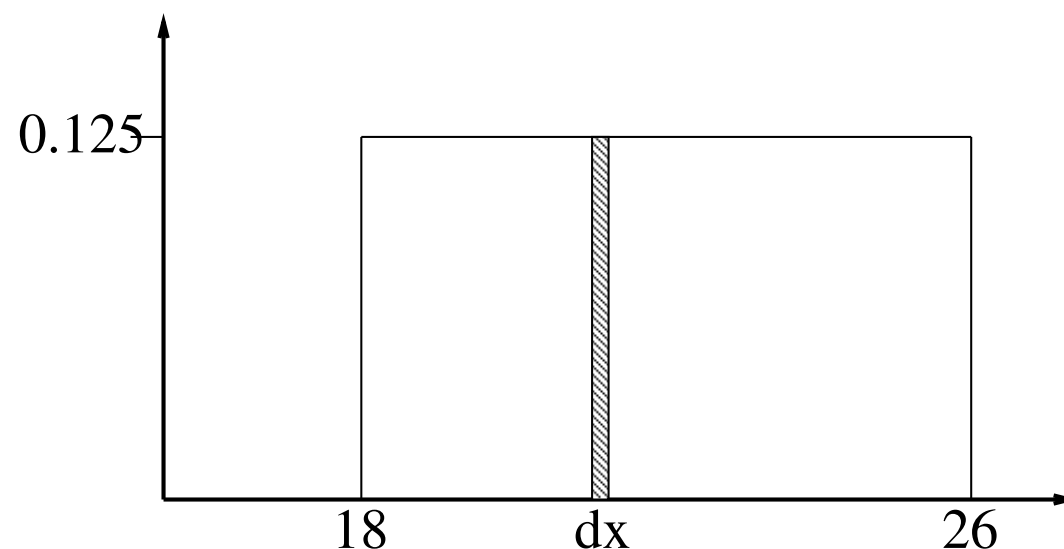
<i>Weather =</i>	<i>sunny</i>	<i>rain</i>	<i>cloudy</i>	<i>snow</i>
<i>Cavity = true</i>	0.144	0.02	0.016	0.02
<i>Cavity = false</i>	0.576	0.08	0.064	0.08

Every question about a domain can be answered by the joint distribution because every event is a sum of sample points

Probability for continuous variables

Express distribution as a parameterized function of value:

$$P(X = x) = U[18, 26](x) = \text{uniform density between } 18 \text{ and } 26$$



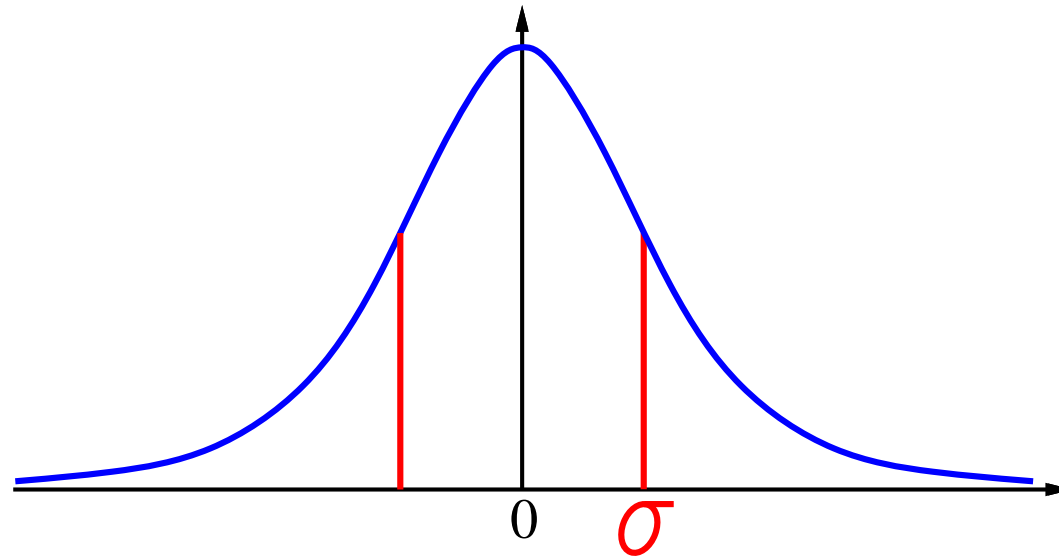
Here P is a **density** and has a unit measure; integrates to 1.

$P(X = 20.5) = 0.125$ really means

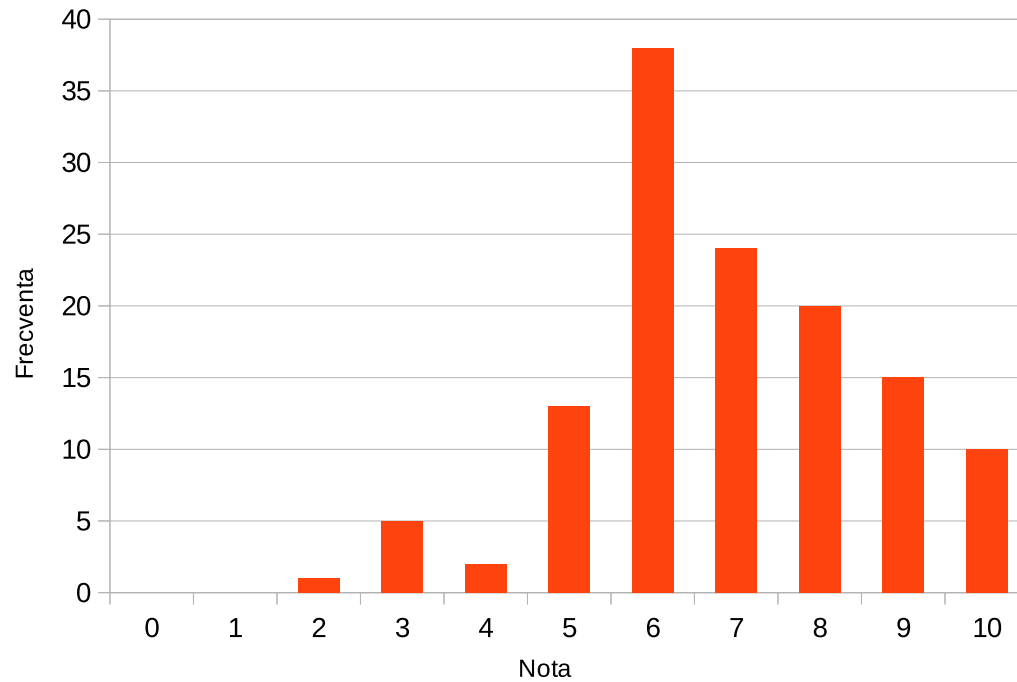
$$\lim_{dx \rightarrow 0} (P(20.5 \leq X \leq 20.5 + dx) / dx) = 0.125$$

Gaussian density

$$P(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$$



Gaussian density - the reality



Conditional probability

Conditional or posterior probabilities

e.g., $P(\text{cavity}|\text{toothache}) = 0.8$

i.e., **given that toothache is all I know**

NOT “if *toothache* then 80% chance of *cavity*”

(Notation for conditional distributions:

$P(\text{Cavity}|\text{Toothache}) = 2\text{-element vector of 2-element vectors})$

If we know more, e.g., *cavity* is also given, then we have

$P(\text{cavity}|\text{toothache}, \text{cavity}) = 1$

Note: the less specific belief **remains valid** after more evidence arrives, but is not always **useful**

New evidence may be irrelevant, allowing simplification, e.g.,

$P(\text{cavity}|\text{toothache}, \text{49ersWin}) = P(\text{cavity}|\text{toothache}) = 0.8$

This kind of inference, sanctioned by domain knowledge, is crucial

Conditional probability

Definition of conditional probability:

$$P(a|b) = \frac{P(a \wedge b)}{P(b)} \text{ if } P(b) \neq 0$$

Product rule gives an alternative formulation:

$$P(a \wedge b) = P(a|b)P(b) = P(b|a)P(a)$$

A general version holds for whole distributions, e.g.,

$$\mathbf{P}(Weather, Cavity) = \mathbf{P}(Weather|Cavity)\mathbf{P}(Cavity)$$

(View as a 4×2 set of equations, **not** matrix mult.)

Chain rule is derived by successive application of product rule:

$$\begin{aligned} \mathbf{P}(X_1, \dots, X_n) &= \mathbf{P}(X_1, \dots, X_{n-1}) \mathbf{P}(X_n|X_1, \dots, X_{n-1}) \\ &= \mathbf{P}(X_1, \dots, X_{n-2}) \mathbf{P}(X_{n-1}|X_1, \dots, X_{n-2}) \mathbf{P}(X_n|X_1, \dots, X_{n-1}) \\ &= \dots \\ &= \prod_{i=1}^n \mathbf{P}(X_i|X_1, \dots, X_{i-1}) \end{aligned}$$

Inference by enumeration

Start with the joint distribution:

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	.108	.012	.072	.008
$\neg$ <i>cavity</i>	.016	.064	.144	.576

For any proposition ϕ , sum the atomic events where it is true:

$$P(\phi) = \sum_{\omega: \omega \models \phi} P(\omega)$$

Inference by enumeration

Start with the joint distribution:

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	.108	.012	.072	.008
$\neg$ <i>cavity</i>	.016	.064	.144	.576

For any proposition ϕ , sum the atomic events where it is true:

$$P(\phi) = \sum_{\omega: \omega \models \phi} P(\omega)$$

$$P(\text{toothache}) = 0.108 + 0.012 + 0.016 + 0.064 = 0.2$$

This procedure is sometimes called **marginalization**

Inference by enumeration

Start with the joint distribution:

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	.108	.012	.072	.008
$\neg$ <i>cavity</i>	.016	.064	.144	.576

For any proposition ϕ , sum the atomic events where it is true:

$$P(\phi) = \sum_{\omega: \omega \models \phi} P(\omega)$$

$$P(\text{cavity} \vee \text{toothache}) = 0.108 + 0.012 + 0.072 + 0.008 + 0.016 + 0.064 = 0.28$$

Inference by enumeration

Start with the joint distribution:

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	.108	.012	.072	.008
$\neg$ <i>cavity</i>	.016	.064	.144	.576

Can also compute conditional probabilities:

$$\begin{aligned} P(\neg \text{cavity} | \text{toothache}) &= \frac{P(\neg \text{cavity} \wedge \text{toothache})}{P(\text{toothache})} \\ &= \frac{0.016 + 0.064}{0.108 + 0.012 + 0.016 + 0.064} = 0.4 \end{aligned}$$

Normalization

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	.108	.012	.072	.008
$\neg$ <i>cavity</i>	.016	.064	.144	.576

Denominator $.108 + .012 + .016 + .064$ works as a **normalization constant** α

$$\begin{aligned}
 \mathbf{P}(Cavity|toothache) &= \alpha \mathbf{P}(Cavity, toothache) \\
 &= \alpha [\mathbf{P}(Cavity, toothache, catch) + \mathbf{P}(Cavity, toothache, \neg catch)] \\
 &= \alpha [\langle 0.108, 0.016 \rangle + \langle 0.012, 0.064 \rangle] \\
 &= \alpha \langle 0.12, 0.08 \rangle = \langle 0.6, 0.4 \rangle
 \end{aligned}$$

General idea: compute distribution on query variable by fixing **evidence variables** and summing over **hidden variables**

Inference by enumeration, contd.

Let $\mathbf{A}$ be the set of all variables. Typically, we want the posterior joint distribution of the query variables $\mathbf{Q}$ given specific values $\mathbf{e}$ for the evidence variables $\mathbf{E}$

Let the hidden variables be $\mathbf{H} = \mathbf{A} - \mathbf{Q} - \mathbf{E}$

Then the required summation of joint entries is done by summing out the hidden variables:

$$\mathbf{P}(\mathbf{Q}|\mathbf{E} = \mathbf{e}) = \alpha \mathbf{P}(\mathbf{Q}, \mathbf{E} = \mathbf{e}) = \alpha \sum_{\mathbf{h}} \mathbf{P}(\mathbf{Q}, \mathbf{E} = \mathbf{e}, \mathbf{H} = \mathbf{h})$$

The terms in the summation are joint entries because $\mathbf{Q}$, $\mathbf{E}$, and $\mathbf{H}$ together exhaust the set of random variables

Obvious problems:

- 1) Worst-case time complexity $O(d^n)$ where d is the largest arity
- 2) Space complexity $O(d^n)$ to store the joint distribution
- 3) How to find the numbers for $O(d^n)$ entries???

Quiz

In the movie "Supravietuitorul", two characters, Comisarul and Cristian Vasile, play Russian Roulette pointing their revolvers to each other. Each round, the number of cartridges in a revolver is increased, then the chamber is rolled. What is the probability of the following event: both characters reach round 3 of the game?

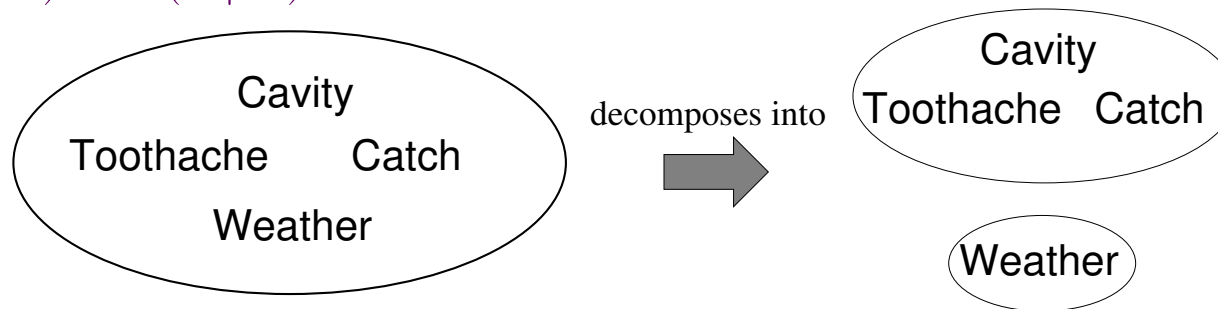
Independence

A and B are (unconditionally) independent iff

$$\mathbf{P}(A|B) = \mathbf{P}(A) \quad \text{or} \quad \mathbf{P}(B|A) = \mathbf{P}(B) \quad \text{or} \quad \mathbf{P}(A, B) = \mathbf{P}(A)\mathbf{P}(B)$$

A and B are conditionally independent given X iff

$$\mathbf{P}(A|B, X) = \mathbf{P}(A|X)$$



$$\begin{aligned} &\mathbf{P}(\textit{Toothache}, \textit{Catch}, \textit{Cavity}, \textit{Weather}) \\ &= \mathbf{P}(\textit{Toothache}, \textit{Catch}, \textit{Cavity})\mathbf{P}(\textit{Weather}) \end{aligned}$$

32 entries reduced to 12; for n independent biased coins, $2^n \rightarrow n$

Absolute independence powerful but rare. E. g., dentistry is a large field with hundreds of variables, none of which are independent.

Conditional independence

$\mathbf{P}(\textit{Toothache}, \textit{Cavity}, \textit{Catch})$ has $2^3 - 1 = 7$ independent entries

If I have a cavity, the probability that the probe catches in it doesn't depend on whether I have a toothache:

$$(1) \ P(\textit{catch}|\textit{toothache}, \textit{cavity}) = P(\textit{catch}|\textit{cavity})$$

The same independence holds if I haven't got a cavity:

$$(2) \ P(\textit{catch}|\textit{toothache}, \neg \textit{cavity}) = P(\textit{catch}|\neg \textit{cavity})$$

Catch is **conditionally independent** of *Toothache* given *Cavity*:

$$\mathbf{P}(\textit{Catch}|\textit{Toothache}, \textit{Cavity}) = \mathbf{P}(\textit{Catch}|\textit{Cavity})$$

(once I know **Cavity**, learning **Toothache** gives me no extra info)

Equivalent statements:

$$\mathbf{P}(\textit{Toothache}|\textit{Catch}, \textit{Cavity}) = \mathbf{P}(\textit{Toothache}|\textit{Cavity})$$

$$\mathbf{P}(\textit{Toothache}, \textit{Catch}|\textit{Cavity}) = \mathbf{P}(\textit{Toothache}|\textit{Cavity})\mathbf{P}(\textit{Catch}|\textit{Cavity})$$

Conditional independence contd.

Write out full joint distribution using chain rule:

$$\begin{aligned} & \mathbf{P}(Toothache, Catch, Cavity) \\ &= \mathbf{P}(Toothache|Catch, Cavity)\mathbf{P}(Catch, Cavity) \\ &= \mathbf{P}(Toothache|Catch, Cavity)\mathbf{P}(Catch|Cavity)\mathbf{P}(Cavity) \\ &= \mathbf{P}(Toothache|Cavity)\mathbf{P}(Catch|Cavity)\mathbf{P}(Cavity) \end{aligned}$$

I.e., $2 + 2 + 1 = 5$ independent numbers (equations 1 and 2 remove 2)

In most cases, the use of conditional independence reduces the size of the representation of the joint distribution from exponential in n to linear in n .

Conditional independence is our most basic and robust form of knowledge about uncertain environments.

Bayes' Rule

Product rule $P(a \wedge b) = P(a|b)P(b) = P(b|a)P(a)$

$$\Rightarrow \text{Bayes' rule } P(a|b) = \frac{P(b|a)P(a)}{P(b)} \quad (\text{posterior} = \text{likelihood} * \text{prior} / \text{norm})$$

or in distribution form

$$\mathbf{P}(Y|X) = \frac{\mathbf{P}(X|Y)\mathbf{P}(Y)}{\mathbf{P}(X)} = \alpha \mathbf{P}(X|Y)\mathbf{P}(Y)$$

Useful for assessing **diagnostic** probability from **causal** probability:

$$P(Cause|Effect) = \frac{P(Effect|Cause)P(Cause)}{P(Effect)}$$

E.g., let M be meningitis ($P(m) = 0.0001$), S be stiff neck ($P(s) = 0.1$).

If $P(s|m) = 0.8$

$$P(m|s) = \frac{P(s|m)P(m)}{P(s)} = \frac{0.8 \times 0.0001}{0.1} = 0.0008 \text{ (still small!)}$$

Bayesian networks

A simple, graphical notation for conditional independence assertions and hence for compact specification of full joint distributions

Syntax:

- a set of nodes, one per variable

- a directed, acyclic graph (link $\approx$ “directly influences”)

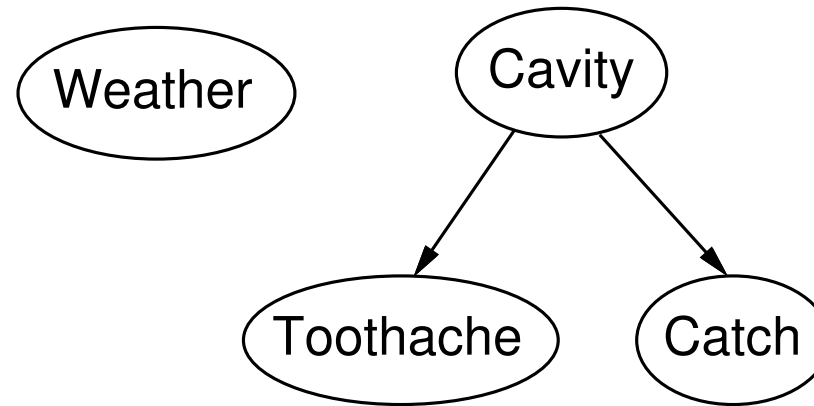
- a conditional distribution for each node given its parents:

$$\mathbf{P}(X_i | \text{Parents}(X_i))$$

In the simplest case, conditional distribution represented as a **conditional probability table** (CPT) giving the distribution over X_i for each combination of parent values

Example

Topology of network encodes conditional independence assertions:



Weather is independent of the other variables

Toothache and *Catch* are conditionally independent given *Cavity*

Example

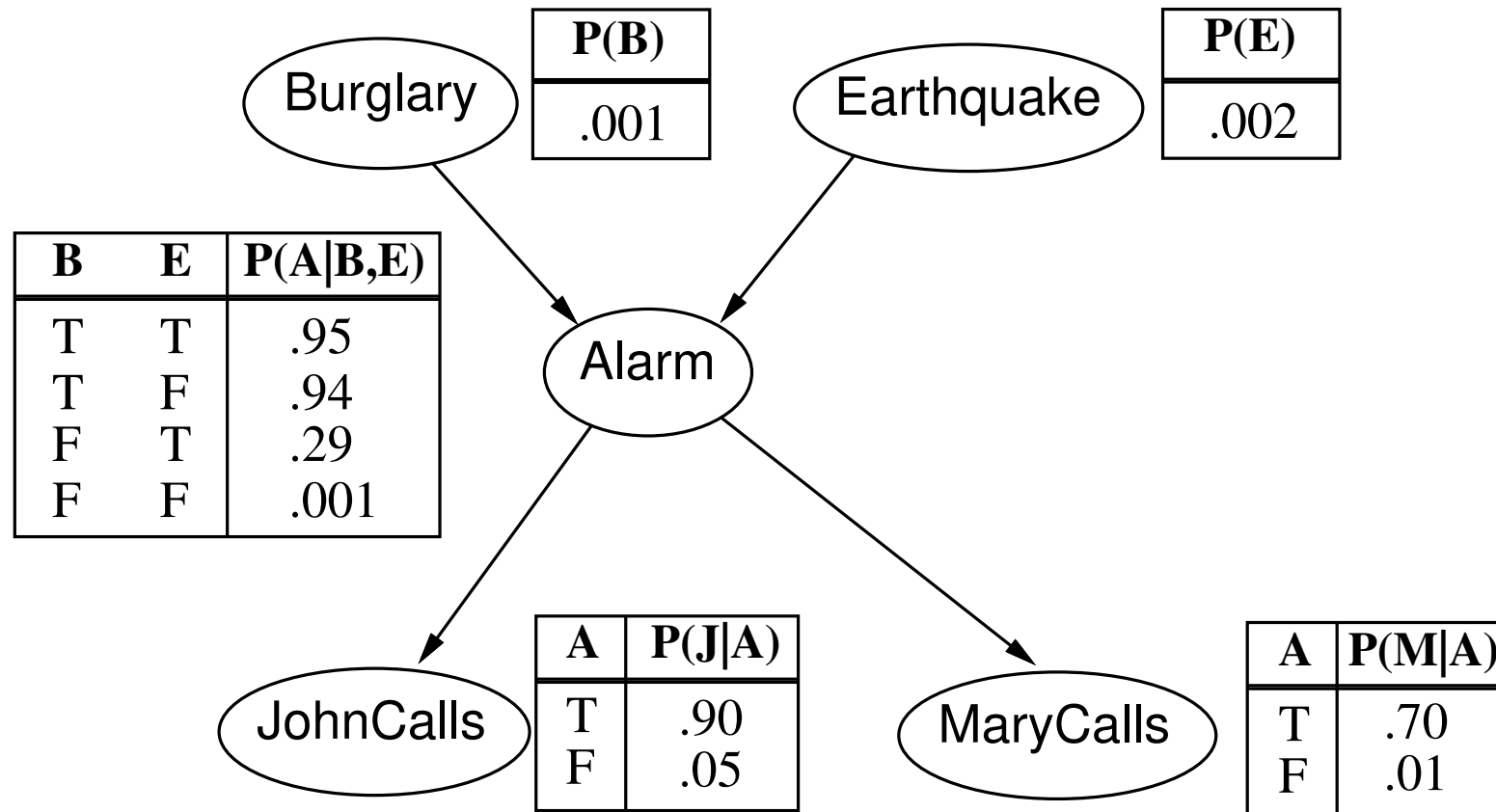
I installed a new alarm in my house in order to detect burglars and asked my two neighbors, John and Mary, to call me if they hear the alarm ringing. Neither of my two neighbors is 100% reliable when talking about hearing the alarm. Sometimes the alarm is set off by minor earthquakes. I'm at work, neighbor John calls to say my alarm is ringing, but neighbor Mary doesn't call. Is there a burglar?

Variables: *Burglar*, *Earthquake*, *Alarm*, *JohnCalls*, *MaryCalls*

Network topology reflects “causal” knowledge:

- A burglar can set the alarm off
- An earthquake can set the alarm off
- The alarm can cause Mary to call
- The alarm can cause John to call

Example contd.



Compactness

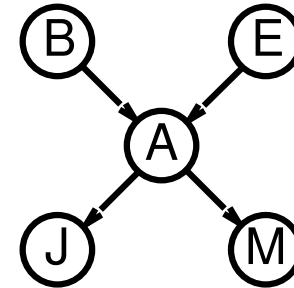
A CPT for Boolean X_i with k Boolean parents has 2^k rows for the combinations of parent values

Each row requires one number p for $X_i = \text{true}$ (the number for $X_i = \text{false}$ is just $1 - p$)

If each variable has no more than k parents, the complete network requires $O(n \cdot 2^k)$ numbers

I.e., grows linearly with n , vs. $O(2^n)$ for the full joint distribution

For burglary net, $1 + 1 + 4 + 2 + 2 = 10$ numbers (vs. $2^5 - 1 = 31$)



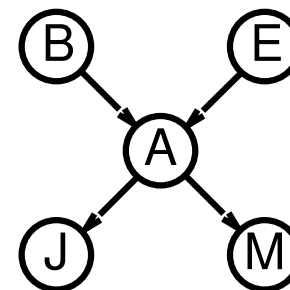
Global semantics

Global semantics defines the full joint distribution as the product of the local conditional distributions:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(X_i))$$

e.g., $P(j \wedge m \wedge a \wedge \neg b \wedge \neg e)$

=



Global semantics

Global semantics defines the full joint distribution as the product of the local conditional distributions:

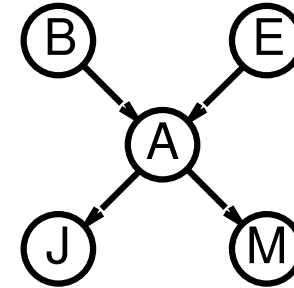
$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(X_i))$$

e.g., $P(j \wedge m \wedge a \wedge \neg b \wedge \neg e)$

$$= P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b)P(\neg e)$$

$$= 0.9 \times 0.7 \times 0.001 \times 0.999 \times 0.998$$

$$\approx 0.00063$$



Summary

Probability is a rigorous formalism for uncertain knowledge

Joint probability distribution specifies probability of every atomic event

Queries can be answered by summing over atomic events

Independence and conditional independence provide the tools for reducing the joint size

Bayes nets provide a natural representation for (causally induced) conditional independence

Topology + CPTs = compact representation of joint distribution

Generally easy for (non)experts to construct

INDEPENDENCE IN BAYESIAN NETWORKS. EXACT INFERENCE IN BAYESIAN NETWORKS

CHAPTER 14.2.2, 14.4
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ An algorithm for establishing independence in Bayes Networks
- ◇ Exact inference in Bayesian nets: enumeration vs. variable elimination

Quiz

Show FROM THE FIRST PRINCIPLES that: $P(\neg a \mid b) = 1 - P(a \mid b)$.

We'll try to prove the equivalent form $P(b) = P(a \wedge b) + P(\neg a \wedge b)$

"Show from the first principles" means that each step has to be justified using axioms or previously established theorems.

$$\begin{aligned} &P(b) \\ &= P(\text{false} \vee b) \text{ (Why?)} \\ &= P((a \wedge \neg a) \vee b) \\ &= P((a \wedge b) \vee (\neg a \wedge b)) \\ &= P(a \wedge b) + P(\neg a \wedge b) - P((a \wedge b) \wedge (\neg a \wedge b)) \text{ (Why?)} \\ &= P(a \wedge b) + P(\neg a \wedge b) - 0 \text{ (Why?)} \\ &= P(a \wedge b) + P(\neg a \wedge b) \end{aligned}$$

Quiz

A medical test for a certain disease has a sensitivity (i.e., $P(t|b)$) of 99.75%. Its specificity (i.e., $P(\neg t|\neg b)$) is also 99.75%. John has been tested for the disease with this test and the result is positive. The disease prevalence (frequency) is 0.000001. (So the disease is rare: it hits only one individual in 1,000,000 people). Knowing the above, which is the probability that John actually has the disease?

$$\begin{aligned} P(b|t) &= \frac{P(t|b) * P(b)}{P(t)} = \\ &= \frac{P(t|b) * P(b)}{P(t \wedge b) + P(t \wedge \neg b)} \\ &= \frac{P(t|b) * P(b)}{P(t|b) * P(b) + P(t|\neg b) * P(\neg b)} \\ &= 0.0004 \end{aligned}$$

Remember: Global semantics

Global semantics defines the full joint distribution as the product of the local conditional distributions:

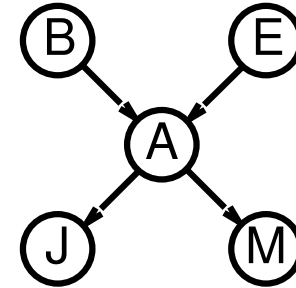
$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(X_i))$$

e.g., $P(j \wedge m \wedge a \wedge \neg b \wedge \neg e)$

$$= P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b)P(\neg e)$$

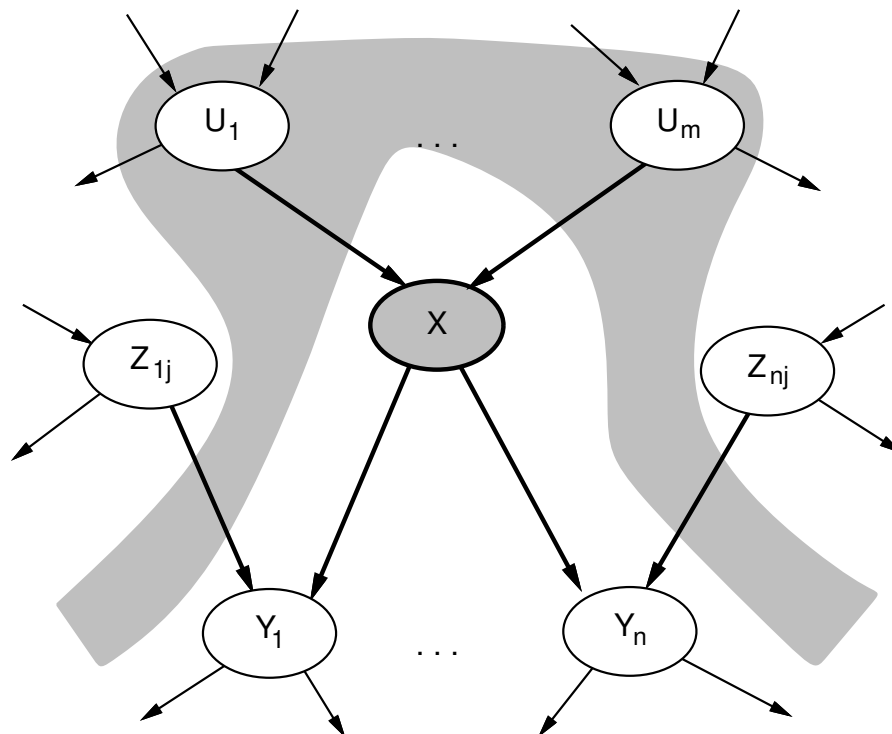
$$= 0.9 \times 0.7 \times 0.001 \times 0.999 \times 0.998$$

$$\approx 0.00063$$



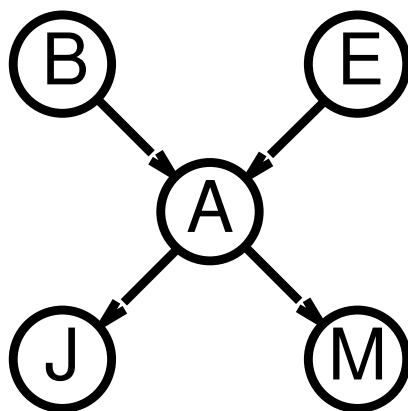
Local semantics

Local semantics: each node is conditionally independent of its nondescendants given its parents



Local semantics

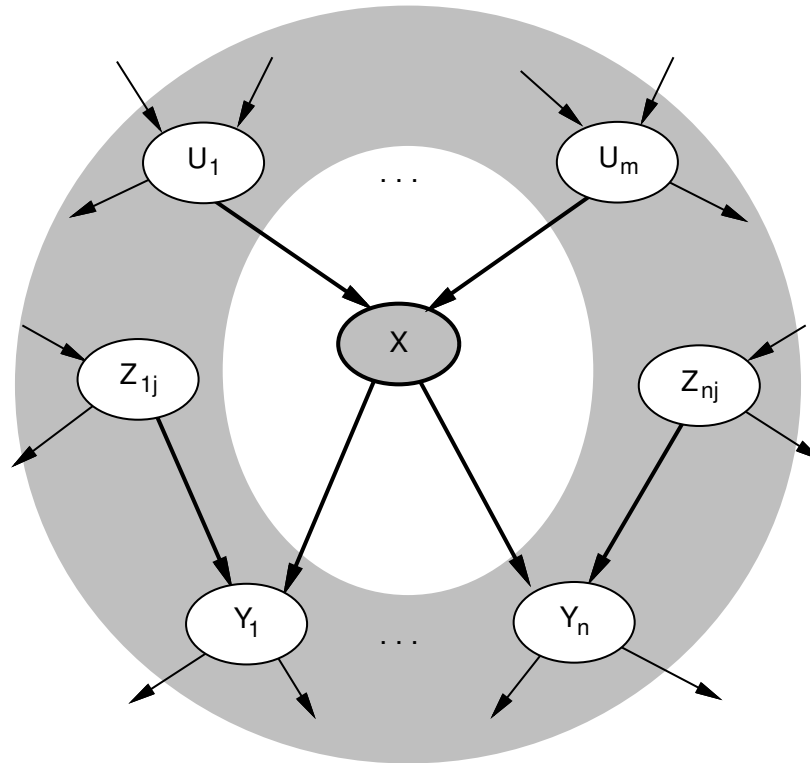
Local semantics: each node is conditionally independent of its nondescendants given its parents



E.g., J is independent of B , E and M given A .

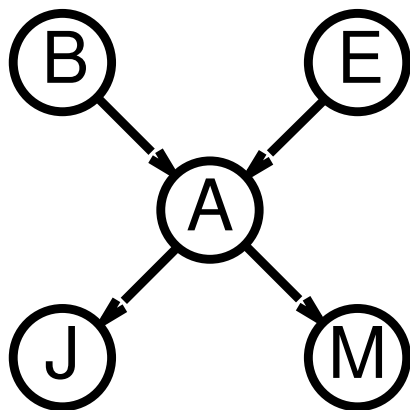
Local semantics: Markov blanket

Each node is conditionally independent of all others given its
Markov blanket: parents + children + children's parents



Local semantics: Markov blanket

Each node is conditionally independent of all others given its Markov blanket



E.g., B is independent of J and M given A and E .

Local and global semantics

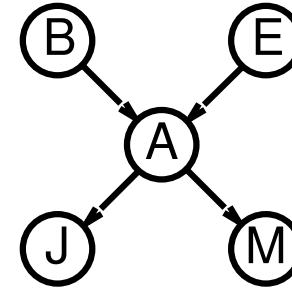
Theorem: Local semantics $\Leftrightarrow$ global semantics

Irrelevant variables

Consider the query $\mathbf{P}(JohnCalls|Burglary = true)$

$$\mathbf{P}(J|b) = \alpha P(b) \sum_e P(e) \sum_a P(a|b, e) \mathbf{P}(J|a) \sum_m P(m|a)$$

Sum over m is identically 1; M is **irrelevant** to the query



Theorem 1: Y is irrelevant unless $Y \in Ancestors(\{X\} \cup \mathbf{E})$, where $\{X\}$ is the set of query variables

Here:

$X = JohnCalls$

$\mathbf{E} = \{Burglary\}$

$Ancestors(\{X\} \cup \mathbf{E}) = \{Alarm, Earthquake\}$

Therefore:

$MaryCalls$ is irrelevant

Irrelevant variables contd.

Definition (moral graph of a Bayes net): marry all pairs of parents in the graph of ancestors of the (checked+evidence) variables, then drop arrows

Definition (m-separation): **A** is m-separated from **B** by **C** iff it is separated by **C** in the moral graph

Theorem 2: **Y** is irrelevant if m-separated from **X** by **E**

Consider the query $\mathbf{P}(\textit{JohnCalls} | \textit{Alarm} = \textit{true})$

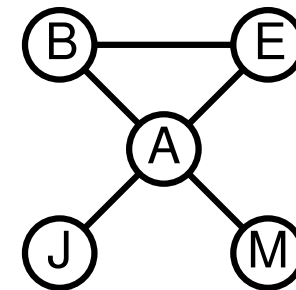
Here:

$X = \textit{JohnCalls}$

$\mathbf{E} = \{\textit{Alarm}\}$

$\textit{Ancestors}(\{X\} \cup \mathbf{E}) = \{\textit{Burglary}, \textit{Earthquake}\}$

For this query, both *Burglary* and *Earthquake* are irrelevant



Establishing independence

1. ancestors graph for evidence variables and variables to be checked for independence
2. moral graph: draw an edge between every pair of parents of every node with more than one parent
3. de-orient all arcs
4. eliminate evidence nodes and all edges incident on them
5. verify the connectivity of variables to be checked

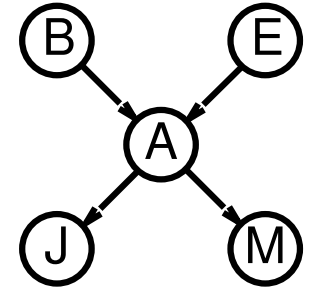
Independence of the checked variables based on the Bayes Network topology **alone**:

- disconnected: they are guaranteed to be independent
- connected by a path: the network does not require the variables to be independent (still, their independence could hold if established **numerically**)

Establishing independence: example

Query: $P(\text{JohnCalls} | \text{Alarm} = \text{true})$

Are *Burglary* and *Earthquake* irrelevant?



1. ancestors graph for evidence variables and variables to be checked
JohnCalls, Alarm, Burglary, Earthquake
2. moral graph: draw an edge between every pair of parents of every node with more than one parent
edge between *Burglary* and *Earthquake*
3. de-orient all arcs
4. eliminate evidence nodes and all edges incident on them
JohnCalls insulated, *Burglary* connected to *Earthquake*
5. verify the connectivity of variables to be checked

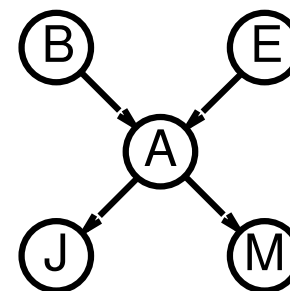
JohnCalls disconnected from both *Burglary* and *Earthquake*, therefore
 $P(J | A, B, E) = P(J | A)$

Inference by enumeration

Slightly smarter way to sum out variables from the joint without actually constructing its explicit representation

Simple query on the burglary network:

$$\begin{aligned}\mathbf{P}(B|j, m) &= \mathbf{P}(B, j, m) / P(j, m) \\ &= \alpha \mathbf{P}(B, j, m) \\ &= \alpha \sum_e \sum_a \mathbf{P}(B, e, a, j, m)\end{aligned}$$



Rewrite full joint entries using product of CPT entries

$$\begin{aligned}\mathbf{P}(B|j, m) &= \alpha \sum_e \sum_a \mathbf{P}(B)P(e)\mathbf{P}(a|B, e)P(j|a)P(m|a) \\ &= \alpha \mathbf{P}(B) \sum_e P(e) \sum_a \mathbf{P}(a|B, e)P(j|a)P(m|a)\end{aligned}$$

Recursive depth-first enumeration: $O(n)$ space, $O(d^n)$ time

Enumeration algorithm (Top Down)

function **ENUMERATION-ASK**($X, \mathbf{e}, bn$) **returns** a distribution over X

inputs: X , the query variable

$\mathbf{e}$, observed values for variables $\mathbf{E}$

bn , a Bayesian network with variables $\{X\} \cup \mathbf{E} \cup \mathbf{Y}$

$Q(X) \leftarrow$ a distribution over X , initially empty

for each value x_i of X **do**

 extend $\mathbf{e}$ with value x_i for X

$Q(x_i) \leftarrow \text{ENUMERATE-ALL}(\text{VARS}[bn], \mathbf{e})$

return $\text{NORMALIZE}(Q(X))$

function **ENUMERATE-ALL**($vars, \mathbf{e}$) **returns** a real number

if $\text{EMPTY?}(vars)$ **then return** 1.0

$Y \leftarrow \text{FIRST}(vars)$

if Y has value y in $\mathbf{e}$

then return $P(y \mid Pa(Y)) \times \text{ENUMERATE-ALL}(\text{REST}(vars), \mathbf{e})$

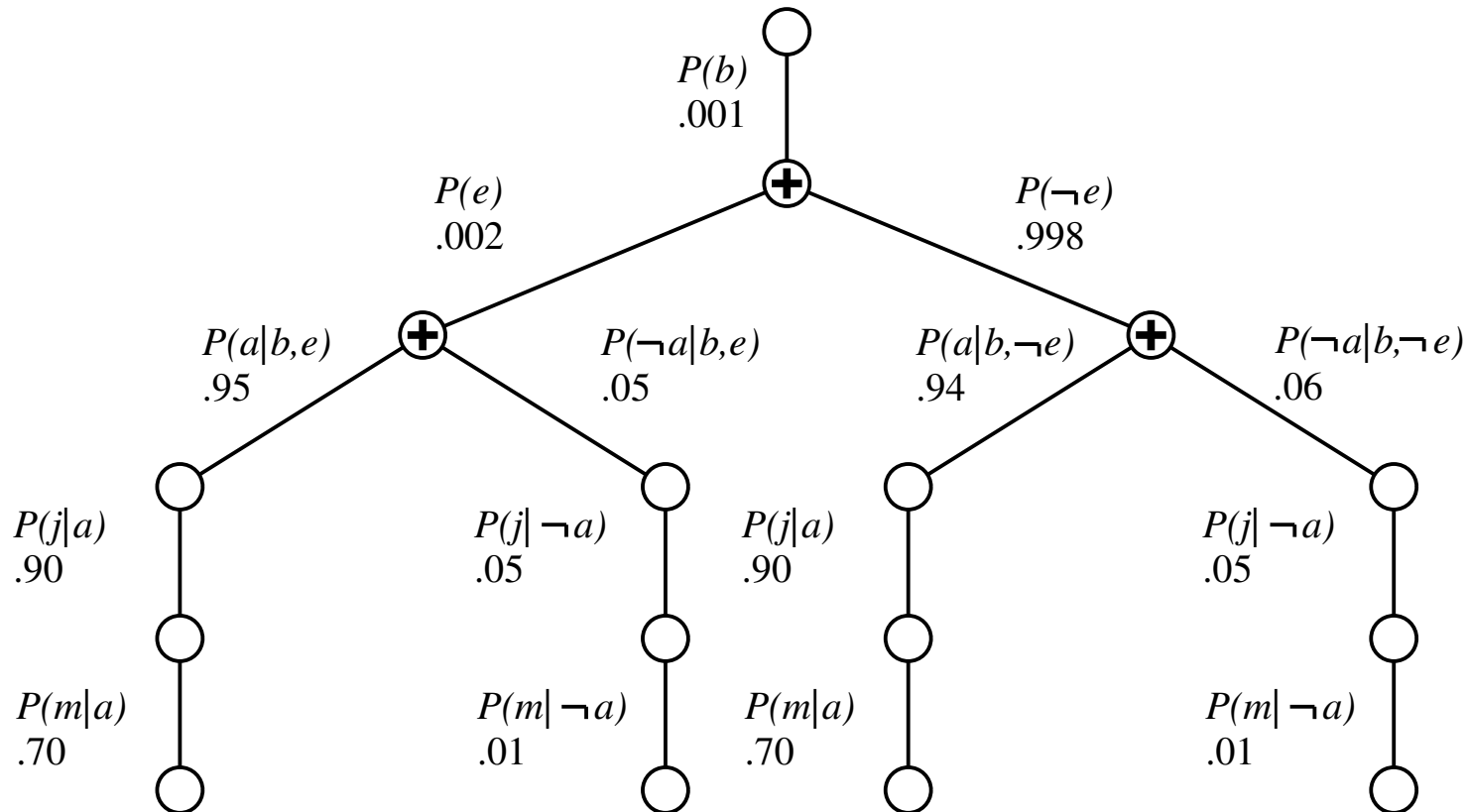
else return $\sum_y P(y \mid Pa(Y)) \times \text{ENUMERATE-ALL}(\text{REST}(vars), \mathbf{e}_y)$

 where $\mathbf{e}_y$ is $\mathbf{e}$ extended with $Y = y$

Evaluation tree

Example: $P(b|j, m) = \alpha P(b) \sum_e P(e) \sum_a P(a|b, e) P(j|a) P(m|a)$

Multiply along a branch, sum up in $\oplus$ nodes



Inefficient! E.g., repeatedly computes $P(j|a)P(m|a)$ for each value of e

Inference by variable elimination

Variable elimination: carry out summations right-to-left, storing intermediate results (**factors**) to avoid recomputation

$$\begin{aligned}\mathbf{P}(B|j, m) &= \alpha \underbrace{\mathbf{P}(B)}_{\mathbf{f}_1(B)} \underbrace{\sum_e P(e)}_{\mathbf{f}_2(E)} \underbrace{\sum_a \mathbf{P}(a|B, e)}_{\mathbf{f}_3(A, B, E)} \underbrace{P(j|a)}_{\mathbf{f}_4(A)} \underbrace{P(m|a)}_{\mathbf{f}_5(A)} \\ &= \alpha \mathbf{f}_1(B) \times \sum_e \mathbf{f}_2(E) \times \sum_a \mathbf{f}_3(A, B, E) \times \mathbf{f}_4(A) \times \mathbf{f}_5(A)\end{aligned}$$

Remarks:

1. $\mathbf{f}_5(A)$ does not need any summing up. It will be stored as a 2x1 vector $(P(m|a) \ P(m|\neg a))^T = (.70 \ .01)^T$. So is $\mathbf{f}_4(A) = (.90 \ .05)^T$
2. $\mathbf{f}_3(A, B, E)$ will be stored as a 2x2x2 matrix. When summing up on A we get a 2x2 matrix

Variable elimination: Pointwise product

Pointwise product ("produs punctual") of factors $\mathbf{f}_1$ and $\mathbf{f}_2$:

$$\mathbf{f}_1(x_1, \dots, x_j, \underline{y_1, \dots, y_k}) \times \mathbf{f}_2(\underline{y_1, \dots, y_k}, z_1, \dots, z_l) \\ = \mathbf{f}(x_1, \dots, x_j, y_1, \dots, y_k, z_1, \dots, z_l)$$

E.g., $\mathbf{f}_1(a, b) \times \mathbf{f}_2(b, c) = \mathbf{f}(a, b, c)$

A	B	$\mathbf{f}_1(A, B)$	B	C	$\mathbf{f}_2(B, C)$	A	B	C	$\mathbf{f}_3(A, B, C)$
T	T	.3	T	T	.2	T	T	T	$.3 \times .2 = .06$
T	F	.7	T	F	.8	T	T	F	$.3 \times .8 = .24$
F	T	.9	F	T	.6	T	F	T	$.7 \times .6 = .42$
F	F	.1	F	F	.4	T	F	F	$.7 \times .4 = .28$
						F	T	T	$.9 \times .2 = .18$
						F	T	F	$.9 \times .8 = .72$
						F	F	T	$.1 \times .6 = .06$
						F	F	F	$.1 \times .4 = .04$

Obs: 2^{j+k} entries for $\mathbf{f}_1$ and 2^{k+l} entries $\mathbf{f}_2$ give 2^{j+k+l} entries for $\mathbf{f}_3$

Variable elimination: Summing out

Summing out a variable from a product of factors:

move any constant factors outside the summation

add up submatrices in the pointwise product of remaining factors

Ex. $f(B, C)$

$$\begin{aligned} &= \sum_a \mathbf{f}_3(A, B, C) \\ &= \mathbf{f}_3(a, B, C) + \mathbf{f}_3(\neg a, B, C) \\ &= \begin{pmatrix} .06 & .24 \\ .42 & .28 \end{pmatrix} + \begin{pmatrix} .18 & .72 \\ .06 & .04 \end{pmatrix} \\ &= \begin{pmatrix} .24 & .96 \\ .48 & .32 \end{pmatrix} \end{aligned}$$

Obs: any factor that does not depend on the variable to be summed out can be moved outside the summation.

Variable elimination algorithm (Bottom Up)

```
function ELIMINATION-ASK( $X, \mathbf{e}, bn$ ) returns a distribution over  $X$   
  inputs:  $X$ , the query variable  
            $\mathbf{e}$ , evidence specified as an event  
            $bn$ , a belief network specifying joint distribution  $\mathbf{P}(X_1, \dots, X_n)$   
  
   $factors \leftarrow []$ ;  $vars \leftarrow \text{REVERSE}(\text{VARS}[bn])$   
  for each  $var$  in  $vars$  do  
     $factors \leftarrow [\text{MAKE-FACTOR}(var, \mathbf{e}) | factors]$   
    if  $var$  is a hidden variable then  $factors \leftarrow \text{SUM-OUT}(var, factors)$   
  return  $\text{NORMALIZE}(\text{POINTWISE-PRODUCT}(factors))$ 
```

Remark: pointwise product operations are performed ONLY when summing up is needed and only on the summing variables

Summary

Independence based on the Bayes Network topology: ancestors graph, moral graph, de-orient, eliminate evidence and verify connectivity

Exact inference by variable elimination is still expensive, even if optimized.

Possible solution: approximate via sampling

BUILDING BAYESIAN NETWORKS. APPROXIMATE INFERENCE IN BAYESIAN NETWORKS

CHAPTER 14.2.1, 14.5
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Building Bayesian Networks
- ◇ Approximate inference in Bayesian Networks

Constructing Bayesian networks

Questions:

◇ Is the arrow direction in a Bayesian network relevant?

No, provided that we have the appropriate values for the probabilities (Bayes' rule might help us here)

◇ How can we figure out whether an arrow exists?

We search for correlation(s) (e.g., blue-eyed ladies and gentlemen)

Constructing Bayesian networks

Need a method such that a series of locally testable assertions of conditional independence guarantees the required global semantics

1. Choose an ordering of variables $X_1, \dots, X_n$
2. For $i = 1$ to n
 - add X_i to the network
 - select parents from $X_1, \dots, X_{i-1}$ such that
$$\mathbf{P}(X_i | \text{Parents}(X_i)) = \mathbf{P}(X_i | X_1, \dots, X_{i-1})$$

This choice of parents guarantees the global semantics:

$$\begin{aligned}\mathbf{P}(X_1, \dots, X_n) &= \prod_{i=1}^n \mathbf{P}(X_i | X_1, \dots, X_{i-1}) \quad (\text{chain rule}) \\ &= \prod_{i=1}^n \mathbf{P}(X_i | \text{Parents}(X_i)) \quad (\text{by construction})\end{aligned}$$

Example

Suppose we choose the ordering M, J, A, B, E

MaryCalls

JohnCalls

$$P(J|M) = P(J)?$$

Example

Suppose we choose the ordering M, J, A, B, E

MaryCalls

JohnCalls

$P(J|M) = P(J)$? No

Example

Suppose we choose the ordering M, J, A, B, E

MaryCalls

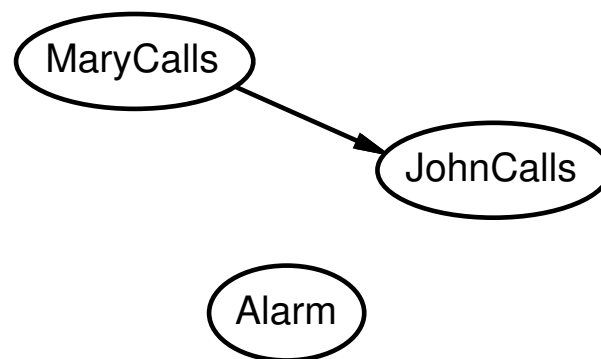
JohnCalls

$P(J|M) = P(J)$? No

(if Mary calls, the alarm has probably gone off, so John is more likely to call)

Example

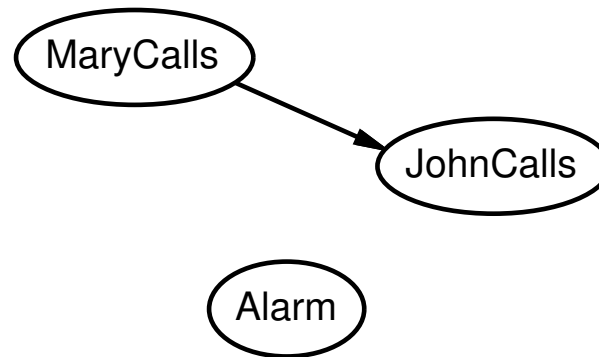
Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

Example

Suppose we choose the ordering M, J, A, B, E

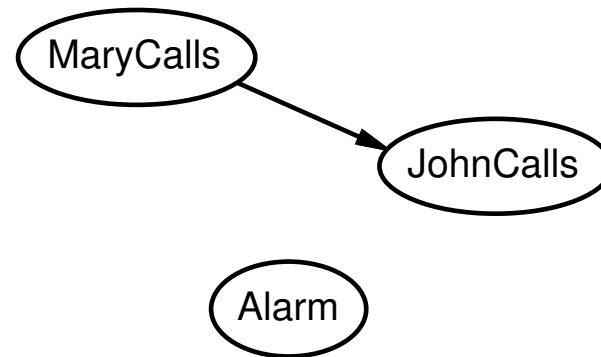


$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$?

Example

Suppose we choose the ordering M, J, A, B, E

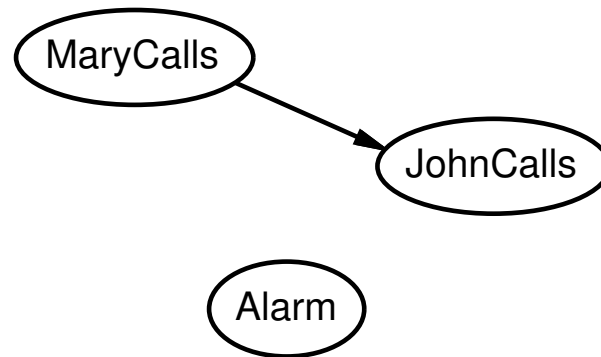


$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

Example

Suppose we choose the ordering M, J, A, B, E



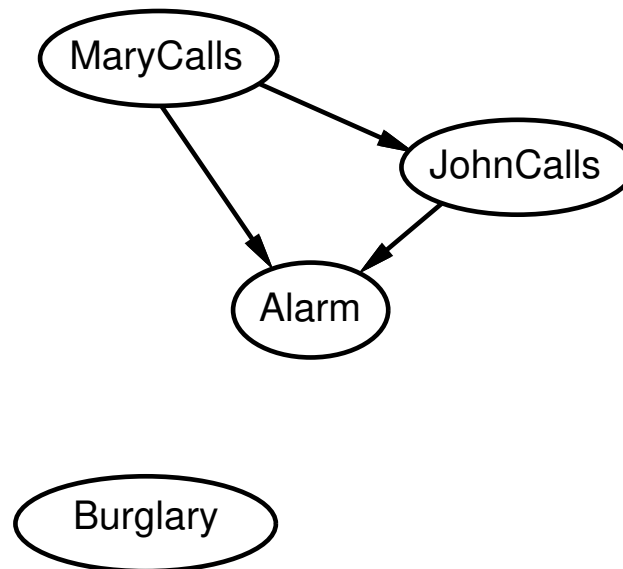
$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

(if both John and Mary call, it is more likely that alarm has gone off than in the situation that at most one called)

Example

Suppose we choose the ordering M, J, A, B, E

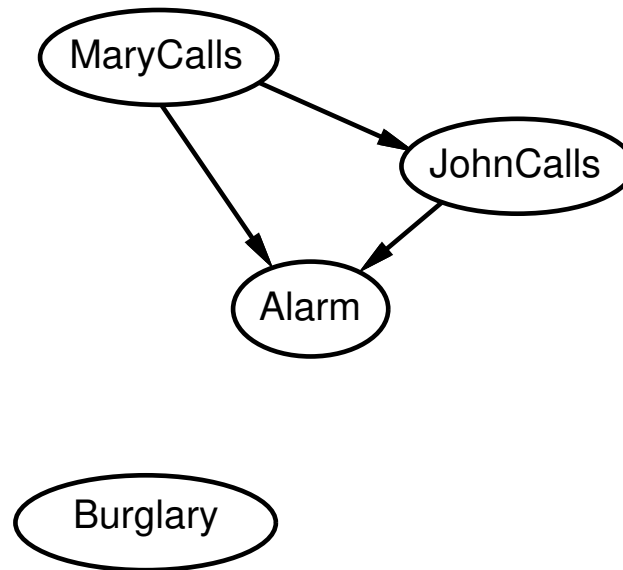


$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

Example

Suppose we choose the ordering M, J, A, B, E



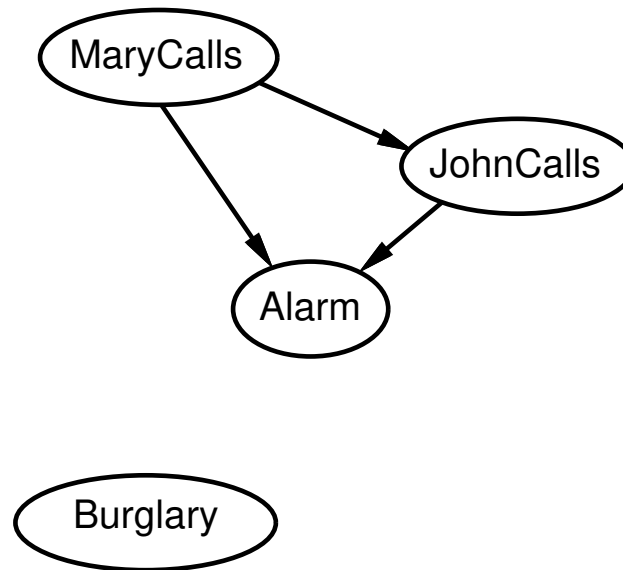
$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

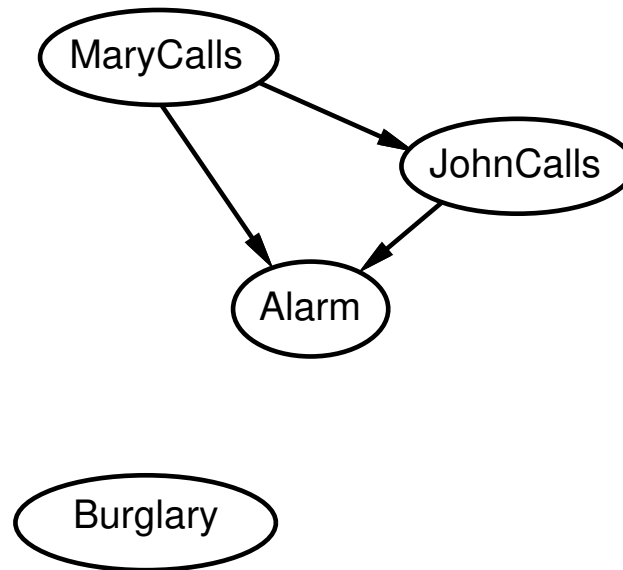
$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

(once we know the alarm has gone off, the fact that John calls does not give more info about the burglary)

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

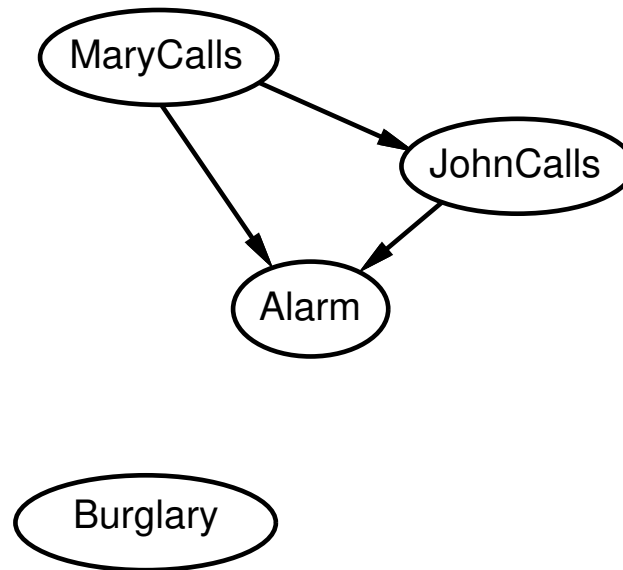
$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

$P(B|A, J, M) = P(B)$?

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

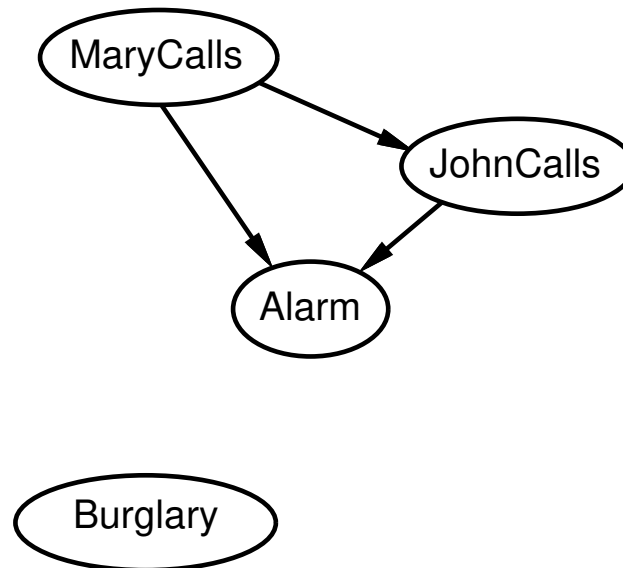
$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

$P(B|A, J, M) = P(B)$? No

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

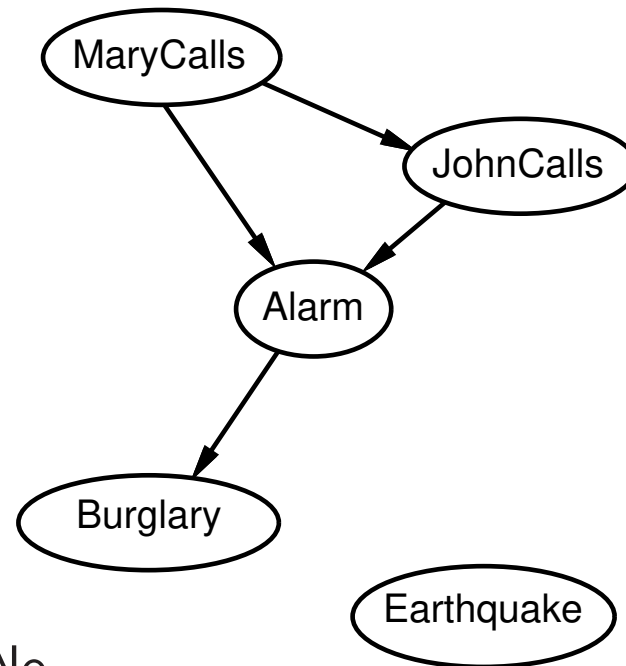
$P(B|A, J, M) = P(B|A)$? Yes

$P(B|A, J, M) = P(B)$? No

(if both call and the alarm has gone off, it is likely we have a burglary)

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

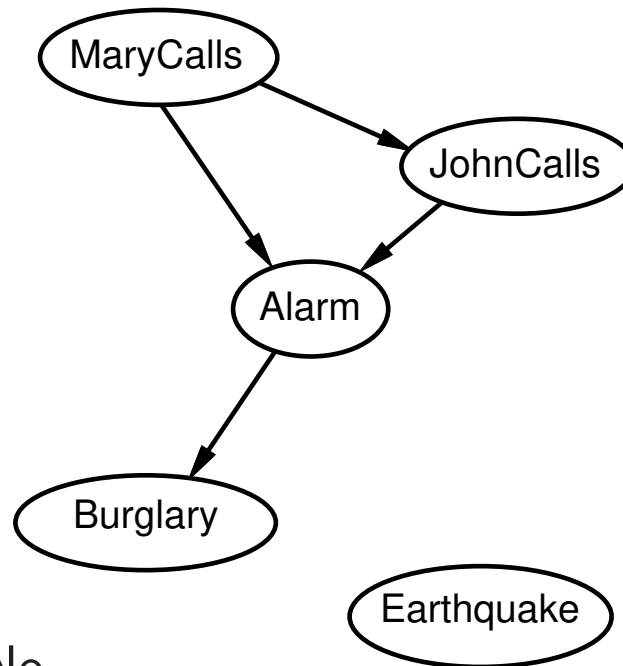
$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

$P(B|A, J, M) = P(B)$? No

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

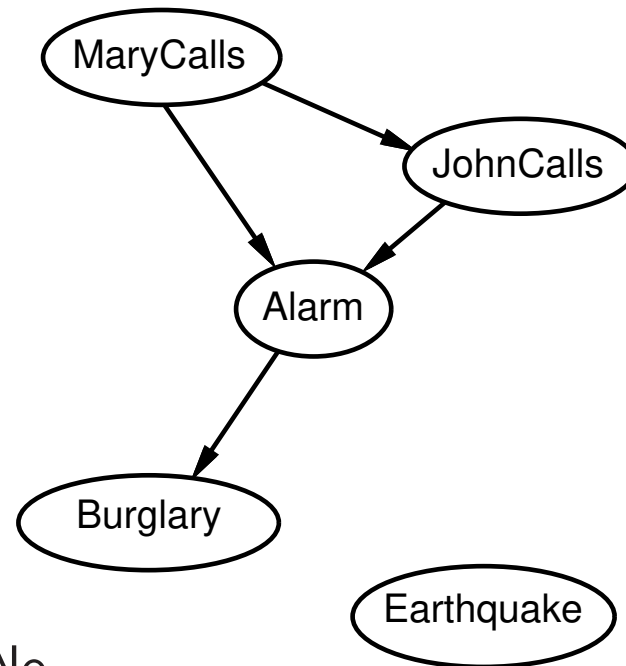
$P(B|A, J, M) = P(B|A)$? Yes

$P(B|A, J, M) = P(B)$? No

$P(E|B, A, J, M) = P(E|A)$?

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

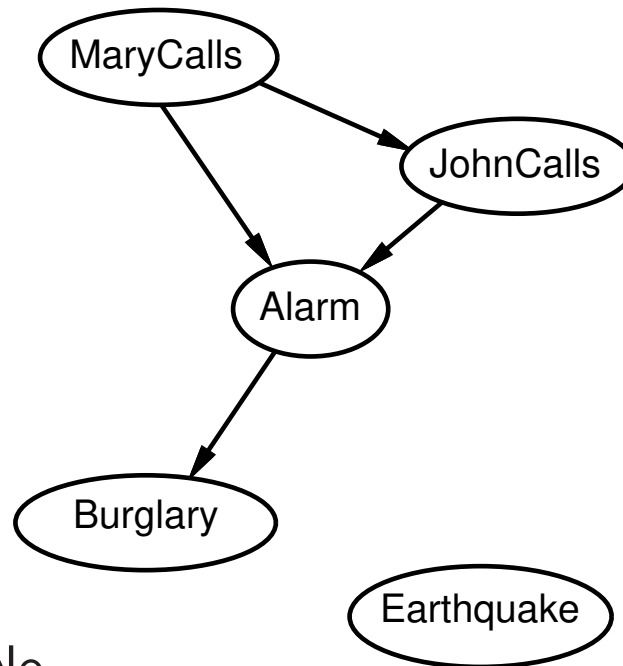
$P(B|A, J, M) = P(B|A)$? Yes

$P(B|A, J, M) = P(B)$? No

$P(E|B, A, J, M) = P(E|A)$? No

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

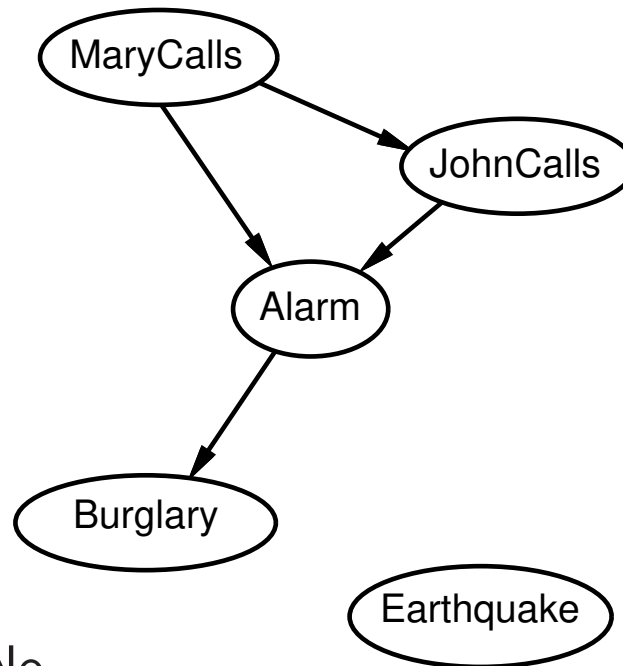
$P(B|A, J, M) = P(B)$? No

$P(E|B, A, J, M) = P(E|A)$? No

(alarm indicates Earthquake, but if burglary, it is likely the alarm goes off)

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

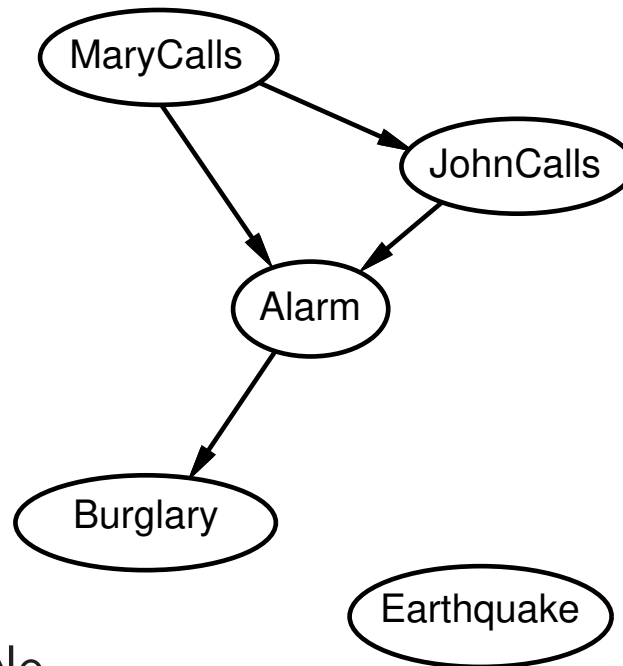
$P(B|A, J, M) = P(B)$? No

$P(E|B, A, J, M) = P(E|A)$? No

$P(E|B, A, J, M) = P(E|A, B)$?

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

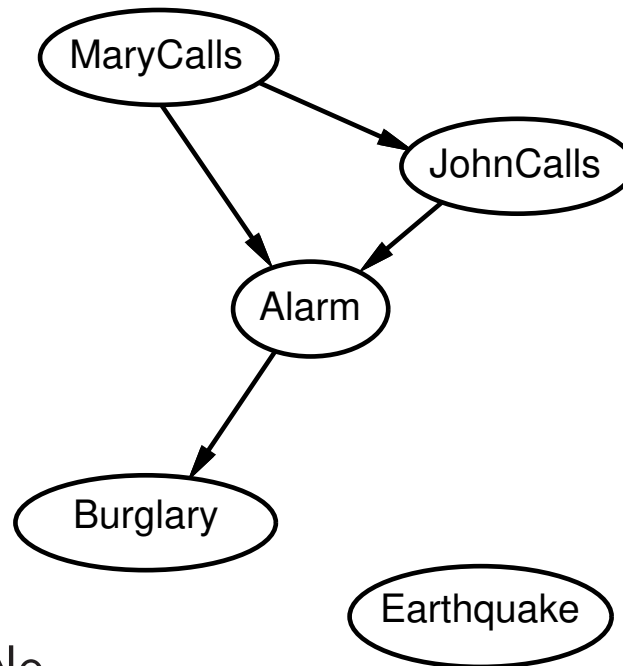
$P(B|A, J, M) = P(B)$? No

$P(E|B, A, J, M) = P(E|A)$? No

$P(E|B, A, J, M) = P(E|A, B)$? Yes

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

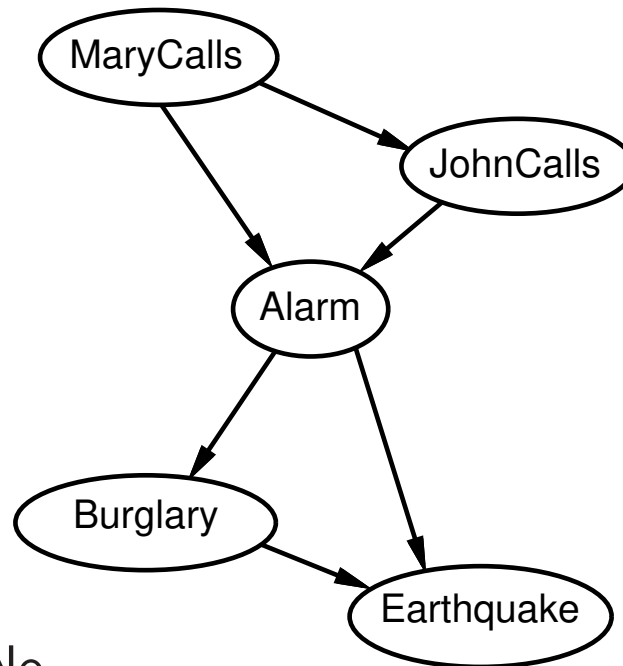
$P(B|A, J, M) = P(B)$? No

$P(E|B, A, J, M) = P(E|A)$? No

$P(E|B, A, J, M) = P(E|A, B)$? Yes (Why?)

Example

Suppose we choose the ordering M, J, A, B, E



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

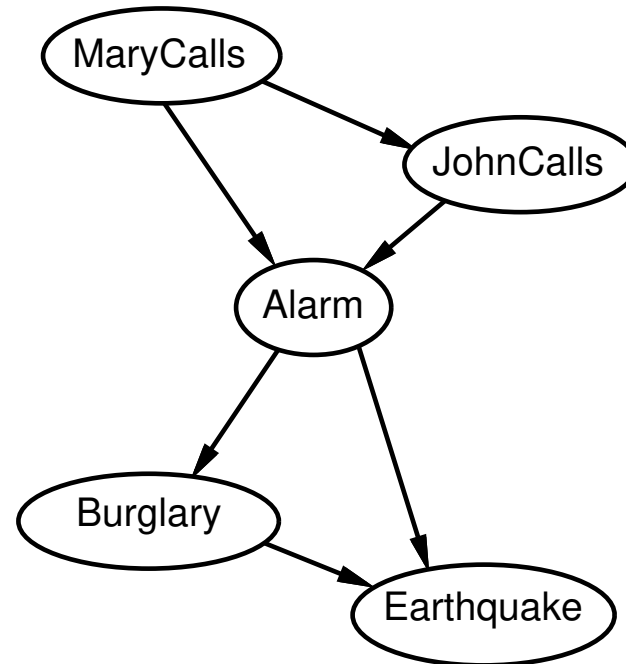
$P(B|A, J, M) = P(B|A)$? Yes

$P(B|A, J, M) = P(B)$? No

$P(E|B, A, J, M) = P(E|A)$? No

$P(E|B, A, J, M) = P(E|A, B)$? Yes

Example contd.



Deciding conditional independence is hard in noncausal directions

(Causal models and conditional independence seem hardwired for humans!)

Assessing conditional probabilities is hard in noncausal directions

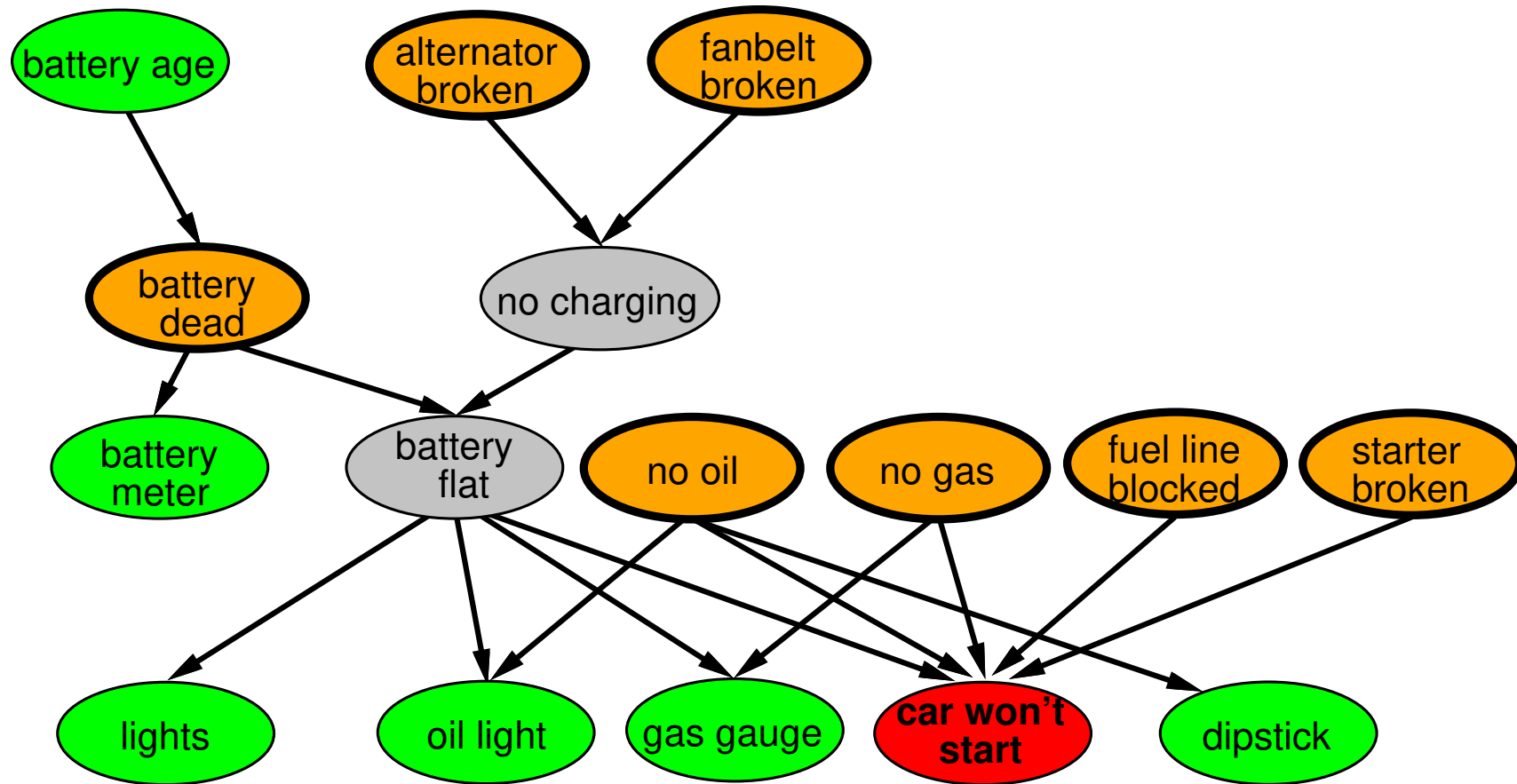
Network is less compact: $1 + 2 + 4 + 2 + 4 = 13$ numbers needed

Example: Car diagnosis

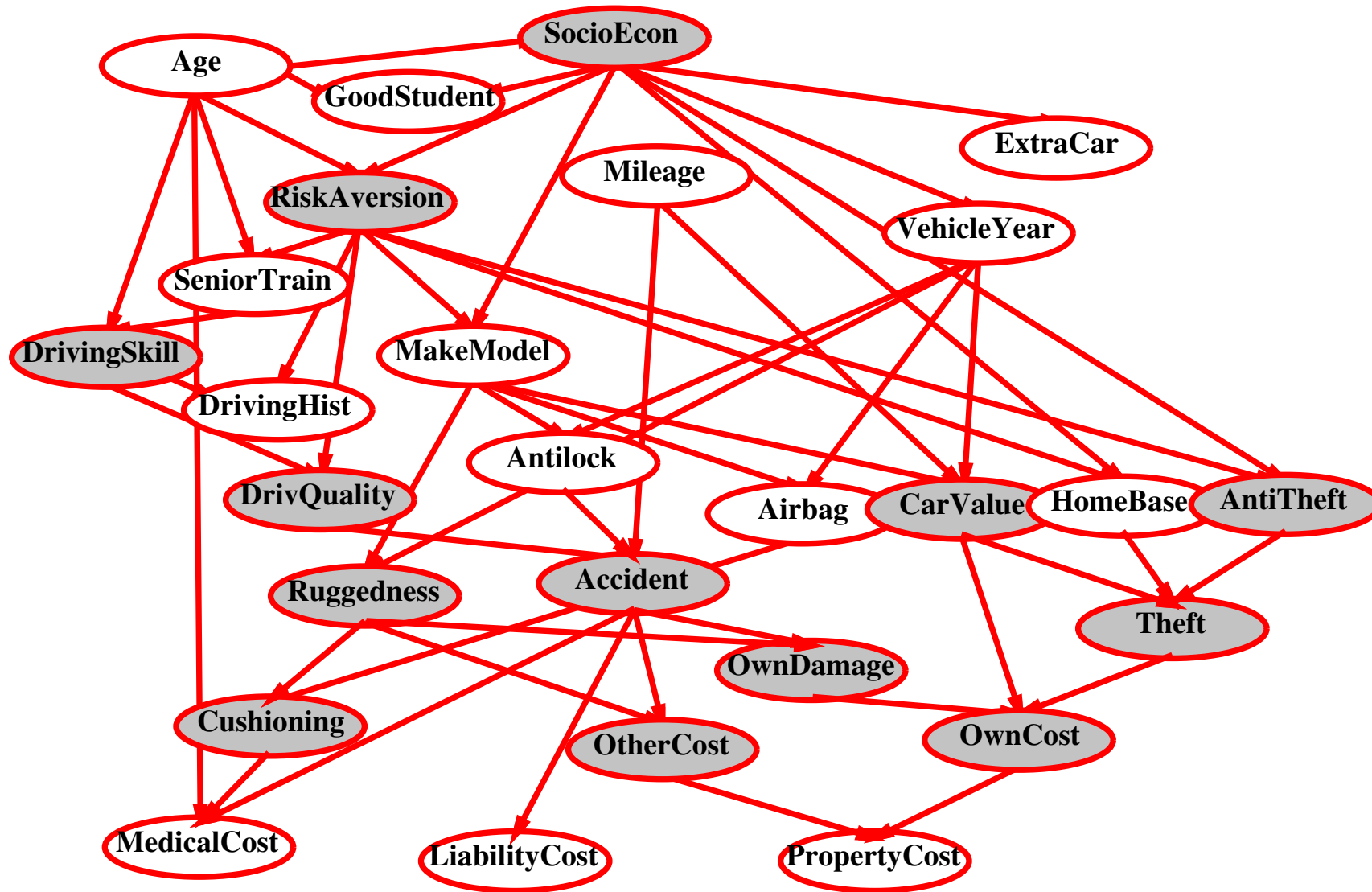
Initial evidence: car won't start

Testable variables (green), "broken, so fix it" variables (orange)

Hidden variables (gray) ensure sparse structure, reduce parameters



Example: Car insurance



Quiz

Prince Charming goes to the gym each Tuesday and spends there 15 min. He is expected to arrive each moment between 2 and 3 p.m. Sleeping Beauty goes to the gym each Tuesday and spends there 12 min. She is expected to arrive each moment between 2 and 3 p.m. What is the probability they meet each other at the gym?

Inference by stochastic simulation

Basic idea:

- 1) Draw N samples from a sampling distribution S
- 2) Compute an approximate posterior probability $\hat{P}$
- 3) Show this converges to the true probability P

0.5



Outline:

- Sampling from an empty network
- Rejection sampling: reject samples disagreeing with evidence
- Likelihood weighting: use evidence to weight samples
- Markov chain Monte Carlo (MCMC): sample from a stochastic process whose stationary distribution is the true posterior

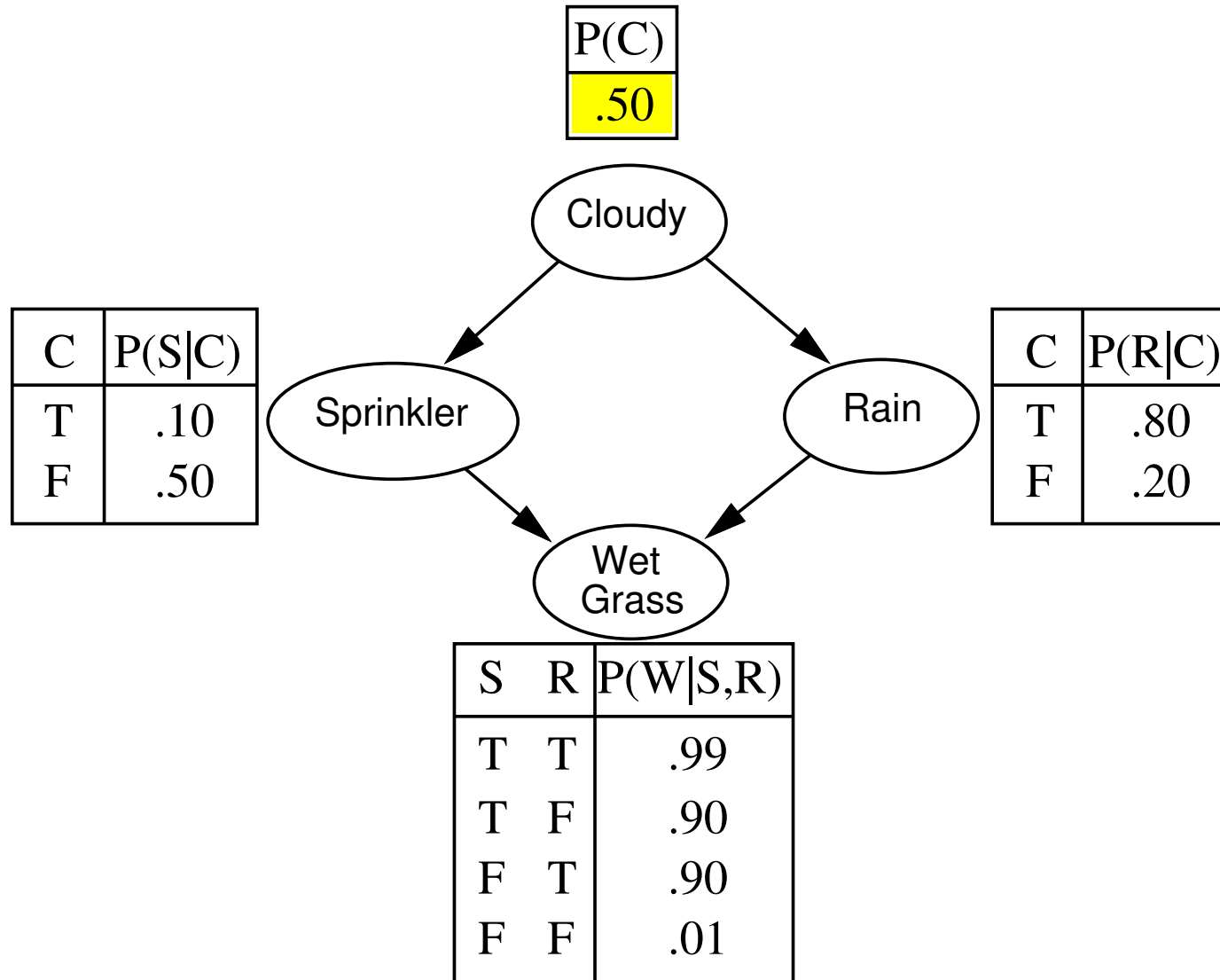
Sampling from an empty network

```
function PRIOR-SAMPLE(bn) returns an event sampled from bn  
  inputs: bn, a belief network specifying joint distribution  $\mathbf{P}(X_1, \dots, X_n)$   
  x  $\leftarrow$  an event with n elements  
  for i = 1 to n do  
    xi  $\leftarrow$  a random sample from  $\mathbf{P}(X_i \mid \text{parents}(X_i))$   
      given the values of Parents(Xi) in x  
  return x
```

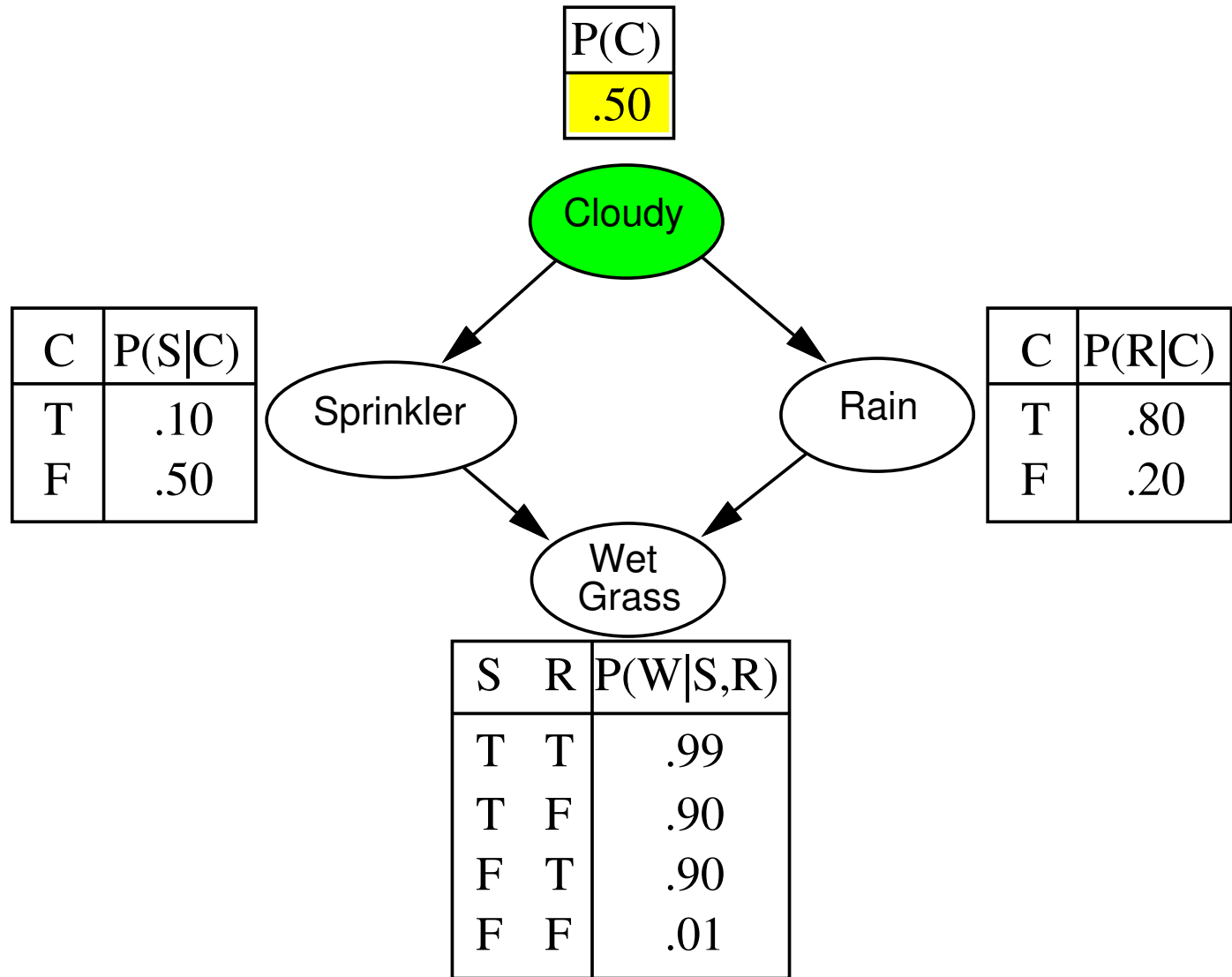
Remarks:

1. sampling is performed in topological order
2. the distribution for one variable depends on the values already assigned to its parents

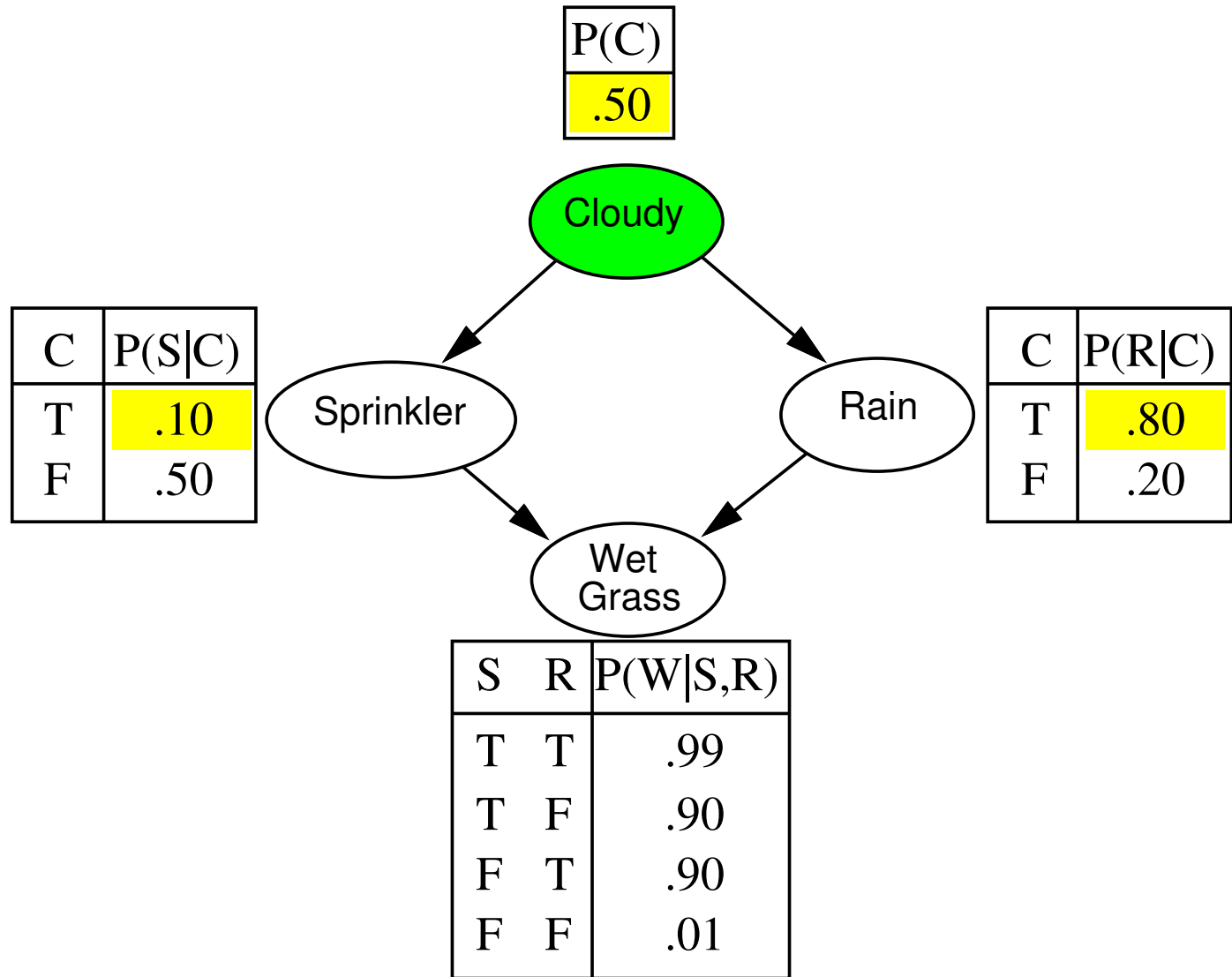
Example (assume C, S, R, W as order)



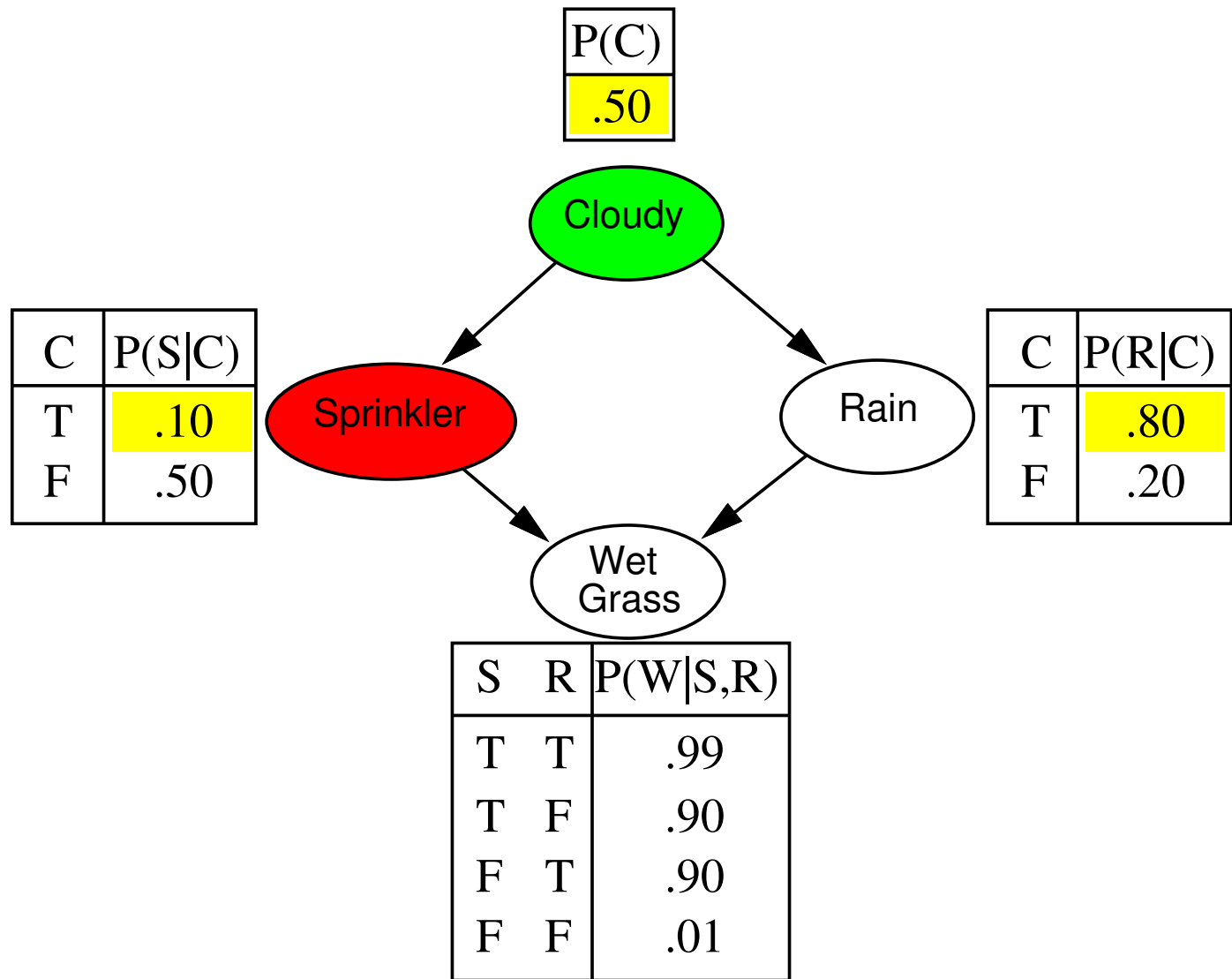
Example



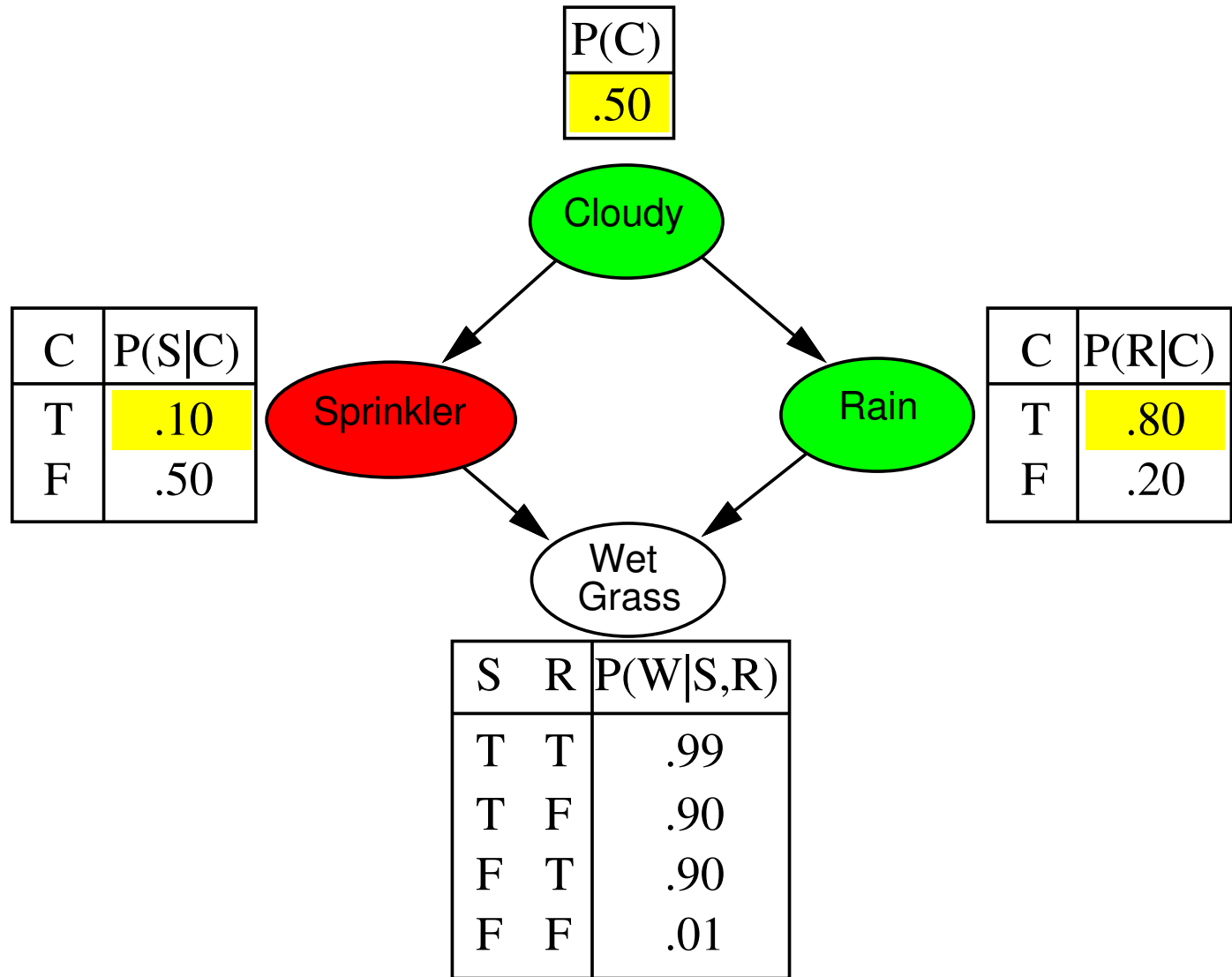
Example



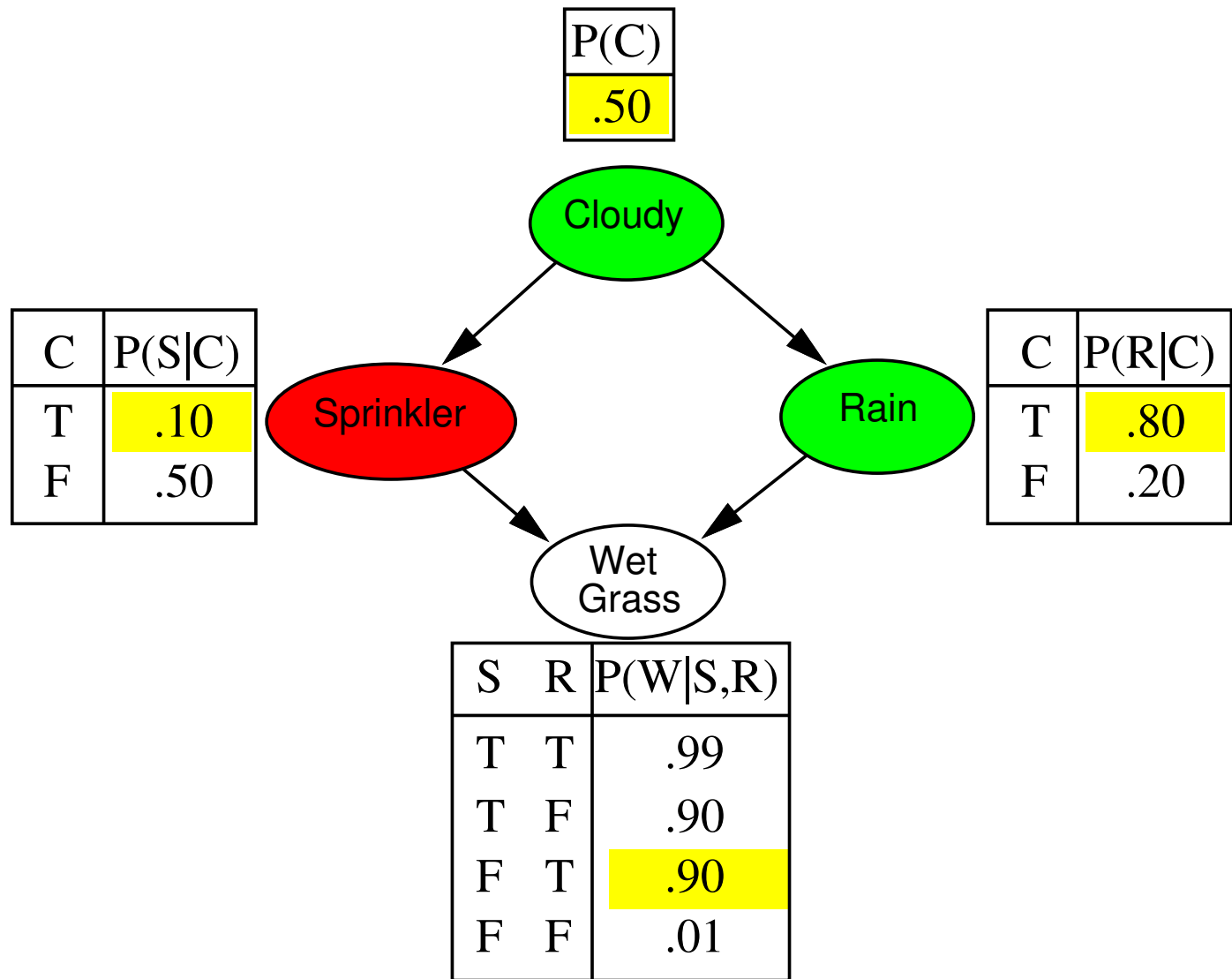
Example



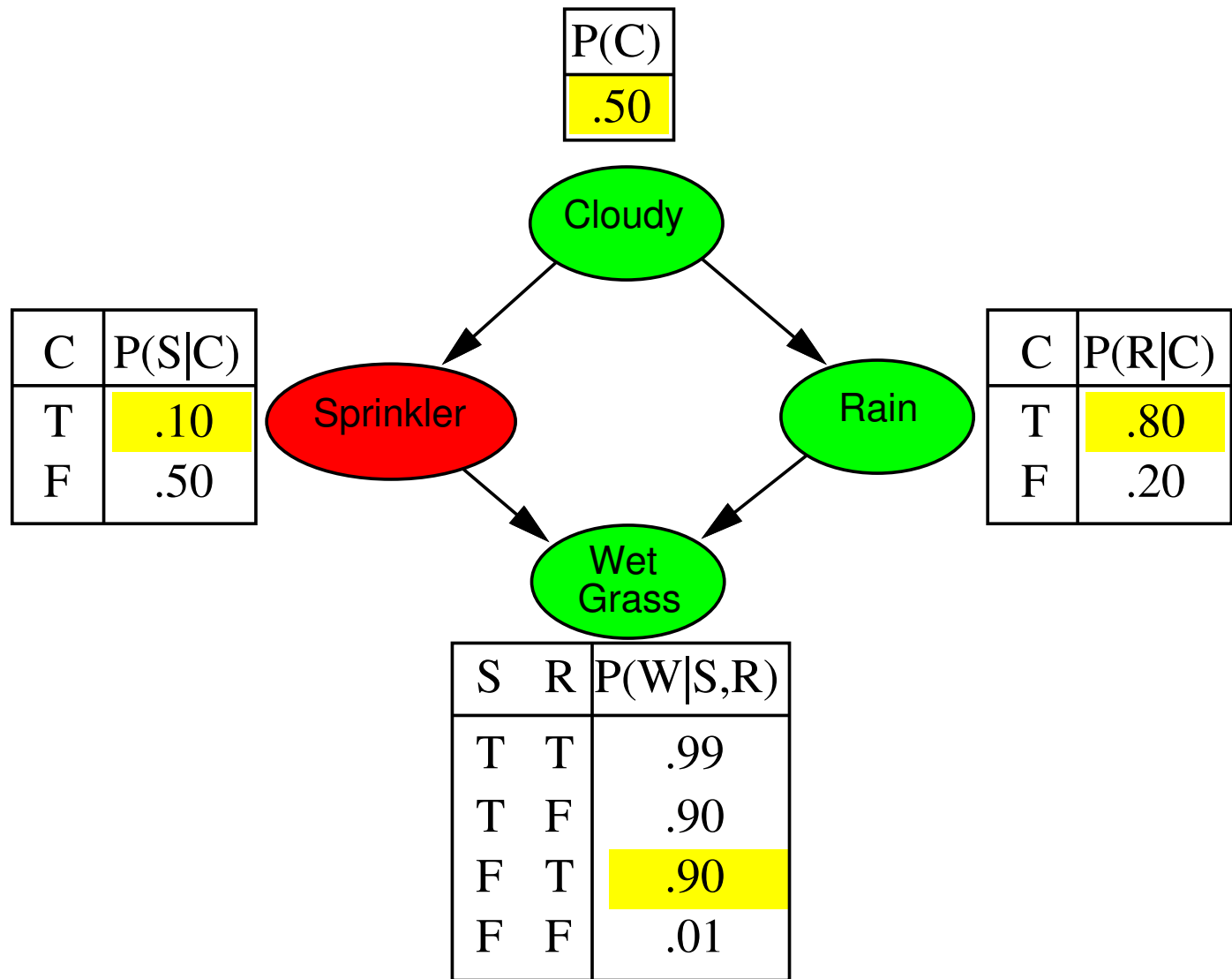
Example



Example



Example



Example

Event: $[true, false, true, true]$

Generate 1000 such events. If 511 of them have $Rain=true$, then estimate $P(Rain=true)$ as 0.511.

Sampling from an empty network contd.

Probability that PRIORSAMPLE generates a particular event

$$S_{PS}(x_1 \dots x_n) = \prod_{i=1}^n P(x_i | \text{parents}(X_i)) = P(x_1 \dots x_n)$$

i.e., the true prior probability

E.g., $S_{PS}(t, f, t, t) = 0.5 \times 0.9 \times 0.8 \times 0.9 = 0.324 = P(t, f, t, t)$

Let $N_{PS}(x_1 \dots x_n)$ be the number of samples generated for event $x_1, \dots, x_n$

Then we have

$$\begin{aligned} \lim_{N \rightarrow \infty} \hat{P}(x_1, \dots, x_n) &= \lim_{N \rightarrow \infty} (N_{PS}(x_1, \dots, x_n) / N) \\ &= S_{PS}(x_1, \dots, x_n) \\ &= P(x_1 \dots x_n) \end{aligned}$$

That is, estimates derived from PRIORSAMPLE are **consistent**

Shorthand: $\hat{P}(x_1, \dots, x_n) \approx P(x_1 \dots x_n)$

Problem: hopelessly expensive if $P(\mathbf{e})$ is small

Rejection sampling

$\hat{\mathbf{P}}(X|\mathbf{e})$ estimated from samples agreeing with $\mathbf{e}$; the other samples are simply discarded. Typically used for estimating conditional probabilities.

```
function REJECTION-SAMPLING( $X, \mathbf{e}, bn, N$ ) returns an estimate of  $P(X|\mathbf{e})$ 
  local variables:  $\mathbf{N}$ , a vector of counts over  $X$ , initially zero
  for  $j = 1$  to  $N$  do
     $\mathbf{x} \leftarrow \text{PRIOR-SAMPLE}(bn)$ 
    if  $\mathbf{x}$  is consistent with  $\mathbf{e}$  then
       $\mathbf{N}[x] \leftarrow \mathbf{N}[x] + 1$  where  $x$  is the value of  $X$  in  $\mathbf{x}$ 
  return NORMALIZE( $\mathbf{N}[X]$ )
```

E.g., estimate $\mathbf{P}(\text{Rain}|\text{Sprinkler} = \text{true})$ using 100 samples

27 samples have $\text{Sprinkler} = \text{true}$

Of these, 8 have $\text{Rain} = \text{true}$ and 19 have $\text{Rain} = \text{false}$.

$$\hat{\mathbf{P}}(\text{Rain}|\text{Sprinkler} = \text{true}) = \text{NORMALIZE}(\langle 8, 19 \rangle) = \langle 0.296, 0.704 \rangle$$

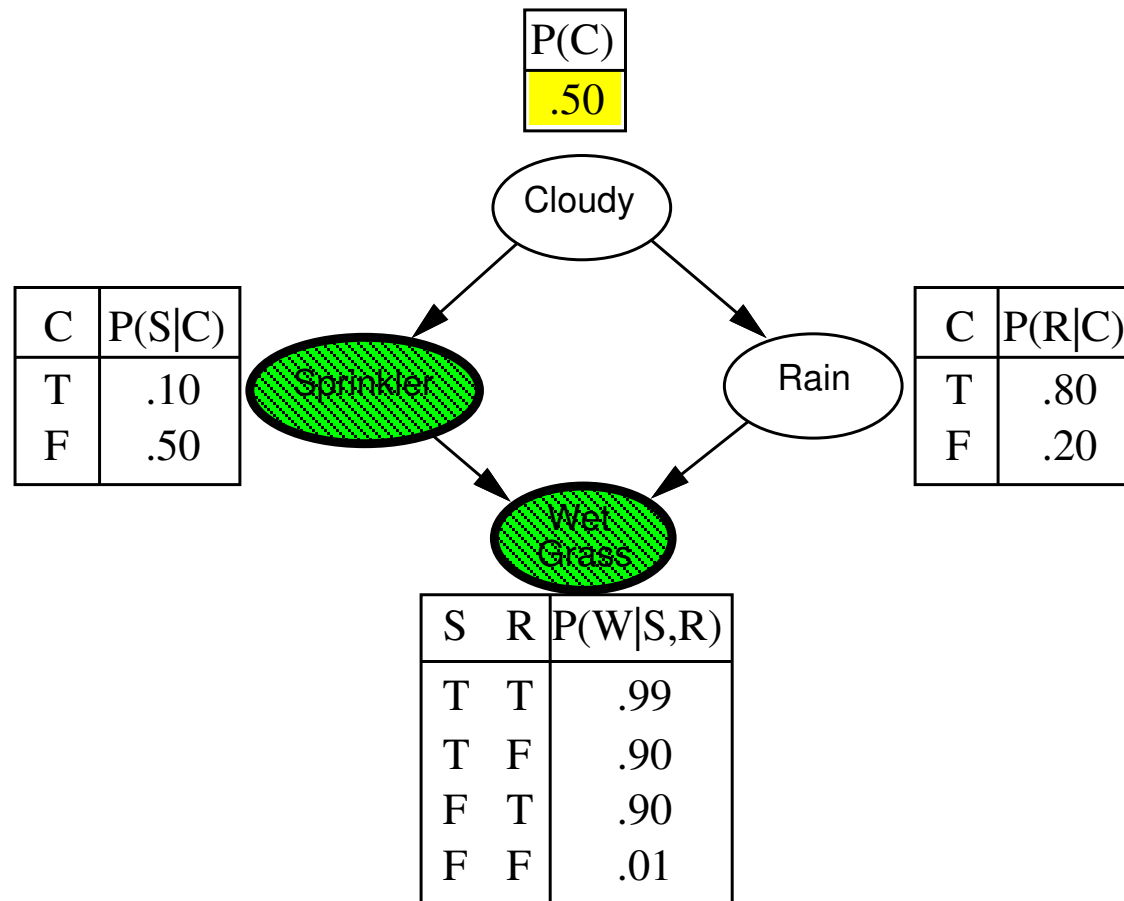
Likelihood weighting

Fix evidence variables, sample only nonevidence variables, weight each sample by its likelihood to be consistent with evidence (NO inconsistent samples!)

```
function LIKELIHOOD-WEIGHTING( $X, \mathbf{e}, bn, N$ ) returns an estimate of  $P(X|\mathbf{e})$   
  local variables:  $\mathbf{W}$ , a vector of weighted counts over  $X$ , initially zero  
  for  $j = 1$  to  $N$  do  
     $\mathbf{x}, w \leftarrow \text{WEIGHTED-SAMPLE}(bn)$   
     $\mathbf{W}[x] \leftarrow \mathbf{W}[x] + w$  where  $x$  is the value of  $X$  in  $\mathbf{x}$   
  return NORMALIZE( $\mathbf{W}[X]$ )
```

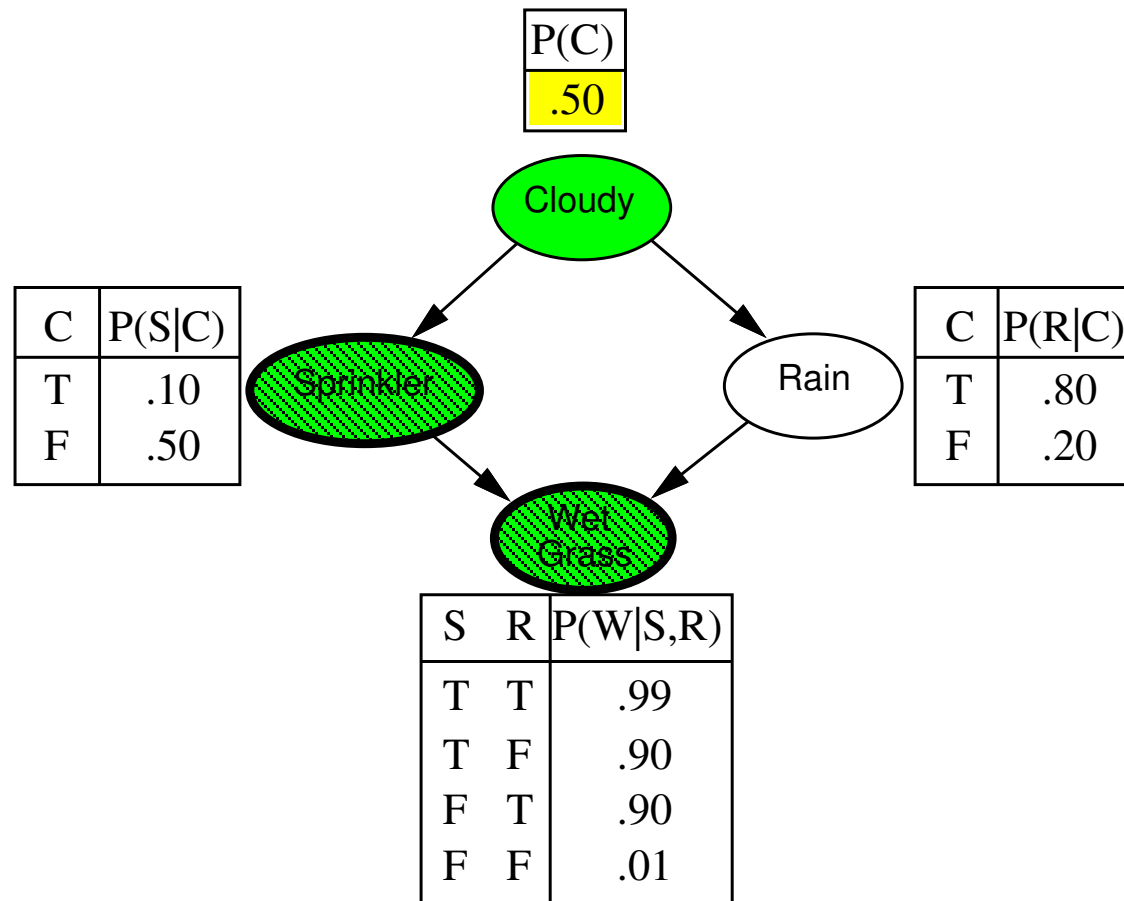
```
function WEIGHTED-SAMPLE( $bn, \mathbf{e}$ ) returns an event and a weight  
   $\mathbf{x} \leftarrow$  an event with  $n$  elements;  $w \leftarrow 1$   
  for  $i = 1$  to  $n$  do  
    if  $X_i$  has a value  $x_i$  in  $\mathbf{e}$   
      then  $w \leftarrow w \times P(X_i = x_i \mid \text{parents}(X_i))$   
      else  $x_i \leftarrow$  a random sample from  $\mathbf{P}(X_i \mid \text{parents}(X_i))$   
  return  $\mathbf{x}, w$ 
```


Likelihood weighting example



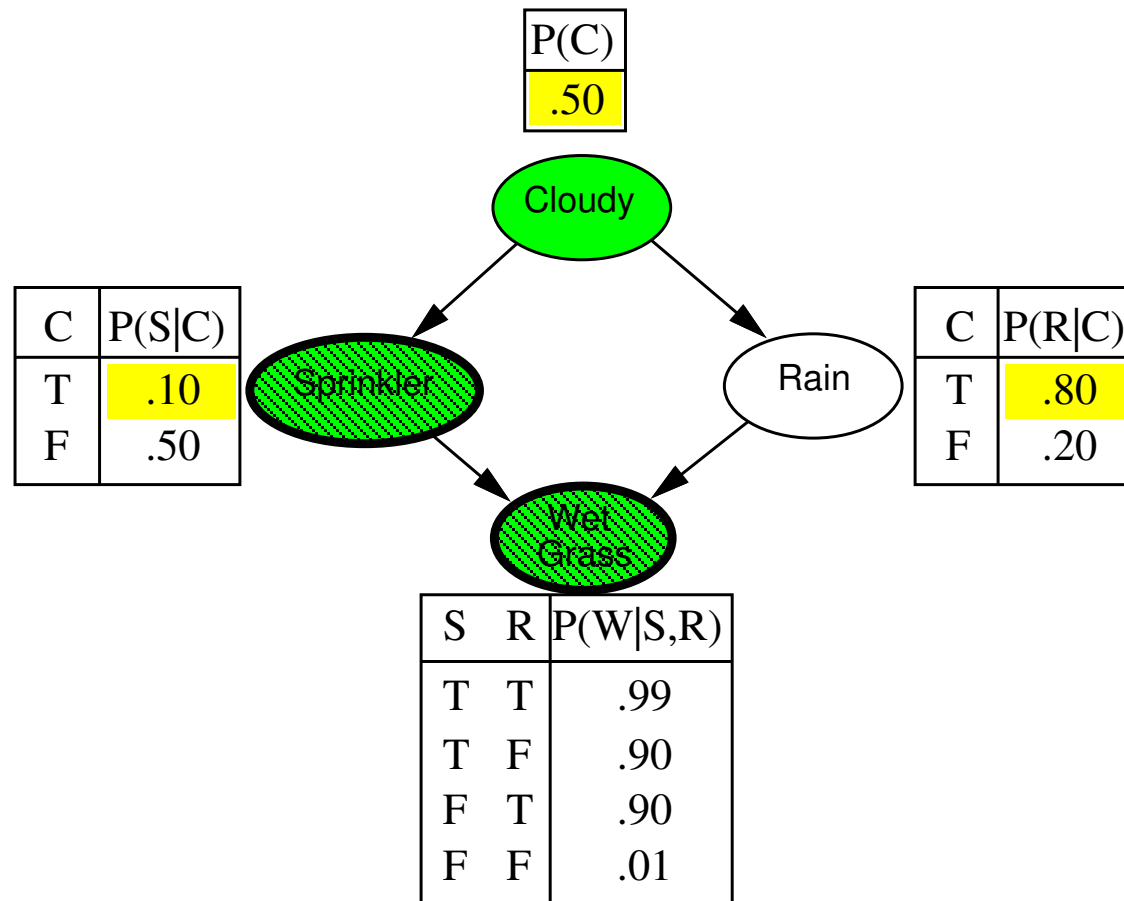
$$w = 1.0$$

Likelihood weighting example



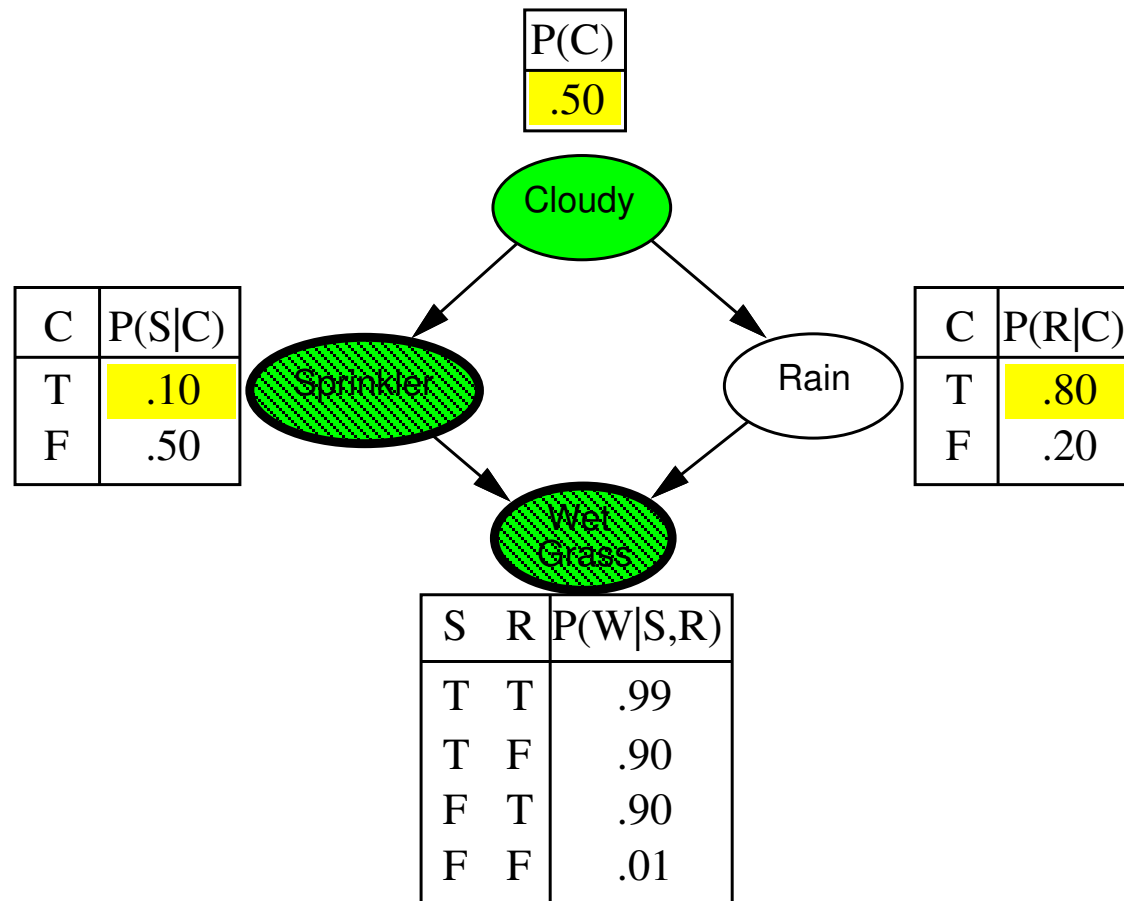
$$w = 1.0$$

Likelihood weighting example



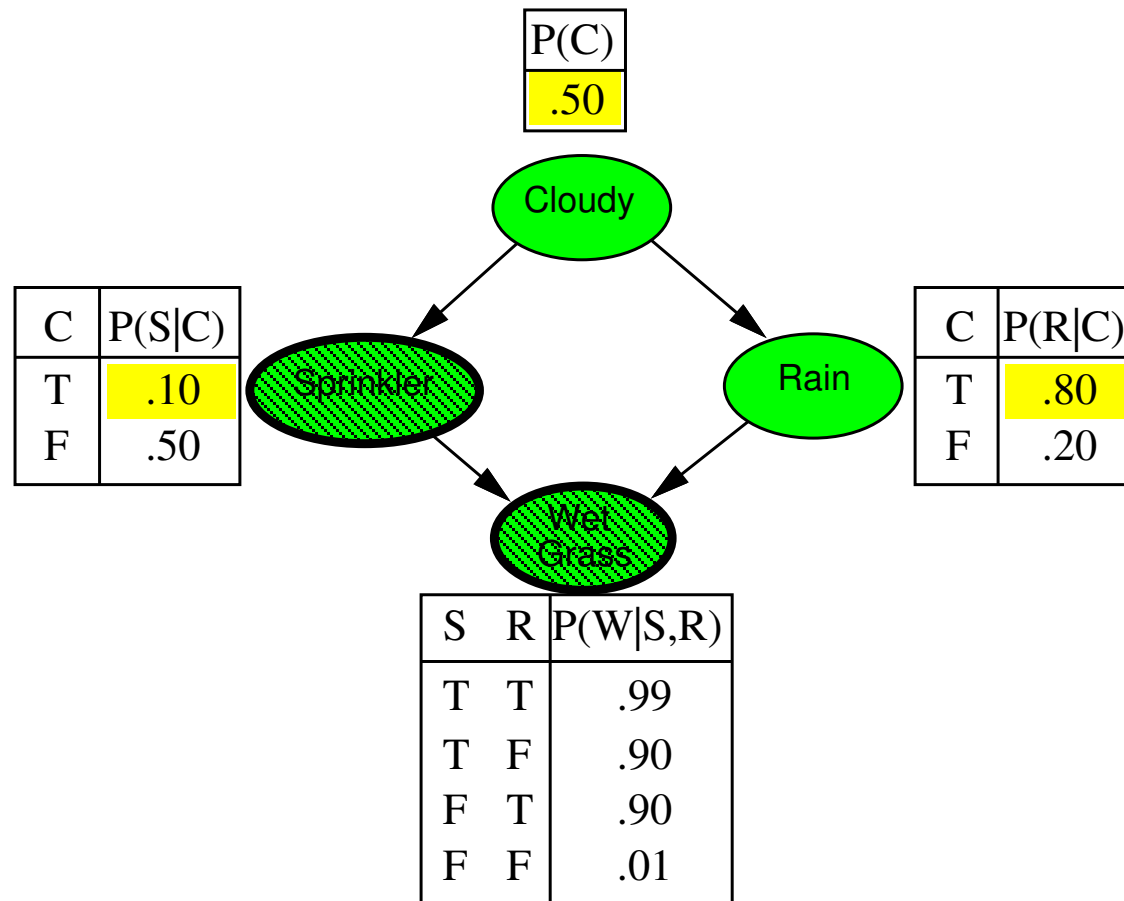
$$w = 1.0$$

Likelihood weighting example



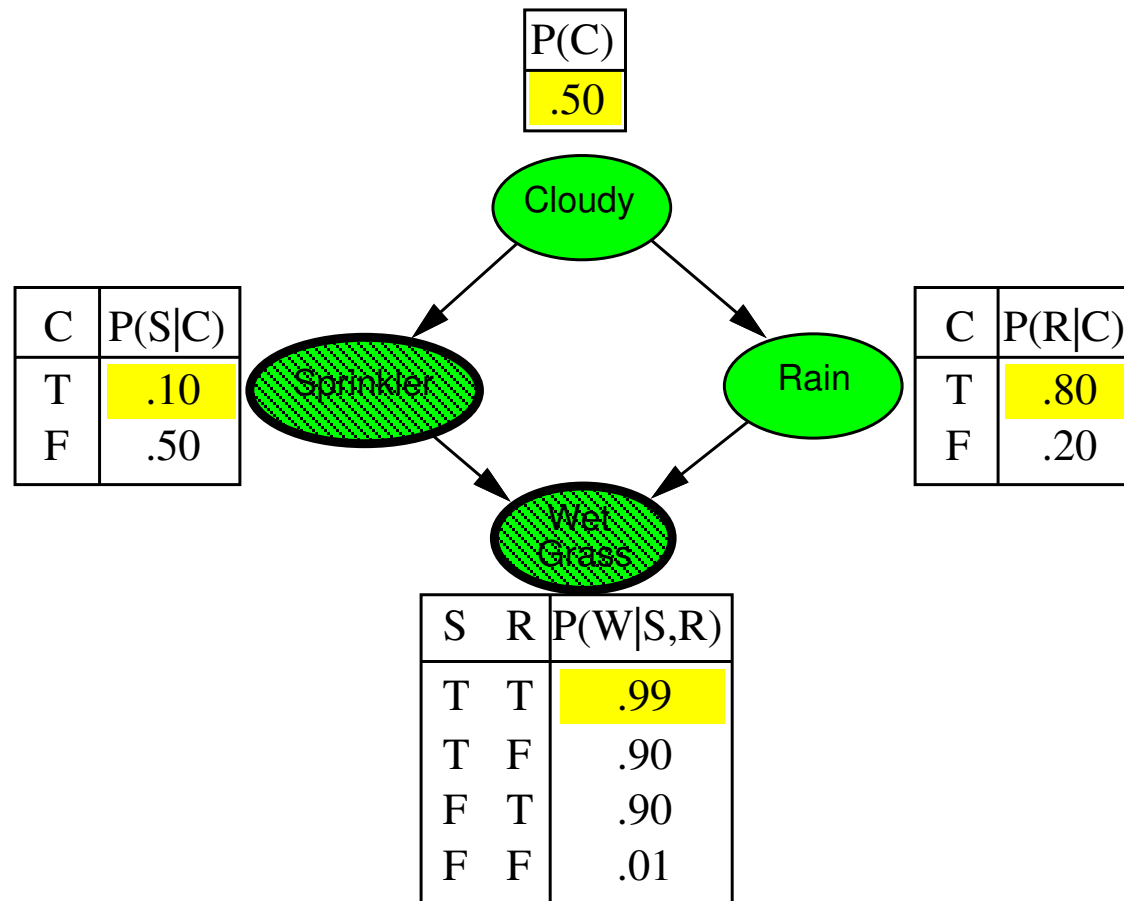
$$w = 1.0 \times 0.1$$

Likelihood weighting example



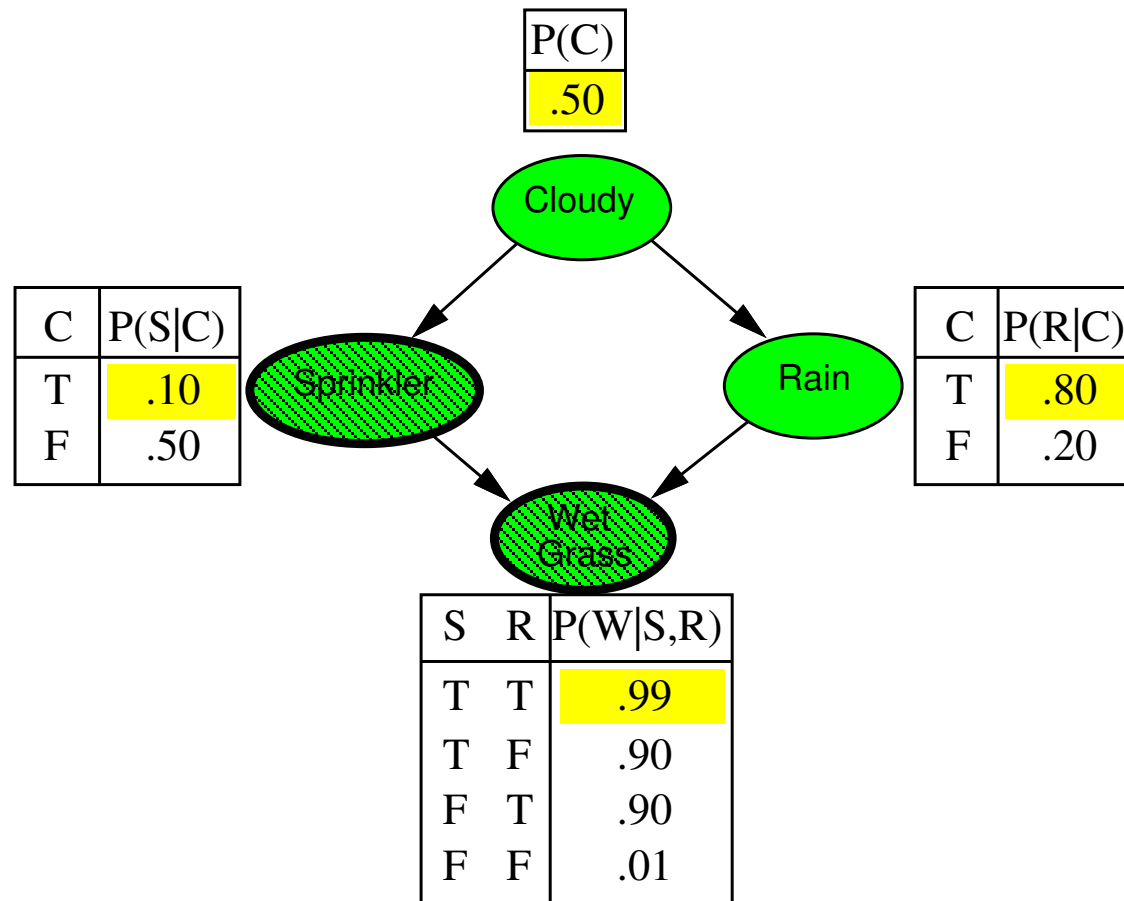
$$w = 1.0 \times 0.1$$

Likelihood weighting example



$$w = 1.0 \times 0.1$$

Likelihood weighting example



$$w = 1.0 \times 0.1 \times 0.99 = 0.099$$

The value returned is $[true, true, true, true]$ with weight 0.099

Approximate inference using MCMC

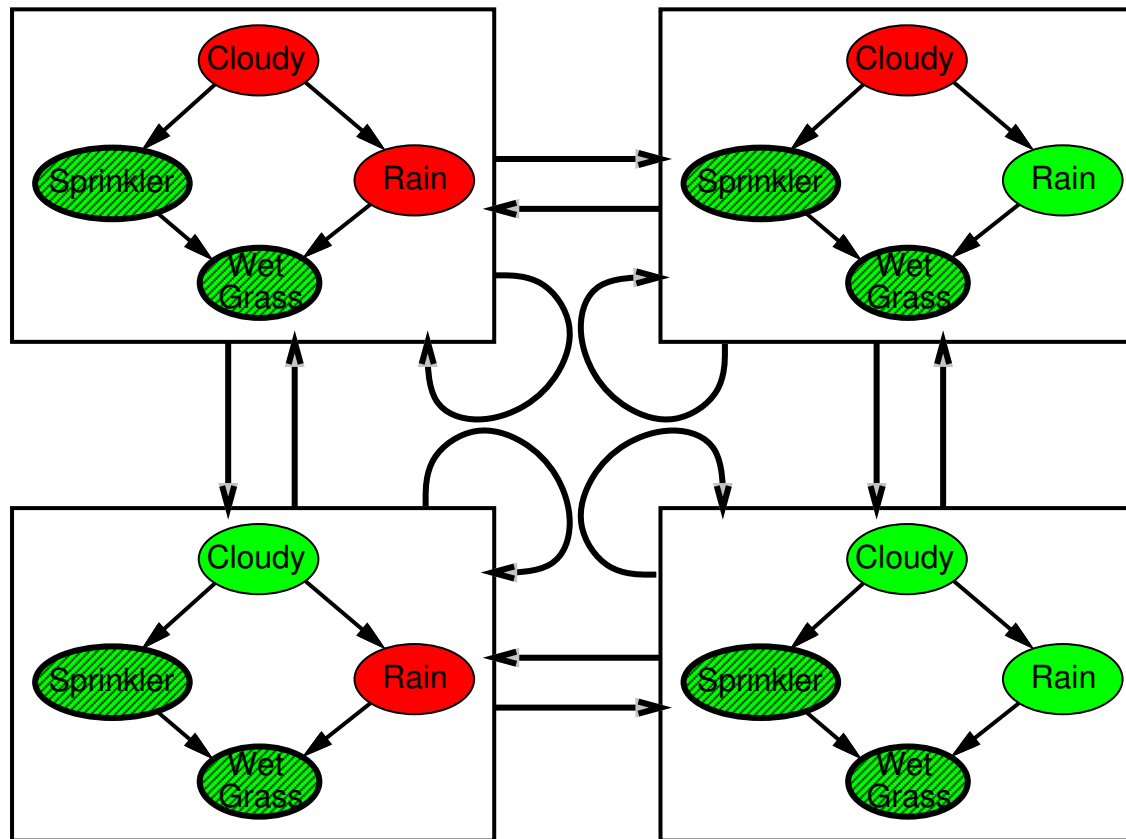
Each sample is generated by making a random change of the previous one
Current "state" of network = current assignments to all variables
Generate next state by sampling one variable given its Markov blanket
Sample each variable in turn, keeping evidence fixed

```
function GIBBS-ASK( $X, \mathbf{e}, bn, N$ ) returns an estimate of  $P(X|\mathbf{e})$ 
  local variables:  $\mathbf{N}$ , a vector of counts over  $X$ , initially zero
                      $\mathbf{Z}$ , the nonevidence variables in  $bn$ 
                      $\mathbf{x}$ , the current state of the network, initially copied from  $\mathbf{e}$ 

  initialize  $\mathbf{x}$  with random values for the variables in  $\mathbf{Z}$ 
  for  $j = 1$  to  $N$  do
    for each  $Z_i$  in  $\mathbf{Z}$  do
      sample the value of  $Z_i$  in  $\mathbf{x}$  from  $\mathbf{P}(Z_i|mb(Z_i))$ 
        given the values of  $MB(Z_i)$  in  $\mathbf{x}$ 
       $\mathbf{N}[x] \leftarrow \mathbf{N}[x] + 1$  where  $x$  is the value of  $X$  in  $\mathbf{x}$ 
  return NORMALIZE( $\mathbf{N}[X]$ )
```


The Markov chain for Gibbs sampling alg.

Goal: $P(r|s, w)$. For $Sprinkler = WetGrass = true$, there are 4 states:



Start from left down corner, roam around for a while, average what you see

MCMC example contd.

To estimate $\mathbf{P}(Rain|Sprinkler = true, WetGrass = true)$:

- Repeat sampling *Cloudy* or *Rain* given its Markov blanket
- Count number of times *Rain* is true and false in the samples

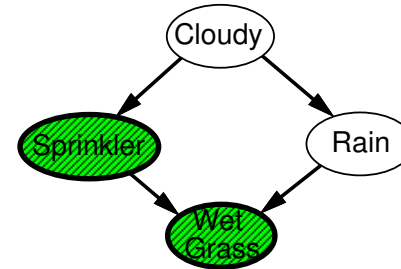
E.g., visit all in all 120 states; 32 have *Rain = true*, 88 have *Rain = false*

$$\begin{aligned}\hat{\mathbf{P}}(Rain|Sprinkler = true, WetGrass = true) \\ &= \text{NORMALIZE}(\langle 32, 88 \rangle) \\ &= \langle 0.267, 0.733 \rangle\end{aligned}$$

Markov blanket sampling

Markov blanket of *Cloudy* is
Sprinkler and *Rain*

Markov blanket of *Rain* is
Cloudy, *Sprinkler*, and *WetGrass*



Probability given the Markov blanket is calculated as follows:

$$P(x'_i | mb(X_i)) = \alpha P(x'_i | parents(X_i)) \prod_{Z_j \in Children(X_i)} P(z_j | parents(Z_j))$$

Remark: the Gibbs sampling process will eventually spend, on the long run, in a state, a fraction of time proportional to its posterior probability.

Naïve Bayes Classifier (NBC)

$$\begin{aligned}\mathbf{P}(C|e_1 \wedge e_2) &= \frac{1}{\mathbf{P}_{(e_1 \wedge e_2)}} \mathbf{P}(C) \mathbf{P}(e_1 \wedge e_2|C) \quad (\textit{Bayes Rule}) \\ &= \frac{1}{\mathbf{P}_{(e_1 \wedge e_2)}} \mathbf{P}(C) \frac{1}{\mathbf{P}_{(C)}} \mathbf{P}(e_1 \wedge e_2 \wedge C) \quad (\textit{def. of cond. prob.}) \\ &= \frac{1}{\mathbf{P}_{(e_1 \wedge e_2)}} \mathbf{P}(e_1|e_2 \wedge C) \mathbf{P}(e_2|C) \mathbf{P}(C) \quad (\textit{chain rule twice}) \\ &= \frac{1}{\mathbf{P}_{(e_1 \wedge e_2)}} \mathbf{P}(C) \mathbf{P}(e_1|C) \mathbf{P}(e_2|C) \quad (\textit{independence assumption})\end{aligned}$$

NBC computes the score $\mathbf{P}(Cause) \prod_i \mathbf{P}(Effect_i|Cause)$ for each *Cause* and puts the new example in the *Class (Cause)* with the highest score

It assumes **conditional independence** of effects (features) given cause (class).

Quick and powerful, especially in text classification and pattern recognition.

NBC Example

A company sells batteries, mice and some other stuff. They use many boxes, which differ in size (small/big), color (black/white) and weight (light/heavy). Because of a mistake, the products have been muddled and put in wrong boxes, which have been closed before being labeled. The employees have opened 20 boxes and looked into them. The results are presented in the table. The next box is big, white and heavy. What does a Naïve Bayes Classifier think we'll find inside this box? Show all your work.

Type	Size		Weight		Color		Total
	Big	Small	Heavy	Light	White	Black	
Battery	8	2	7	3	5	5	10
Mouse	1	5	3	3	2	4	6
Other	0	4	3	1	3	1	4
TOTAL	9	11	13	7	10	10	20

NBC Example

Type	Size		Weight		Color		Total
	Big	Small	Heavy	Light	White	Black	
Battery	8	2	7	3	5	5	10
Mouse	1	5	3	3	2	4	6
Other	0	4	3	1	3	1	4
TOTAL	9	11	13	7	10	10	20

$$P(battery) = 10/20 = 0.5 \quad P(mouse) = 6/20 = 0.3 \quad P(other) = 0.2$$

$$P(big|battery) = 8/10 = 0.8$$

$$P(white|battery) = 5/10 = 0.5$$

$$P(heavy|battery) = 7/10 = 0.7$$

...

NBC Example

$$P(battery) = 10/20 = 0.5 \quad P(mouse) = 6/20 = 0.3 \quad P(other) = 0.2$$

$$P(big|battery) = 8/10 = 0.8$$

$$P(white|battery) = 5/10 = 0.5$$

$$P(heavy|battery) = 7/10 = 0.7$$

...

$$P(battery|big, white, heavy) =$$

$$P(battery) * P(big|battery) * P(white|battery) * P(heavy|battery) =$$

$$0.5 * 0.8 * 0.5 * 0.7 = 0.140$$

$$P(mouse|big, white, heavy) = 0.008$$

$$P(other|big, white, heavy) = 0.000$$

NBC Performance

Advantages:

- ◇ simple
- ◇ rapid and efficient
- ◇ can handle missing data and noise
- ◇ can handle multiple classes easily
- ◇ high performance in text classification

Summary

Building arcs in causal direction in a Bayes net helps obtaining probability values as well as reasoning

Exact inference in Bayes nets can be expensive, even if optimized. Possible solution: approximate via sampling

Rejection Sampling: samples inconsistent with observations are discarded

Likelihood Weighting: adjust sample weight each time we need to generate a value for an evidence variable

MCMC: generate a sample by making a random change to the previous one

LEARNING FROM OBSERVATIONS

CHAPTER 18, SECTIONS 1–3
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Learning: general idea
- ◇ A first example: decision tree learning
- ◇ Measuring learning performance

Learning

Learning is essential for unknown environments or if no algorithm is known, i.e., when designer lacks omniscience (all situations the agent can be in, all changes, or no algorithm available for the problem at hand)

Learning is useful as a system construction method, i.e., expose the agent to reality rather than trying to write it down (if the latter method is chosen, the system might be tedious to write and maintain)

Learning might be useful for discovering new, unsuspected patterns, i.e. to do **data mining**

We need to learn what Learning is

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E . (Tom Mitchell)

Learning: general idea

Simplest form: learn a function from examples only

f is the target function

An example is a pair $(x, f(x))$, e.g., $(\begin{array}{c|c|c} O & O & X \\ \hline & X & \\ \hline X & & \end{array}, +1)$

Problem: find a(n) hypothesis h
such that $h \approx f$
given a training set of examples

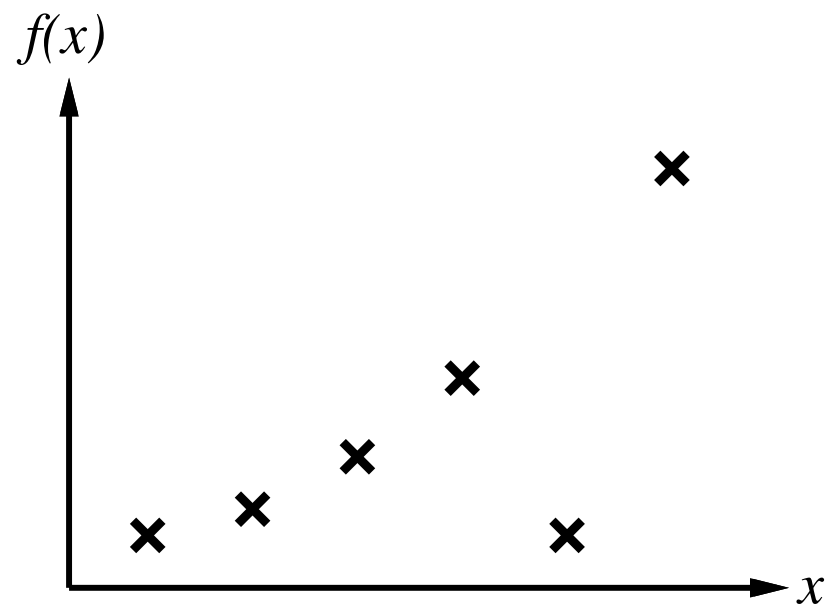
(This is a highly simplified model of real learning:

- Ignores prior knowledge (tabula rasa)
- Assumes a deterministic, observable “environment”
- Assumes labeled examples are given)

Learning: general idea

Construct/adjust h to agree with f on training set
(h is **consistent** if it agrees with f on all examples)

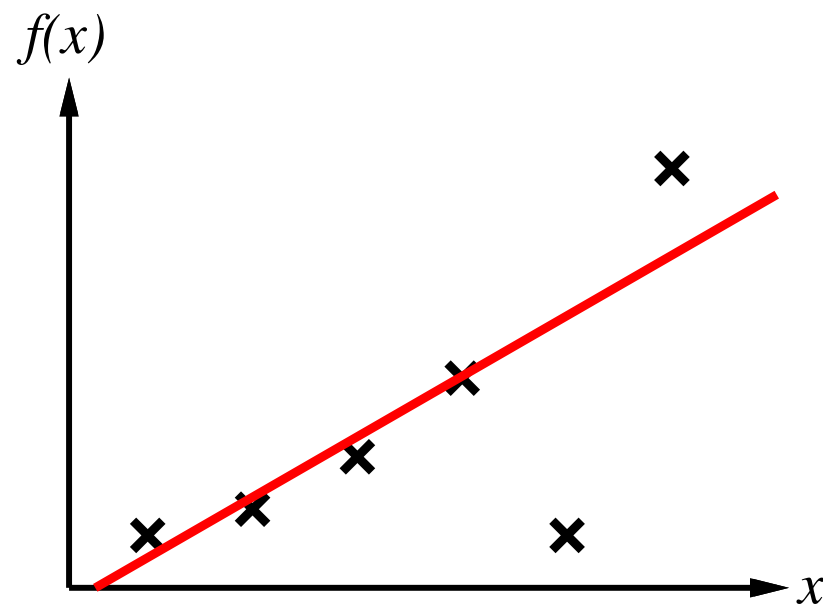
E.g., curve fitting:



Learning: general idea

Construct/adjust h to agree with f on training set
(h is **consistent** if it agrees with f on all examples)

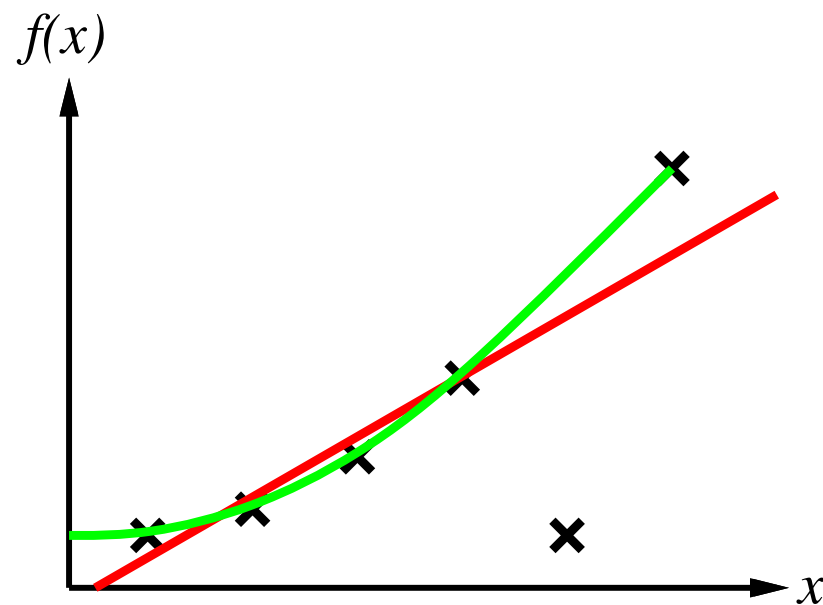
E.g., curve fitting:



Learning: general idea

Construct/adjust h to agree with f on training set
(h is **consistent** if it agrees with f on all examples)

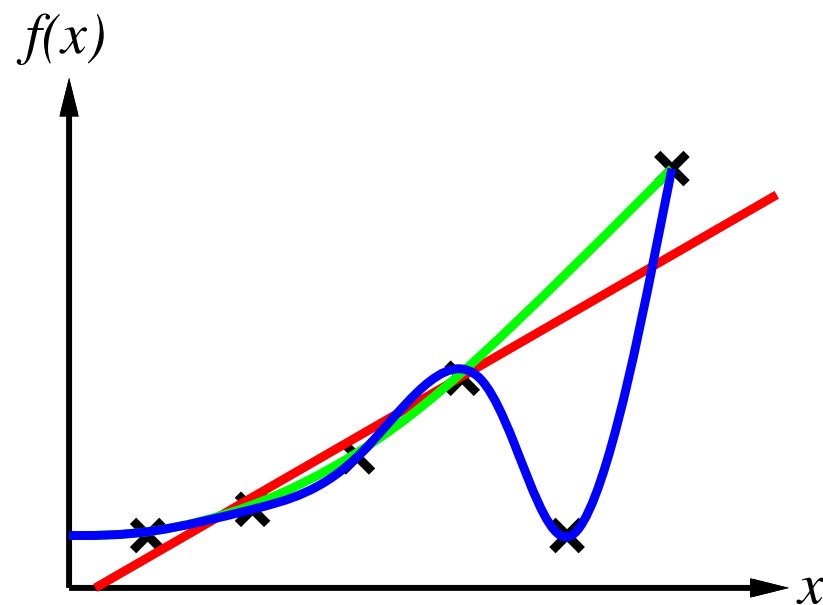
E.g., curve fitting:



Learning: general idea

Construct/adjust h to agree with f on training set
(h is **consistent** if it agrees with f on all examples)

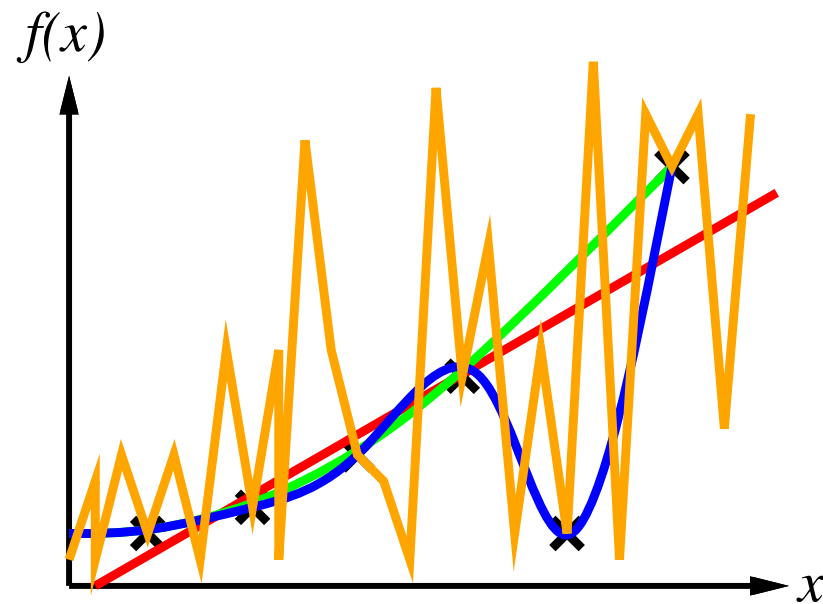
E.g., curve fitting:



Learning: general idea

Construct/adjust h to agree with f on training set
(h is **consistent** if it agrees with f on all examples)

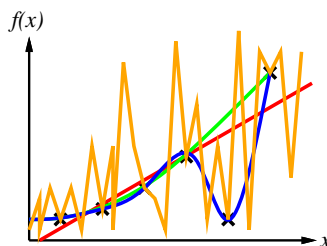
E.g., curve fitting:



Learning: general idea

Construct/adjust h to agree with f on training set
(h is **consistent** if it agrees with f on all examples)

E.g., curve fitting:



Ockham's razor: maximize a combination of consistency and simplicity

Realizable learning problem: the hypothesis space contains the real function

A hypothesis **generalizes** well if it correctly predicts the value of f on **novel** examples

Classification ($f(x)$ a finite set of values): vs. regression ($f(x)$: number)

Learning: general idea

Learning comes in more flavors (supervised vs. unsupervised):

- **Unsupervised learning**: learn patterns in the input even though no explicit feedback is given (e.g., clustering, PCA, association rule learning)
- **Supervised learning**: examples (pairs (input, correct output)) are given; learn a function which maps input to output
- **Semisupervised learning**: only a small fraction of the data is labeled
- **Reinforcement learning**: occasional rewards are presented to the agent; learn a policy (what action to take in every possible situation)

Observations:

- Supervised learning of correct actions requires a “teacher”
- Reinforcement learning is harder, but requires no teacher

Learning: general idea

Learning comes in more flavors (online vs. batch):

- **Batch** aka **offline learning**: train using all examples available, then use
- **Online learning**: train incrementally, as the training examples arrive

Learning comes in more flavors (instance-based vs. model-based)

- **Instance-based**: learn examples by heart, then **compare** the new example with the existing ones
- **Model-based**: build a **model** based on examples

Attribute-based representations

Example = **attributes' values** aka **features** (Boolean, discrete etc.) + **label**
 E.g., situations where Stuart will/won't wait for a table:

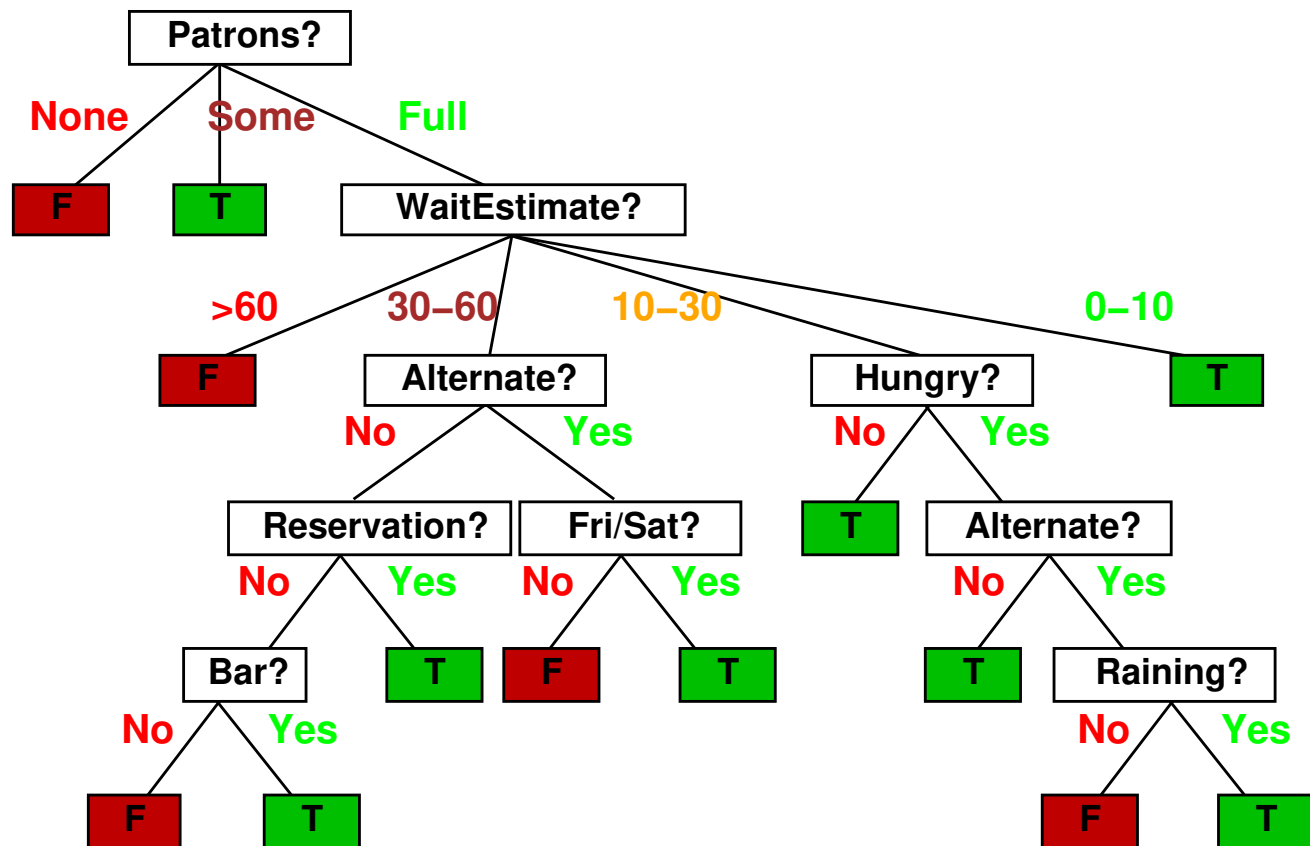
Example	Attributes										Target
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
X_1	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>Some</i>	<i>\$\$\$</i>	<i>F</i>	<i>T</i>	<i>French</i>	<i>0–10</i>	<i>T</i>
X_2	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>Full</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Thai</i>	<i>30–60</i>	<i>F</i>
X_3	<i>F</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>Some</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Burger</i>	<i>0–10</i>	<i>T</i>
X_4	<i>T</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>Full</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Thai</i>	<i>10–30</i>	<i>T</i>
X_5	<i>T</i>	<i>F</i>	<i>T</i>	<i>F</i>	<i>Full</i>	<i>\$\$\$</i>	<i>F</i>	<i>T</i>	<i>French</i>	<i>>60</i>	<i>F</i>
X_6	<i>F</i>	<i>T</i>	<i>F</i>	<i>T</i>	<i>Some</i>	<i>\$\$</i>	<i>T</i>	<i>T</i>	<i>Italian</i>	<i>0–10</i>	<i>T</i>
X_7	<i>F</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>None</i>	<i>\$</i>	<i>T</i>	<i>F</i>	<i>Burger</i>	<i>0–10</i>	<i>F</i>
X_8	<i>F</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>Some</i>	<i>\$\$</i>	<i>T</i>	<i>T</i>	<i>Thai</i>	<i>0–10</i>	<i>T</i>
X_9	<i>F</i>	<i>T</i>	<i>T</i>	<i>F</i>	<i>Full</i>	<i>\$</i>	<i>T</i>	<i>F</i>	<i>Burger</i>	<i>>60</i>	<i>F</i>
X_{10}	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>Full</i>	<i>\$\$\$</i>	<i>F</i>	<i>T</i>	<i>Italian</i>	<i>10–30</i>	<i>F</i>
X_{11}	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>None</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Thai</i>	<i>0–10</i>	<i>F</i>
X_{12}	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>Full</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Burger</i>	<i>30–60</i>	<i>T</i>

Classification of examples is **positive** (T) or **negative** (F) - see column Target

Decision trees

One possible representation for hypotheses

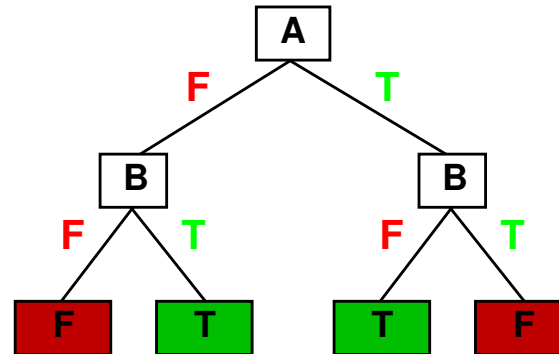
E.g., here is the “true” tree for deciding whether to wait:



Expressiveness

Decision trees can express any function of the input attributes.
E.g., for Boolean functions, truth table row $\rightarrow$ path to leaf:

A	B	A xor B
F	F	F
F	T	T
T	F	T
T	T	F



Trivially, there is a consistent decision tree for any training set with one path to leaf for each example (unless f nondeterministic in x) but it probably won't generalize to new examples

Prefer to find more **compact** decision trees

Hypothesis spaces

How many distinct decision trees with n Boolean attributes??

Hypothesis spaces

How many distinct decision trees with n Boolean attributes??

$\sim$ number of Boolean functions with n variables

Hypothesis spaces

How many distinct decision trees with n Boolean attributes??

$\sim$ number of Boolean functions with n variables

$=$ number of distinct truth tables with 2^n rows

Hypothesis spaces

How many distinct decision trees with n Boolean attributes??

$\sim$ number of Boolean functions with n variables

$=$ number of distinct truth tables with 2^n rows $= 2^{2^n}$

Hypothesis spaces

How many distinct decision trees with n Boolean attributes??

~ number of Boolean functions with n variables

= number of distinct truth tables with 2^n rows = 2^{2^n}

E.g., with 6 Boolean attributes, there are 18,446,744,073,709,551,616 trees

Hypothesis spaces

How many distinct decision trees with n Boolean attributes??

$\sim$ number of Boolean functions with n variables

$=$ number of distinct truth tables with 2^n rows $= 2^{2^n}$

E.g., with 6 Boolean attributes, there are 18,446,744,073,709,551,616 trees

How many purely conjunctive hypotheses (e.g., $Hungry \wedge \neg Rain$)??

Hypothesis spaces

How many distinct decision trees with n Boolean attributes??

$\sim$ number of Boolean functions with n variables

= number of distinct truth tables with 2^n rows = 2^{2^n}

E.g., with 6 Boolean attributes, there are 18,446,744,073,709,551,616 trees

How many purely conjunctive hypotheses (e.g., $Hungry \wedge \neg Rain$)??

Each attribute can be in (positive), in (negative), or out

$\Rightarrow 3^n$ distinct conjunctive hypotheses

More expressive hypothesis space

- increases chance that target function can be expressed 😊
 - increases number of hypotheses consistent w/ training set
- $\Rightarrow$ may get worse predictions 😞

Decision tree learning

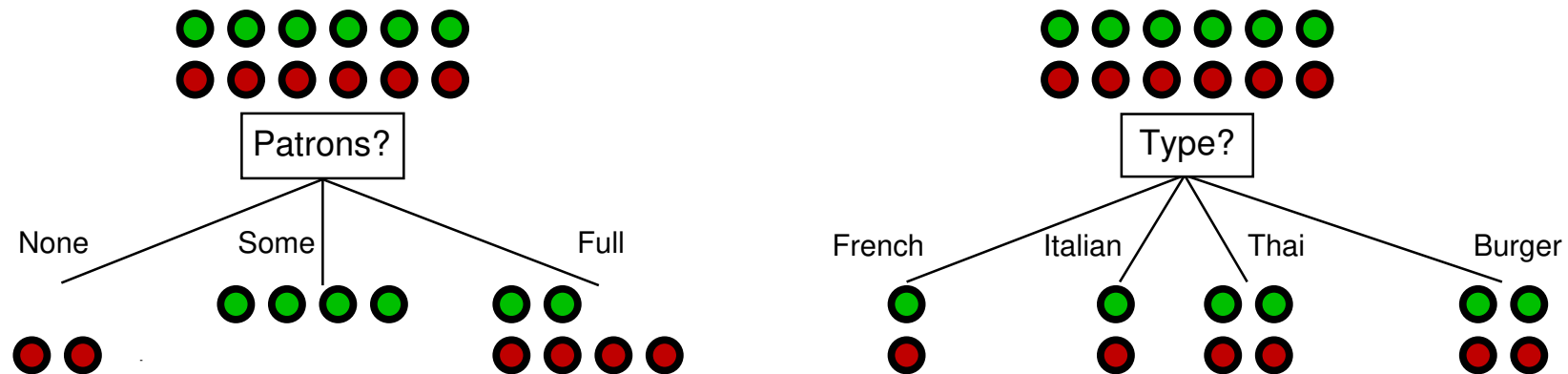
Aim: find a small tree consistent with the training examples

Idea: (recursively) choose “most significant” attribute as root of (sub)tree

```
function DTL(examples, attributes, parent_examples) returns a decision tree
  if examples is empty then return PLURALITY_VALUE(parent_examples)
  else if all examples have the same classification then return the classification
  else if attributes is empty then return PLURALITY_VALUE(examples)
  else
     $A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$ 
    tree  $\leftarrow$  a new decision tree with root test A
    for each value  $v_k$  of A do
       $\text{exs} \leftarrow \{e : e \in \text{examples} \text{ and } e.A = v_k\}$ 
      subtree  $\leftarrow$  DTL(examplesi, attributes − A, examples)
      add a branch to tree with label  $A=v_k$  and subtree subtree
  return tree
```

Choosing an attribute

Idea: a good attribute splits the examples into subsets that are (ideally) “all positive” or “all negative”



Patrons? is a better choice—gives **information** about the classification

Information

Information answers questions

The more clueless I am about the answer initially, the more information is contained in the answer

Scale: 1 bit = answer to Boolean question with prior $\langle 0.5, 0.5 \rangle$

Information in an answer when prior is $\langle P_1, \dots, P_n \rangle$ is

$$H(\langle P_1, \dots, P_n \rangle) = \sum_{i=1}^n -P_i \log_2 P_i$$

(also called **entropy** of the prior)

Entropy measures uncertainty; gaining information reduces entropy

Information contd.

Suppose we have p positive and n negative examples at the root

$\Rightarrow H(\langle p/(p+n), n/(p+n) \rangle)$ bits needed to classify a new example

E.g., for 12 restaurant examples, $p = n = 6$ so we need 1 bit

An attribute A splits the examples E into subsets E_i , each of which (we hope) needs less information to complete the classification

Let E_i have p_i positive and n_i negative examples

$\Rightarrow H(\langle p_i/(p_i+n_i), n_i/(p_i+n_i) \rangle)$ bits needed to classify a new example

$\Rightarrow$ **expected** number of bits per example over all branches is

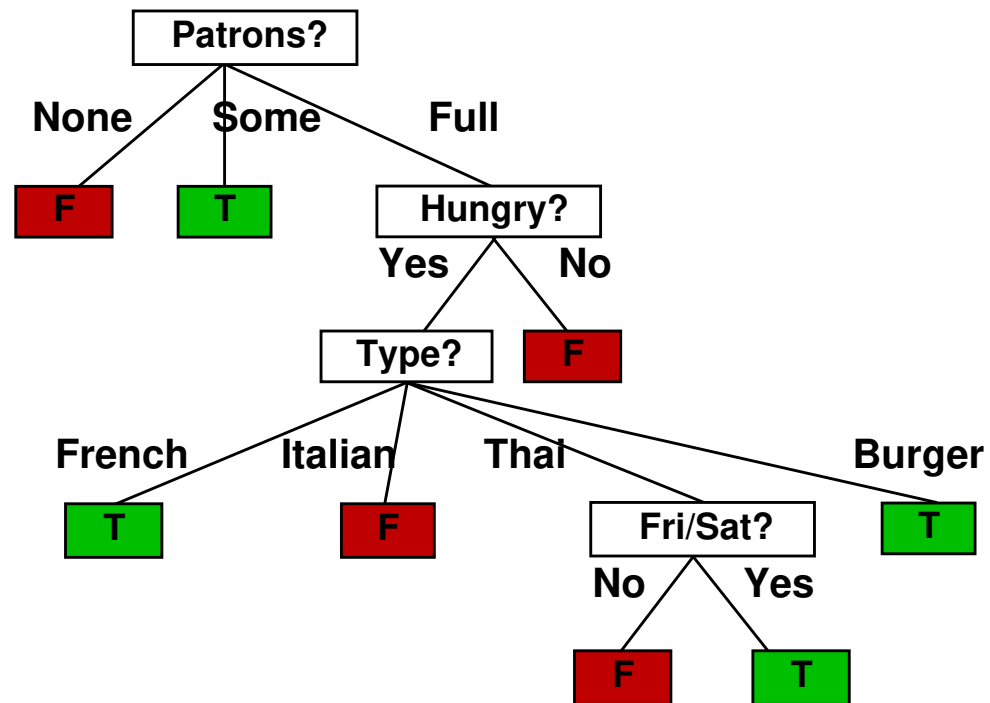
$$Reminder(A) = \sum_i \frac{p_i + n_i}{p + n} H(\langle p_i/(p_i + n_i), n_i/(p_i + n_i) \rangle)$$

For *Patrons*, this is 0.459 bits, for *Type* this is (still) 1 bit

$\Rightarrow$ choose the attribute that minimizes the remaining information needed (i.e., maximizes the $Gain(A) = H(\langle \frac{p}{n+p}, \frac{n}{n+p} \rangle) - Reminder(A)$)

Example contd.

Decision tree learned from the 12 examples:



Substantially simpler than the "true" tree—a more complex hypothesis isn't justified by small amount of data

Gini impurity

Let us assume we classify examples at random, based on the distribution of classes in the training set. Under this assumption, we define:

Gini impurity = probability that an example is misclassified

If p_i is the probability of class i , we can show that:

$$\text{Gini impurity} = 1 - \sum_i p_i^2$$

The Gini impurity of a **split** (on an attribute) is computed in the very same way as the Information Gain, but now the entropy is replaced by the Gini impurity. We'll choose for split the attribute with the best Gini score.

Decision trees

Implementations:

- ◇ ID3
- ◇ C4.5/C5.0/J4.8(Java - Weka)
- ◇ scikit-learn (Decision Trees DTs)

Extensions:

- ◇ missing values: propagate example on all branches and weight with frequency
- ◇ continuous values for attributes → split points
- ◇ multivalued attributes (e.g., SSN) → replace information gain with **gain ratio**, which takes into consideration, besides information, the number of branches and their sizes
- ◇ continuous-valued output: regression trees (each leaf is a linear function)

Summary

For supervised learning, the aim is to find a simple hypothesis that is approximately consistent with training examples

Decision tree learning using information gain

MAKING SIMPLE DECISIONS

CHAPTER 16.1-3,6
(SLIDES ADAPTED FROM S. RUSSELL)

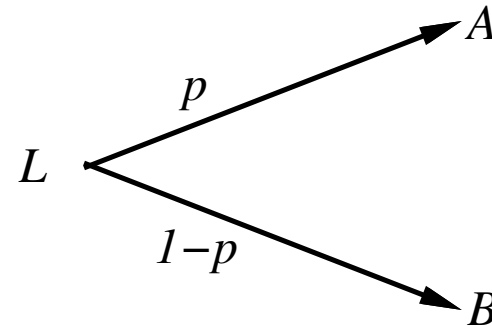
Outline

- ◇ Rational preferences
- ◇ Utilities
- ◇ Money
- ◇ Value of information
- ◇ Assessing probabilities

Preferences

An agent chooses among prizes (A , B , etc.) and lotteries, i.e., situations with uncertain prizes

Lottery $L = [p, A; (1 - p), B]$



Notation:

$A \succ B$	A preferred to B
$A \sim B$	indifference between A and B
$A \not\succ B$	B not preferred to A

Rational preferences

Idea: preferences of a rational agent must obey constraints.

Rational preferences $\Rightarrow$

behavior describable as maximization of expected utility (MEU)

Constraints:

Orderability(must choose)

$$(A \succ B) \vee (B \succ A) \vee (A \sim B)$$

Transitivity

$$(A \succ B) \wedge (B \succ C) \Rightarrow (A \succ C)$$

Continuity

$$A \succ B \succ C \Rightarrow \exists p \ [p, A; 1 - p, C] \sim B$$

Substitutability

$$A \sim B \Rightarrow [p, A; 1 - p, C] \sim [p, B; 1 - p, C]$$

Monotonicity(must prefer the most probable lottery)

$$A \succ B \Rightarrow (p \geq q \Leftrightarrow [p, A; 1 - p, B] \succeq [q, A; 1 - q, B])$$

Decomposability(reduce compound to simple lottery via probabil. theory)

$$[p, A; 1 - p, [q, B; 1 - q, C]] \sim [p, A; (1 - p)q, B; (1 - p)(1 - q), C]$$

Rational preferences contd.

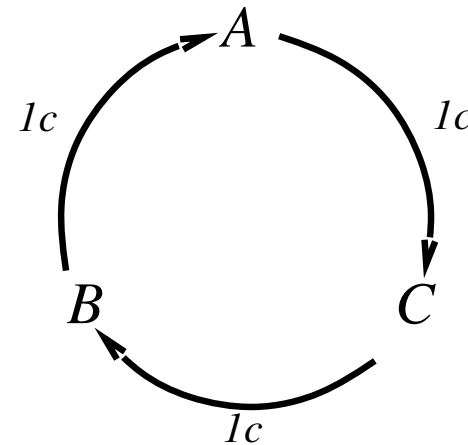
Violating the constraints leads to self-evident irrationality

For example: an agent with intransitive preferences can be induced to give away all its money

If $B \succ C$, then an agent who has C would pay (say) 1 cent to get B

If $A \succ B$, then an agent who has B would pay (say) 1 cent to get A

If $C \succ A$, then an agent who has A would pay (say) 1 cent to get C



Maximizing expected utility

Theorem (von Neumann and Morgenstern): Given preferences satisfying the constraints above, there exists a real-valued function U such that

$$U(A) \geq U(B) \Leftrightarrow A \succsim B$$
$$U([p_1, S_1; \dots; p_n, S_n]) = \sum_i p_i U(S_i)$$

MEU principle:

Choose the action that maximizes expected utility

Note: an agent can be entirely rational (consistent with MEU) without ever representing or manipulating utilities and probabilities

E.g., a lookup table for perfect tictactoe

Utilities

Utilities map states to real numbers. Which numbers?

Standard approach to assessment of human utilities:

compare a given state A to a **standard lottery** L_p that has

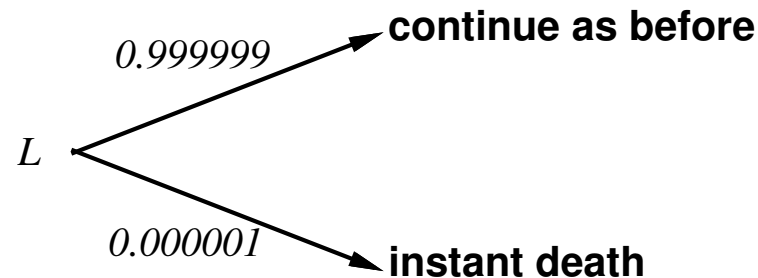
“best possible prize” $u_{\top}$ with probability p

“worst possible catastrophe” $u_{\perp}$ with probability $(1 - p)$

adjust lottery probability p until $A \sim L_p$

pay \$30

$\sim$



Utility scales

Normalized utilities: $u_{\top} = 1.0$, $u_{\perp} = 0.0$

Micromorts: one-millionth chance of death

useful for Russian roulette, paying to reduce product risks, etc.

E.g., in US, 1 micromort = 375 km. Car lifetime = 150000 km.

QALY (quality-adjusted life years): for medical decisions involving a high risk.

E.g., living one year in full health vs. living 2 years on dialysis

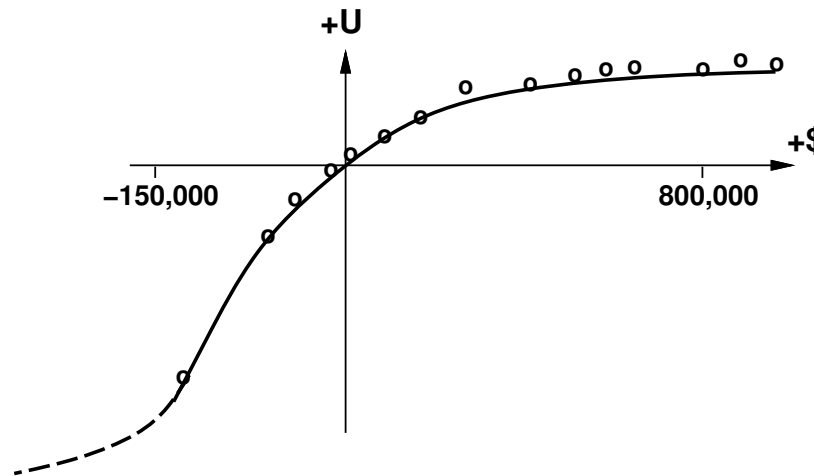
Money

Money does **not** behave as a utility function

Risk-averse: Given a lottery L with expected monetary value $EMV(L)$, usually $U(L) < U(S_{EMV(L)})$, where S_n denotes the state of having $\$n$.

E.g. grab \$1,000,000 prize vs. risk it on a coin flip and win \$2,500,000

Typical empirical data, extrapolated with **risk-seeking** behavior:



Certainty equivalent of L = the value an agent would accept in lieu of L

Insurance premium: difference up to L 's EMV

Value of information

How to compute the value of acquiring each possible piece of information?

Ex: there are n indistinguishable drilling blocks; one has oil worth C , the others have no oil. The price per block is C/n . A consultant offers accurate answer of oil presence in block #3. How much to pay for this info?

Value of information

n indistinguishable blocks; just one has oil worth C ; price per block: C/n

Solution: profit = expected gain - cost

Consultant answer is "Yes":

$$pr_Y = C - \frac{C}{n} = \frac{n-1}{n} \times C$$

Consultant answer is "No" (and investors are risk-neutral):

$$pr_N = \left(\frac{1}{n-1} \times C + \frac{n-2}{n-1} \times 0 \right) - \frac{C}{n} = \frac{1}{n \times (n-1)} \times C$$

Expected profit:

$$\frac{1}{n} \times pr_Y + \frac{n-1}{n} \times pr_N = \frac{1}{n} \times \left(\frac{n-1}{n} \times C \right) + \frac{n-1}{n} \times \left(\frac{1}{n \times (n-1)} \times C \right) = \frac{C}{n}$$

Value of information: same as the value of block itself!

χ^2 test: blue eyed people in two countries

Are there significantly more blue-eyed people in country U compared to S?

Observed	Blue	Other	Total
U	111	189	300
S	17	83	100
TOTAL	128	272	400

Expected	Blue	Other
U	96	204
S	32	68

Chi	Blue	Other
U	2.34	1.10
S	7.03	3.31

Expected value of blue-eyed people in U: $300/400 \times 128 = 96$

Deviation: $\frac{(\text{observed} - \text{expected})^2}{\text{expected}}$ (e.g. $(111 - 96)^2 / 96 = 2.34$)

Total deviation Δ : 13.78 (measures deviation from the **absence** of pattern)

Compare this to the value 6.6349 (for $\alpha = 0.01$ level of significance) $\rightarrow$ reject the **null hypothesis**. So: based on the observed data, there exists a connection between country and eye color (the lack of connection is unlikely).

The higher Δ value, the more likely the data supports the connection.

Summary

Utilities can express rational preferences

Rational decisions: maximize the expected utility

Money?

Value of information

EVALUATING HYPOTHESES

CHAPTER 18.3,4
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

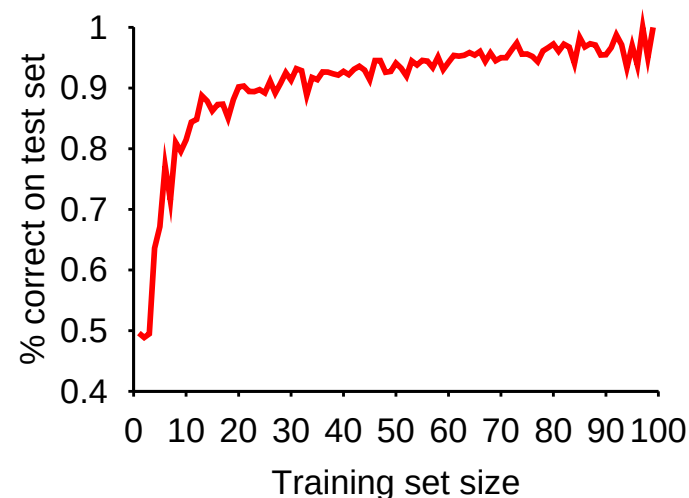
- ◇ Measuring performance
- ◇ Overfitting
- ◇ Validation

How to measure Learning Performance

How do we know that $h \approx f$? One can try h on a new **test set** of examples (with the **same distribution over example space** as the training set)

Learning curve = % correct on test set as a function of training set size

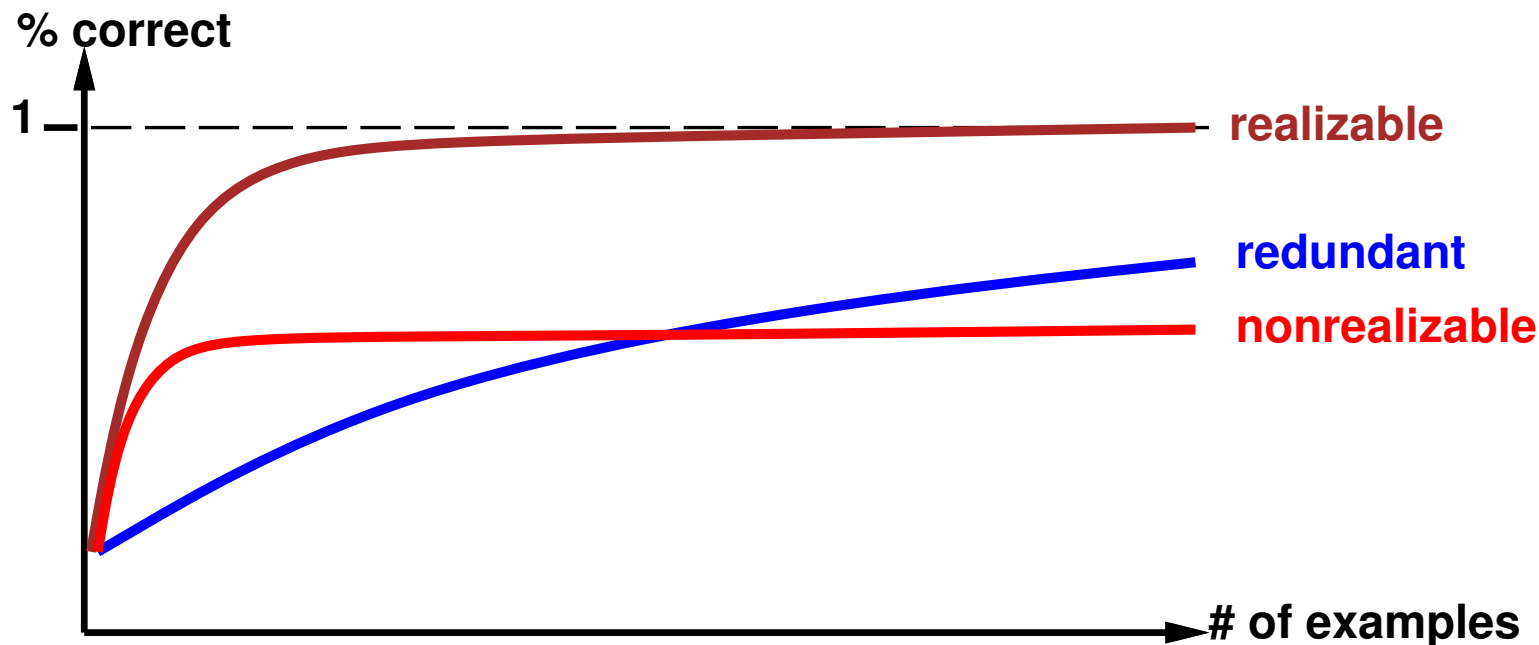
E.g., for the restaurant data with 100 randomly generated examples (randomly split 20 times into train + test data), graph averaged over 20 trials



How to measure Learning Performance

Learning curve depends on some factors:

- **realizable** (can express target function) vs. **non-realizable**
non-realizability can be due to missing attributes
or restricted hypothesis class (e.g., thresholded linear function)
- redundant expressiveness (e.g., loads of irrelevant attributes)



How to measure Learning Performance

Confusion matrix for a 2-class problem:

	Real: P	Real: N
Alg: P	TP	FP
Alg: N	FN	TN

	Real: P	Real: N
Alg: P	Hit	False Alarm
Alg: N	Miss	Correct Rejection

Performance measures:

◇ Accuracy $A = \frac{TP+TN}{TP+FP+FN+TN}$

◇ Precision $Pr = \frac{TP}{TP+FP}$

◇ Recall $Rc = \frac{TP}{TP+FN}$

◇ F-measure $F = \frac{2*Pr*Rc}{Pr+Rc}$ (aka F_1 measure, a particular case of F_β measure)

Overfitting

Overfitting = discover patterns which happen to exist in the training data, but do not exist in general

Overfitting leads to GOOD performance on the training data, but POOR performance on previously unseen data

Overfitting:

- ◇ more likely if the hypothesis space and number of inputs grow
- ◇ less likely if the number of training examples increases

Overfitting

Fighting overfitting in case of decision tree: prune nodes having only leaf descendants if those descendants contain irrelevant information.

An attribute is considered irrelevant if it splits the examples into subsets with roughly the same proportion of positive examples as the whole set.

Note: The price for simpler trees could be the presence of NOISE in leaves (now, we give up the purity property and accept to have examples from more than one class in a leaf).

To test for irrelevance, we employ the χ^2 test

Remember χ^2 test: blue eyed people in U and S

Are there significantly more blue-eyed people in country U compared to S?

Observed	Blue	Other	Total
U	111	189	300
S	17	83	100
TOTAL	128	272	400

Expected	Blue	Other
U	96	204
S	32	68

Chi	Blue	Other
U	2.34	1.10
S	7.03	3.31

Expected value of blue-eyed people in U: $300/400 \times 128 = 96$

Deviation: $\frac{(\text{observed} - \text{expected})^2}{\text{expected}}$ (e.g. $(111 - 96)^2 / 96 = 2.34$)

Total deviation Δ : 13.78 (measures deviation from the **absence** of pattern)

Compare this to the value 6.6349 (for $\alpha = 0.01$ level of significance) $\rightarrow$ reject the **null hypothesis**. So: based on the observed data, there exists a connection between country and eye color (the lack of connection is unlikely).

The higher Δ value, the more likely the data supports the connection.

χ^2 test for decision tree learning

Null hypothesis: an attribute is irrelevant (hence the information gain for an infinitely large sample would be zero).

E.g., if value k for an attribute splits the example set into subsets containing p_k and n_k examples respectively, then the expected numbers $\hat{p}_k$ and $\hat{n}_k$ are:

$$\hat{p}_k = p \times \frac{p_k + n_k}{p + n}$$

$$\hat{n}_k = n \times \frac{p_k + n_k}{p + n}$$

Deviation:

$$\Delta = \sum_k \left(\frac{(p_k - \hat{p}_k)^2}{\hat{p}_k} + \frac{(n_k - \hat{n}_k)^2}{\hat{n}_k} \right)$$

Evaluating and Choosing the Best Hypothesis

Question: can we assume the data in the past can help in building models to be used in the future?

Each example (before we see it) is regarded as a random variable E_j whose observed value $e_j = (x_j, y_j)$ is sampled from a distribution over examples that remains **stationary** over time.

Assumption: data (1) independent and (2) identically distributed:

$$1) \mathbf{P}(E_j | E_{j-1}, E_{j-2}, \dots) = \mathbf{P}(E_j)$$

$$2) \mathbf{P}(E_j) = \mathbf{P}(E_{j-1}) = \mathbf{P}(E_{j-2}) = \dots$$

These assumptions connect past and future: so, using examples from the past, we can build a hypothesis which fits data from the future.

Evaluating and Choosing the Best Hypothesis

Question: given 2 hypotheses h_1 and h_2 , which one better represents f ?

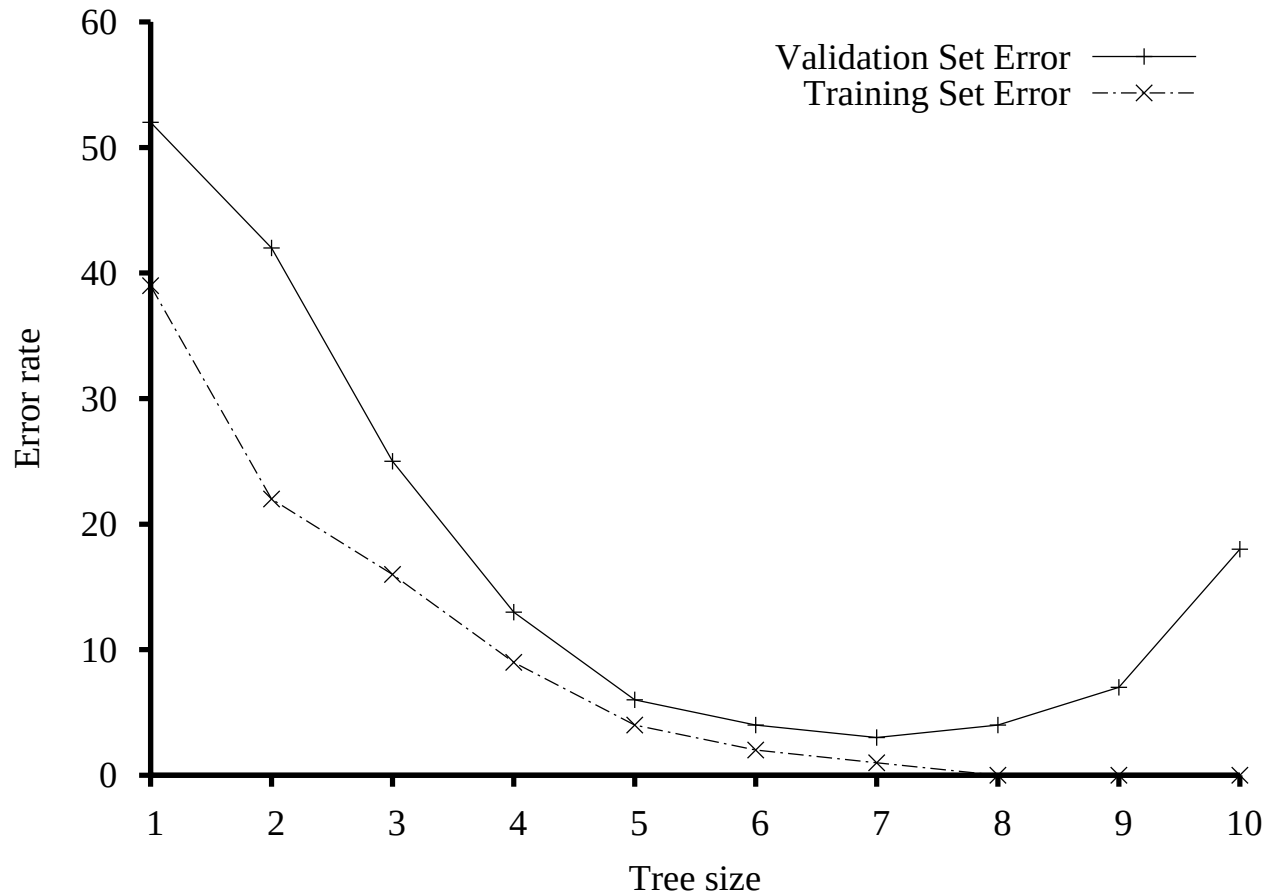
Error rate: proportion of times when $y \neq h(x)$ for example (x, y) . Unfortunately, small error rate computed on the training set does not mean good generalization (i.e., small error rate on new examples). What to do?

◇ **Holdout cross-validation:** randomly split the data at hand into a **training** set used for producing h and a **testing** set used for evaluating its performance.

◇ **K-fold cross validation:** divide the set of training data into k groups. Leave one group out and perform the training on the other $k - 1$ groups, then evaluate the performance on the group left out. Repeat this for each group and report the average performance. Typical values for k : 5, 10, n (the latter is called "Leave One Out Cross Validation").

Warning: beware of peeking! Don't mix model **evaluation** and **selection**! The test data set MUST be kept separated and used for evaluation only!

Overfitting and Cross Validation



Stop increasing complexity when the validation set error start increasing, even though the error on train data keeping decreasing.

Summary

Measuring performance: Accuracy, Precision, Recall, F-measure etc. ()

Overfitting: good score on training data, poor score on new data. Avoid by keeping the model simple, by cross validation etc.

χ^2 test helps in eliminating irrelevant tests, thus shrinking the tree

Cross validation helps in choosing hypotheses

Learning performance = precision measured on SEPARATE test set

REGRESSION AND CLASSIFICATION

CHAPTER 18.4,6
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Model selection
- ◇ Loss
- ◇ Regularization
- ◇ Regression and Classification

Model selection: Complexity v. goodness of fit

Model selection: define hypothesis space

Optimization: find the best hypothesis within that space

E.g.: for fitting a polynomial on a set of points:

- ◇ model selection: choose the size (i.e., polynomial degree)
- ◇ optimization: for that size, pick up parameters to get the optimal fit (i.e., polynomial coefficients)

We need a tradeoff between underfit (model too simplistic to express the real function f) and overfit (model too complex, likely to fail on novel examples).

Model selection: Complexity v. goodness of fit

To obtain the underfit/overfit tradeoff:

loop until model starts to overfit do

 increase the model complexity (in our case, *size*)

 for each model of that *size*

 use cross-validation to get the average error rate on training and
 validation set

Model selection: Complexity v. goodness of fit

```
function MODEL-SELECTION(Learner, k, examples) returns a hypothesis
  local variables: errT, an array, indexed by size, storing training-set error rates
                    errV, an array, indexed by size, storing validation-set error rates
  for size = 1 to  $\infty$  do
    errT[size], errV[size]  $\leftarrow$  CROSS-VALIDATION(Learner, size, k, examples)
    if errT has converged then do
      best_size  $\leftarrow$  the value of size with minimum errV[size]
    return Learner(best_size, examples)

function CROSS-VALIDATION(Learner, size, k, examples) returns two values:
  average training set error rate, average validation set error rate
  fold_errT  $\leftarrow$  0; fold_errV  $\leftarrow$  0
  for fold = 1 to k do
    training_set, validation_set  $\leftarrow$  PARTITION(examples, fold, k)
    h  $\leftarrow$  Learner(size, training_set)
    fold_errT  $\leftarrow$  fold_errT + ERROR-RATE(h, training_set)
    fold_errV  $\leftarrow$  fold_errV + ERROR-RATE(h, validation_set)
  return fold_errT/k, fold_errV/k
```

Loss

Which spam/ham classifier is better: 1% error rate, mostly due to labeling spam as ham or 0.5% error rate mostly due to labeling ham as spam

Loss: Amount of utility lost by predicting $\hat{y} = h(x)$ instead of $y = f(x)$.

$$L(x, y, \hat{y}) = \text{Utility}(\text{result of using } y \text{ given an input } x) \\ - \text{Utility}(\text{result of using } \hat{y} \text{ given an input } x)$$

Loss

Loss: Amount of utility lost by predicting $\hat{y} = h(x)$ instead of $y = f(x)$.

Usual loss functions:

◇ $L_1(y, \hat{y}) = |y - \hat{y}|$ (absolute value)

◇ $L_2(y, \hat{y}) = (y - \hat{y})^2$ (squared error)

◇ $L_{0/1}(y, \hat{y}) = \begin{cases} 0, & \text{if } y = \hat{y} \\ 1, & \text{otherwise} \end{cases}$ (0/1 loss)

Generalization and Empirical Loss

Generalization loss: if ε is the set of all possible input–output examples

$$GenLoss_L(h) = \sum_{(x,y) \in \varepsilon} L(y, h(x))P(x, y)$$

This is a theoretical measure. Usually, $P(x, y)$ is not known. What to do?

Estimate $GenLoss_L(h)$ with the **Empirical Loss** on a set E of N examples:

$$EmpLoss_{L,E}(h) = \frac{1}{N} \sum_{(x,y) \in E} L(y, h(x))$$

We will choose h which minimizes the **empirical loss**

Small-scale v. large-scale Learning

Small-scale learning: 10 to 10K examples. Generalization error due to f not in the hypotheses space and from a too large variance due to the relatively small number of examples

Large-scale learning: 1M+ examples. Generalization error due to limits of computation

Regularization

Problem: how to avoid overfitting when doing model selection. Solutions:

- ◇ cross-validation on model size (see above)
- ◇ **Regularization**: explicitly penalize complex hypotheses

To achieve this, we'll search for hypothesis h with minimum total $Cost(h)$:

$$Cost(h) = EmpLoss(h) + \lambda Complexity(h)$$

E.g.: As a regularization function (complexity measure), one may use the sum of squares of coefficients of a polynomial

Note: Cross-validation still employed to choose the hypothesis which generalizes best, but we optimize via λ rather than *size*: pick the λ with the best score on the validation set

Feature selection

Problem: how to avoid overfitting when doing model selection. Solution:

◇ **Feature selection**: discard irrelevant attributes in order to reduce the dimensions the models deal with

E.g.: χ^2 pruning. Null hypothesis: an attribute is irrelevant (hence the information gain for an infinitely large sample would be zero).

Example: if value k for an attribute splits the example set in subsets with p_k and n_k examples respectively, then the expected numbers $\hat{p}_k$ and $\hat{n}_k$ are:

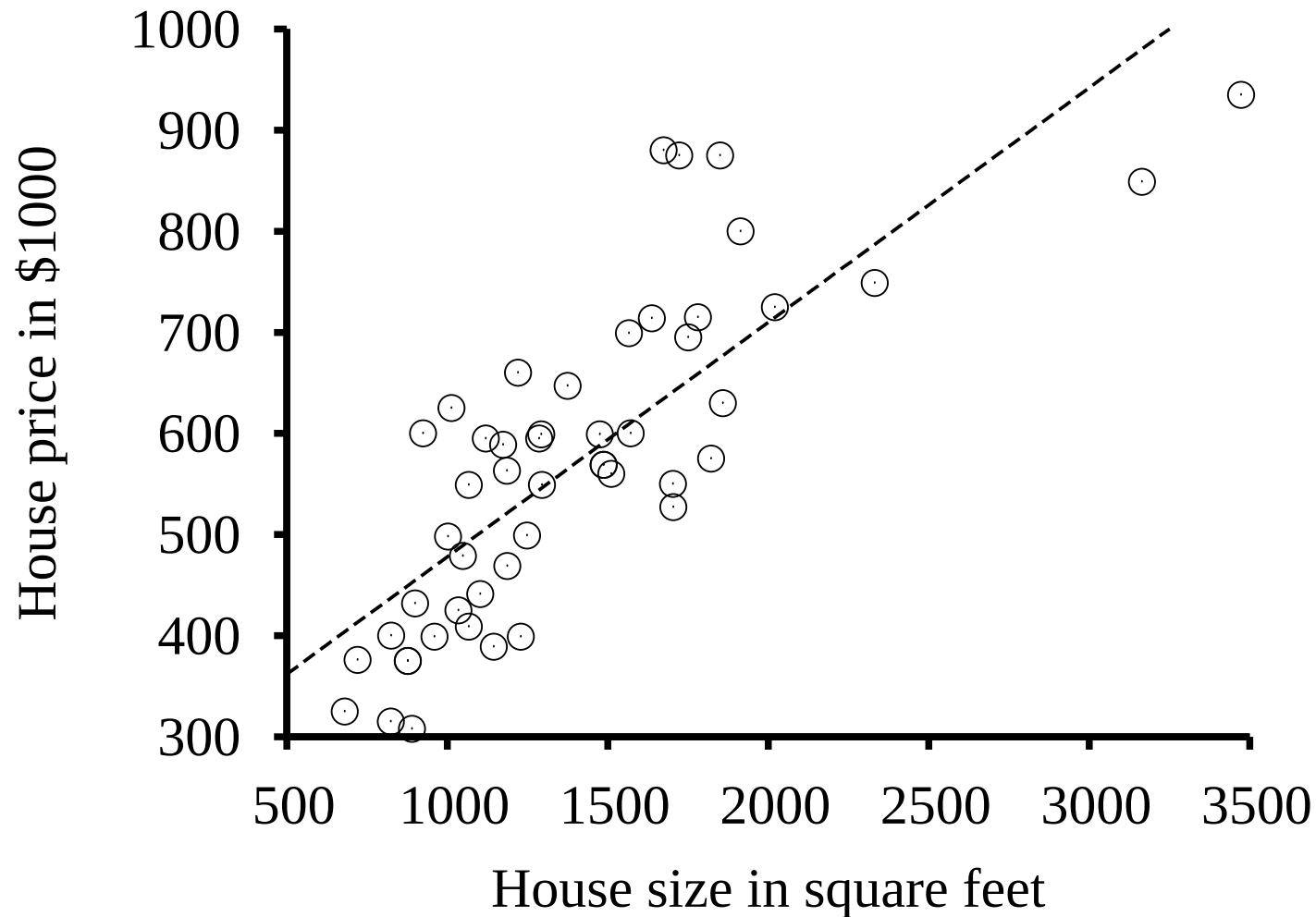
$$\hat{p}_k = p \times \frac{p_k + n_k}{p + n}$$

$$\hat{n}_k = n \times \frac{p_k + n_k}{p + n}$$

Deviation (if the attribute is not relevant, this is small):

$$\Delta = \sum_{k=1}^n \left(\frac{(p_k - \hat{p}_k)^2}{\hat{p}_k} + \frac{(n_k - \hat{n}_k)^2}{\hat{n}_k} \right)$$

Univariate Linear Regression (ULR)



Estimate Berkeley house price as a function of floor space.

Univariate Linear Regression

Hypothesis space: univariate linear functions $h_{\mathbf{w}}(x) = w_1x + w_0$.

Linear regression: learn $h_{\mathbf{w}}$ (i.e., find w_0 and w_1) that best fits the data.

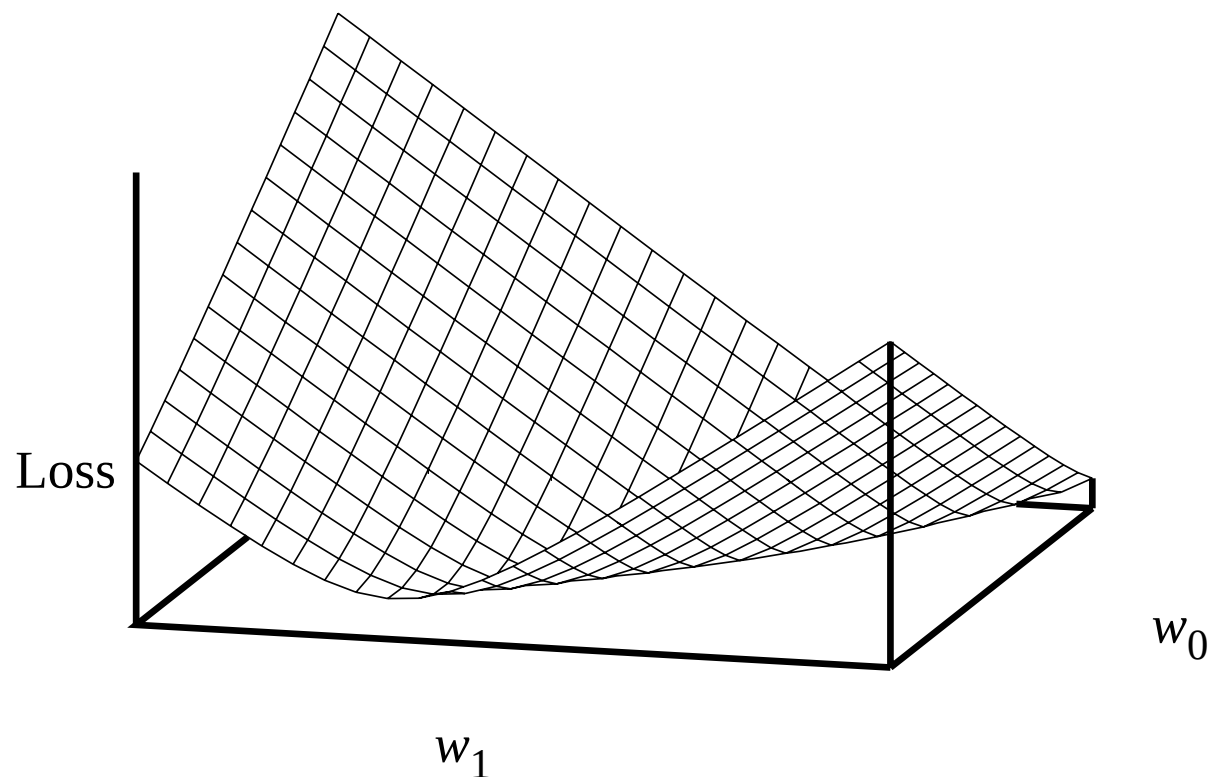
Loss L_2 (Gauss): $Loss(h_{\mathbf{w}}) = \sum_{j=1}^N L_2(y_j, h_{\mathbf{w}}(x_j)) = \sum_{j=1}^N (y_j - (w_1x_j + w_0))^2$
(here, N = number of training examples)

Solution:

- take the partial derivatives with respect to w_1 and w_0 and equal them to 0
- get a system of 2 equations with a unique solution (Exercise!)

E.g., for the Berkeley house prices example: $w_0 = 246, w_1 = 0.232$

Univariate Linear Regression



L_2 convex for every linear regression problem; hence no local minima!

Nonlinear models

For non-linear models (often no closed-form solution): **gradient descent**.

$\mathbf{w} \leftarrow$ any point in the parameter space

loop until convergence do

for each w_i in $\mathbf{w}$ do $w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} Loss(\mathbf{w})$

Note: α is called learning rate (constant or decaying value)

E.g., for univariate regression, for **one example (x, y)** , this will give:

$$w_0 \leftarrow w_0 + \alpha(y - h_{\mathbf{w}}(x))$$

$$w_1 \leftarrow w_1 + \alpha(y - h_{\mathbf{w}}(x)) \times x$$

Q: How do weights change if $h_{\mathbf{w}}(x) > y$, assuming $\alpha > 0$?

Nonlinear models

$\mathbf{w} \leftarrow$ any point in the parameter space

loop until convergence do

for each w_i in $\mathbf{w}$ do $w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} Loss(\mathbf{w})$

Extend to N examples: we get batch gradient descent learning rule for ULR:

$$w_0 \leftarrow w_0 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j))$$

$$w_1 \leftarrow w_1 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j)) \times x_j$$

Convergence guaranteed for α small enough, but possibly slow

Stochastic gradient descent: update using one example at a time. Faster than batch; convergence not guaranteed for fixed α , only for decaying α

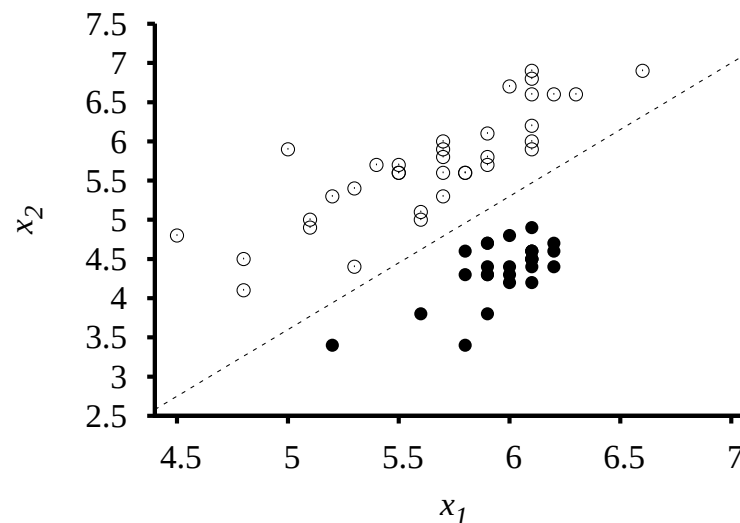
Linear Classification: build a decision boundary

Linear functions can also do classification, by building a decision boundary

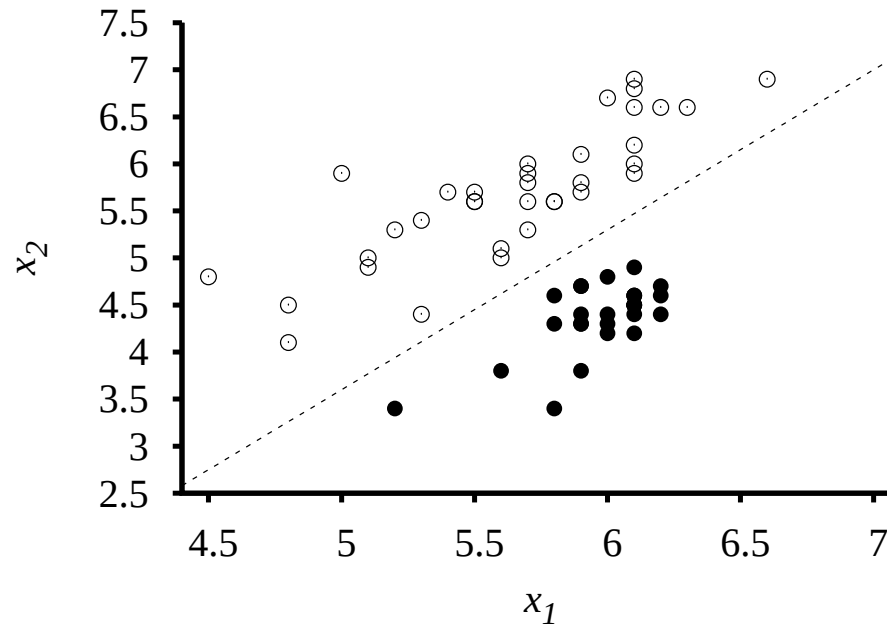
Decision boundary = surface which separates the two classes

Linear separator = linear decision boundary

E.g. Earthquake dataset:



Linear Classification: build a decision boundary

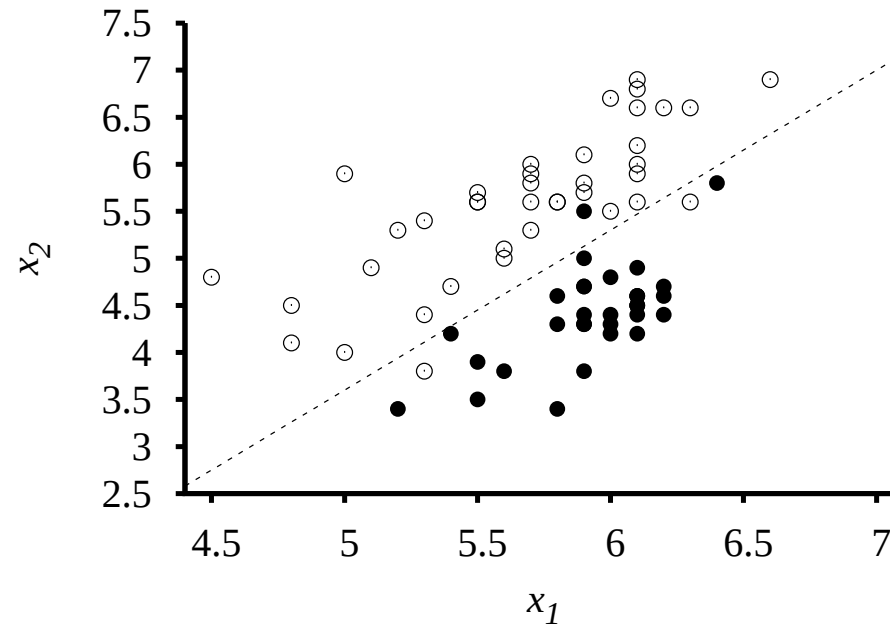


Features: x_1 body wave magnitude and x_2 surface wave magnitude

Classes: earthquake (white); nuclear test (black). They are **linearly separable**.
(See the dotted line. The linear separator equation: $-4.9 + 1.7x_1 - x_2 = 0$)

Classification hypothesis (by convention $x_0 = 1$):
$$h_{\mathbf{w}}(\mathbf{x}) = \begin{cases} 1, & \text{if } \mathbf{w} \cdot \mathbf{x} \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Linear Classification



Slightly different dataset. Classes are not linearly separable anymore (there exists no linear decision boundary for this dataset).

Linear Classification: Threshold vs. Logistic

Classification hypothesis: $h_{\mathbf{w}}(\mathbf{x}) = 1$ if $\mathbf{w} \cdot \mathbf{x} \geq 0$ and 0 otherwise

We can see the classification hypothesis h as passing the linear function $\mathbf{w} \cdot \mathbf{x}$ through a **threshold function**:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x})$$

How to find $\mathbf{w}$ which minimizes loss?

- ◇ closed-form? (N/A)
- ◇ gradient descent? (gradient 0 or undefined)
- ◇ employ the **perceptron weight update rule**: for a single example $(\mathbf{x}, y)$:

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$

(it converges if the data are linearly separable)

Linear Classification: Threshold vs. Logistic

We can also build the classification hypothesis h by passing the linear function $\mathbf{w} \cdot \mathbf{x}$ through a smoother function, called the **logistic function**:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Logistic}(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}\mathbf{x}}}$$

Logistic regression = fitting the weights with this models to minimize loss. It could be done by gradient descent, with the following weight update rule:

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x}))(1 - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$

Linear Classification: Threshold vs. Logistic

Logistic function:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Logistic}(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}\mathbf{x}}}$$

Weight update rule:

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x}))(1 - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$

Notes:

- ◇ the following equality was used: $g'(z) = g(z)(1 - g(z))$
- ◇ hard threshold approximated with a continuous, differentiable function
- ◇ its output is a number in $(0, 1)$ which could be interpreted as the **probability** of belonging to class 1. So, the hypothesis creates a **soft boundary** in the input space.

Summary

- ◇ Model selection: define hypothesis space
- ◇ Model optimization: find the best model in that space
- ◇ Loss: take into account utility of misclassification
- ◇ Generalization loss (known prior probability distribution for examples) v. empirical loss (estimate generalization loss)
- ◇ Regularization: take into account both loss and hypothesis complexity
- ◇ Feature selection: discard irrelevant attributes
- ◇ Regression v. Classification (find function v. class)
- ◇ Threshold v. Logistic: perform classification by applying the respective function to $\mathbf{w} \cdot \mathbf{x}$

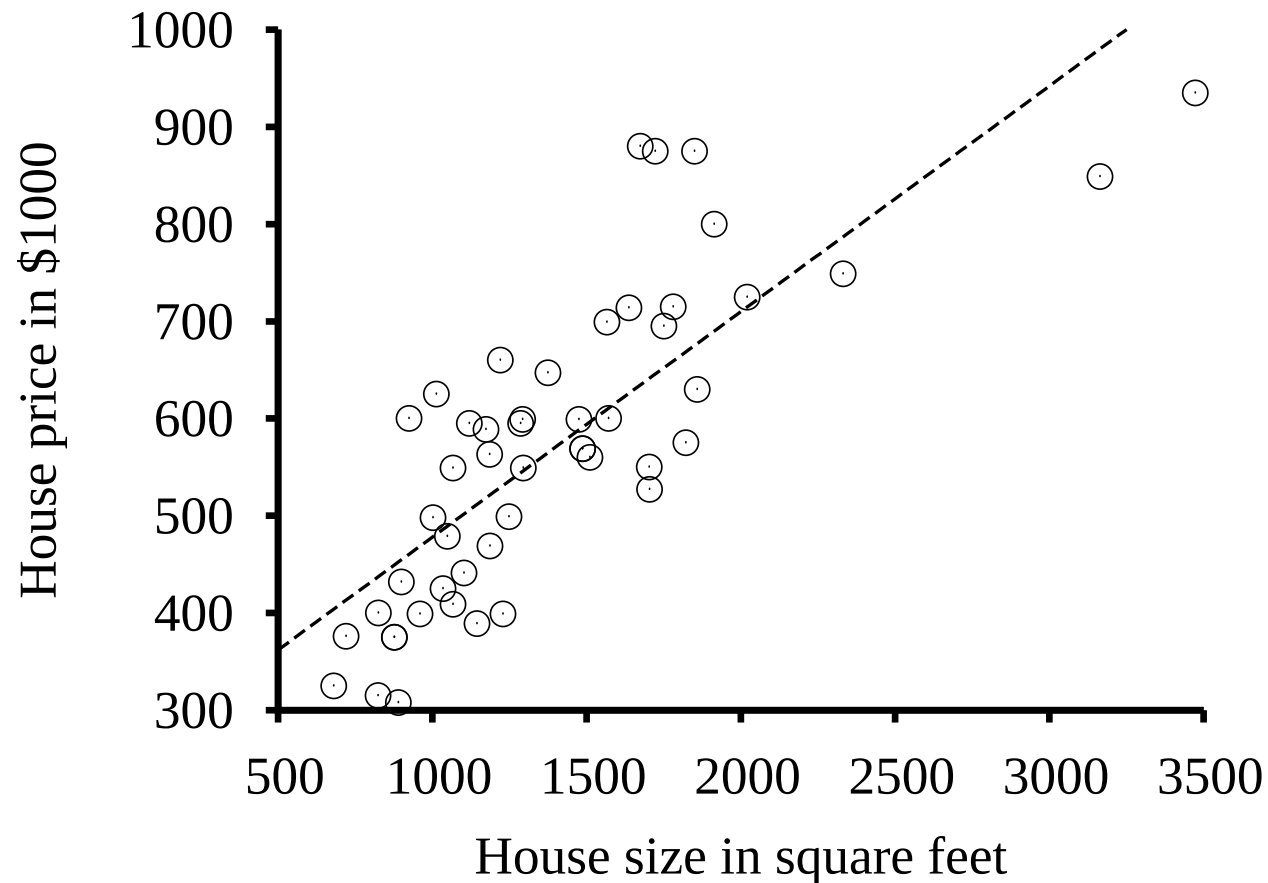
REGRESSION AND CLASSIFICATION II

CHAPTER 18.6.2-4
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Multivariate Linear Regression
- ◇ Classification via Logistic Regression

Remember: Univariate Linear Regression (ULR)



Estimate Berkeley house price as a function of floor space: find out parameters w_0 and w_1 which minimize the L_2 loss for hypothesis $h_{\mathbf{w}}(x) = w_1x + w_0$

Multivariate linear regression (MLR)

But: price might depend on more than one feature, e.g. floor space and distance to the Ocean. What to do?

Extend ULR for the case of an example $\mathbf{x}_j$ which is an n -element vector.
Hypothesis space:

$$h_{\mathbf{w}}(\mathbf{x}_j) = \mathbf{w} \cdot \mathbf{x}_j = \mathbf{w}^\top \mathbf{x}_j = \sum_i w_i x_{j,i}$$

(with dummy attribute $x_{j,0} = 1$ added to handle the intercept w_0)

◇ Solution 1 (numerical): gradient descent with update rule:

$$w_i \leftarrow w_i + \alpha \sum_j x_{j,i} (y_j - h_{\mathbf{w}}(\mathbf{x}_j))$$

Multivariate linear regression (MLR)

Hypothesis space:

$$h_{sw}(\mathbf{x}_j) = \mathbf{w} \cdot \mathbf{x}_j = \mathbf{w}^\top \mathbf{x}_j = \sum_i w_i x_{j,i}$$

◇ Solution 2 (closed-form): $\mathbf{w}^*$ which minimizes the squared error is:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

where

$\mathbf{y}$ = the vector of desired outputs for the training examples

$\mathbf{X}$ = matrix of inputs with one n -dimensional example per row (data matrix)

Multivariate linear regression (MLR)

◇ Regularization mandatory to avoid overfitting as in multi-dimensional spaces it is possible that an irrelevant dimension appears to be relevant purely by chance.

Regularization function:

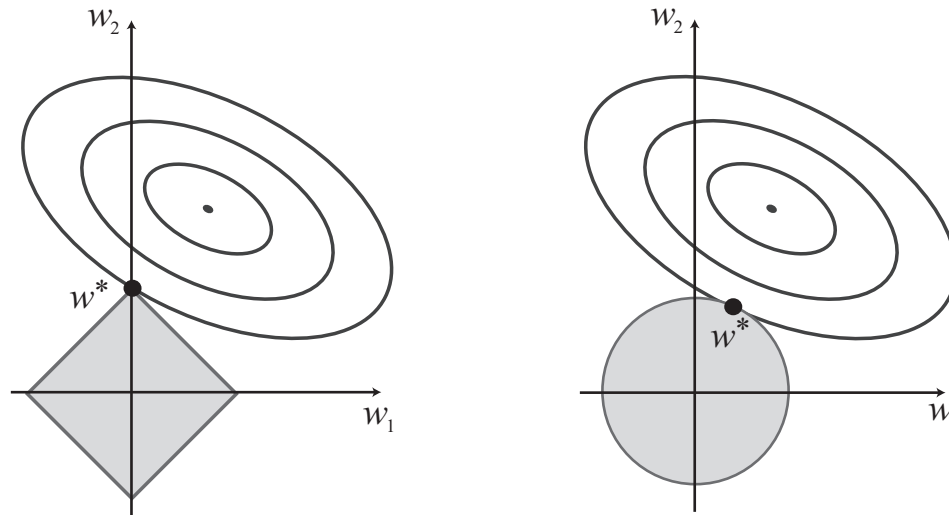
$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q$$

Q: Which q ?

Using L_1 as loss function often leads to a **sparse** model (with many zero coefficients, i.e., features deemed irrelevant), which is less likely to overfit and usually more comprehensible.

Multivariate linear regression: choosing L

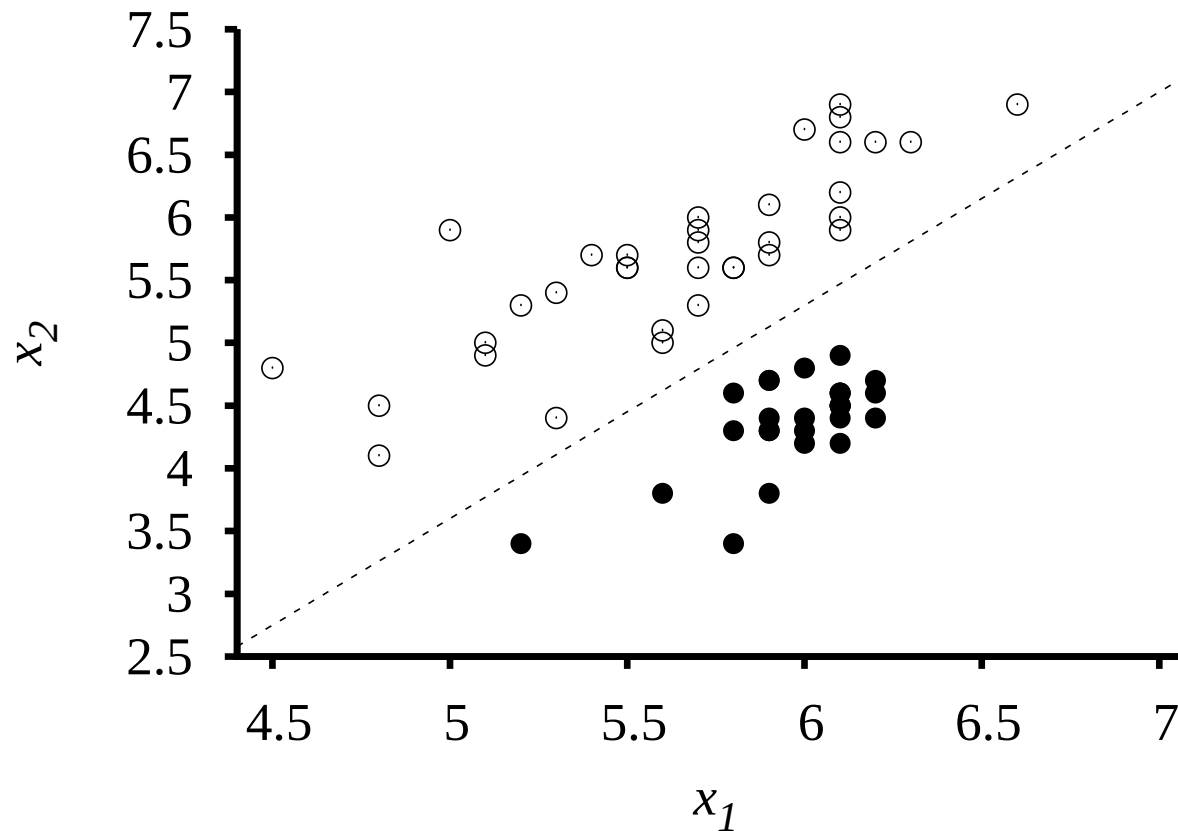
Minimizing $Loss(\mathbf{w}) + \lambda Complexity(\mathbf{w})$ equivalent to minimizing $Loss(\mathbf{w})$, subject to the constraint $Complexity(\mathbf{w}) \leq c$ for some constant c depending on λ .



L_1 tends to produce a **sparse** model L_2 usually does not

Diamond-shaped box = set of points $\mathbf{w}$ with $L_1 < c$, for some constant c .
For an arbitrary position of the loss contours (ovals), it's likely the point in the box closest to the loss minimum is a corner (so some dimensions are 0)

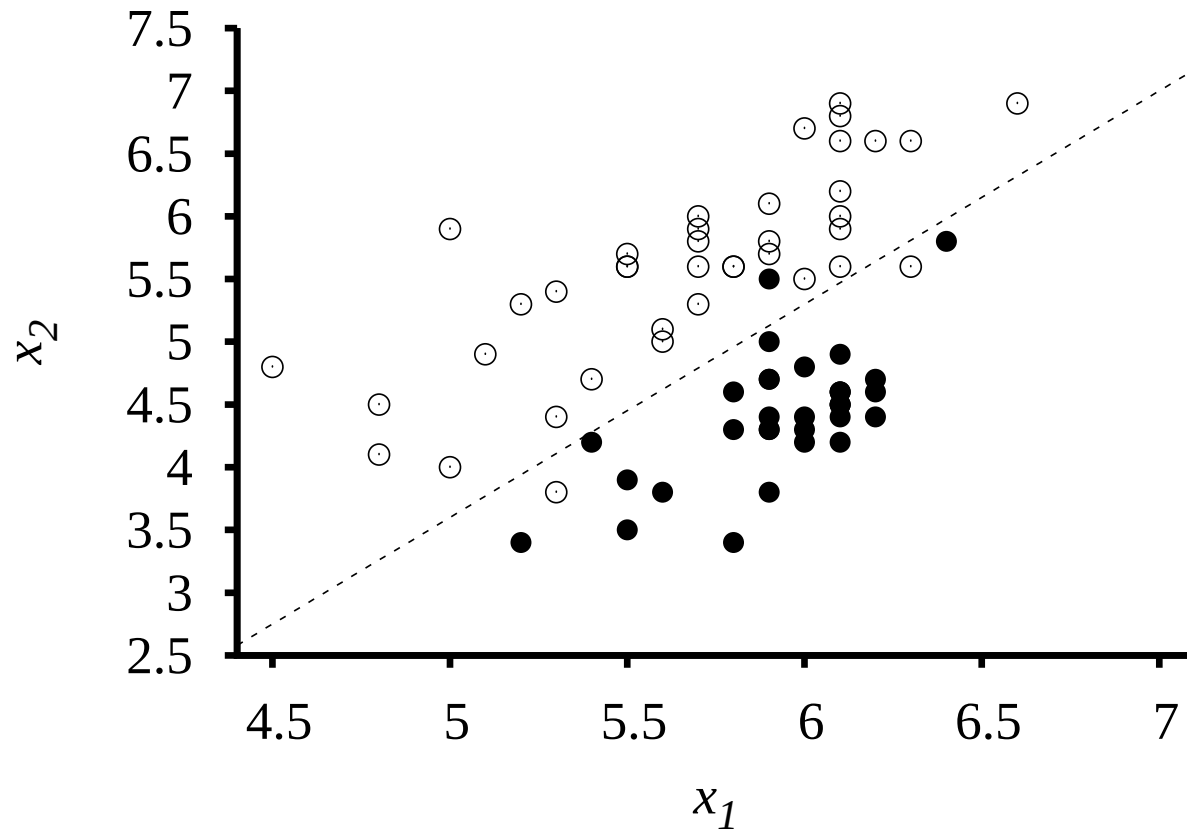
Linear Classification: build a decision boundary



Features: wave magnitudes x_1 (body) and x_2 (surface)

Classes: earthquake (white); nuclear test (black). Linearly separable.

Linear Classification



Slightly different dataset. Classes not linearly separable any more.

Linear Classification: Threshold vs. Logistic

We can see the classification hypothesis h as passing the linear function $\mathbf{w} \cdot \mathbf{x}$ through a threshold function:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x})$$

To find $\mathbf{w}$ which minimizes loss, if we consider a single example $(\mathbf{x}, y)$, we can use the perceptron weight update rule:

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$

Obs:

1. closed-form or gradient descent based methods do not work, as the gradient is 0 everywhere except points for which $\mathbf{w} \cdot \mathbf{x} = 0$
2. the perceptron rule does work, provided linear separability of data
3. the rule is applied **one example at a time**, not in a batch manner

Linear Classification: Threshold v. Logistic

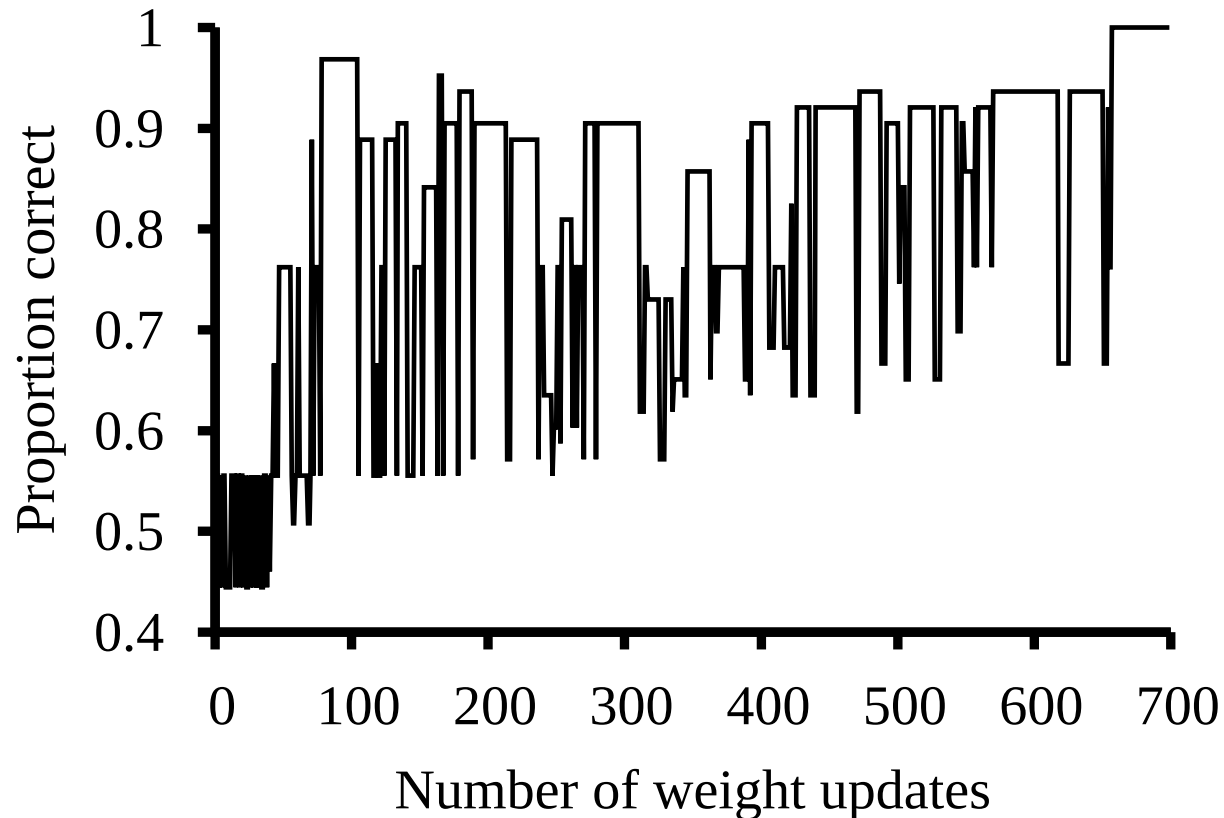
$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$

Q1: Which *Loss* function is used in this case?

Q2: What happens to w_i if $y = 1$ and $h_{\mathbf{w}}(\mathbf{x}) = 0$?

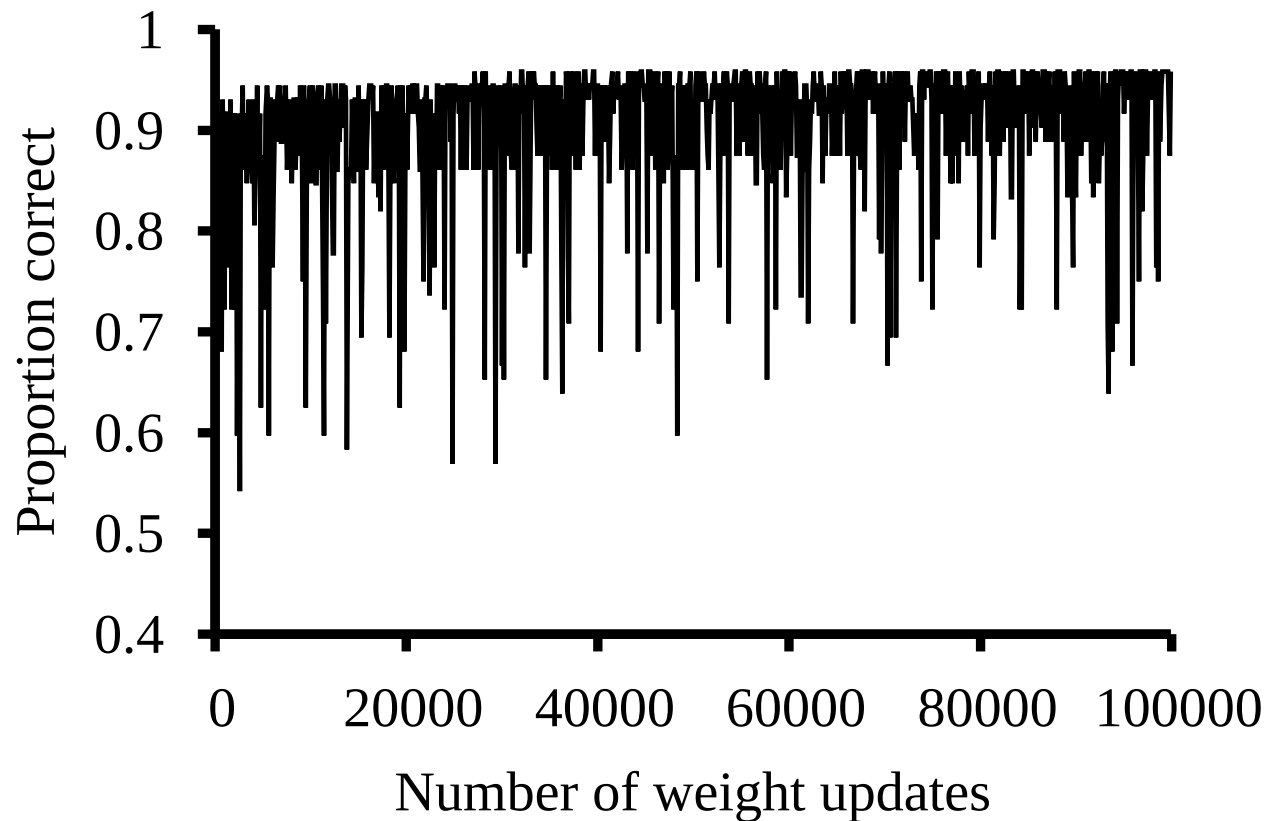
Q3: What happens to w_i if $y = 0$ and $h_{\mathbf{w}}(\mathbf{x}) = 1$?

Linear Classification: Threshold v. Logistic



Training curve (learning and performance evaluating done on the same training set) for the perceptron rule applied to the no-noise, earthquake dataset. Converges to a zero-error linear separator

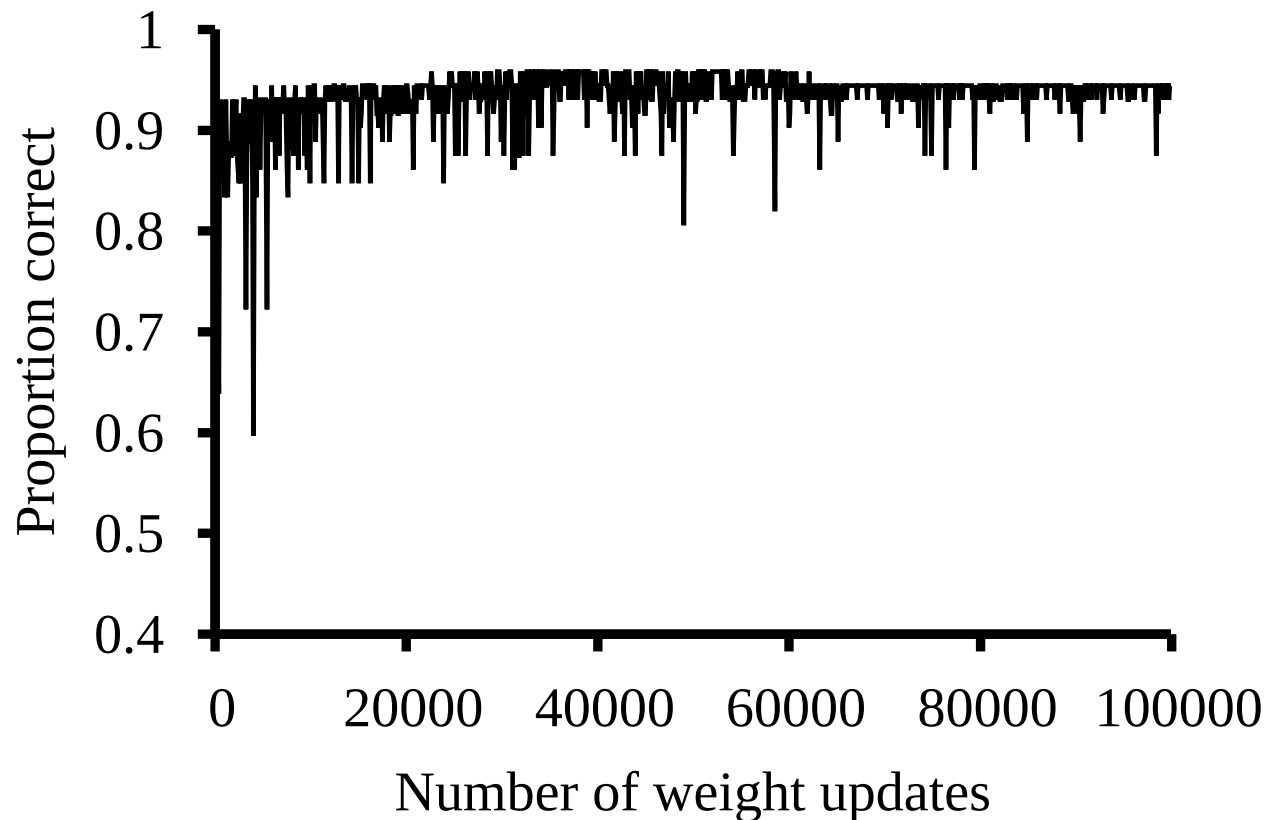
Linear Classification: Threshold v. Logistic



Perceptron rule applied to the noise, earthquake dataset.

No convergence if α constant.

Linear Classification: Threshold v. Logistic



Perceptron rule applied to the noise, earthquake dataset with $\alpha = \frac{1000}{1000+t}$.

Converges for α decaying with iteration number as $O(\frac{1}{t})$

Linear Classification: Threshold v. Logistic

The classification hypothesis can be built by passing the linear function $\mathbf{w} \cdot \mathbf{x}$ through a differentiable function (the logistic function):

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Logistic}(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1+e^{-\mathbf{w} \cdot \mathbf{x}}}$$

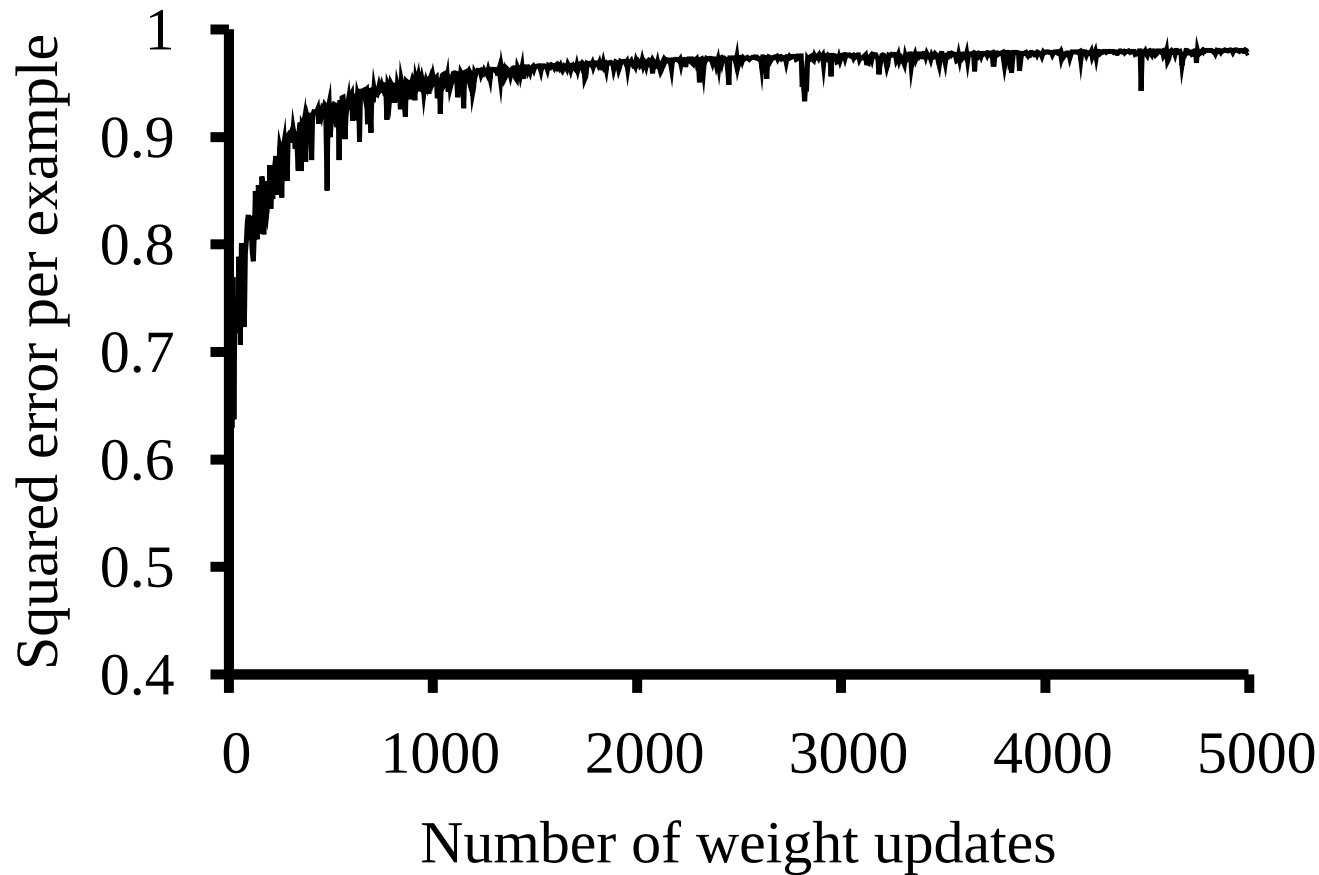
The output can be regarded as probability to belong to the class labeled 1.

Assuming loss L_2 , we can use the following weight update rule:

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x})) \times h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$

(For the proof, remember that $g'(z) = g(z)(1-g(z))$. See also the textbook)

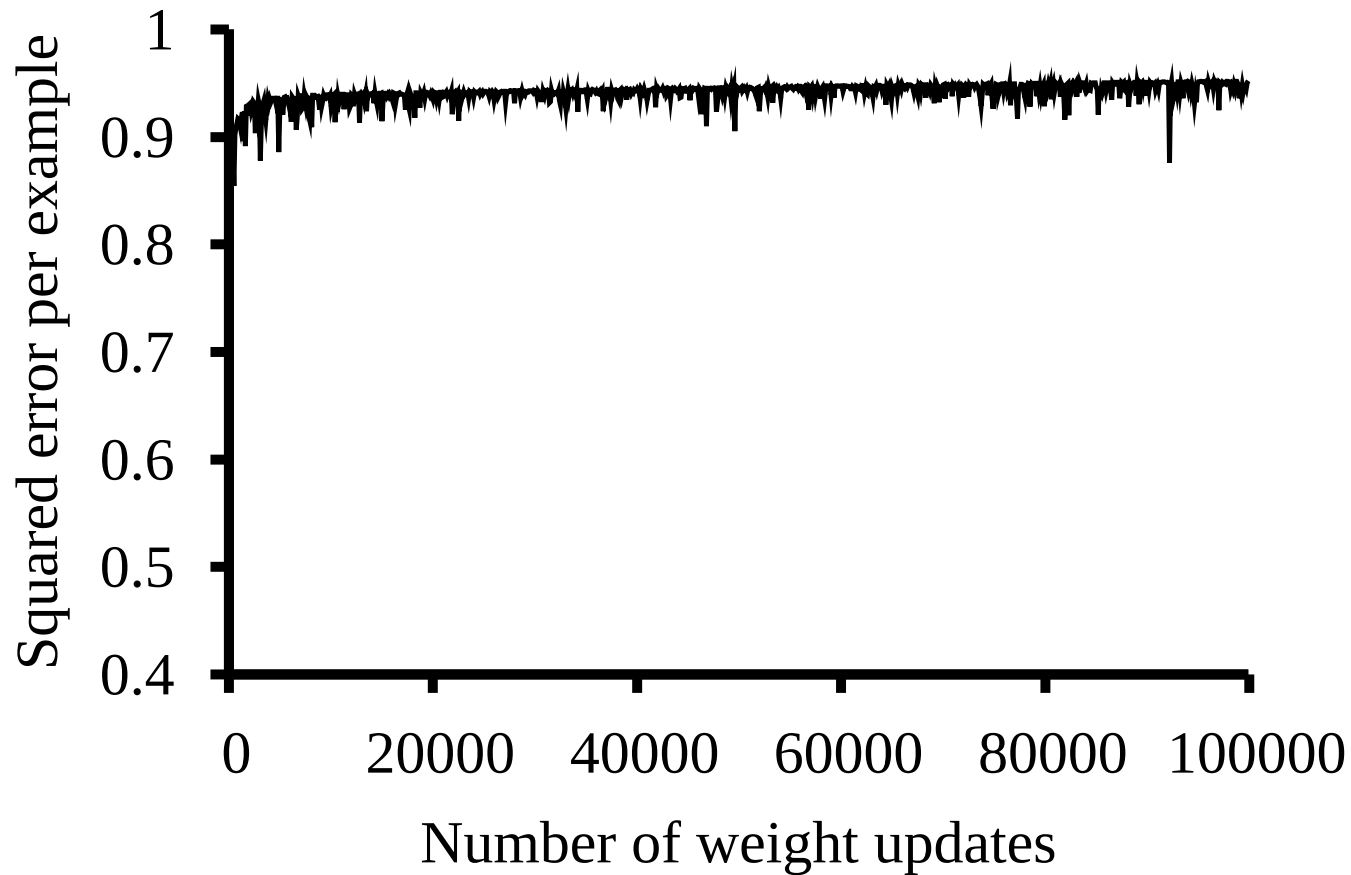
Linear Classification: Threshold v. Logistic



Logistic regression for the no-noise, earthquake dataset.

Converges, slower than the perceptron version: 5000 v. 1000, less oscillations

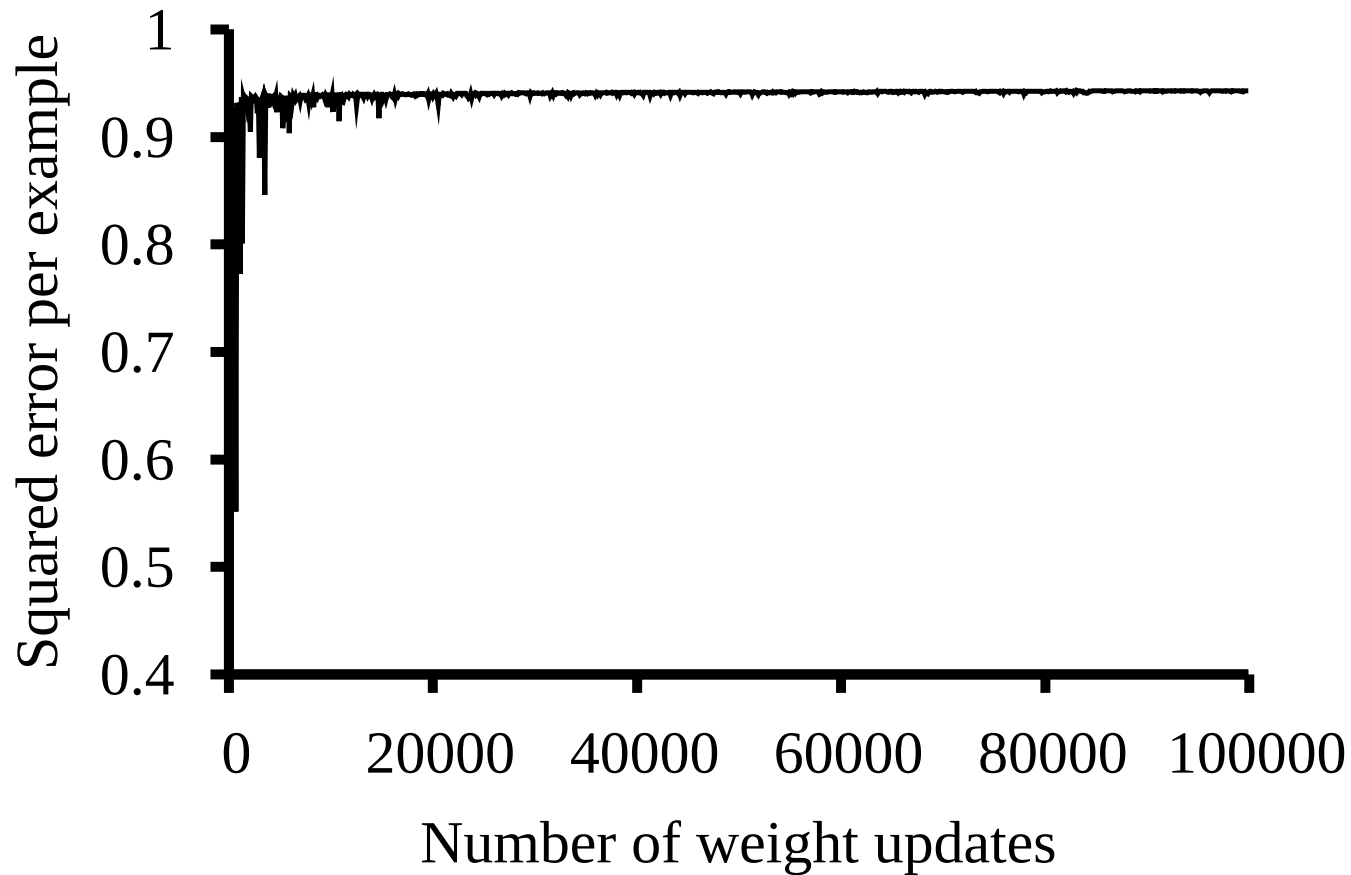
Linear Classification: Threshold v. Logistic



Logistic regression for the noisy, non-separable earthquake dataset

Converges, quick, no oscillations.

Linear Classification: Threshold v. Logistic



Logistic regression for the noisy, earthquake dataset with α decay

Converges, quick, no oscillations.

Summary

- ◇ Multivariate Linear Regression: one example is an n -element vector $\mathbf{x}_j$
- ◇ Possible solutions for MLR: gradient descent or closed-form
- ◇ Regularization must be done to avoid overfitting
- ◇ Logistic Regression employs the continuous, differentiable *logistic* function instead of the threshold function in order to do linear classification

NONPARAMETRIC METHODS ENSEMBLE LEARNING STATISTICAL LEARNING

CHAPTER 18.8-9, 18.10; 20, SECTIONS 1–2
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Nearest Neighbor model for classification and regression
- ◇ Ensemble Learning
- ◇ Learning as probabilistic inference
- ◇ Bayesian learning
- ◇ Maximum *a posteriori* and maximum likelihood learning

Parametric vs. non-parametric learning

Parametric: the model has a fixed number of parameters (it summarizes **all** training examples with a set of parameters of fixed size)

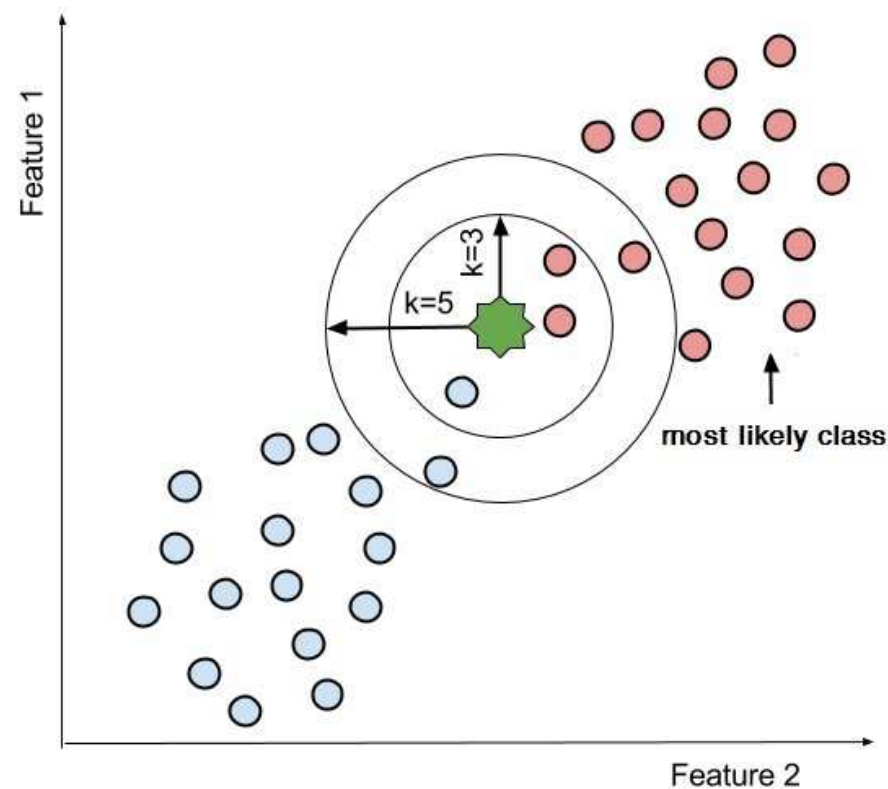
E.g., linear regression, neural networks

Non-parametric: no bounded number of parameters

E.g., **instance based learning:** retain within itself all training examples and uses all of them to predict the next one

Nearest neighbor model

Idea: stick with the set of the k nearest neighbors of $\mathbf{x}_q$ (henceforth $NN(k, \mathbf{x}_q)$)



Source: https://www.researchgate.net/publication/322358139_Eye_Tracking_with_EEG_life-style/figures?lo=1

Nearest neighbor model

How do we perform:

◇ **Classification**: to establish the label of $\mathbf{x}_q$: use the plurality of labels of its k closest neighbors (we use an odd value for k to avoid ties)

◇ **Regression**: to get the value y_q corresponding to $\mathbf{x}_q$: select the k closest neighbors of $\mathbf{x}_q$ and:

(i) compute their mean

OR

(ii) perform linear regression over them

Nearest neighbor model

How to measure the distance from a query point $\mathbf{x}_q$ to an example point $\mathbf{x}_j$?

$$L^p(\mathbf{x}_j, \mathbf{x}_q) = (\sum_i |x_{j,i} - x_{q,i}|^p)^{1/p}$$

The L^p norm (aka Minkowski distance) generalizes the following distances:

Manhattan ($p = 1$); Euclidian ($p = 2$); Hamming ($p = 1$, binary attributes)

Typically, $p = 2$ is preferred when the dimensions are similar in nature (e.g., length and width), while $p = 1$ is used if they are different (e.g., blood sugar and blood pressure).

Normalization

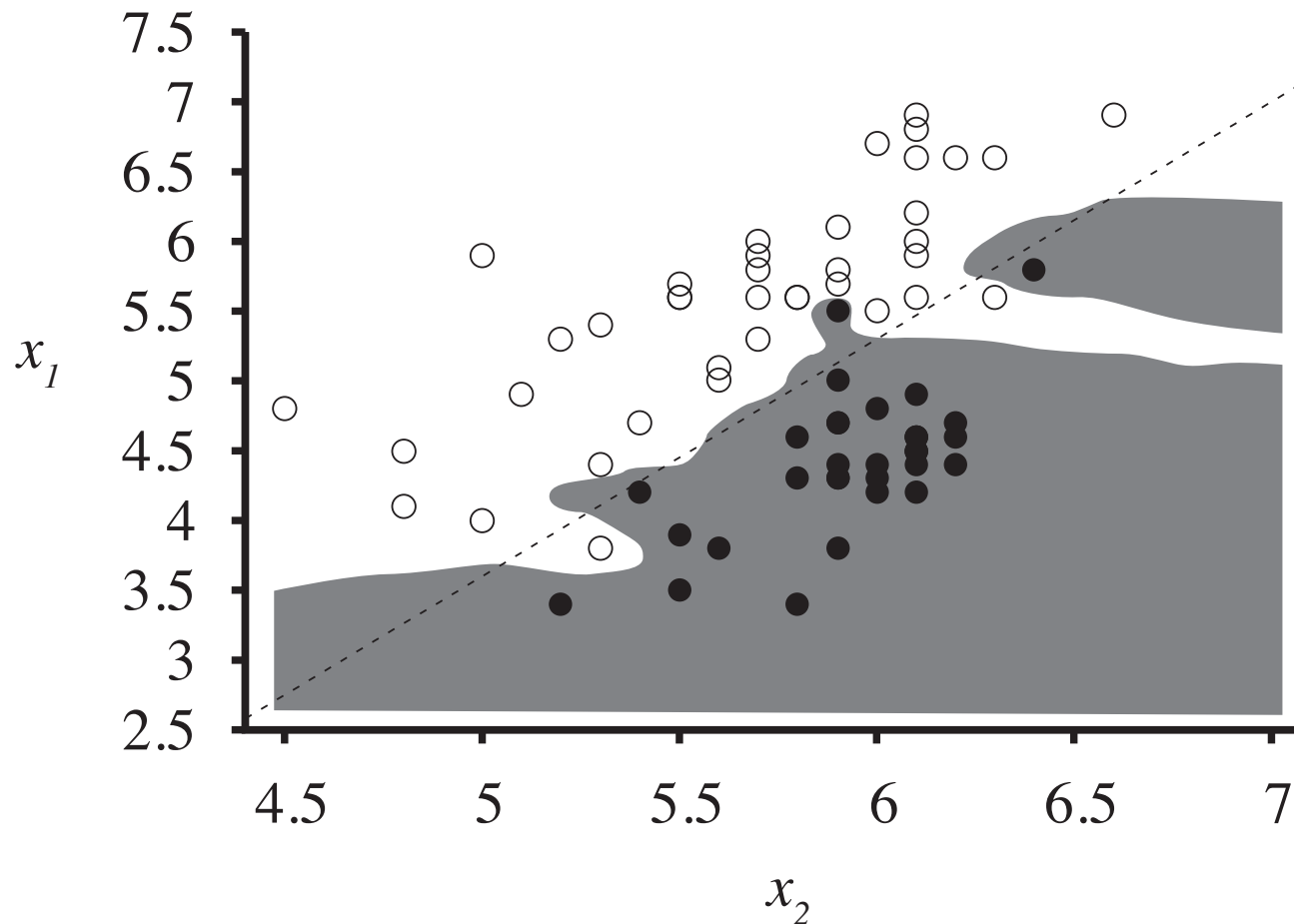
Usually, a change in the scale of a dimension (i.e., feature) can result in changing the set of the k nearest neighbors of a point.

E.g., given $P_1(0, 0)$, $P_2(2, 3)$, $P_3(0, 5)$, the nearest neighbor of P_1 is P_2 (distance 3.61). If we scale the values across both dimensions to get the same range for all of them, point (p, q) is mapped into $(\frac{p - \min_i(p_i)}{\max_i(p_i) - \min_i(p_i)}, \frac{q - \min_i(q_i)}{\max_i(q_i) - \min_i(q_i)})$, the nearest neighbor for P_1 becomes P_3 (distance 1).

To avoid this phenomenon, one can perform a **normalization**. This could be done via the **min-max scaling** described above or via **standardization**:

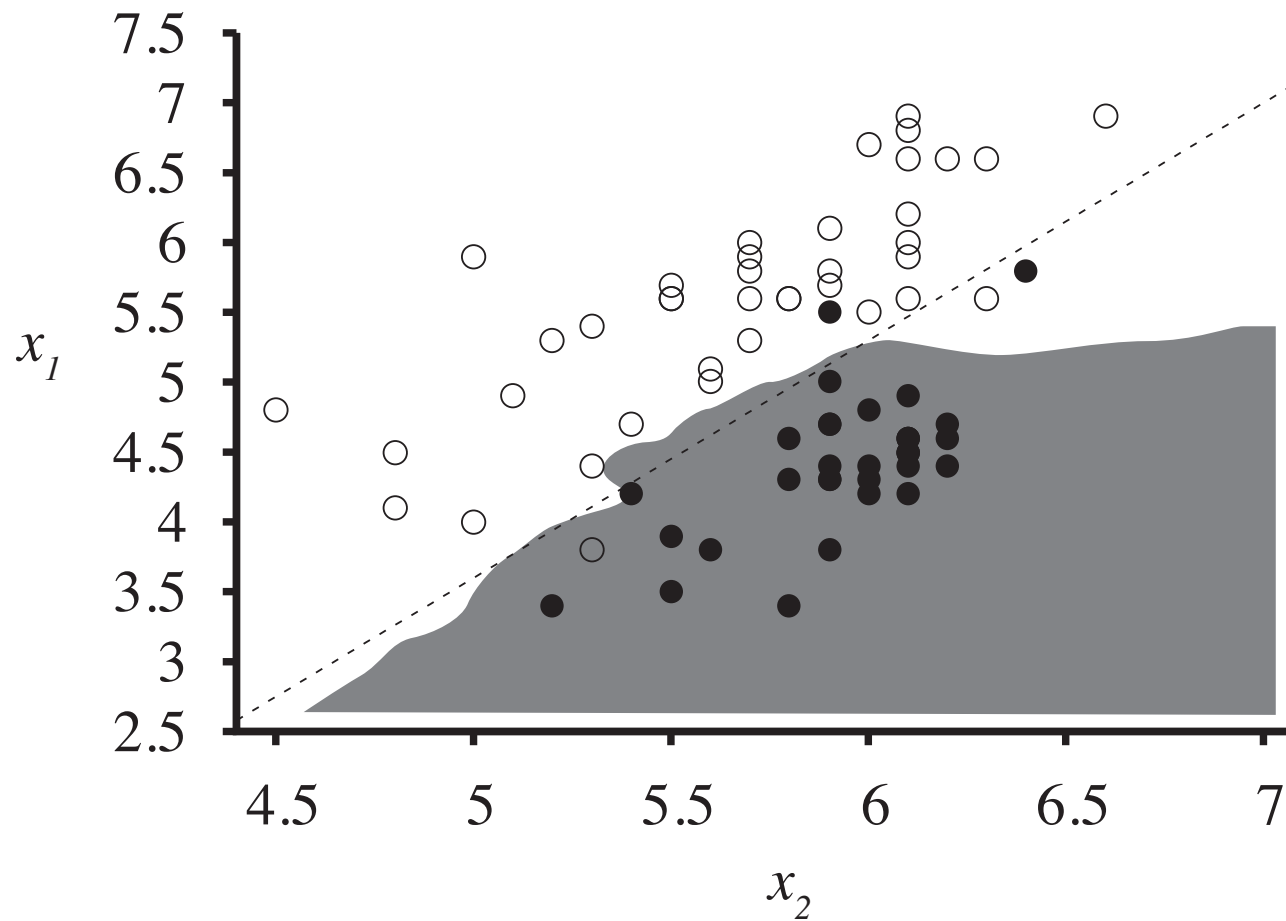
1. get the mean μ_i and standard deviation σ_i of each feature values
2. rescale the values: $x_{j,i}$ is mapped into $\frac{x_{j,i} - \mu_i}{\sigma_i}$

Nearest neighbor model: KNN algorithm



Earthquake data, non-linearly separable version. k -nearest neighbor model for $k = 1$ (explosion class in gray). Overfitting is obvious.

Nearest neighbor model: KNN algorithm



Earthquake data. $k = 5$. No overfitting.

Higher k values likely to underfit. To select a proper k : do cross-validation

Curse of Dimensionality

The nearest neighbors in high-dimensional spaces are usually not very near!

For a data set of P points uniformly distributed throughout the interior of an n -dimensional unit hypercube, we define the k -neighborhood of a point as the smallest hypercube containing the k -nearest neighbors.

Let l be the average side length of a neighborhood. Neighborhood volume: l^n . Hypercube volume (with P points): 1.

Since, on average, we have $l^n/1 = k/P$, it follows that $l = (k/P)^{1/n}$

E.g., let's assume $k = 10$, $P = 1.000.000$. For different values of n , we get:

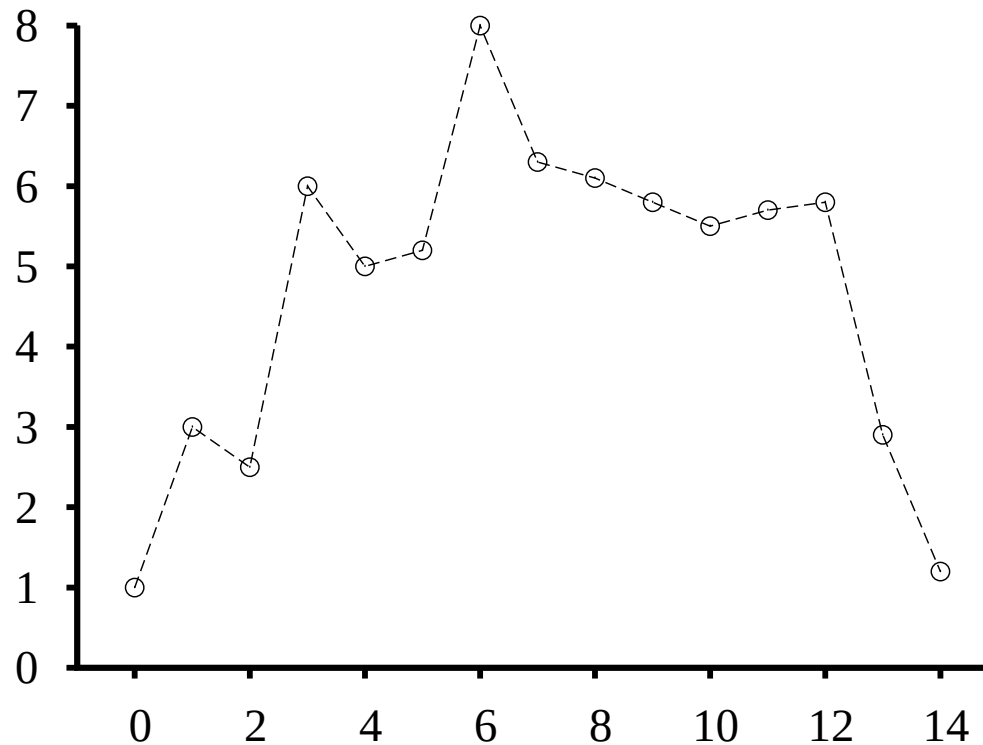
$$n = 2: l = 0.003$$

$$n = 3: l = 0.02$$

$$n = 200: l = 0.94$$

The higher the number of dimensions, the more "extremistic" your examples!

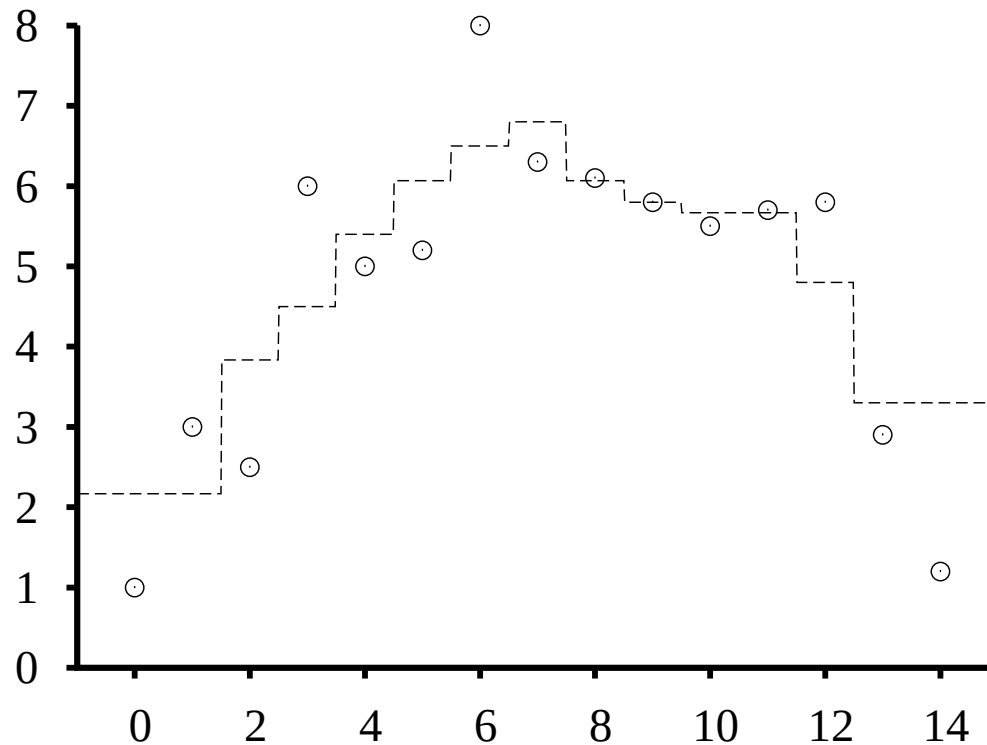
KNN for Regression



"Connect the dots": create $h(x)$ by solving a linear regression problem for just two neighbors in the training set: those closest to the query point x_q .

Not good generalization for noisy data (spiky function).

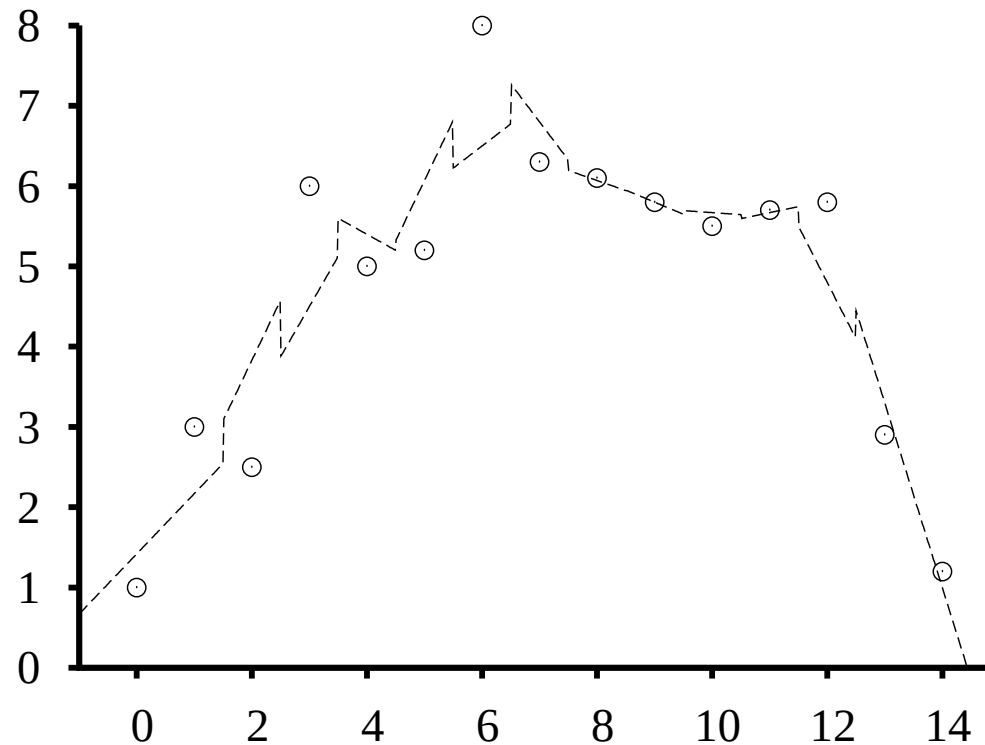
KNN for Regression



Average of k nearest neighbors (3 for this example).

Smoother than above, but poor estimates around outliers (0 and 14), where the trend gets ignored.

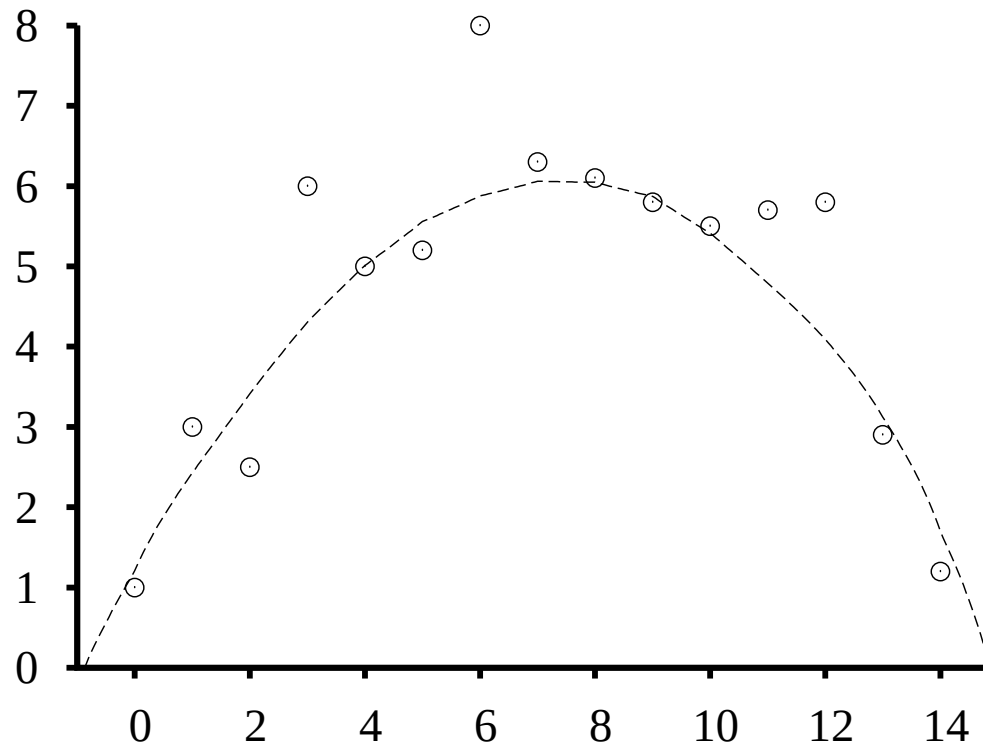
KNN for Regression



k nearest neighbors linear regression: best line through k examples.

Trends better estimated for outliers, but still discontinuous.

KNN for Regression



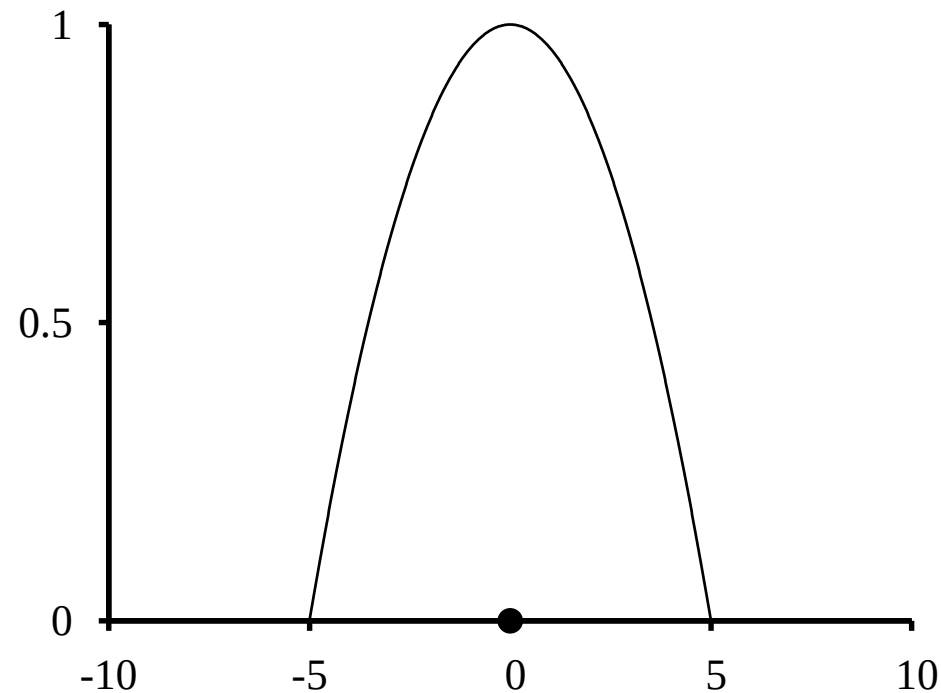
Locally **weighted** regression: assign **higher weights** to the examples closer to the query point x_q .

No discontinuities of $h(x)$.

Kernel example: quadratic, width $k = 10$

Kernel function $\mathcal{K}$: gives the weight of an example. The function should be:

(1) symmetric around 0; (2) maximum at 0; (3) have area under it bounded



Shape: not so important. Width: important (big $\rightarrow$ underfitting; small $\rightarrow$ overfitting) (cross-validation to choose it)

Locally weighted regression with kernels

Using gradient descent, chose $\mathbf{w}^*$ which minimizes:

$$\sum_j \mathcal{K}(\text{Distance}(\mathbf{x}_q, \mathbf{x}_j))(y_j - \mathbf{w} \cdot \mathbf{x}_j)^2$$

where *Distance* is any of the distance functions used for neighbor selection.

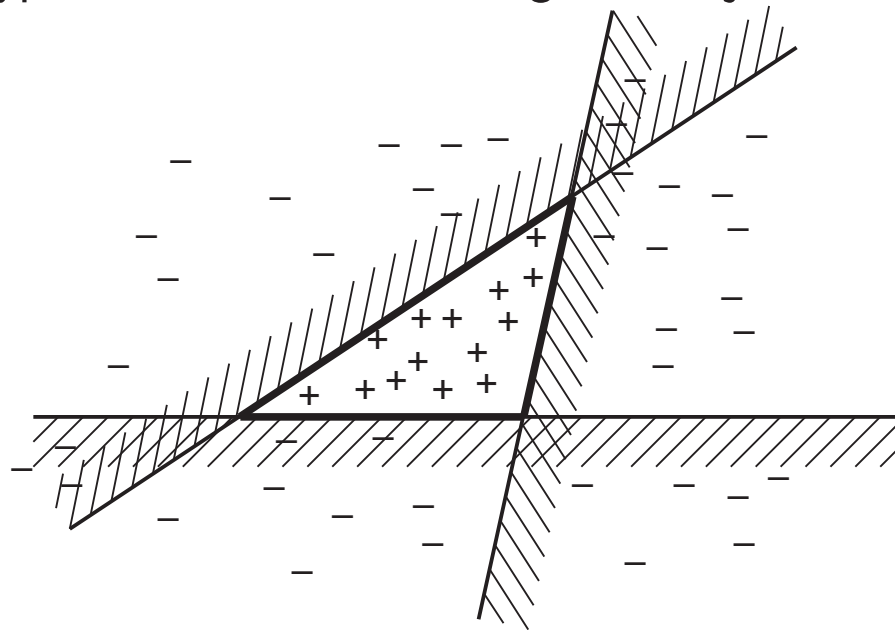
The hypothesis in $\mathbf{x}_q$ is:

$$h(\mathbf{x}_q) = \mathbf{w}^* \cdot \mathbf{x}_q$$

Ensemble Learning: expressivity

Select an **ensemble of hypotheses** and combine their predictions by voting.

- ◇ The probability to get a wrong result is diminished.
- ◇ Improve hypothesis expressivity. E.g., classify as positive each example which is classified as positive by **all** three linear threshold hypotheses. So, we can have a hypothesis such as a triangle, not just half-planes



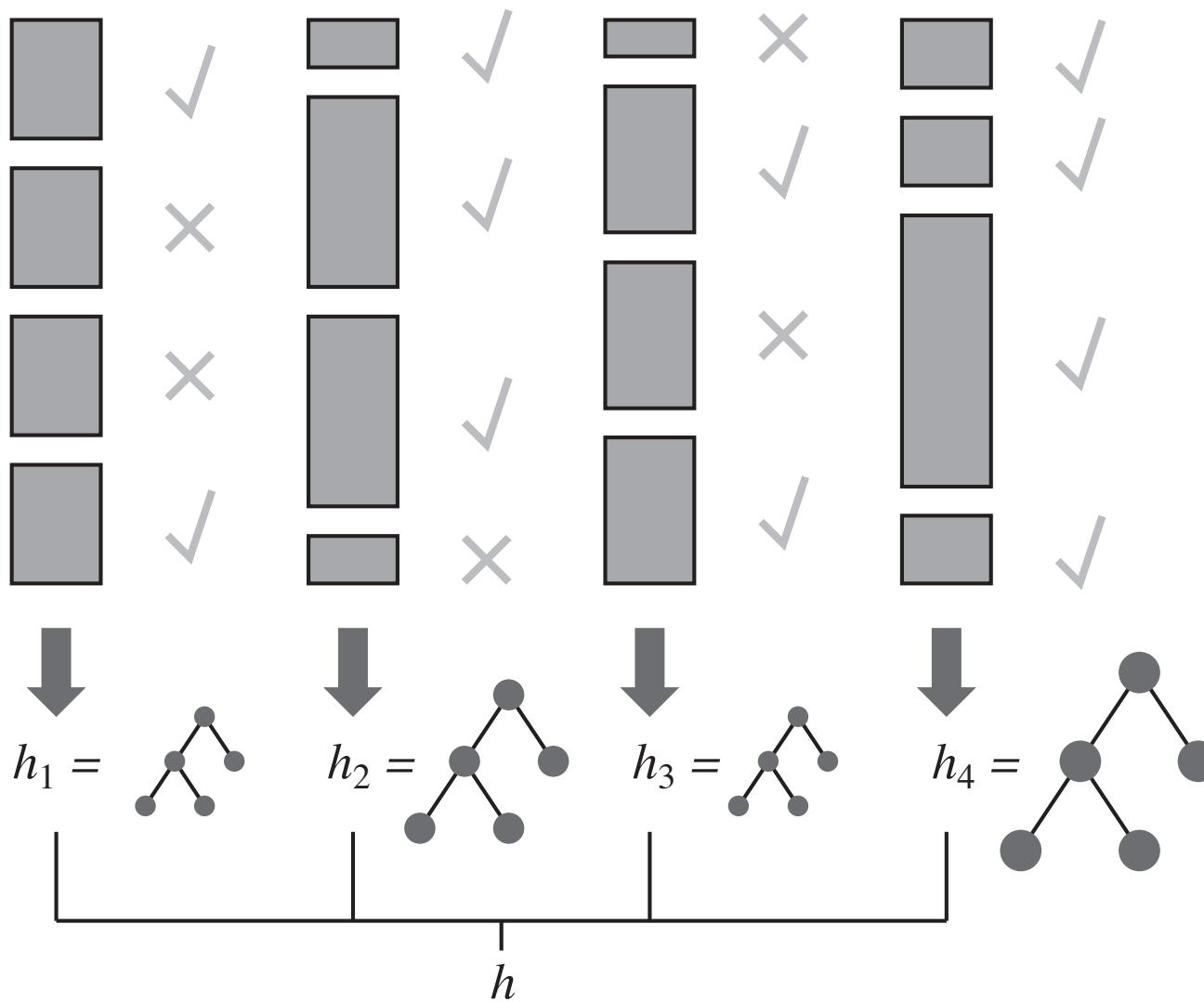
Example: boosting and weighted training set

Weighted training set: each example j has an associated weight $w_j \geq 0$.

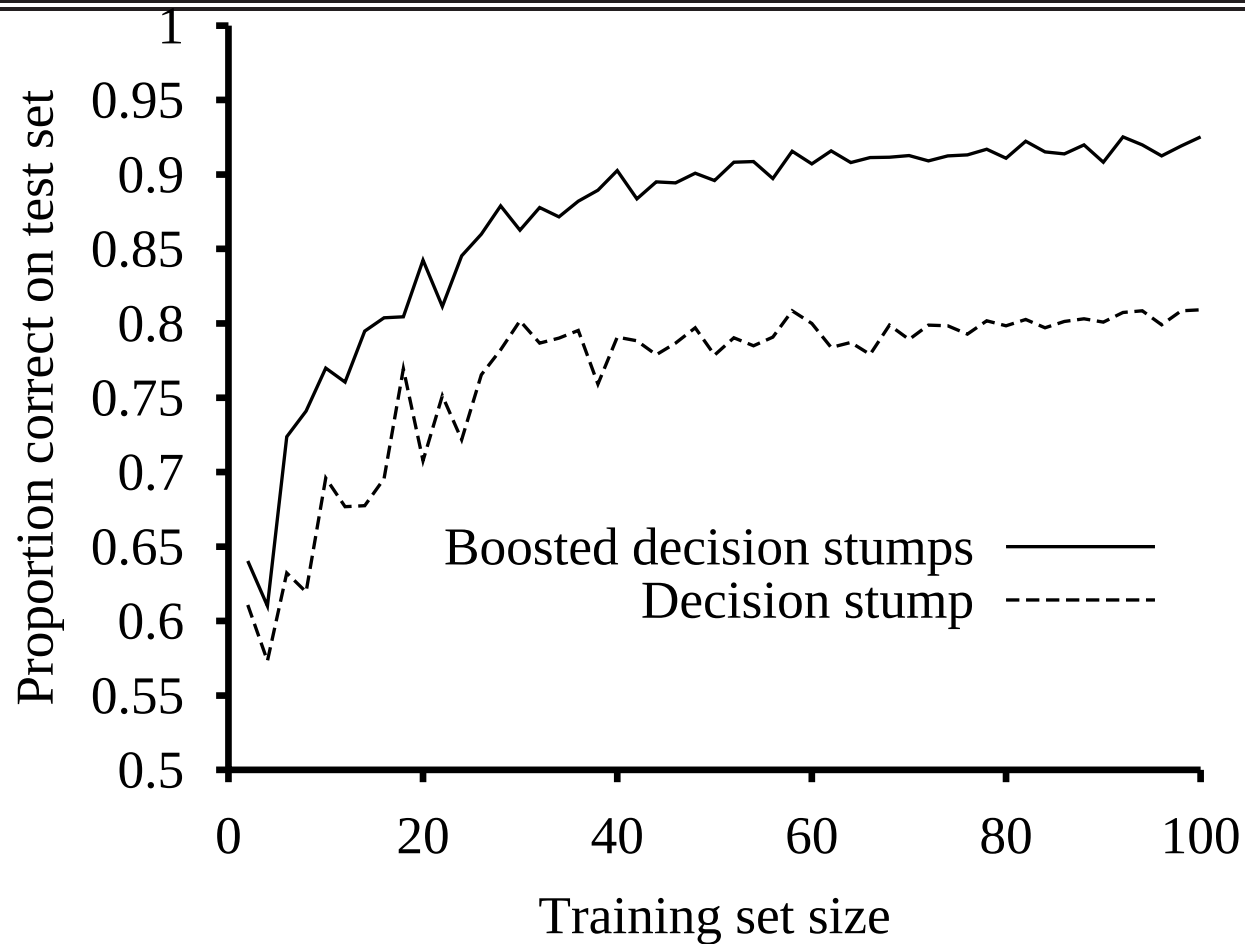
1. assign $w_j = 1$ for all examples and generate h_1 , which misclassifies some;
2. increase the weights of examples misclassified by h_1 , decrease the weights of the ones correctly classified by it and generate h_2 based on this new set; (e.g., replicate examples in the training set according to the weights)
3. continue until K hypotheses are generated (K given as input).

The final ensemble hypothesis is a weighted-majority combination of the K generated hypotheses, each weighted according to how well it performed on the training set.

Boosting and weighted training set



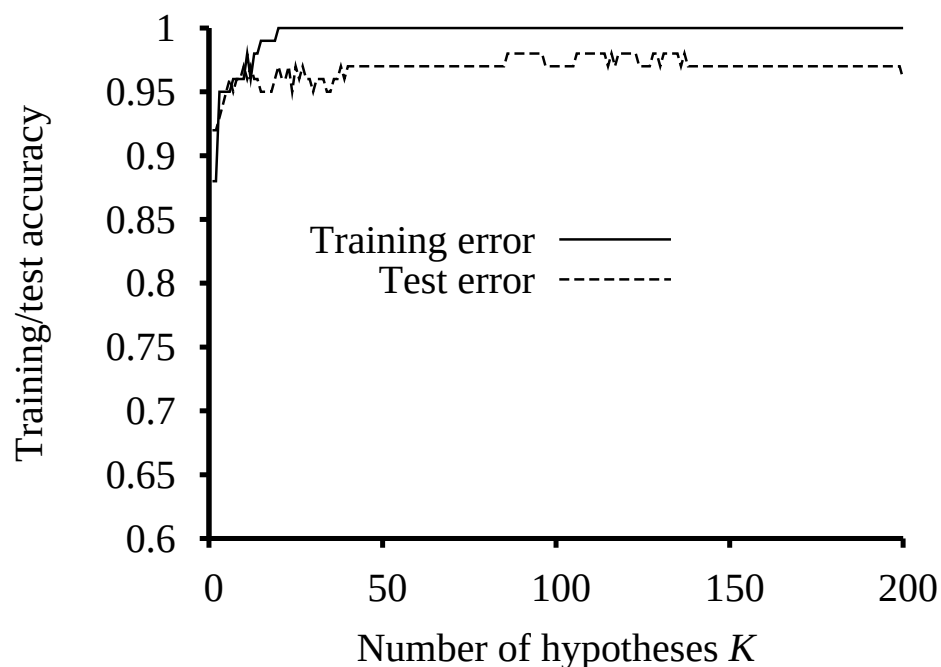
Ensemble Learning: AdaBoost algorithm



"AdaBoost-ed" (K=5) v. not boosted decision stumps (restaurant dataset)

Ensemble Learning: AdaBoost

If L is a **weak learning** algorithm (it always returns a hypothesis with an accuracy on the training set ϵ —slightly higher than random guessing), then AdaBoost will return a hypothesis that classifies the training data perfectly for K large enough



Note: Test set accuracy improves even after the training accuracy reaches 1

Bayes Learning Example Scenario

Suppose there are five kinds of bags of candies (and every bag is BIG):

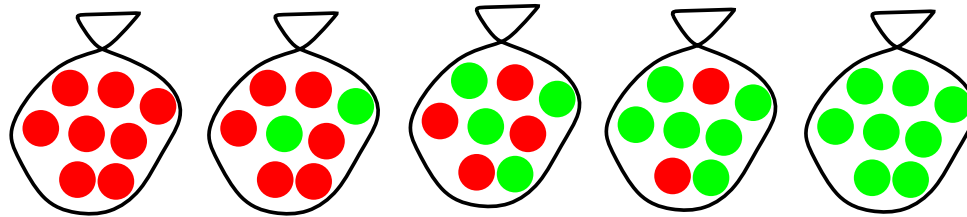
h_1 : 100% cherry candies

h_2 : 75% cherry candies + 25% lime candies

h_3 : 50% cherry candies + 50% lime candies

h_4 : 25% cherry candies + 75% lime candies

h_5 : 100% lime candies



Then we observe candies drawn from some bag: ● ● ● ● ● ● ● ● ● ●

Q1: What kind of bag is it? **Q2:** What flavour will the next candy be?

Note: sometimes, we assume the producer discloses the priors for $h_1, \dots, h_5$ (e.g., 10, 20, 40, 20, 10% respectively).

Full Bayesian learning

In statistical learning:

- data = evidence (instantiations of random variables in the domain)
- hypotheses = probabilistic theories of how the domain works

In our scenario:

- H is the hypothesis variable, with values $h_1, h_2, \dots$ and prior $\mathbf{P}(H)$
- j th observation d_j gives the outcome of random variable D_j (with possible values: either *cherry* or *lime*). $\mathbf{d} = d_1, \dots, d_N$ will be the training data

Bayesian learning = compute a probability distribution over the hypothesis space and make predictions on that basis

Full Bayesian learning (Q1: bag type)

Bayesian learning = compute a PD over the hypothesis space

Given the data so far, each hypothesis has a posterior probability:

$$P(h_i|\mathbf{d}) = \alpha P(\mathbf{d}|h_i)P(h_i)$$

We assume independent and identically distributed data (the bags are BIG!)

Under the i.i.d. assumption, the data **likelihood** $P(\mathbf{d}|h_i)$ is

$$P(\mathbf{d}|h_i) = \prod_j P(d_j|h_i)$$

E.g., for our example, we get $P(\mathbf{d}|h_3) = 0.5^{10}$

Full Bayesian learning (Q2: next candy's type)

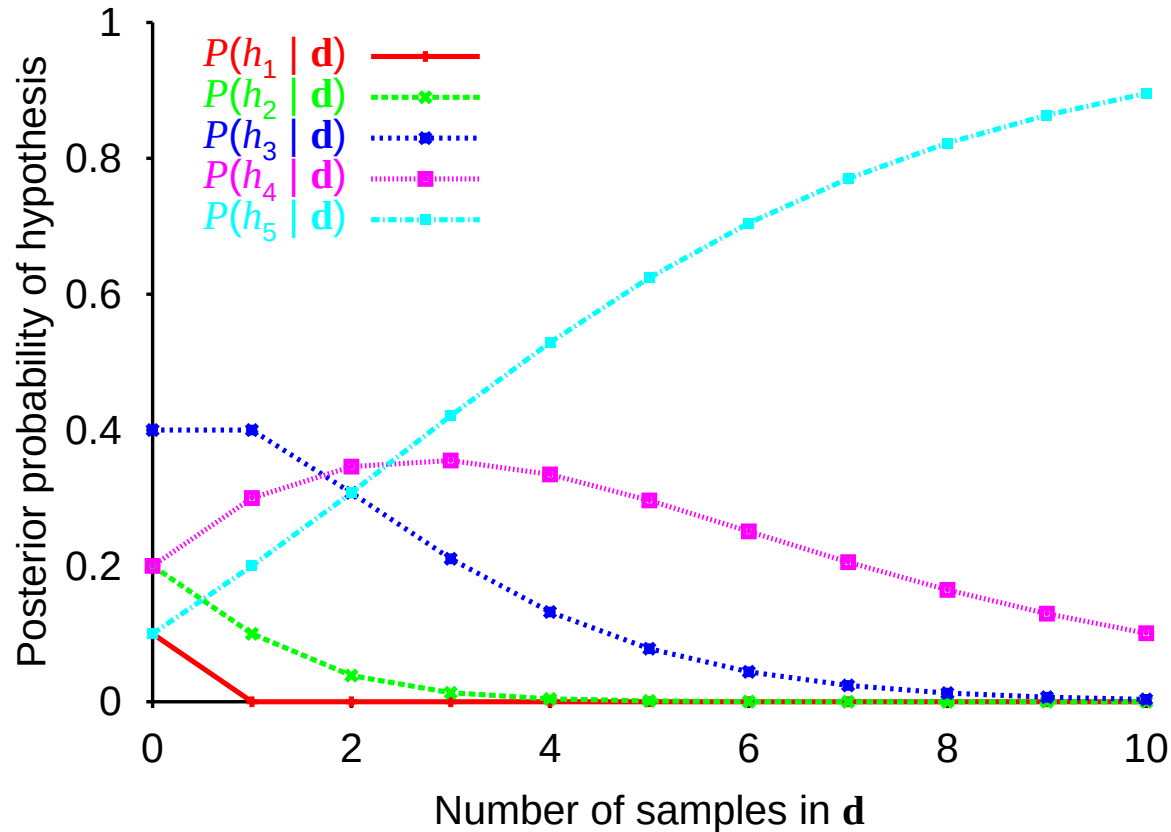
Prediction of X = likelihood-weighted avg of posterior probabilities of h_i :

$$\mathbf{P}(X|\mathbf{d}) = \sum_i \mathbf{P}(X|\mathbf{d}, h_i)P(h_i|\mathbf{d}) = \sum_i \mathbf{P}(X|h_i)P(h_i|\mathbf{d})$$

Notes:

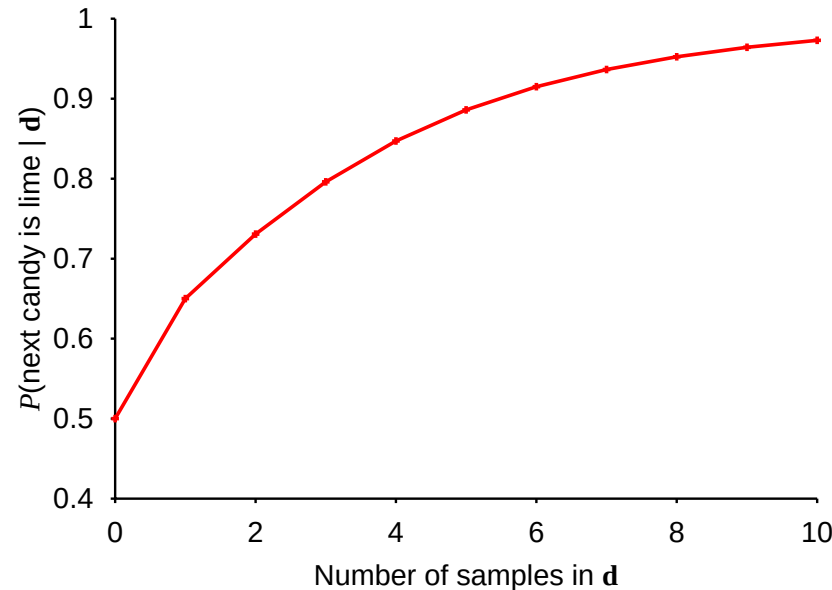
1. $\mathbf{P}(X|h_i)$: likelihood; $P(h_i|\mathbf{d})$: posterior probability
2. hypotheses act like intermediaries between raw data $\mathbf{d}$ and prediction X
3. in full Bayesian learning, we don't need to pick one best-guess hypothesis; we take into consideration **all hypotheses**

Posterior probability of hypotheses



$$P(h_2|li_1) = \frac{P(li_1|h_2)*P(h_2)}{P(li_1|h_1)*P(h_1)+...+P(li_1|h_5)*P(h_5)} = \frac{0.25*0.2}{0*0.1+...+1*0.1} = 0.1$$

Prediction probability



$$\begin{aligned} P(li_2|li_1) &= P(li_1|h_1) * P(h_1|li_1) + \dots + P(li_1|h_5) * P(h_5|li_1) \\ &= 0 * 0 + 0.25 * 0.1 + 0.5 * 0.4 + 0.75 * 0.3 + 1 * 0.2 \\ &= 0.650 \end{aligned}$$

The probability approaches 1: Bayesian prediction eventually agrees with the true h . Any wrong h will vanish.

MAP approximation

Note: summing over the hypothesis space is often intractable
(e.g., 18,446,744,073,709,551,616 Boolean functions of 6 attributes)

Simplification #1: make predictions based on the **maximum a posteriori** hypothesis h_{MAP} , i. e., the hypothesis which maximizes $P(h_i|\mathbf{d})$.

E.g., for our example, we get $h_{MAP} = h_5$ after 3 limes. We predict d_4 is *lime* with probability 1.0 (compare to 0.796 in full Bayesian learning)

- ◇ prior probabilities are used to **penalize complexity** (to avoid overfitting), as complex hypotheses usually have lower priors since there are more of them
- ◇ maximizing $P(\mathbf{d}|h_i)P(h_i)$ is minimizing $-(\log P(\mathbf{d}|h_i) + \log P(h_i))$: bits encoding data given hypothesis + bits encoding hypothesis. This is the basic idea of the **minimum description length** learning

ML approximation

Assume a uniform prior over the space of hypotheses (obs.: for large data sets, priors are actually irrelevant).

Simplification #2: **Maximum likelihood** (ML) learning: choose the hypothesis h_{ML} which maximizes $P(\mathbf{d}|h_i)$

I.e., simply get the best fit to the data; identical to MAP once we assume a uniform prior over the space of hypotheses (which is reasonable if all hypotheses are of the same complexity)

ML is the "standard" (non-Bayesian) statistical learning method. It offers a good approximation for Bayesian and MAP learning for **large** datasets.

Summary

- ◇ Nearest Neighbor model take into consideration only the closest neighbors; could be used for classification and regression
- ◇ Ensemble Learning helps boosting weak algorithms
- ◇ Full Bayesian learning gives best possible predictions but might be intractable
- ◇ MAP balances complexity with accuracy on training data
- ◇ ML assumes uniform prior, OK for large data sets

NEURAL NETWORKS I

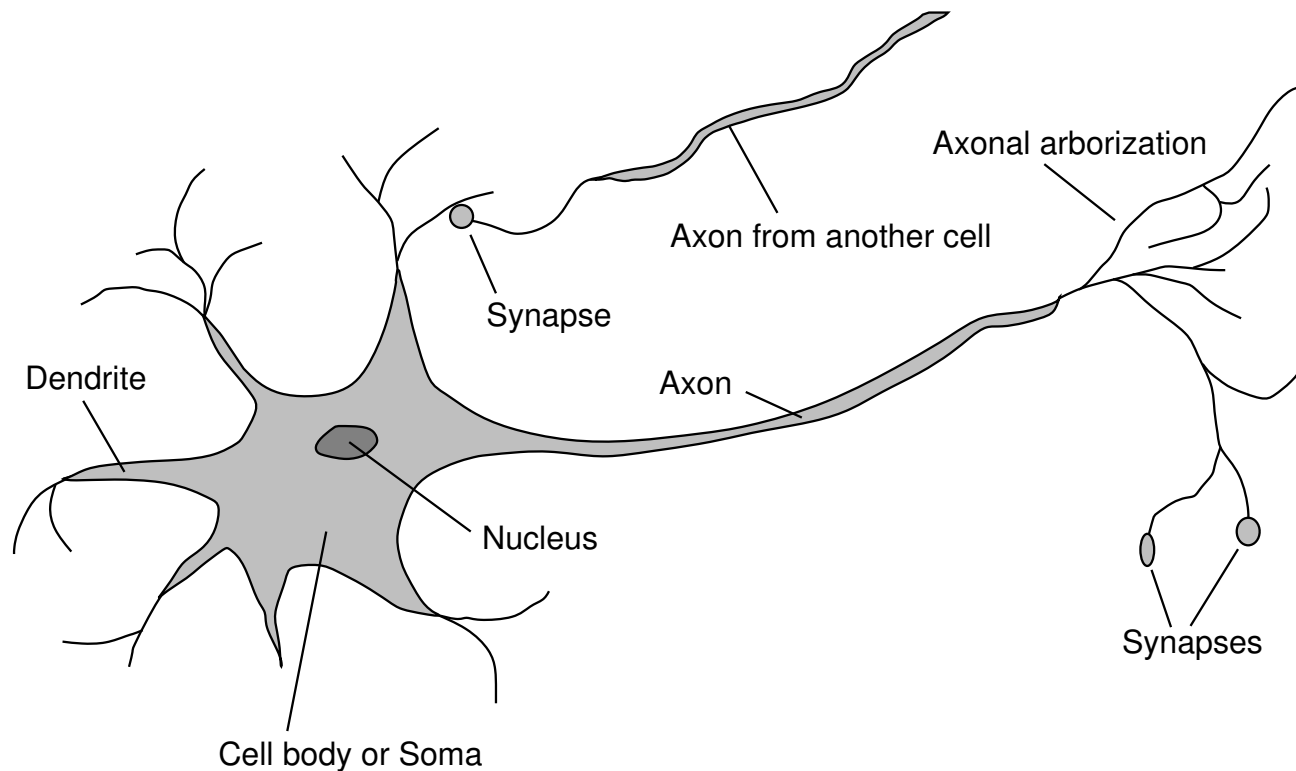
CHAPTER 18–7; CHAPTER 21 (AIMA 4TH)
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Neural networks basics
- ◇ Perceptrons

Brains

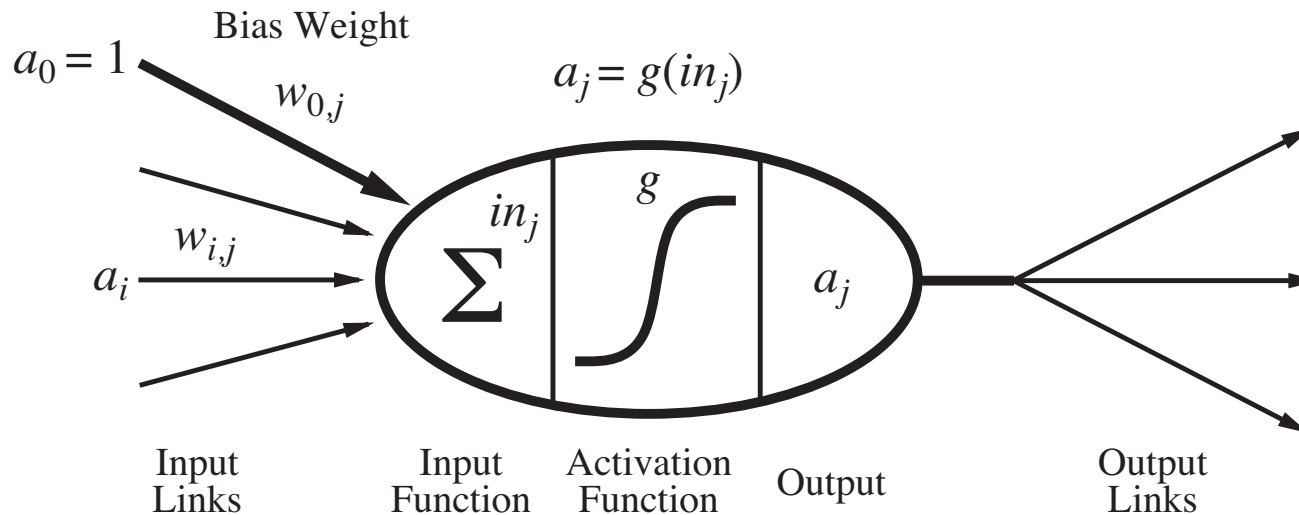
10^{11} neurons of > 20 types, 10^{14} synapses, 1ms–10ms cycle time
Signals are noisy “spike trains” of electrical potential



McCulloch–Pitts “unit” (1943!)

Output is a “squashed” linear function of the inputs:

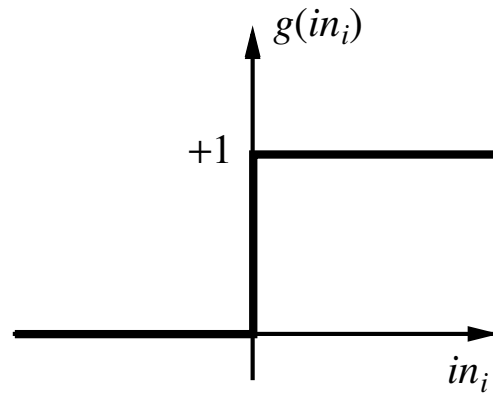
$$a_i \leftarrow g(in_i) = g\left(\sum_j w_{j,i} a_j\right)$$



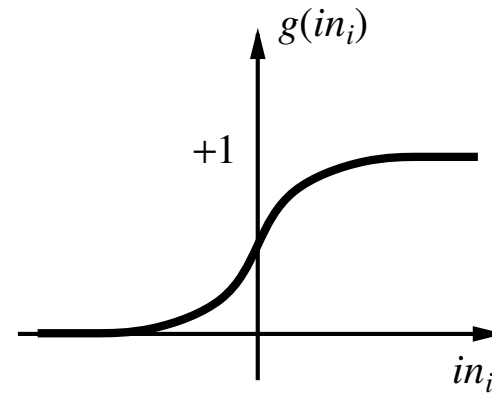
Bias: a_0 (typically, 1 or -1) and $w_{0,i}$

A gross oversimplification of real neurons, but its purpose is to develop understanding of what networks of simple units can do

Activation functions



(a)



(b)

(a) **step** aka **hard threshold function** (the unit using it is called "perceptron")

(b) **sigmoid**, a sort of logistic function: $1/(1 + e^{-x})$ (sigmoid perceptron)

(c) **ReLU** $\max(0, x)$ (Rectified Linear Unit)

(d) **softplus** $\log(1 + e^x)$

(e) **tanh** $(e^{2x} - 1)/(e^{2x} + 1)$

Changing the bias weight $w_{0,i}$ moves the threshold location

Network structures

Feed-forward: connections form a dag

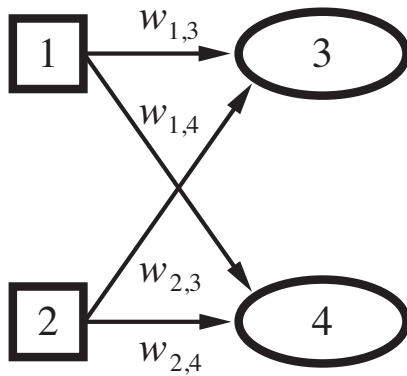
- single-layer perceptrons
- multi-layer networks

Feed-forward networks implement functions, have no internal state

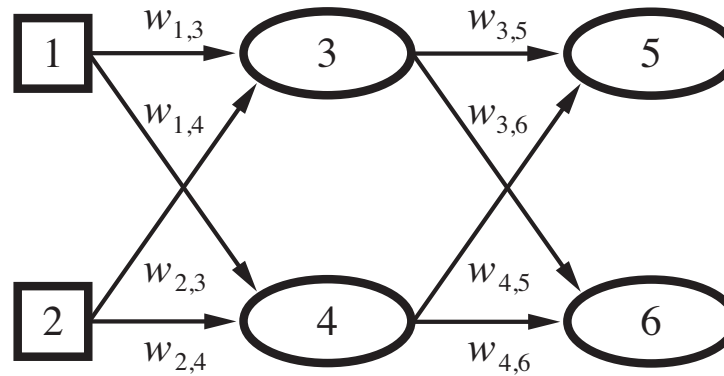
Recurrent: feed its outputs back to its inputs

- recurrent neural nets have directed cycles with delays
 - ⇒ have internal state (like flip-flops), can oscillate etc.
- Long Short Term Memory

Feed-forward example



(a)



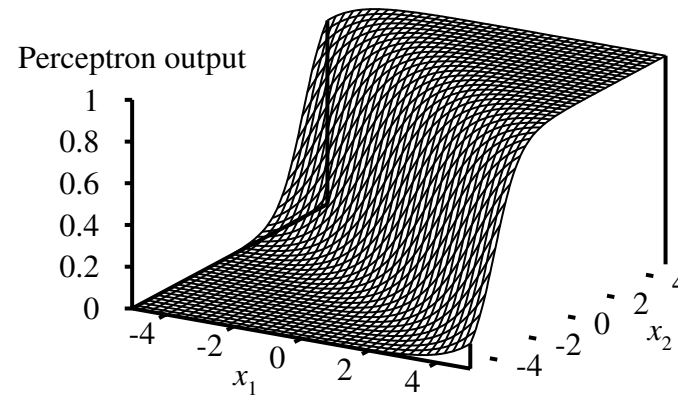
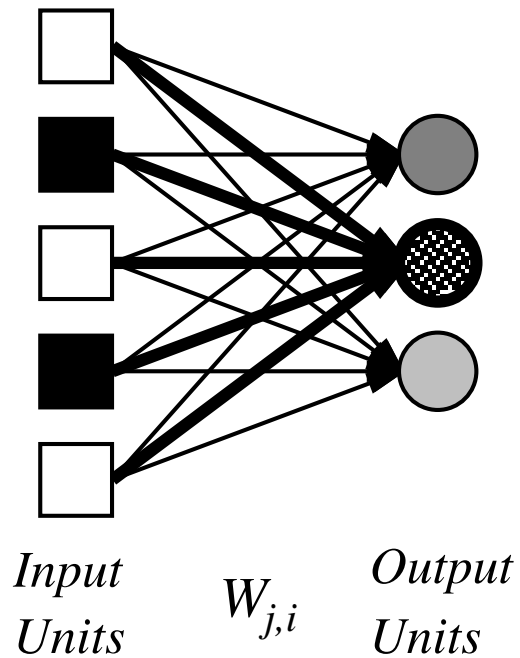
(b)

Feed-forward network = a family of nonlinear functions parameterized by w :

$$\begin{aligned} a_5 &= g(w_{0,5} + w_{3,5} \cdot a_3 + w_{4,5} \cdot a_4) \\ &= g(w_{0,5} + w_{3,5} \cdot g(w_{0,3} + w_{1,3} \cdot a_1 + w_{2,3} \cdot a_2) + \\ &\quad w_{4,5} \cdot g(w_{0,4} + w_{1,4} \cdot a_1 + w_{2,4} \cdot a_2)) \end{aligned}$$

Adjusting weights changes the function: do learning this way!

Single-layer perceptrons



No hidden layers, just input and output layers. Could "split" example space.

Output units all operate separately – no shared weights (e.g., digit recognition)

Adjusting weights moves the location, orientation, and steepness of cliff

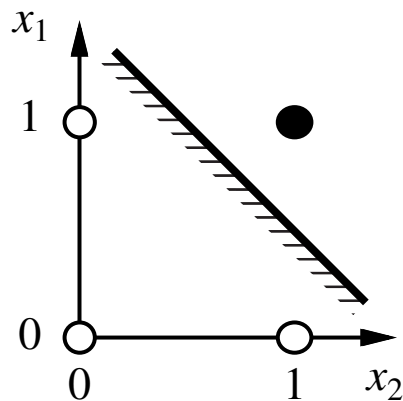
Expressiveness of perceptrons

Consider a perceptron with $g = \text{step function}$ (Rosenblatt, 1957, 1960)

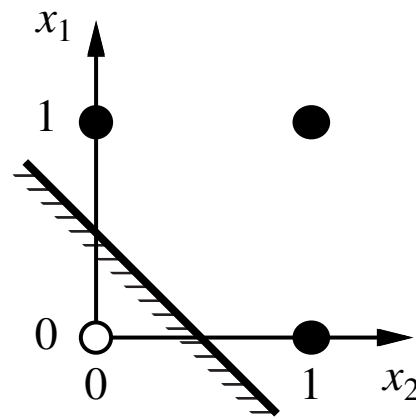
Can represent AND, OR, NOT, majority, etc., but not XOR

Represents a **linear separator** in input space:

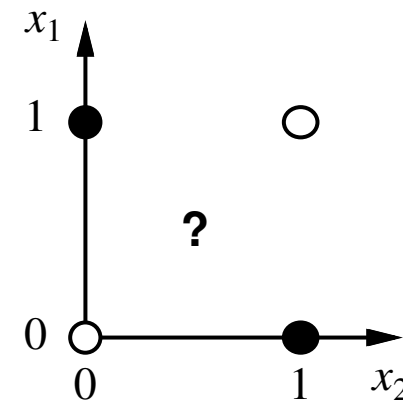
$$\sum_j w_j x_j > 0 \quad \text{or} \quad \mathbf{w} \cdot \mathbf{x} > 0$$



(a) x_1 **and** x_2



(b) x_1 **or** x_2



(c) x_1 **xor** x_2

(we switched from W to w notation)

Perceptron learning

How to learn? by adjusting weights to reduce **error on training set**.

We assume a **logistic** activation function (continuous and differentiable!)

We aim to minimize the following **Loss function** for an example $(\mathbf{x}, y)$:

$$E = \frac{1}{2}Err^2 \equiv \frac{1}{2}(y - h_{\mathbf{w}}(\mathbf{x}))^2$$

Perform optimization search in the weight space by **gradient descent**:

$$\frac{\partial E}{\partial w_j} = Err \times \frac{\partial Err}{\partial w_j} = Err \times \frac{\partial}{\partial w_j} \left(y - g\left(\sum_{j=0}^n w_j x_j\right) \right) = -Err \times g'(\mathbf{w} \cdot \mathbf{x}) \times x_j$$

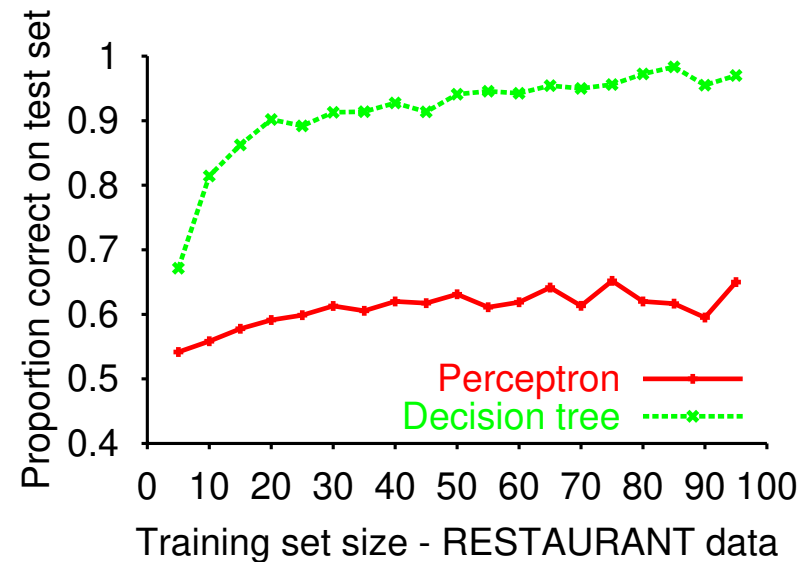
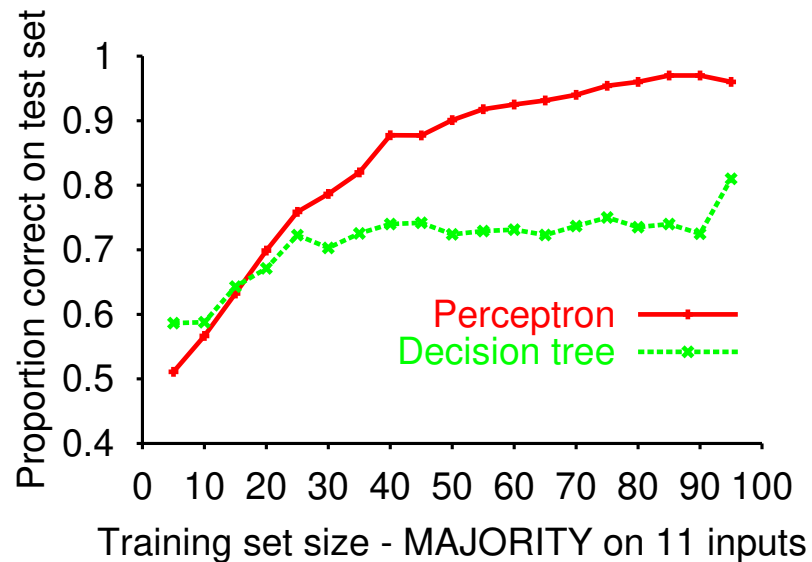
Simple weight update rule (with **learning rate** α):

$$w_i \leftarrow w_i + \alpha \times Err \times h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$

+ve err $\Rightarrow$ output $\nearrow \Rightarrow$ weights of +ve inputs $\nearrow$, of -ve inputs $\searrow$

Perceptron learning contd.

Perceptron learning rule converges to a consistent function **for any linearly separable data set**

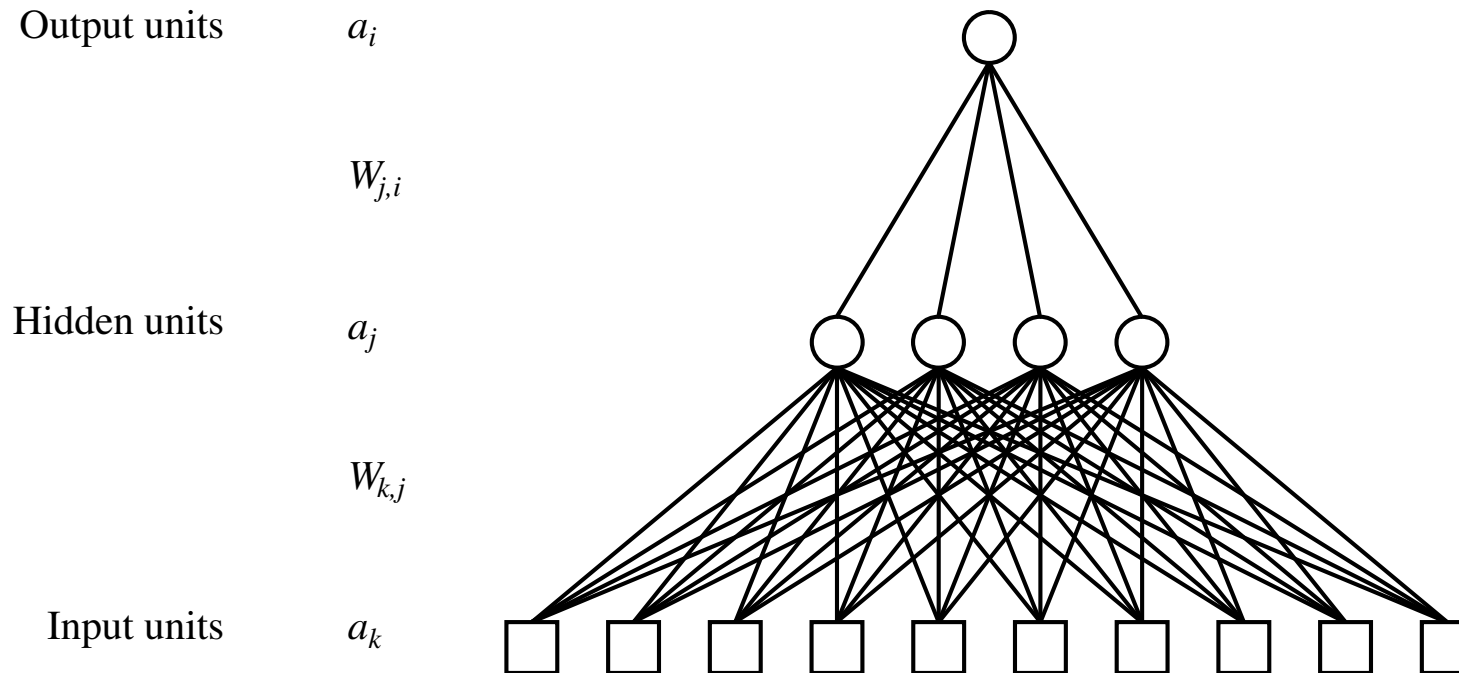


Perceptron learns majority function easily (e.g, $w_0 = -n/2$, the rest 1),
DTL is hopeless (i.e., exponential)

DTL learns restaurant function easily, perceptron cannot represent it

Multilayer feed-forward neural networks

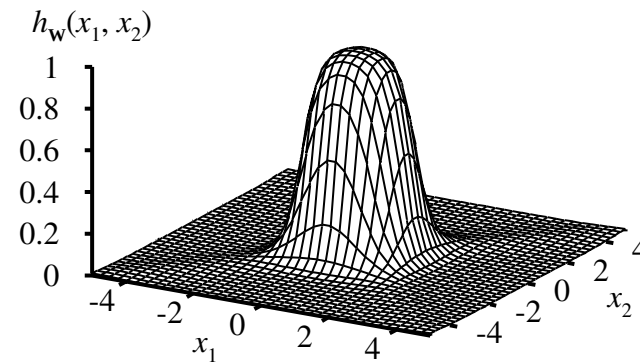
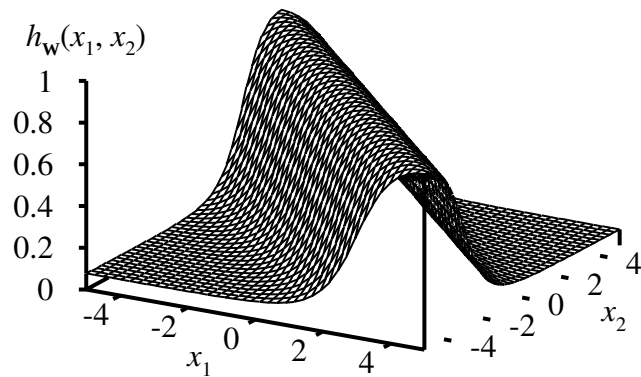
They have **hidden layers**. Usually, layers are fully connected. The number of **hidden units** typically chosen by hand.



Adding hidden layers enlarges the space of hypotheses the network can represent

Expressiveness of MLNNs

All continuous functions with 2 layers, all functions with 3 layers



Combine two opposite-facing threshold functions and threshold the result to make a ridge

Combine two orthogonal ridges to make a bump

Add bumps of various sizes and locations to fit any surface

Summary

Most brains have lots of neurons; each neuron $\approx$ linear-threshold unit (?)

Perceptrons (one-layer networks) insufficiently expressive

Engineering, cognitive modelling, and neural system modelling subfields have largely diverged

NEURAL NETWORKS II

SUPPORT VECTOR MACHINES

CHAPTER 18–7; CHAPTER 21 (AIMA 4TH)
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Training multilayer networks
- ◇ Support Vector Machines

Back-propagation learning for MLNNs

Network = function $h_{\mathbf{x}}(\mathbf{x})$ parameterized by the link weights $\mathbf{w}$.

Q: How do we update the weights?

For the output layer: same rule as for single-layer perceptron:

$$w_{j,i} \leftarrow w_{j,i} + \alpha \times a_j \times \Delta_i$$

where

$$\Delta_i = Err_i \times g'(in_i)$$

$$in_i = \sum_m w_{m,j} a_m$$

Back-propagation learning for MLNNs

Q: How do we update the weights?

For the hidden layer(s): **back-propagate** the error from the output layer:

$$\Delta_j = g'(in_j) \sum_k w_{j,k} \Delta_k$$

Update rule for weights in hidden layer:

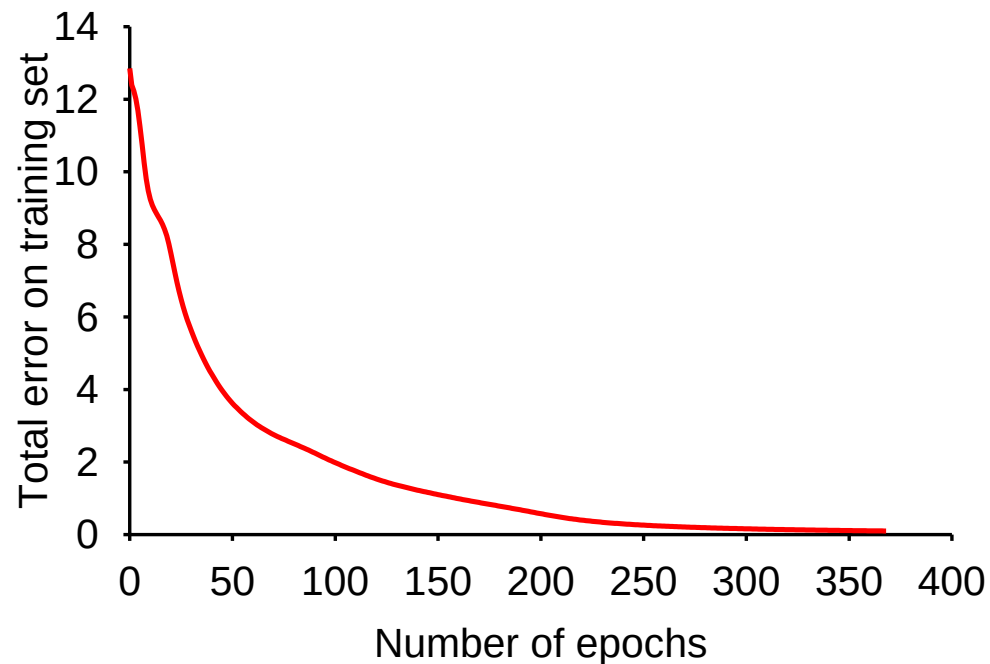
$$w_{i,j} \leftarrow w_{i,j} + \alpha \times a_i \times \Delta_j$$

(Note: most neuroscientists deny that back-propagation occurs in the brain)

Back-propagation learning contd.

At each **epoch**, sum gradient updates for all examples and apply

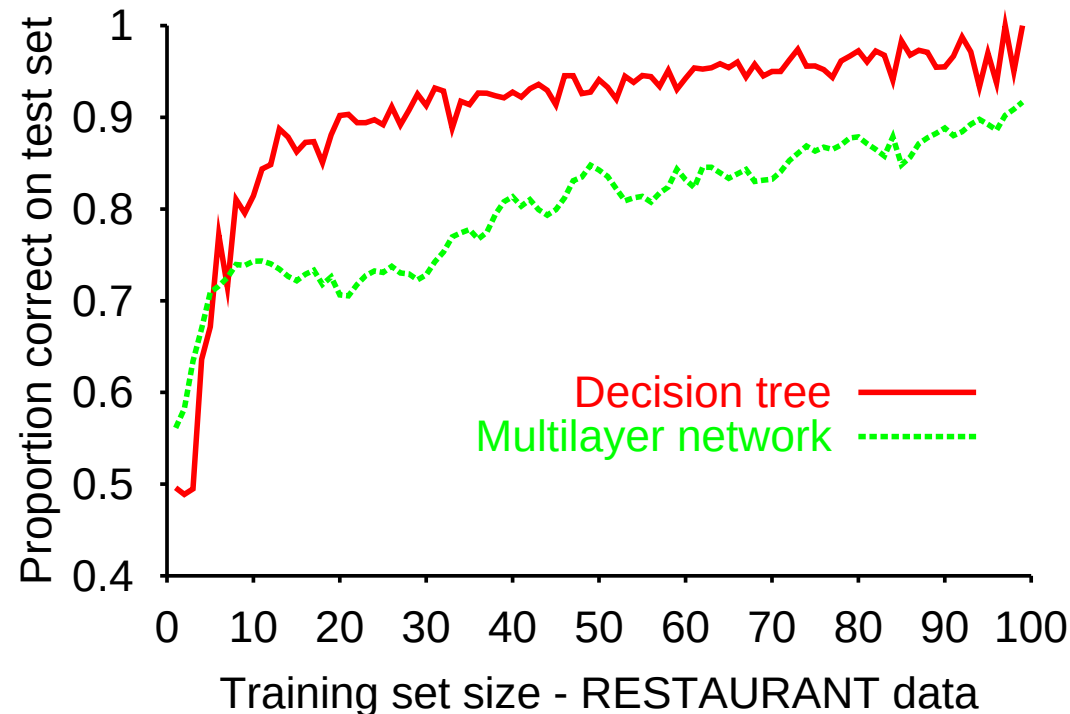
Training curve for 100 restaurant examples: finds exact fit



Typical problems: slow convergence, local minima

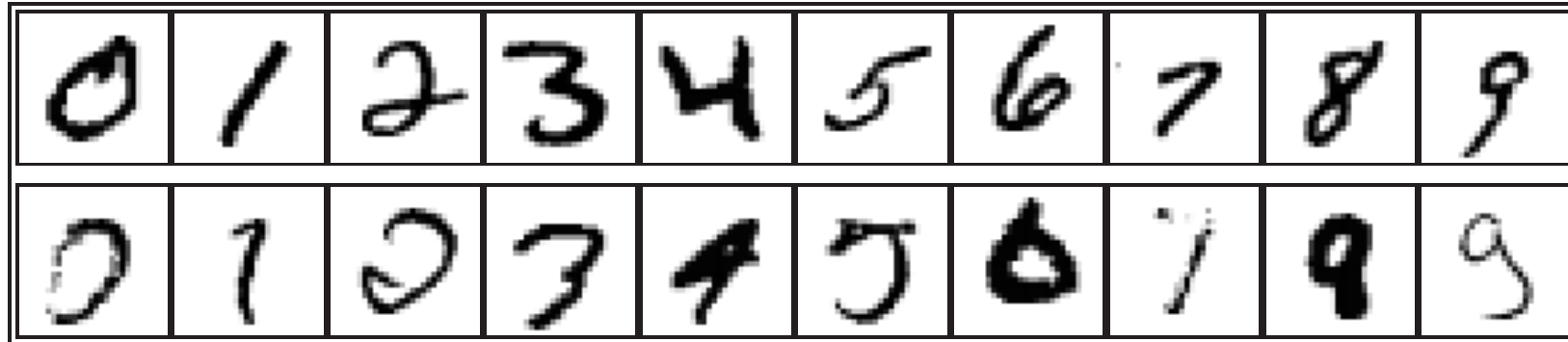
Back-propagation learning contd.

Learning curve for MLP with 4 hidden units:



MLPs are quite good for complex pattern recognition tasks, but the resulting hypotheses cannot be understood easily (unlike the DT case, where extracting IF-THEN rules for classification is trivial)

Handwritten digit recognition



3-nearest-neighbor = 2.4% error

400–300–10 unit MLP = 1.6% error

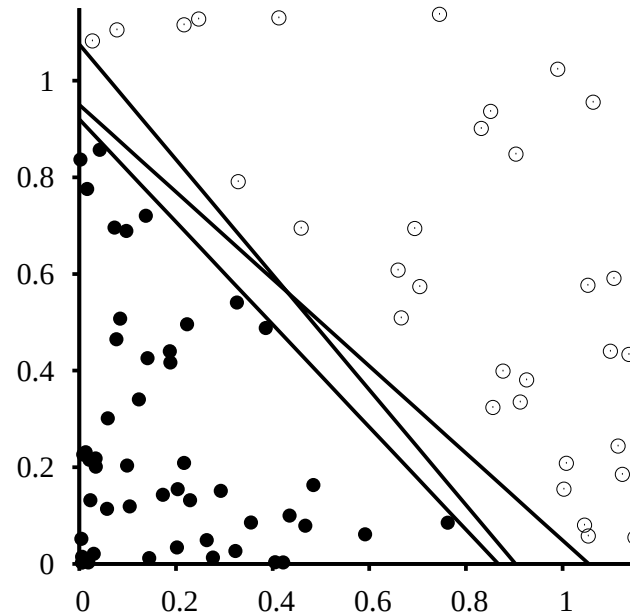
LeNet: 768–192–30–10 unit MLP = 0.9% error

kernel machines: $\approx$ 0.6% error

Support Vector Machines SVM

- ◇ "off-the-shelf" performant supervised learning approach
- ◇ build a **maximum margin separator**: decision boundary with the largest distance to example points (hence good generalization)
- ◇ nonparametric: retain potentially all training examples, but usually rely on a small fraction of them (so are both flexible and overfitting-resistant)
- ◇ unlike in logistic regression, consider some example more important than the others; the separator will depend on these examples
- ◇ employ the **kernel trick**: if data are not linearly separable, the original data space is mapped into a higher-dimensional one, where the problem becomes linearly separable. The linear separator in the high dimensional space is nonlinear in the original space (so the hypotheses space is expanded compared to the methods which stick with linear representations)

Maximum margin separator

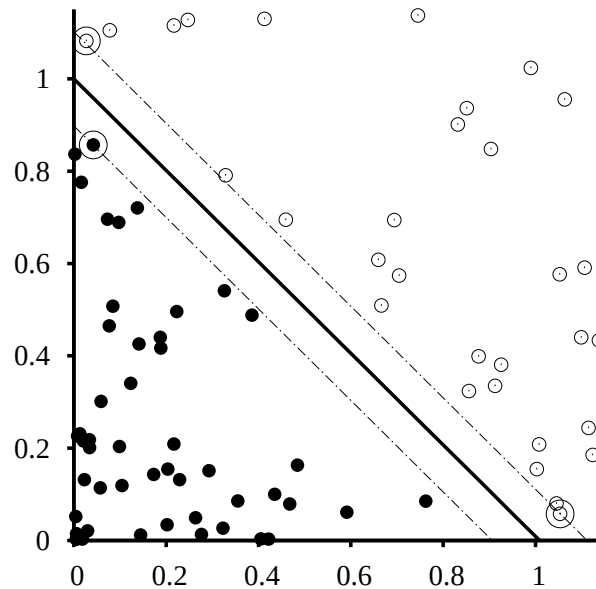


Three possible decision boundaries, each a linear separator consistent with all examples. The lowest line is very close to 5 examples in the black class.

Logistic regression finds a separation line which depends on all examples

Maximum margin separator

A. Linearly separable case: minimize **generalization loss**, not empirical loss



We can do this by maximizing the **margin** between the 2 regions of examples (the width of the area bounded by the dashed lines). Separator: $\mathbf{w} \cdot \mathbf{x} + b = 0$

Support Vectors: points with circles (the ones closest to the separator)

Finding the maximum margin separator

Can be done by solving "the dual optimization problem", which is in fact a **quadratic programming optimization problem**:

$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k)$$

subject to constraints $\alpha_j \geq 0$ and $\sum_j \alpha_j y_j = 0$ (where $\mathbf{w} = \sum_j \alpha_j \mathbf{x}_j$)

Key points:

- ◇ The expression has a single global maximum; this can be found efficiently
- ◇ Data enter in it only as dot products of vector pairs; we get the separator:

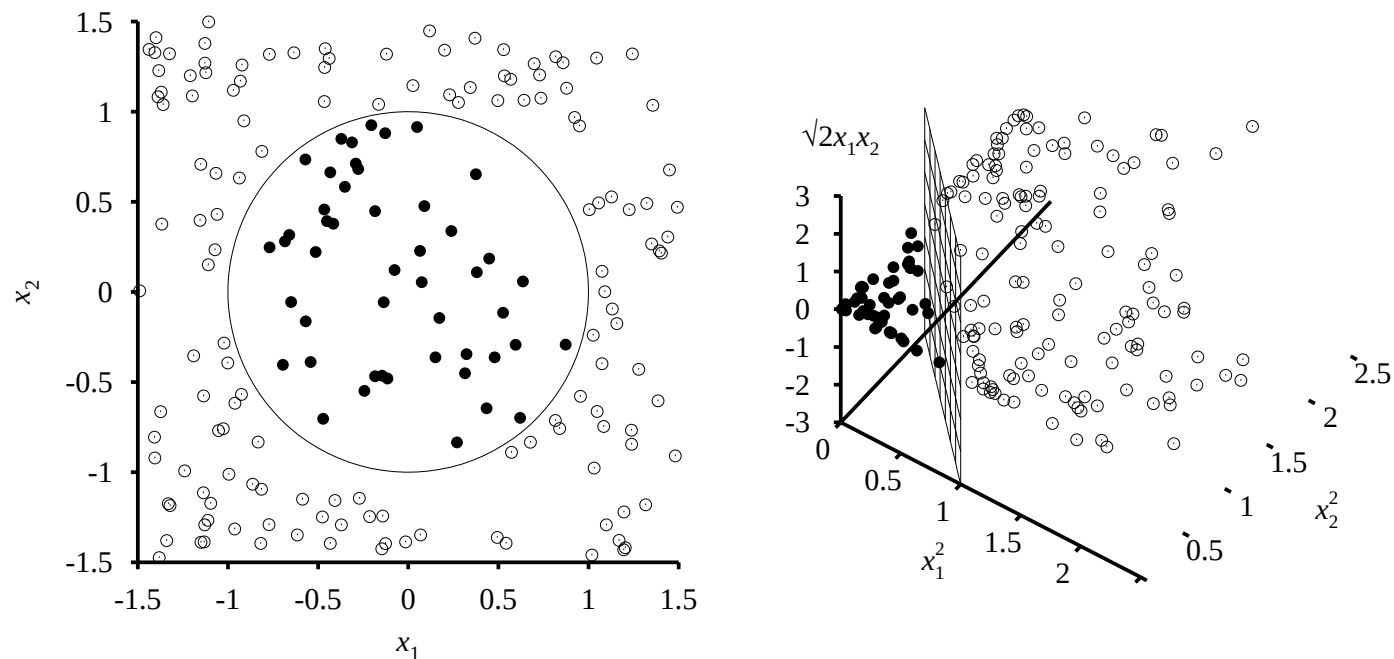
$$h(\mathbf{x}) = \operatorname{sign}(\sum_j \alpha_j y_j (\mathbf{x} \cdot \mathbf{x}_j) - b)$$

(Note: traditionally, in SVM class labels are +1 and -1)

- ◇ Most α_j are zero except for **support vectors**. We need to store only them.

Kernel trick: map to feature space

B. Not linearly separable case: maps the input space into another (usually higher-dimensional) space, where the problem is linearly separable. E.g., map $\mathbf{x} = (x_1, x_2)$ to $F(\mathbf{x}) = (f_1, f_2, f_3)$ with $f_1 = x_1^2, f_2 = x_2^2, f_3 = \sqrt{2}x_1x_2$



Kernel trick: map to feature space

- ◇ We can compute the separator by replacing $\mathbf{x}_j \cdot \mathbf{x}_k$ with $F(\mathbf{x}_j) \cdot F(\mathbf{x}_k)$
- ◇ **Kernel trick**: compute $F(\mathbf{x}_j) \cdot F(\mathbf{x}_k)$ without actually computing F ! For our example, this gives $(\mathbf{x}_j \cdot \mathbf{x}_k)^2$, which will be called a **kernel function** K . (Exercise: prove that $F(\mathbf{x}_j) \cdot F(\mathbf{x}_k) = (\mathbf{x}_j \cdot \mathbf{x}_k)^2$)
- ◇ The algorithm can learn in higher-dimensional space, but computes only $K(\mathbf{x}_j, \mathbf{x}_k)$, not the whole list of features for each data point in the new space.
- ◇ The separator could be computed efficiently for feature spaces having billions of dimensions. It typically depends on just a small fraction of the training set.
- ◇ Many kernels are available: $(\mathbf{x}_j \cdot \mathbf{x}_k)^2$, polynomial $((1 + \mathbf{x}_j \cdot \mathbf{x}_k)^d)$, radial basis function $e^{(-\gamma \|\mathbf{x}_j - \mathbf{x}_k\|^2)}$ etc.
- ◇ **Mercer theorem**: correspondence kernel functions $\rightarrow$ feature spaces

Summary

Multi-layer networks are sufficiently expressive; can be trained by gradient descent, i.e., error back-propagation

Many applications: speech, driving, handwriting, fraud detection, etc.

Support Vector Machines maximize generalization loss and could be combined with the kernel trick

TEMPORAL PROBABILITY MODELS

HIDDEN MARKOV MODELS

CHAPTER 15.1–3
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Time and uncertainty
- ◇ Hidden Markov models

Time and uncertainty: deal with world changes

Dynamic/static world: the value of a random variable does/does not change over time

E.g., diabetes management v. vehicle diagnosis (bloodSugar varies according to what was eaten v. what is broken remains broken)

Idea: "slice" time into discrete moments and have variables for each of them:

- $\mathbf{X}_t$ = set of unobservable state variables at time t
e.g., *BloodSugar_t*, *StomachContents_t*, etc.
(notation: $\mathbf{X}_{a:b} = \mathbf{X}_a, \mathbf{X}_{a+1}, \dots, \mathbf{X}_{b-1}, \mathbf{X}_b$)
- $\mathbf{E}_t$ = set of observable evidence variables at time t
e.g., *MeasuredBloodSugar_t* (possibly noisy!), *PulseRate_t*, *FoodEaten_t*

Construct a Bayes net from these variables: how to choose parents?

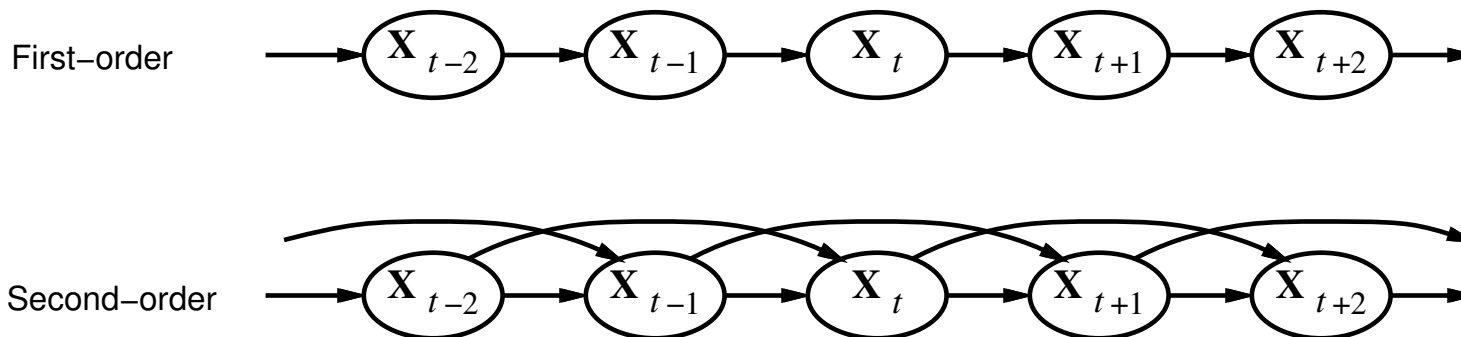
Markov processes (Markov chains)

Transition model: $\mathbf{P}(\mathbf{X}_t | \mathbf{X}_{0:t-1})$ + sensor model: $\mathbf{P}(\mathbf{E}_t | \mathbf{X}_{0:t}, \mathbf{E}_{0:t-1})$

Markov assumption: $\mathbf{X}_t$ depends on **bounded** subset of $\mathbf{X}_{0:t-1}$

◇ First-order Markov process (chain): $\mathbf{P}(\mathbf{X}_t | \mathbf{X}_{0:t-1}) = \mathbf{P}(\mathbf{X}_t | \mathbf{X}_{t-1})$

◇ Second-order Markov process (chain): $\mathbf{P}(\mathbf{X}_t | \mathbf{X}_{0:t-1}) = \mathbf{P}(\mathbf{X}_t | \mathbf{X}_{t-2}, \mathbf{X}_{t-1})$

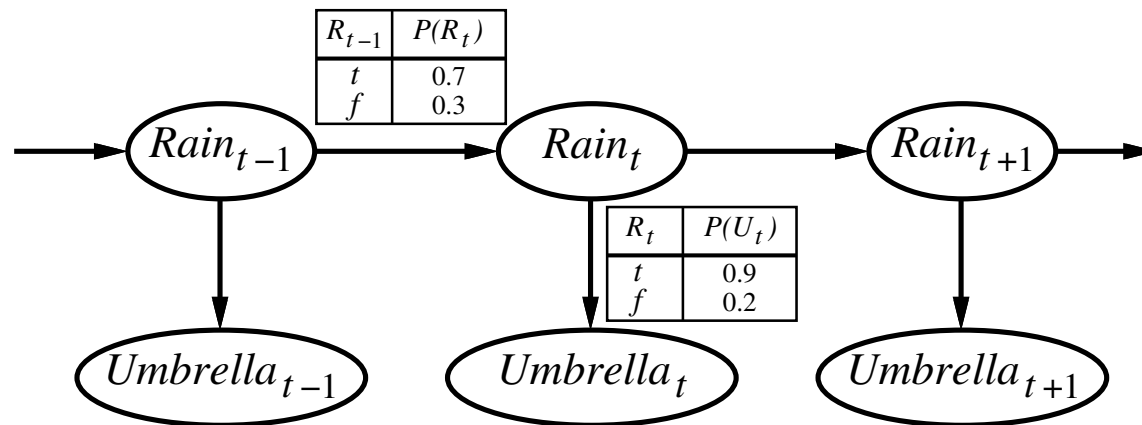


Sensor Markov assumption: $\mathbf{P}(\mathbf{E}_t | \mathbf{X}_{0:t}, \mathbf{E}_{0:t-1}) = \mathbf{P}(\mathbf{E}_t | \mathbf{X}_t)$

Stationary process: both transition $\mathbf{P}(\mathbf{X}_t | \mathbf{X}_{t-1})$ and sensor models $\mathbf{P}(\mathbf{E}_t | \mathbf{X}_t)$ are **fixed** for all t . The state changes according to laws which **do not** change.

Example

A guy at an underground facility wants to know if it's raining on a specific day. All he can see each day is if his commanding officer carries an umbrella.



Upper table: transition model; down table: sensor model

First-order Markov assumption not exactly true in real world! Possible fixes:

1. **Increase order** of Markov process (R_t also depends on R_{t-2})
2. **Increase the number of state variables**, e.g., add $Temp_t$, $Pressure_t$, according to the physics of the phenomenon

Inference tasks

Task 1: **Filtering** (aka state estimation): $P(\mathbf{X}_t | \mathbf{e}_{1:t})$

E.g., Probability to rain today, given all evidence till today

means computing the **belief state**, needed for making rational decisions

Task 2: **Prediction**: $P(\mathbf{X}_{t+k} | \mathbf{e}_{1:t})$ for $k > 0$

E.g., Probability to rain the day after tomorrow, given all evidence till today

evaluation of possible action sequences;

like filtering without the evidence

Task 3: **Smoothing**: $P(\mathbf{X}_k | \mathbf{e}_{1:t})$ for $0 \leq k < t$

E.g., Probability that it rained the last Monday, given all evidence till today

better estimate of past states, essential for learning

Task 4: **Most likely explanation**: $\arg \max_{\mathbf{x}_{1:t}} P(\mathbf{x}_{1:t} | \mathbf{e}_{1:t})$

E.g., The most probable sequence of states which generated the sequence of evidence till today. If we see umbrella in the first 3 days and no umbrella on the fourth, it should infer the sequence *Rain, Rain, Rain, $\neg$ Rain*

speech recognition, decoding with a noisy channel

Some useful formulas

$$(F1) \quad \mathbf{P(a, b|c) = P(a|b, c) * P(b|c)}$$

$$(F1R) \quad \mathbf{P(a|b, c) = \frac{1}{P(b|c)} * P(a, b|c) = \alpha P(a, b|c)}$$

$$(F2) \quad \mathbf{P(a|e) = \sum_i P(a|i, e) * P(i|e)}$$

Task 1: Filtering

Aim: a **recursive** state estimation algorithm (store state estimate and update it; don't go back on the entire history of percepts, or else the cost for the update step will be $O(t)$): $\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t+1}) = f(\mathbf{e}_{t+1}, \mathbf{P}(\mathbf{X}_t|\mathbf{e}_{1:t}))$

$$\begin{aligned}\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t+1}) &= \mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t}, \mathbf{e}_{t+1}) && (\text{divide evidence}) \\ &= \mathbf{P}(\mathbf{X}_{t+1}, \mathbf{e}_{1:t}|\mathbf{e}_{t+1})/\mathbf{P}(\mathbf{e}_{1:t}|\mathbf{e}_{t+1}) && (F1R) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1}, \mathbf{e}_{1:t})\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t}) && (Bayes) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1})\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t}) && (\text{Markov assumption for sensors})\end{aligned}$$

I.e., sensor model + one step prediction.

Task 1: Filtering

$$\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t+1}) = \alpha \mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1})\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t})$$

(F2) helps to get the one step prediction:

$$\begin{aligned}\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t+1}) &= \alpha \mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1}|\mathbf{x}_t, \mathbf{e}_{1:t}) P(\mathbf{x}_t|\mathbf{e}_{1:t}) && (F2) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1}|\mathbf{x}_t) P(\mathbf{x}_t|\mathbf{e}_{1:t}) && (Markov)\end{aligned}$$

I.e., sensor model + (transition model + current state distribution)

$$\mathbf{f}_{1:t+1} = \alpha \text{FORWARD}(\mathbf{f}_{1:t}, \mathbf{e}_{t+1}) \text{ where } \mathbf{f}_{1:t} = \mathbf{P}(\mathbf{X}_t|\mathbf{e}_{1:t}) \text{ and } \mathbf{f}_{1:0} = \mathbf{P}(\mathbf{X}_0)$$

The MESSAGE $\mathbf{f}_{1:t}$ is propagated forward along the sequence, modified by each transition and updated by each new observation.

The time and space needed are **constant**

Task 1: Filtering

One step prediction equation:

$$\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t}) = \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1}|\mathbf{x}_t)P(\mathbf{x}_t|\mathbf{e}_{1:t})$$

Update equation:

$$\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t+1}) = \alpha \mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1})\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t})$$

So:

predict

loop

update with the new piece of evidence (using the previous prediction)

predict (based on the updated probabilities)

Filtering example

Let's say the underground guy sees an umbrella on both day #1 and #2.
We assume $\mathbf{P}(R_0) = \langle 0.5, 0.5 \rangle$

The prediction from $t = 0$ to $t = 1$ will be:

$$\begin{aligned}\mathbf{P}(R_1) &= \sum_{r_0} \mathbf{P}(R_1|r_0)P(r_0) \\ &= \langle 0.7, 0.3 \rangle * 0.5 + \langle 0.3, 0.7 \rangle * 0.5 \\ &= \langle 0.5, 0.5 \rangle\end{aligned}$$

We know that $U_1 = \textit{true}$. Update with the evidence for $t = 1$:

$$\begin{aligned}\mathbf{P}(R_1|u_1) &= \alpha \mathbf{P}(u_1|R_1)\mathbf{P}(R_1) \\ &= \alpha \langle 0.9, 0.2 \rangle \langle 0.5, 0.5 \rangle \\ &= \alpha \langle 0.45, 0.1 \rangle \\ &\approx \langle 0.818, 0.182 \rangle\end{aligned}$$

Filtering example

The prediction from $t = 1$ to $t = 2$ will be:

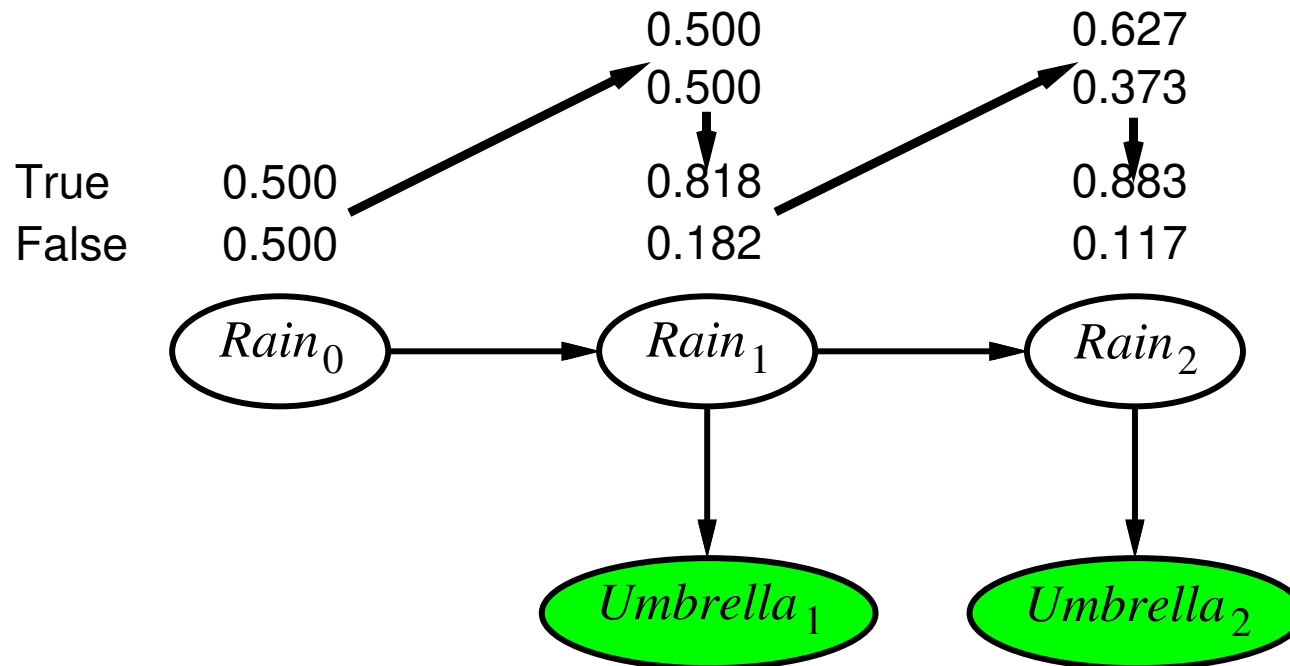
$$\begin{aligned}\mathbf{P}(R_2|u_1) &= \sum_{r_1} \mathbf{P}(R_2|r_1)P(r_1|u_1) \\ &= \langle 0.7, 0.3 \rangle \times 0.818 + \langle 0.3, 0.7 \rangle \times 0.182 \\ &\approx \langle 0.627, 0.373 \rangle\end{aligned}$$

We know that $U_2 = \textit{true}$. Update with the evidence for $t = 2$:

$$\begin{aligned}\mathbf{P}(R_2|u_1, u_2) &= \alpha \mathbf{P}(u_2|R_2) \mathbf{P}(R_2|u_1) \\ &= \alpha \langle 0.9, 0.2 \rangle \langle 0.627, 0.373 \rangle \\ &= \alpha \langle 0.565, 0.075 \rangle \\ &\approx \langle 0.883, 0.117 \rangle\end{aligned}$$

(higher than 0.818!)

Filtering example



One can regard the filtered estimate $P(\mathbf{X}_t | \mathbf{e}_{1:t})$ as a message $f_{1:t}$ propagated forward along the sequence, modified by each transition and updated by each new observation.

Task 2: Prediction

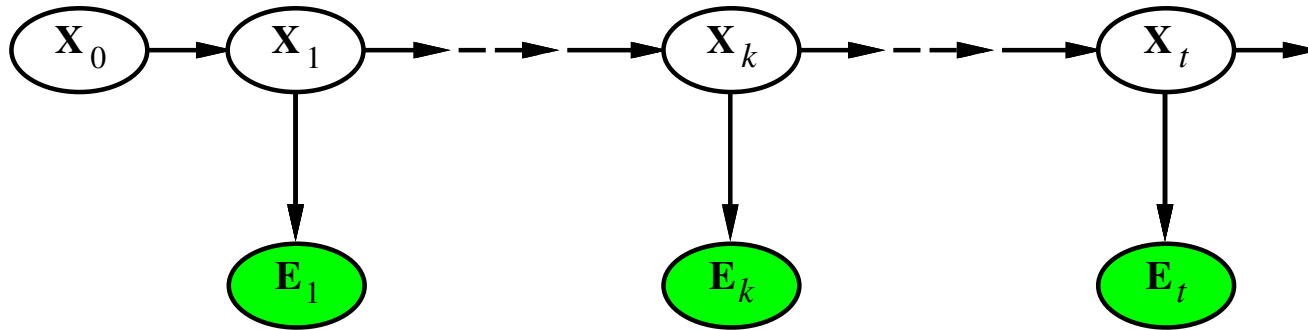
Filtering, but without adding new evidence:

$$\mathbf{P}(\mathbf{X}_{t+k+1}|\mathbf{e}_{1:t}) = \sum_{\mathbf{x}_{t+k}} \mathbf{P}(\mathbf{X}_{t+k+1}|\mathbf{x}_{t+k})P(\mathbf{x}_{t+k}|\mathbf{e}_{1:t})$$

Obs:

1. this uses no sensor model, just the transition model
2. as $k \rightarrow \infty$, $P(\mathbf{x}_{t+k}|\mathbf{e}_{1:t})$ tends to a fix point ($\langle 0.5, 0.5 \rangle$), the **stationary distribution** of the Markov chain. The future obscures in an amount of time called *mixing time*, which depends on how **stochastic** the chain is

Task 3: Smoothing



Divide evidence into pieces up to k and beyond k :

$$\begin{aligned}
 \mathbf{P}(\mathbf{X}_k | \mathbf{e}_{1:t}) &= \mathbf{P}(\mathbf{X}_k | \mathbf{e}_{1:k}, \mathbf{e}_{k+1:t}) && \text{(divide evidence)} \\
 &= \alpha \mathbf{P}(\mathbf{e}_{1:k}, \mathbf{e}_{k+1:t} | \mathbf{X}_k) \mathbf{P}(\mathbf{X}_k) && \text{(Bayes)} \\
 &= \alpha \mathbf{P}(\mathbf{e}_{k+1:t} | \mathbf{X}_k, \mathbf{e}_{1:k}) \mathbf{P}(\mathbf{e}_{1:k} | \mathbf{X}_k) \mathbf{P}(\mathbf{X}_k) && (F1) \\
 &= \alpha \mathbf{P}(\mathbf{e}_{k+1:t} | \mathbf{X}_k, \mathbf{e}_{1:k}) \mathbf{P}(\mathbf{X}_k | \mathbf{e}_{1:k}) && \text{(Bayes, new } \alpha) \\
 &= \alpha \mathbf{P}(\mathbf{X}_k | \mathbf{e}_{1:k}) \mathbf{P}(\mathbf{e}_{k+1:t} | \mathbf{X}_k) && \text{(independence)} \\
 &= \alpha \mathbf{f}_{1:k} \times \mathbf{b}_{k+1:t}
 \end{aligned}$$

Task 3: Smoothing

$$\begin{aligned}\mathbf{P}(\mathbf{X}_k | \mathbf{e}_{1:t}) &= \alpha \mathbf{P}(\mathbf{X}_k | \mathbf{e}_{1:k}) \mathbf{P}(\mathbf{e}_{k+1:t} | \mathbf{X}_k) \\ &= \alpha \mathbf{f}_{1:k} \times \mathbf{b}_{k+1:t}\end{aligned}$$

Backward message computed using a backwards recursion:

$$\begin{aligned}\mathbf{P}(\mathbf{e}_{k+1:t} | \mathbf{X}_k) &= \sum_{\mathbf{x}_{k+1}} \mathbf{P}(\mathbf{e}_{k+1:t} | \mathbf{X}_k, \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} | \mathbf{X}_k) && (F2) \\ &= \sum_{\mathbf{x}_{k+1}} P(\mathbf{e}_{k+1:t} | \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} | \mathbf{X}_k) && (\text{independence}) \\ &= \sum_{\mathbf{x}_{k+1}} P(\mathbf{e}_{k+1}, \mathbf{e}_{k+2:t} | \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} | \mathbf{X}_k) && (\text{divide } \mathbf{e}) \\ &= \sum_{\mathbf{x}_{k+1}} P(\mathbf{e}_{k+1} | \mathbf{x}_{k+1}) P(\mathbf{e}_{k+2:t} | \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} | \mathbf{X}_k) && (\mathbf{e}_{k+1} \text{ independent} \\ &&& \text{of } \mathbf{e}_{k+2:t} \text{ given } \mathbf{X}_k)\end{aligned}$$

I.e., sensor and transition models + recursive call $\mathbf{b}_{k+1:t} = \text{BACKWARD}(\mathbf{b}_{k+2:t}, \mathbf{e}_{k+1})$

Smoothing example

Obs: the backward phase is initialized with $\mathbf{b}_{t+1:t} = \mathbf{P}(e_{t+1:t}|\mathbf{X}_t) = \mathbf{P}(\cdot|\mathbf{X}_t)\mathbf{1}$

Smoothed estimate for the probability of rain at $k = 1$, given the umbrella observations on days #1 and #2:

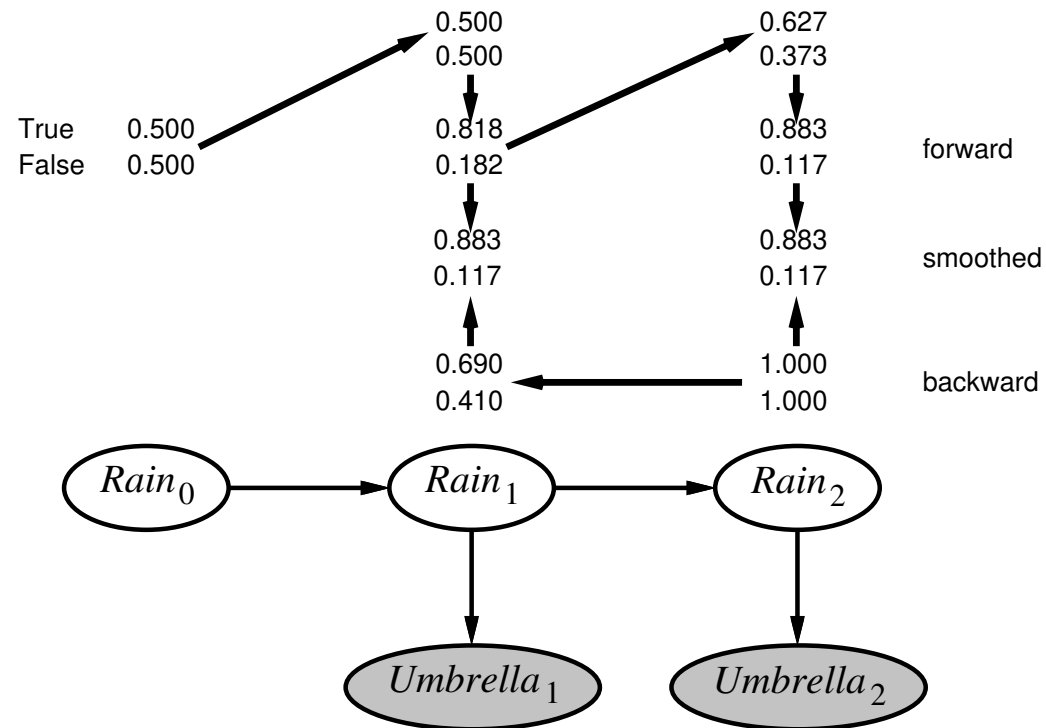
$\mathbf{P}(R_1|u_1, u_2) = \alpha \mathbf{P}(R_1|u_1) \mathbf{P}(u_2|R_1)$ where

$$\mathbf{P}(R_1|u_1) = \langle 0.818, 0.182 \rangle \quad (\text{forward})$$

$$\begin{aligned} \mathbf{P}(u_2|R_1) &= \sum_{r_2} P(u_2|r_2)P(r_2)\mathbf{P}(r_2|R_1) \quad (\text{backward}) \\ &= (0.9 \times 1 \times \langle 0.7, 0.3 \rangle) + (0.2 \times 1 \times \langle 0.3, 0.7 \rangle) \\ &= \langle 0.63, 0.27 \rangle + \langle 0.06, 0.14 \rangle \\ &= \langle 0.69, 0.41 \rangle \end{aligned}$$

$$\mathbf{P}(R_1|u_1, u_2) = \alpha \langle .818, .182 \rangle \times \langle .69, .41 \rangle \approx \langle .883, .117 \rangle \quad (\text{higher than .818!})$$

Smoothing example



Forward-backward algorithm: cache forward messages along the way and adjust them with the backward messages

Task 4: Most likely explanation

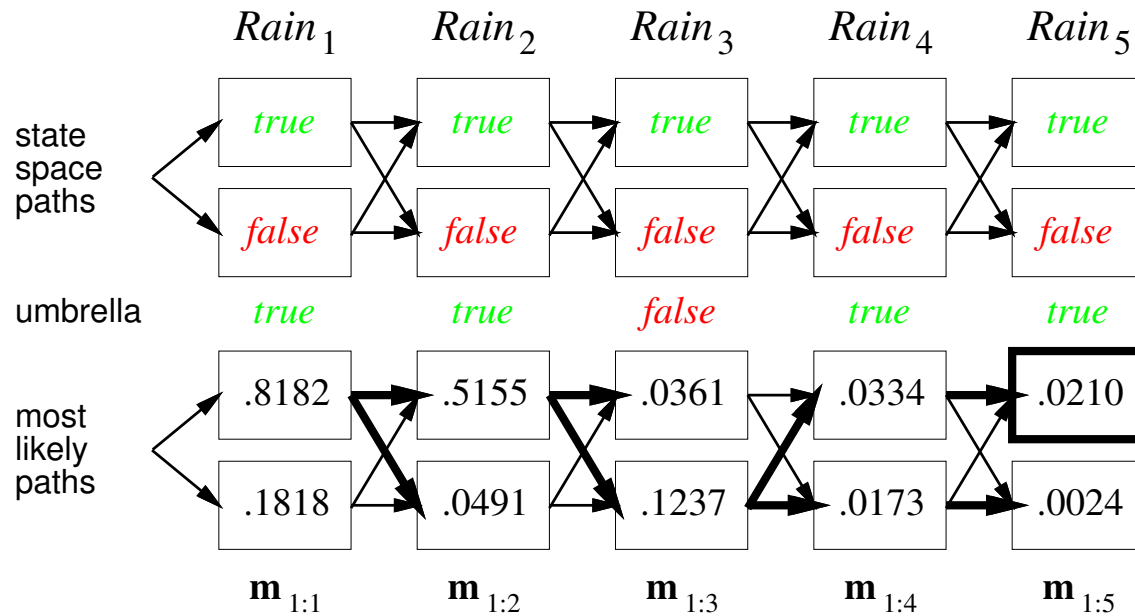
The guard observes the umbrella sequence $[true, true, false, true, true]$.
What is the weather **sequence** most likely to explain this?

Most likely sequence $\neq$ sequence of most likely states! (distributions over single time steps vs. joint probability distributions over all time steps)

Idea: Sequence = path in a graph (nodes=possible states at each time step)

Task 4: Most likely explanation

Idea: Sequence = path in a graph (nodes=possible states at each time step)



Task 4: Most likely explanation

Likelihood of a path = product of the transition probabilities along it and the probabilities of the given observations at each state

Obs: there is a recursive relationship between the most likely path to each state $\mathbf{x}_{t+1}$ and the most likely paths to each state $\mathbf{x}_t$:

$$\begin{aligned} & \max_{\mathbf{x}_1 \dots \mathbf{x}_t} \mathbf{P}(\mathbf{x}_1, \dots, \mathbf{x}_t, \mathbf{X}_{t+1} | \mathbf{e}_{1:t+1}) \\ &= \mathbf{P}(\mathbf{e}_{t+1} | \mathbf{X}_{t+1}) \max_{\mathbf{x}_t} \left(\mathbf{P}(\mathbf{X}_{t+1} | \mathbf{x}_t) \max_{\mathbf{x}_1 \dots \mathbf{x}_{t-1}} \mathbf{P}(\mathbf{x}_1, \dots, \mathbf{x}_{t-1}, \mathbf{x}_t | \mathbf{e}_{1:t}) \right) \end{aligned}$$

This is the idea of the **Viterbi algorithm**.

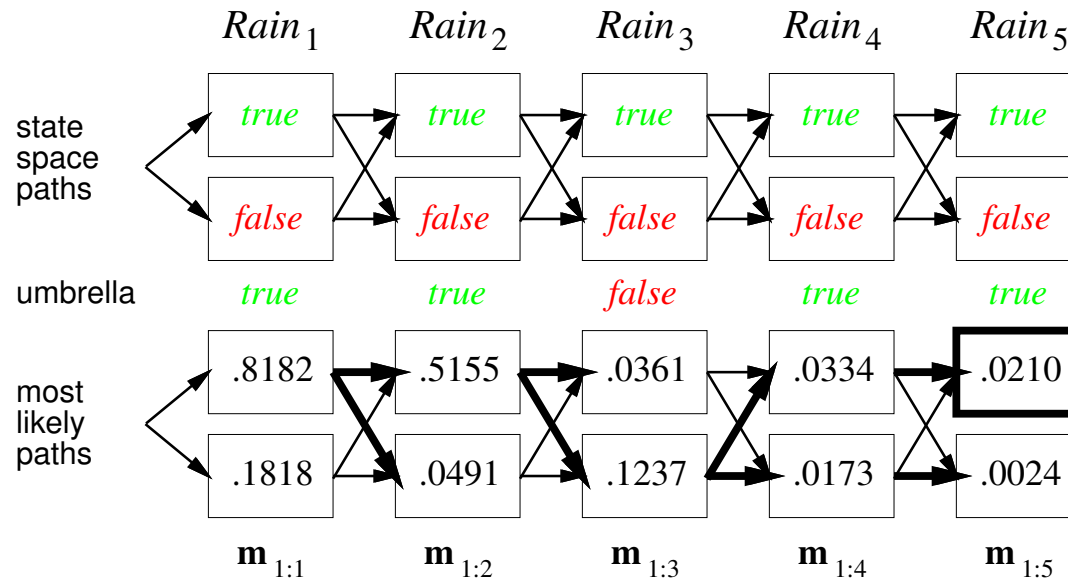
Most likely explanation: Viterbi algorithm

It works similar to filtering, except:

1. $\mathbf{f}_{1:t} = \mathbf{P}(\mathbf{x}_t | \mathbf{e}_{1:t})$ is replaced by $\mathbf{m}_{1:t} = \max_{\mathbf{x}_1 \dots \mathbf{x}_{t-1}} \mathbf{P}(\mathbf{x}_1, \dots, \mathbf{x}_{t-1}, \mathbf{X}_t | \mathbf{e}_{1:t})$
i.e., $\mathbf{m}_{1:t}(i)$ gives the probability of the most likely path to state i .
2. the update step has sum replaced by max:

$$\mathbf{m}_{1:t+1} = \mathbf{P}(\mathbf{e}_{t+1} | \mathbf{X}_{t+1}) \max_{\mathbf{x}_t} (\mathbf{P}(\mathbf{X}_{t+1} | \mathbf{x}_t) \mathbf{m}_{1:t})$$

Viterbi example



$$\mathbf{m}_{1:1} = \mathbf{P}(R_1|u_1) = \langle .8182, .1818 \rangle \text{ (by filtering)}$$

$$\begin{aligned} \mathbf{m}_{1:2}^{r_2} &= P(u_2|r_2) \max_{r_1}(\mathbf{P}(R_2|r_1)\langle .8182, .1818 \rangle) \\ &= .9 \max_{r_1}(\langle .7, .3 \rangle \langle .8182, .1818 \rangle) = .9 \max(\langle .5727, .0545 \rangle) = .5155 \end{aligned}$$

$$\begin{aligned} \mathbf{m}_{1:2}^{\neg r_2} &= P(u_2|\neg r_2) \max_{r_1}(\mathbf{P}(\neg r_2|r_1)\langle .8182, .1818 \rangle) \\ &= .2 \max_{r_1}(\langle .3, .7 \rangle \langle .8182, .1818 \rangle) = .2 \max(\langle .2455, .1273 \rangle) = .0491 \end{aligned}$$

Solution (by algorithm): $[true, true, false, true, true]$

HMM example: Robot localization by filtering

Hidden Markov Model (HMM) = temporal probabilistic model in which the state of the process is described by a **single discrete** random variable, whose possible values are the possible states of the world.

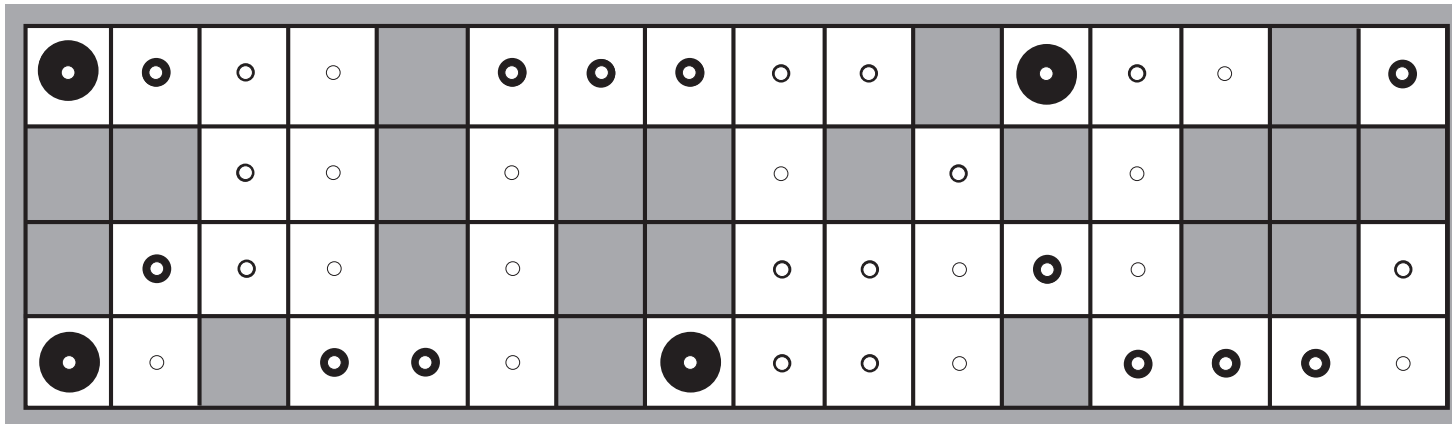
State variable X_t : robot location on a discrete grid.

Domain of X_t : set of empty squares $\{s_1, \dots, s_n\}$

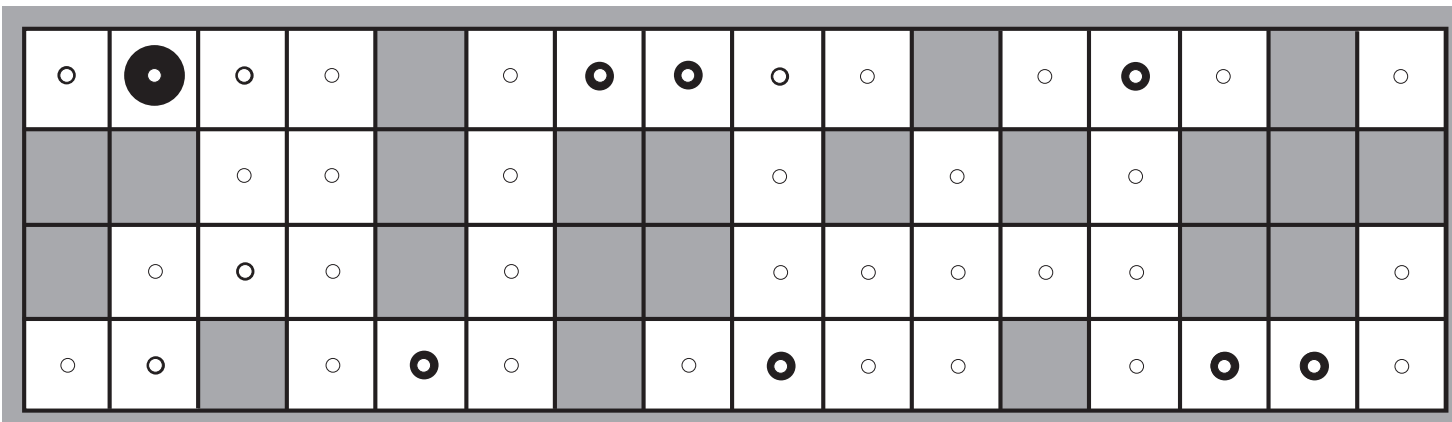
Transition model for action *Move*: the robot is equally likely to get to any neighbor square

Four independent sensors, one per cardinal direction, each with error rate ϵ

HMM example: Robot localization by filtering

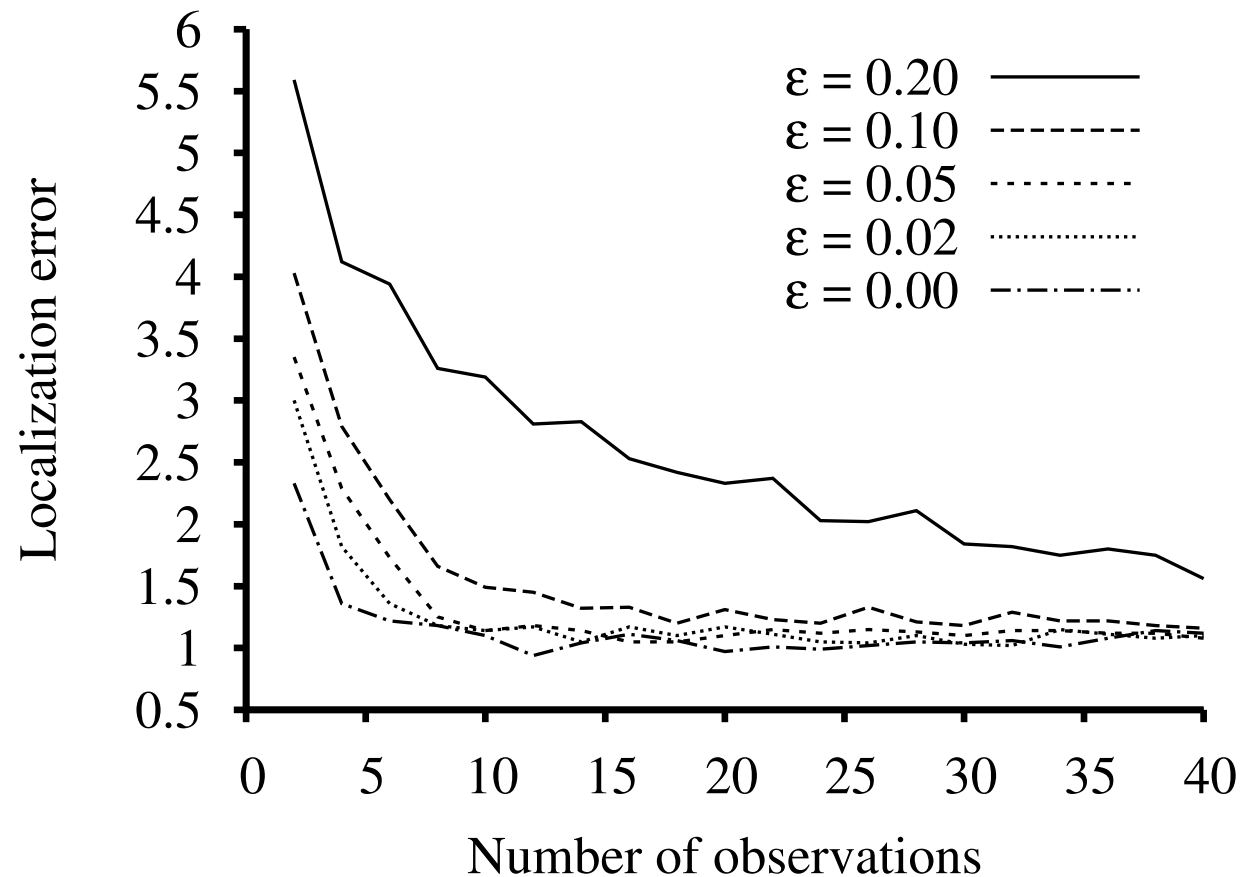


(a) Posterior distribution over robot location after $E_1 = \text{NSW}$



(b) Posterior distribution over robot location after $E_1 = \text{NSW}, E_2 = \text{NS}$

HMM example: Robot localization by filtering



ϵ = probability that one of the four sensor is wrong

(Q: in how many cases, in percent, at least one sensor is wrong?)

Summary

Tasks: filtering, prediction, smoothing, most likely sequence

Hidden Markov models have a single discrete state variable

KALMAN FILTERS. COMPLEX DECISIONS

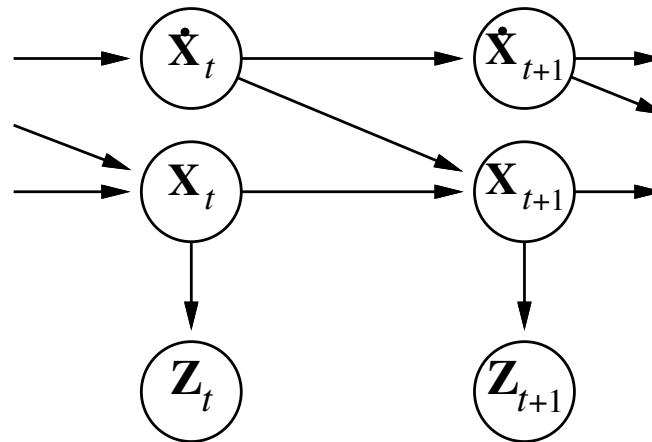
CHAPTER 15.4, 17 SECTIONS 1–3
(SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Kalman filters (a brief mention)
- ◇ Sequential decision problems
- ◇ Value iteration and Policy iteration

Kalman filters

For systems described by continuous variables (e.g., bird flying: $(X, Y, Z), (\dot{X}, \dot{Y}, \dot{Z})$)

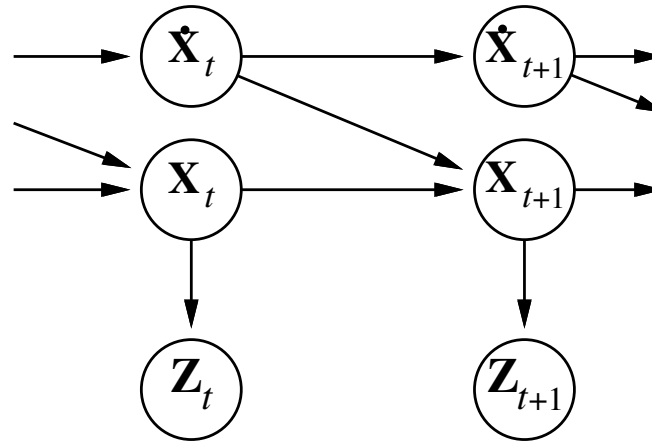


($\mathbf{X}_t$ = position vector, $\dot{\mathbf{X}}_t$ = velocity vector, $\mathbf{Z}_t$ = position measurement)

Both transition model and sensor models are linear Gaussian:

$\mathbf{X}_{t+1}$ is a linear function of $\mathbf{X}_t$ plus a Gaussian noise

Kalman filters



X position update: $X_{t+\Delta} = X_t + \dot{X}_t \Delta$ (Δ time interval, $\dot{X}$ constant in it)

Linear Gaussian transition model (includes Gaussian noise):

$$P(X_{t+\Delta} = x_{t+\Delta} | X_t = x_t, \dot{X}_t = \dot{x}_t) = N(x_t + \dot{x}_t \Delta, \sigma^2)(x_{t+\Delta})$$

Simple 1-D example (e.g., consumer confidence)

Let us consider a Gaussian random walk of a single continuous variable X_t , with a noisy observation Z_t . Assume variances: σ_x^2 (transition), σ_z^2 (sensor).

Transition model: $P(x_{t+1}|x_t) = \alpha e^{-1/2(\frac{(x_{t+1}-x_t)^2}{\sigma_x^2})}$ (with $P(x_0) = N(\mu_0, \sigma_0^2)$)

Sensor model: $P(z_t|x_t) = \alpha e^{-1/2(\frac{(z_t-x_t)^2}{\sigma_z^2})}$

Simple 1-D example (e.g., consumer confidence)

How does the filtering algorithm work in this case?

One-step prediction:

$$P(x_1) = \int_{-\infty}^{\infty} P(x_1|x_0)P(x_0)dx = \text{complete square, integrate} \dots = \alpha e^{-\frac{1}{2}\left(\frac{(x_1-\mu_0)^2}{\sigma_0^2+\sigma_x^2}\right)}$$

So: same μ_0 , variance as a sum of σ_0^2 and σ_x^2

Update:

$$P(x_1|z_1) = \alpha P(z_1|x_1)P(x_1)$$

Remark: Both steps are Gaussian

Simple 1-D example

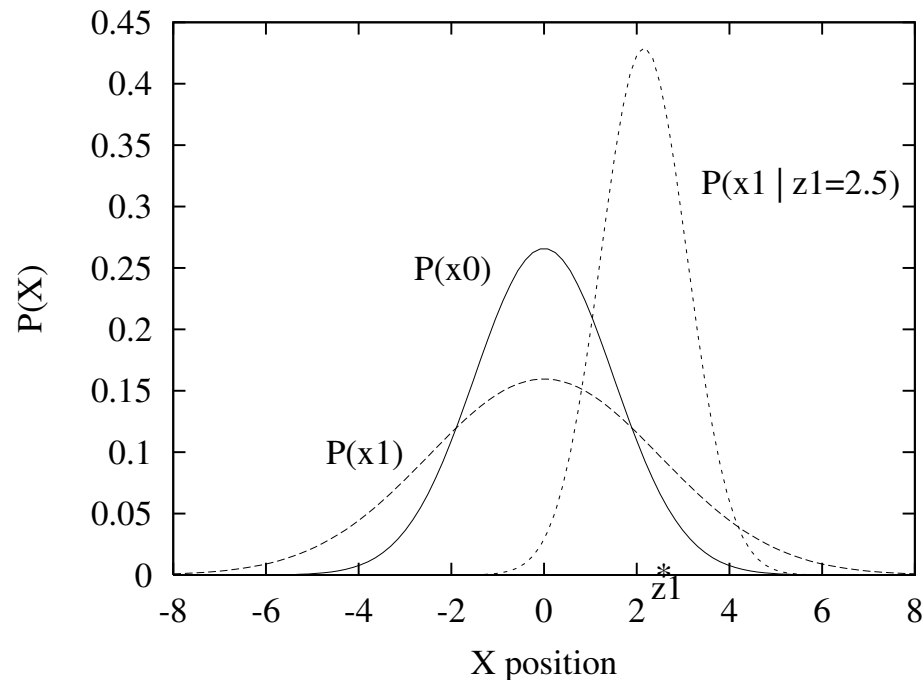
For μ_{t+1} and σ_{t+1}^2 , we proceed as above and get:

$$\mu_{t+1} = \frac{(\sigma_t^2 + \sigma_x^2)z_{t+1} + \sigma_z^2\mu_t}{\sigma_t^2 + \sigma_x^2 + \sigma_z^2} \quad \sigma_{t+1}^2 = \frac{(\sigma_t^2 + \sigma_x^2)\sigma_z^2}{\sigma_t^2 + \sigma_x^2 + \sigma_z^2}$$

Remarks:

1. μ_{t+1} weighted mean of the old mean μ_t and the new observation z_{t+1}
2. unreliable observation (high σ_z^2): the old mean μ_t will have big impact
unreliable old mean μ_t (high σ_t^2) or highly unpredictable process (high σ_x^2):
observation z_{t+1} will have higher impact
3. σ_{t+1}^2 does not depend on observation, so we can compute it in advance

Simple 1-D example

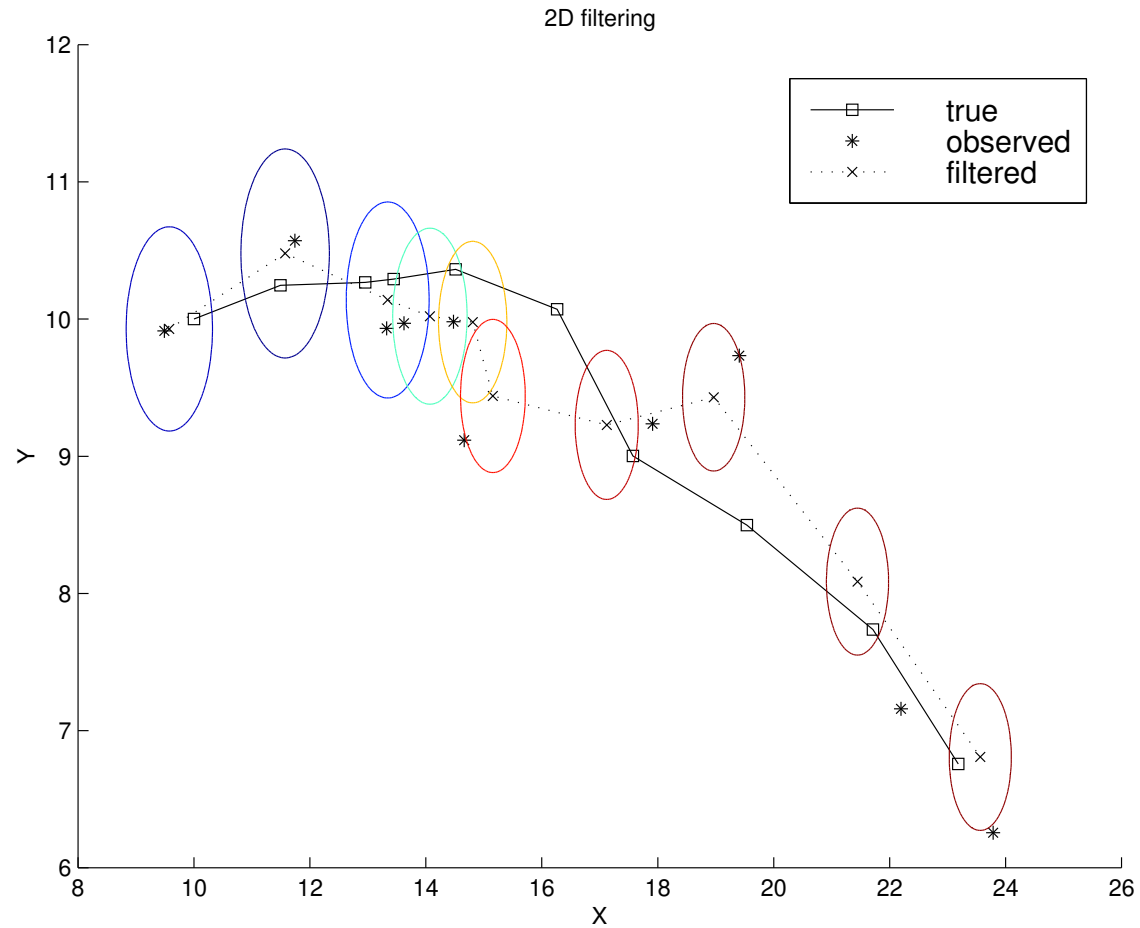


We assume $\mu_0 = 0.0$, $\sigma_0 = 1.0$, $\sigma_x = 2.0$, $\sigma_z = 1.0$, observation $z_1 = 2.5$

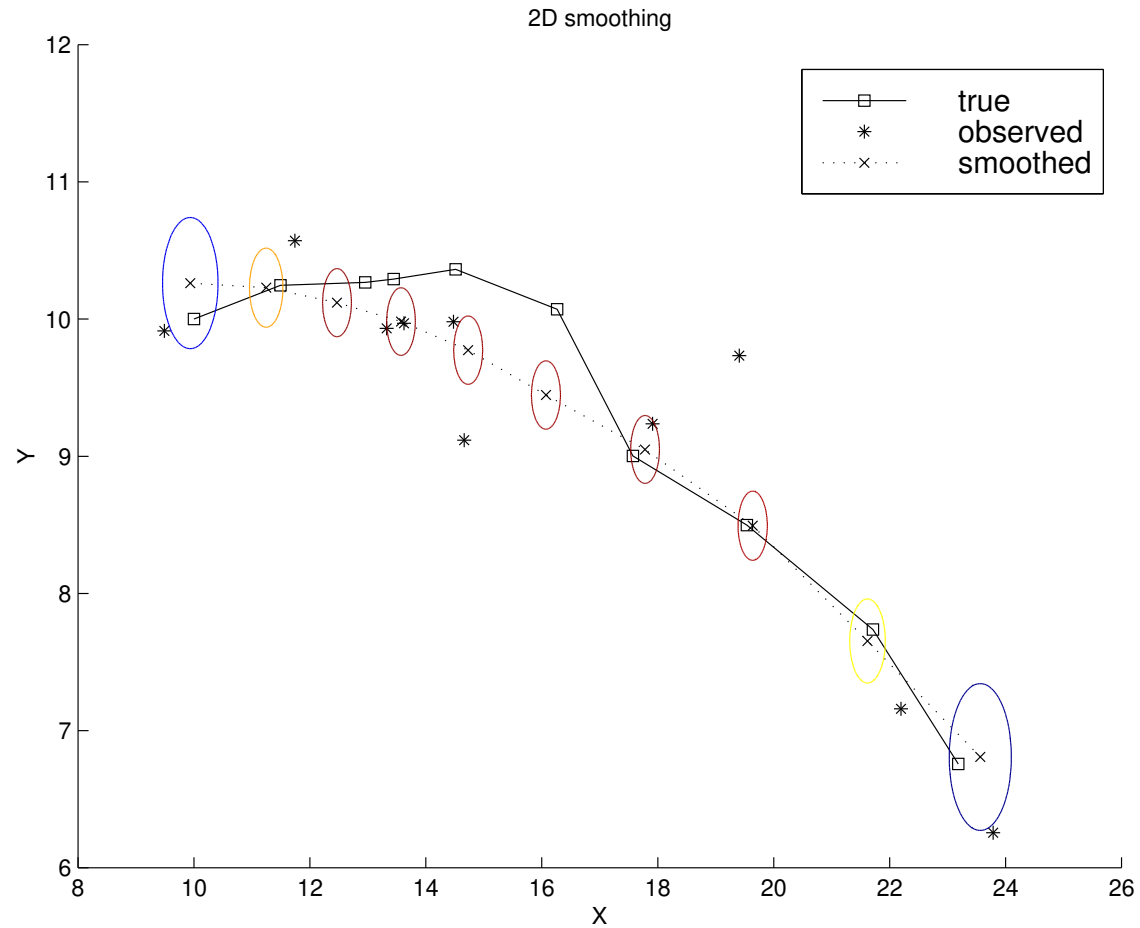
Prediction $P(x_1)$ is flattened in comparison with $P(x_0)$ by σ_z .

$P(x_1|z_1)$ is moved slightly to the left of z_1 (μ_1 influenced by observation z_t)

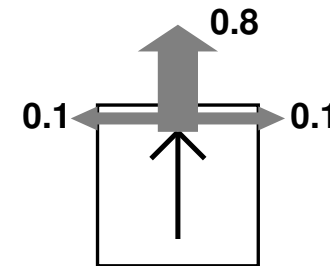
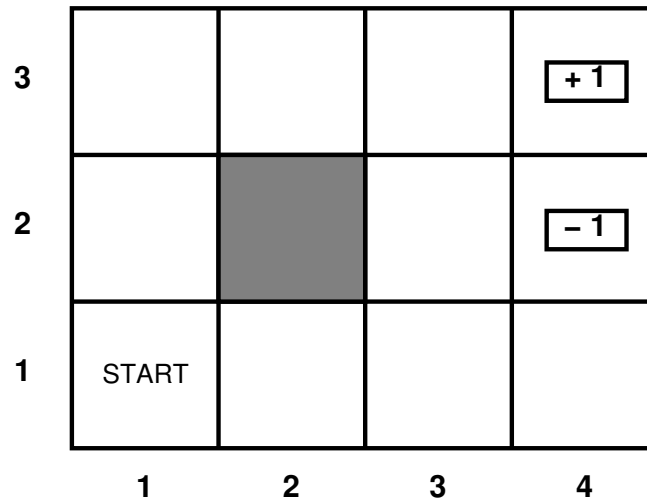
2-D tracking example: filtering



2-D tracking example: smoothing

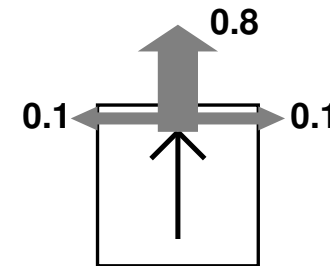
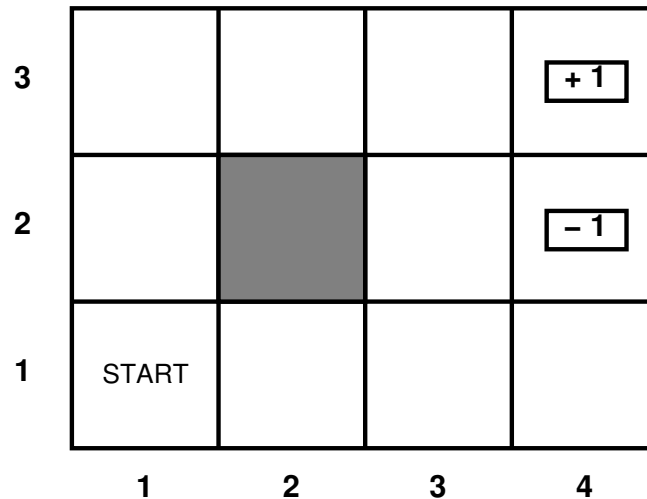


Example of a Markov Decision Process (MDP)



- ◇ States $s \in S$ (e.g., $(1, 1)$: nonterminal; $(4, 3)$ terminal)
 - ◇ Actions $a \in A$ (Unreliable: action "U" could result in moving the agent to its North neighbor with probability 0.8, but could also result in moving it to either its East or its West neighbor, each with probability 0.1)
- Q:** for action sequence UURRR, the probability the agent gets to $(4, 3)$ is:

Example of a Markov Decision Process (MDP)



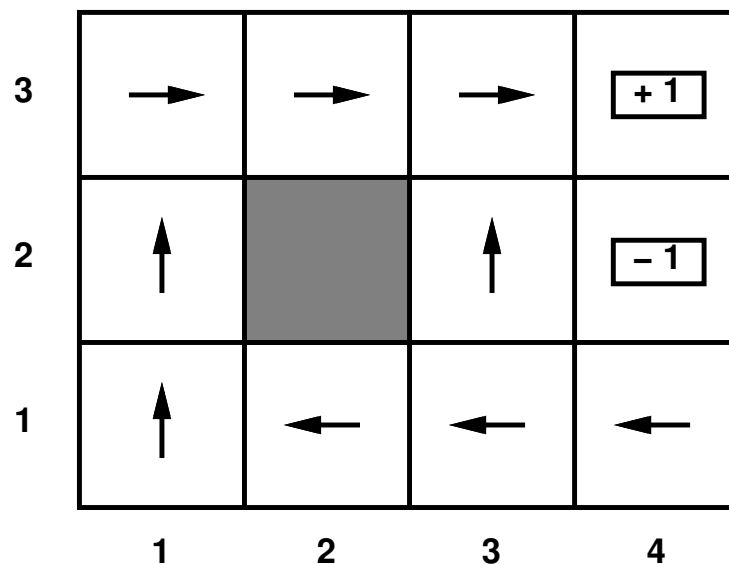
◇ **Transition model** $T(s, a, s') \equiv P(s'|s, a)$ probability that performing action a in state s will result in reaching state s' (Markovian!)

◇ **Reward function** $R(s)$ (or $R(s, a)$, $R(s, a, s')$) = $\begin{cases} -0.04 & \text{for nonterminal states} \\ \pm 1 & \text{for terminal states} \end{cases}$

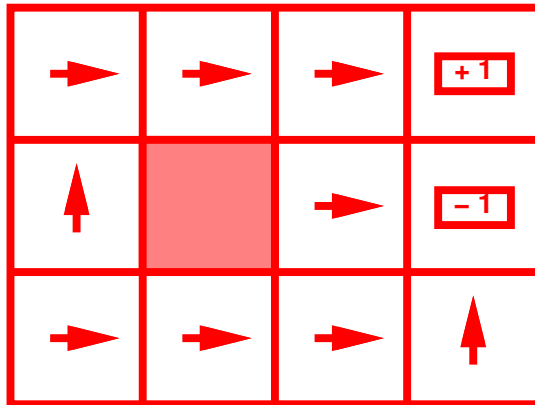
Solving MDPs

Aim: find an optimal *policy* $\pi(s)$ (i.e., best action for every possible state s)
Optimal policy = maximizes (say) the expected sum of rewards.

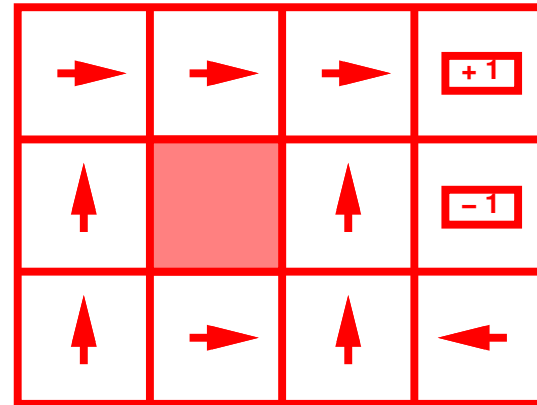
E.g.: optimal policy for $R(s) = -0.04$:



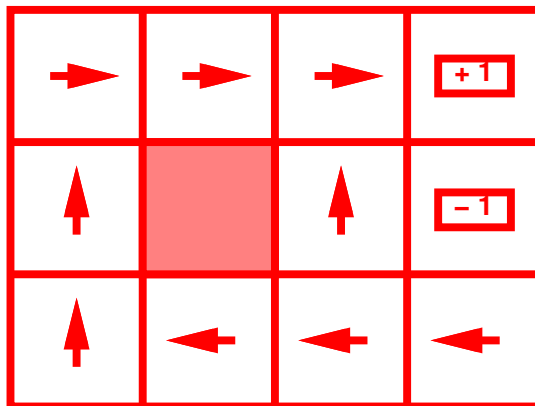
Risk and reward



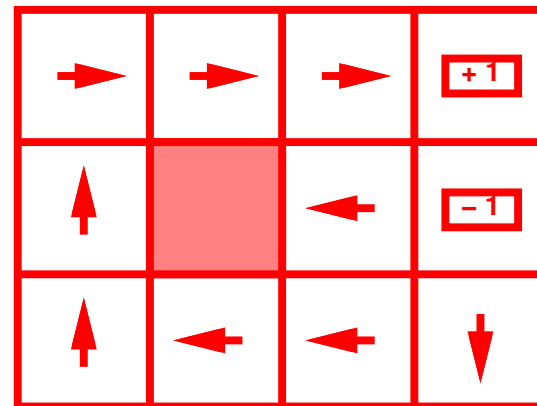
$$r = [-\infty : -1.6284]$$



$$r = [-0.4278 : -0.0850]$$



$$r = [-0.0480 : -0.0274]$$



$$r = [-0.0218 : 0.0000]$$

Get out at any cost / get out, but avoid -1 / avoid -1

Utility of state sequences

We must deal with preferences between reward *sequences*.

Typically consider **stationary preferences** on reward sequences:

$$[r, r_0, r_1, r_2, \dots] \succ [r, r'_0, r'_1, r'_2, \dots] \Leftrightarrow [r_0, r_1, r_2, \dots] \succ [r'_0, r'_1, r'_2, \dots]$$

Theorem: We can assess sequence utility based on rewards in two ways only:

- 1) Additive: $U([s_0, s_1, s_2, \dots]) = R(s_0) + R(s_1) + R(s_2) + \dots$
- 2) Discounted: $U([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$
(γ is a **discount factor** giving the importance of the future)

Utility of state sequences

Expected utility obtained by executing policy π starting in s :

$$U^\pi(s) = E(\sum_{t=0}^{\infty} \gamma^t R(s_t))$$

Optimal policy:

$$\pi^* = \operatorname{argmax}_{\pi} U^\pi(s)$$

Utility of states

Utility of a state (state value) = expected sum of discounted rewards assuming the agent executes an optimal policy

Given the state utilities, choosing the best action is just maximizing the expected utility of the immediate successors

3	0.812	0.868	0.918	+ 1
2	0.762		0.660	- 1
1	0.705	0.655	0.611	0.388
	1	2	3	4

3	→	→	→	+ 1
2	↑		↑	- 1
1	↑	←	←	←
	1	2	3	4

The Bellman equation (dynamic programming)

Relation between state utility and neighbor state utilities (Bellman equation):

$$U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|a, s) * U(s')$$

Utility of a state = current reward + $\gamma \times$ expected utility of the next state
(assuming the agent takes the best action)

E.g.,

$$U(1, 1) = -0.04 + \gamma \max \left\{ \begin{array}{l} 0.8 * U(1, 2) + 0.1 * U(2, 1) + 0.1 * U(1, 1), \quad U \\ 0.8 * U(1, 1) + 0.1 * U(1, 1) + 0.1 * U(1, 2), \quad L \\ 0.8 * U(1, 1) + 0.1 * U(1, 1) + 0.1 * U(2, 1), \quad D \\ 0.8 * U(2, 1) + 0.1 * U(1, 2) + 0.1 * U(1, 1) \end{array} \right\} R$$

One equation per state = n **nonlinear** equations in n unknowns

Value iteration algorithm

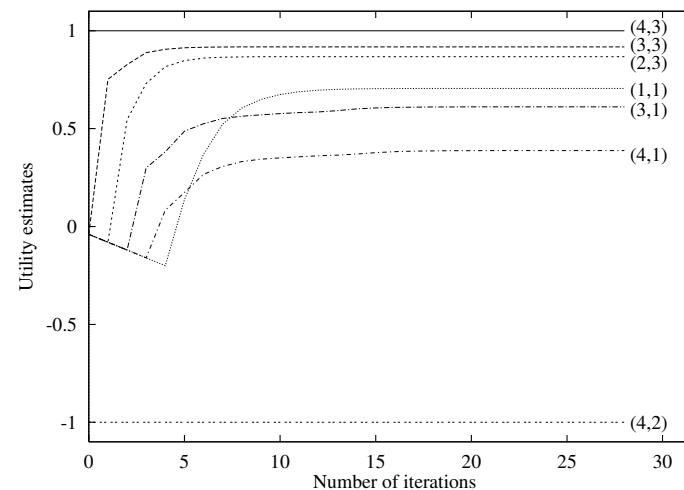
Idea: Update utilities to make them **locally consistent** with Bellman equation

Start with arbitrary utility values

repeat

$$U(s) \leftarrow R(s) + \gamma \max_a \sum_{s'} P(s'|s, a) * U(s') \quad \text{simultaneously for all } s$$

until "no change"



Policy iteration

Howard, 1960: search for optimal policy and utility values simultaneously

Algorithm:

$\pi \leftarrow$ an arbitrary initial policy

repeat until no change in π

 compute utilities given π

 update π as if utilities were correct (i.e., local MEU)

To compute utilities given a fixed π (**value determination**), one should solve:

$$U(s) = R(s) + \gamma \sum_{s'} U(s') T(s, \pi(s), s') \quad \text{for all } s$$

(n simultaneous **linear** (no max!) equations in n unknowns $\rightarrow O(n^3)$)

Modified policy iteration

Policy iteration often converges in few iterations, but each is expensive

Idea: use a few steps of value iteration (with fixed π), starting from the value produced the last time, to produce an approximation of utilities

Reinforcement learning algorithms operate by performing such updates based on the observed transitions made in an initially unknown environment

Summary

- ◇ Kalman filters allow n state variables linear Gaussian
- ◇ Sequential decision problems aim to find a policy which maximizes the expected sum of rewards
- ◇ Value iteration algorithm finds, for each state, the utility which is consistent with the Bellman equation. The optimal policy is then extracted by choosing, for each state, the action which maximizes the expected utility, assessed using the utilities computed above
- ◇ Policy iteration starts with an arbitrary policy, does a number of value iterations, then extracts the optimal policy corresponding to them

COMMUNICATION AND LANGUAGE

CHAPTER 22, 23 (SLIDES ADAPTED FROM S. RUSSELL)

Outline

- ◇ Grammar and Syntactic analysis
- ◇ Parsing
- ◇ Semantics
- ◇ some challenges for the Natural Language Processing

Wumpus lexicon: allowed words

Noun → *stench* | *breeze* | *glitter* | *nothing*
| *wumpus* | *pit* | *pits* | *gold* | *east* | ...

Verb → *is* | *see* | *smell* | *shoot* | *feel* | *stinks*
| *go* | *grab* | *carry* | *kill* | *turn* | ...

Adjective → *right* | *left* | *east* | *south* | *back* | *smelly* | ...

Adverb → *here* | *there* | *nearby* | *ahead*
| *right* | *left* | *east* | *south* | *back* | ...

Pronoun → *me* | *you* | *I* | *it* | ...

Name → *John* | *Mary* | *Boston* | *UCB* | *PAJC* | ...

Article → *the* | *a* | *an* | ...

Preposition → *to* | *in* | *on* | *near* | ...

Conjunction → *and* | *or* | *but* | ...

Digit → **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9**

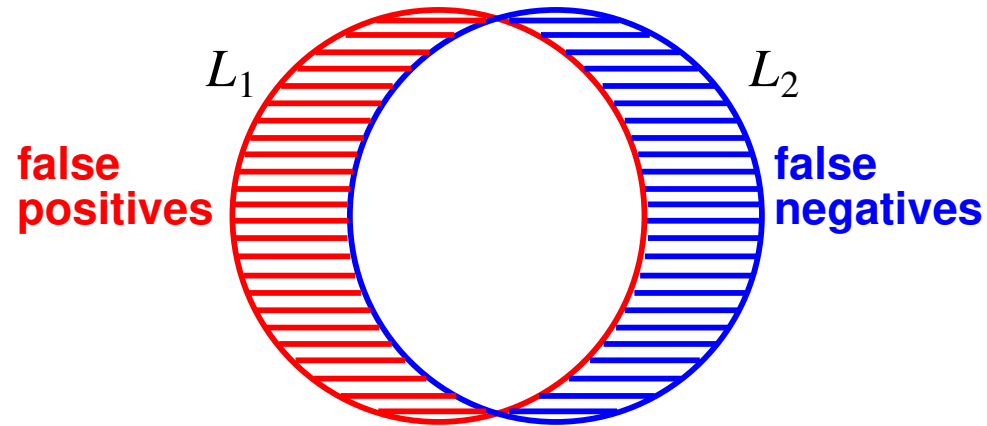
Divided into **closed** (e.g., pronoun) and **open** classes (e.g., noun)

Wumpus grammar: from words to structures

$S \rightarrow NP VP$	I + feel a breeze
$ S Conjunction S$	I feel a breeze + and + I smell a wumpus
$NP \rightarrow Pronoun$	I
$ Noun$	pits
$ Article Noun$	the + wumpus
$ Digit Digit$	3 4
$ NP PP$	the wumpus + to the east
$ NP RelClause$	the wumpus + that is smelly
$VP \rightarrow Verb$	stinks
$ VP NP$	feel + a breeze
$ VP Adjective$	is + smelly
$ VP PP$	turn + to the east
$ VP Adverb$	go + ahead
$PP \rightarrow Preposition NP$	to + the east
$RelClause \rightarrow \textit{that} VP$	that + is smelly

Grammaticality judgements

Formal language L_1 may differ from natural language L_2



Adjusting L_1 to agree with L_2 is a learning problem!

Real grammars 10–500 pages, insufficient even for "proper" English

Parse trees

Exhibit the grammatical structure of a sentence

I shoot the wumpus

Parse trees

Exhibit the grammatical structure of a sentence

Pronoun

I

Verb

shoot

Article

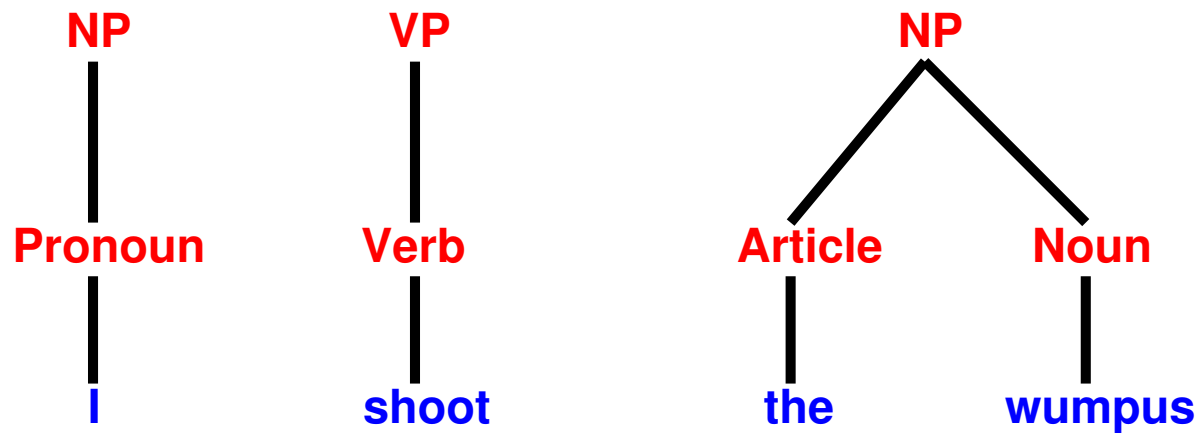
the

Noun

wumpus

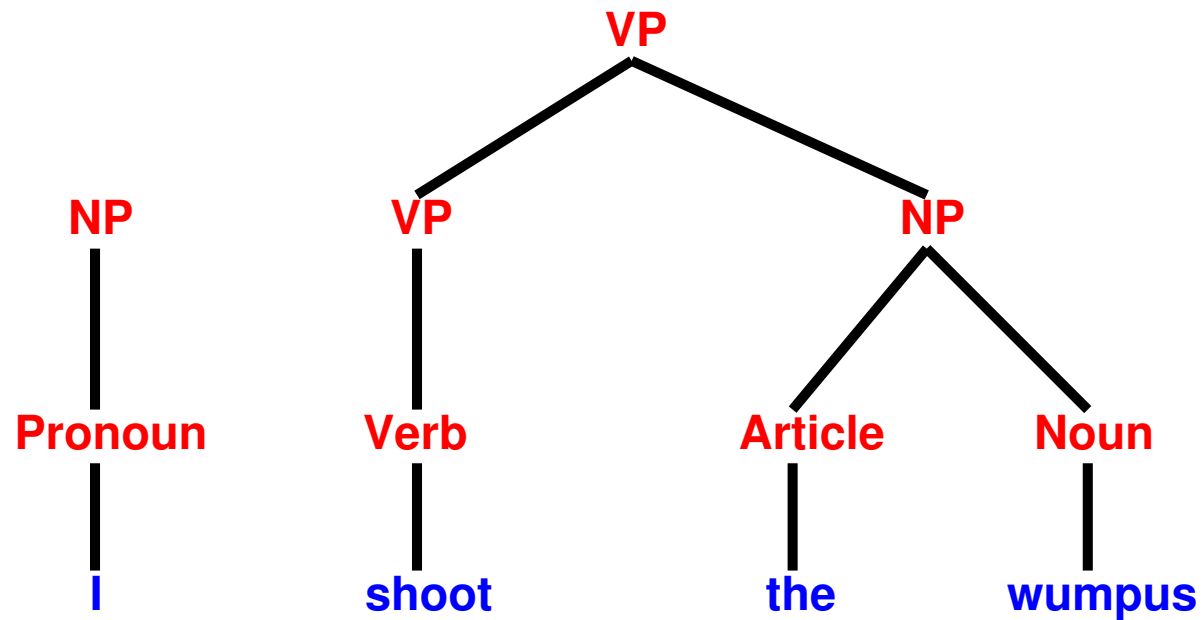
Parse trees

Exhibit the grammatical structure of a sentence



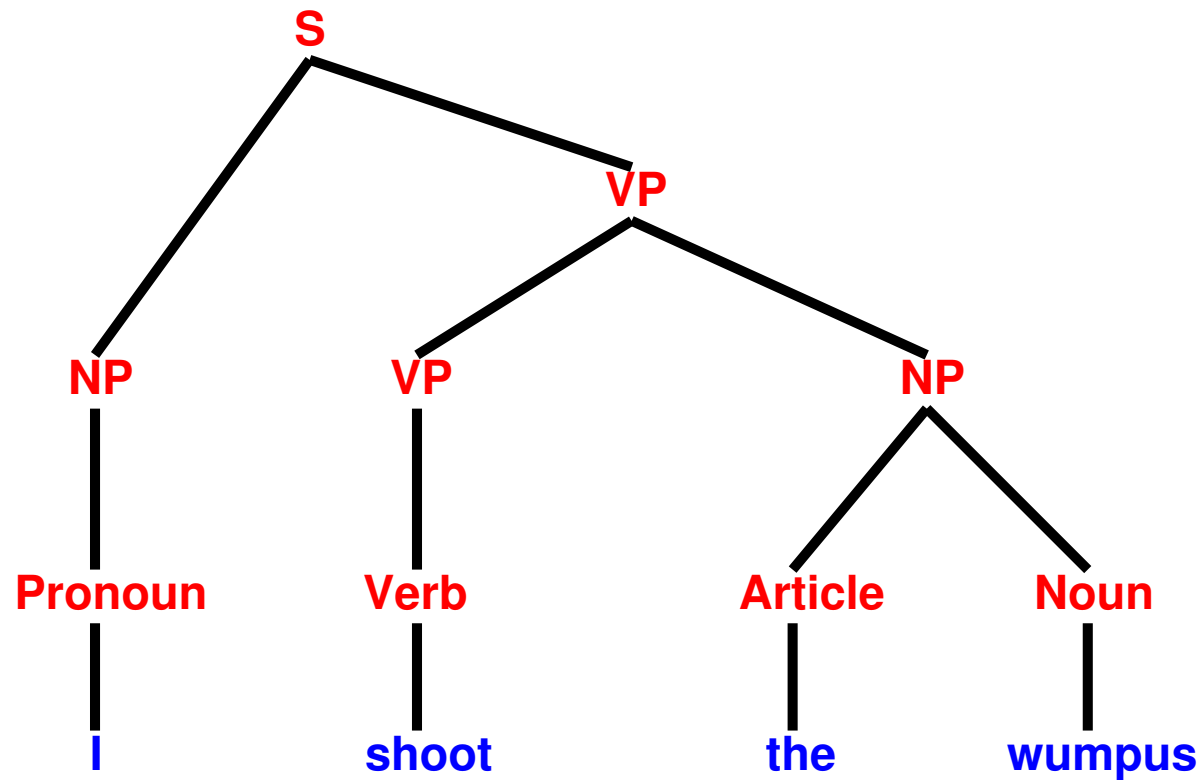
Parse trees

Exhibit the grammatical structure of a sentence

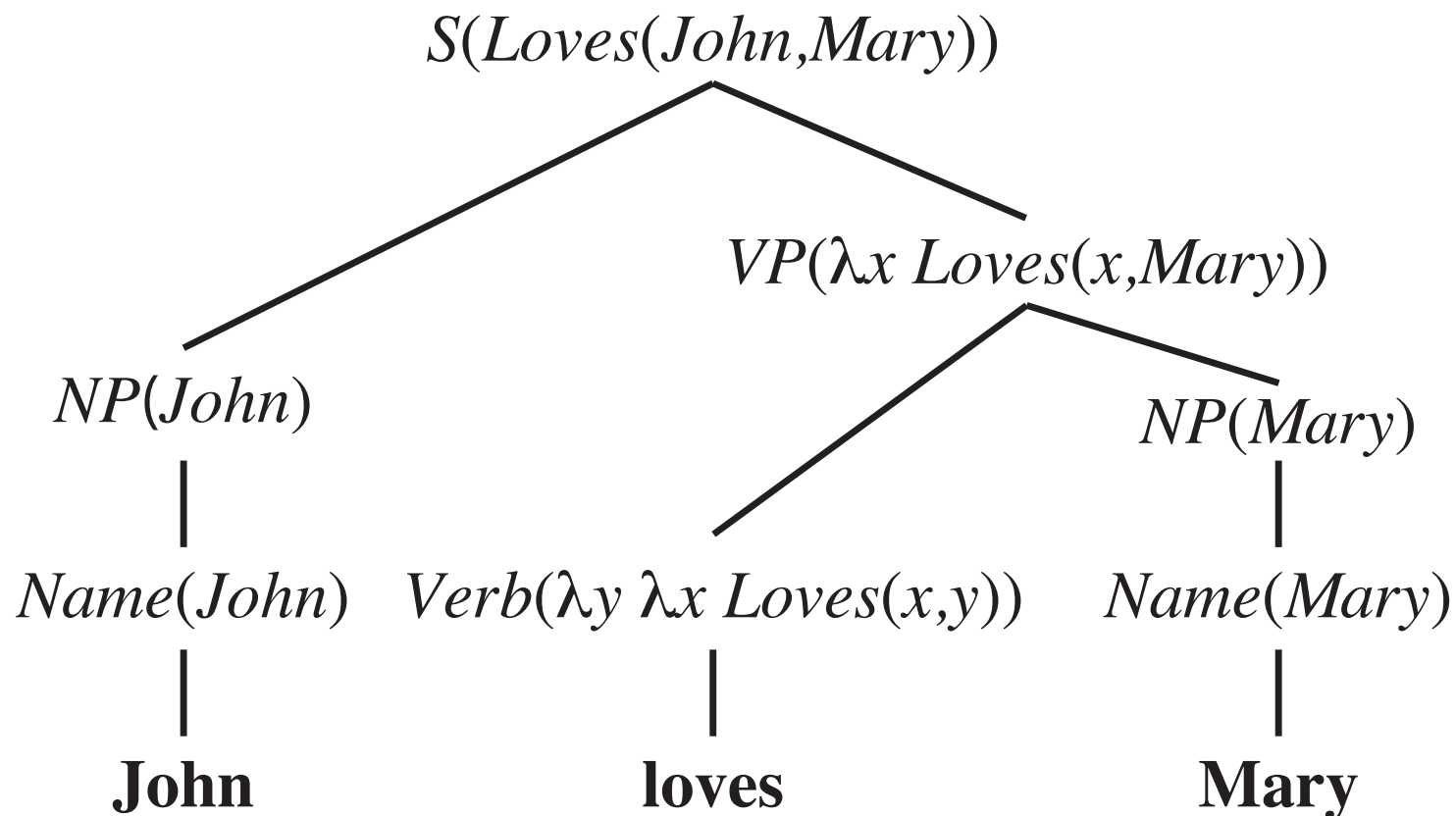


Parse trees

Exhibit the grammatical structure of a sentence



Semantics: John loves Mary



Real language: Challenges

Real human languages provide many problems for NLP:

- ◇ ambiguity
- ◇ anaphora
- ◇ discourse structure
- ◇ metonymy
- ◇ metaphor
- ◇ noncompositionality

Ambiguity

◇ Try to parse:

Helicopter powered by human flies

Professor strikes idle students

◇ Meanings of "with":

I ate spaghetti with meatballs / salad / abandon / a fork / a friend.

◇ Ambiguity types:

- lexical (polysemy): back adj/noun/adv
- syntactic: Vand rucsac de copil de piele
- semantic: coast road (alongside/to)
- referential: a huge rock star

Anaphora

Using pronouns to refer back to entities already introduced in the text:

After Mary proposed to John, **they** found a preacher and got married.

For the honeymoon, **they** went to Hawaii.

Mary saw a ring through the window and asked John for **it**

Mary threw a rock at the window and broke **it**

Mary flew from NY to LA. John flew **there** from Boston.

Metonymy

Using one noun phrase to stand for another:

I've read **Shakespeare**

Chrysler announced record profits

Summary

- ◇ Syntactic analysis could be done via augmented grammars
- ◇ Semantics: via lambda calculus
- ◇ Inherent difficulties are present in the symbolic approach